

MATHEMATICAL LOGIC & FOUNDATIONS OF MATHEMATICS

JOHN HUTCHINSON

Mathematical Sciences Institute
Australian National University

Primary website: <https://johnhutchinson.github.io>
ANU website: <https://maths.anu.edu.au/people/john-hutchinson>

May 27, 2026

Preface

These Notes are the text for a course on Mathematical Logic at the third and fourth year honours level, or first year graduate level, for students with a strong background in pure/theoretical mathematics.

I have tried to provide a clear route through to the major results. There are various optional asides and additional material, but this is indicated as such.

Mathematical logic grew out of the work by Hilbert, Gödel, Tarski, and others in the period 1930–1950 in order to address the paradoxes and problematic foundational issues raised previously by the work of Cantor, Frege and Russell.

The subject is one which is currently taught and researched in University Departments of Mathematics, Computer Science and Philosophy, with varying emphasis on topics and approach. The emphasis here is, naturally, mathematical! But we will also consider the broader insights and applications.

Pure/theoretical mathematics is an abstract discipline, extracting, analysing, extending and axiomatising the essence of ideas that may be common to a variety of disciplines, including the physical and biological sciences, computer science, economics and increasingly the social sciences.

Mathematical logic abstracts further from the reasoning itself which is used in mathematics, providing insight to the limits of human and machine cognition.

“Mathematical logic is the most abstract branch of mathematical thought, the most abstract human discipline.”¹

ARTIFICIAL INTELLIGENCE [AI]

This has an impact on the way the course is taught and assessed, and on the future of mathematical research.

The assessment is discussed in detail on the course homepage. But in essence it will be approximately 20% class participation, 20% assignments, 15% midsemester and 45% final exams. Class participation and discussion is required and is an essential part of the course.

Regarding the future of mathematics research see the [Talk by Terry Tao](#), including the interview in the second half. There is a series of [AMS articles on AI](#), see in particular the articles by the Australian mathematicians Terry Tao, Akshay Venkatesh and Geordie Williamson.

“Our generation is witnessing the coming-of-age of automated theorem proving and proof assistants, which seem destined to transform mathematical practice. Mathematicians of the future may be commonly using proof assistants and automated proof verifiers, with

¹Introduction ([Goldstern and Judah, 1995](#)).

large and growing databases of formally verified proofs, to undertake their mathematical investigations.” ([Hamkins, 2021](#), §5.4)

Contents

Preface	i
Introduction	xi
1 Structures and Languages	1
1.1 Truth and Proof	1
1.2 Mathematical Structures	3
1.3 Language and Syntax	5
1.4 Semantics	13
1.5 Related Reading	18
2 Hilbert's Deductive System	19
2.1 Axioms	19
2.2 Rules of Inference	23
2.3 Syntactic Entailment	24
2.4 Properties of Derivations	25
2.5 Material To Include	28
3 Gödel's Completeness Theorem	29
3.1 Overview	29
3.2 Consistency	29
3.3 Gödel's Completeness Theorem – Preliminaries	33
3.4 Gödel's Completeness Theorem – Proof	34
3.5 Wrap-Up	42
4 Basics: Set Theory	44
4.1 Axiom of Choice	44
4.2 A Little on Ordinals & Well-Orderings	44
4.3 A Little on Cardinals	50
5 Extended Completeness and Applications	54
5.1 Extended Completeness Theorem	54
5.2 Compactness	56
5.3 Lowenheim–Skolem Theorems	57
6 Basics: Peano Axioms; Primitive Recursion; Representability	58
6.1 The Three Topics	58
6.2 Peano Axioms and their Models	58
6.3 Primitive Recursive — Computability	64

6.4	Arithmetic Definability and Representability	70
7	Gödel's Incompleteness Theorem	81
7.1	Gödel Coding and Arithmetic Representation	81
7.2	Gödel's First Incompleteness Theorem	85
7.3	Summary	86
8	Computability	89
8.1	Summary of Computability Section 6.3	89
8.2	Computability Overview	89
8.3	Turing Machines	91
8.4	Computable Functions — Other Approaches	93
8.5	Church–Turing Thesis	93
8.6	Halting Problem	94
8.7	Arithmetic Representation	94
8.8	Entscheidungsproblem	95
8.9	(Optional) More on Computably Enumerable Sets	96
8.10	(Optional) Ackermann Function	97
9	Propositional Logic	102
9.1	The Basics of Propositional Logic	102
9.2	Optional: A Deductive System	105
9.3	Optional: Deduction Theorem	106
9.4	Optional: Completeness Theorem	109
10	Peano Axioms and Arithmetic	114
11	Model Theory	115
12	Axiom of Choice	116
13 (Optional)	Ordinals	117
13.1	Linear Orderings	117
13.2	Ordinals as Sophisticated Counting	119
13.3	Ordinals and Iterated Processes	120
13.4	Equivalent Definitions of Well-Orderings	122
13.5	Properties of Well-Orderings	124
13.6	Transfinite induction and recursion	125
14	Cardinals	126
14.1	Cardinal numbers	126
15	Zermelo–Frankel Set Theory	129
15.1	Metatheory	129
15.2	Structures (V, ϵ)	130
15.3	ZFC Axioms	131
15.4	Classes	137
16	Extras	139

References**140**

Detailed Contents

Preface	i
Introduction	xi
1 Structures and Languages	1
1.1 Truth and Proof	1
1.1.1 Truth	1
1.1.2 First-Order Logic	1
1.1.3 Proof	2
1.1.4 “Truth = Proof”	2
1.2 Mathematical Structures	3
1.2.1 What is a Structure?	3
1.2.2 Examples	3
1.2.3 Fonts & Alphabets – An Aside	4
1.2.4 Variants for Structures	5
1.2.5 Foundational Issues	5
1.3 Language and Syntax	5
1.3.1 Symbols for a Language	6
1.3.2 Structures for a Language	6
1.3.3 Logical Symbols	7
1.3.4 MetaLanguage and MetaLanguage Symbols	7
1.3.5 Defined Logical Symbols	8
1.3.6 Terms	8
1.3.7 Atomic Formulas	9
1.3.8 Formulas	9
1.3.9 Complexity of Terms and Formulas	9
1.3.10 Proving Results about Terms and Formulas	10
1.3.11 Uniqueness of Parsing	11
1.3.12 Other Logics	11
1.3.13 Cardinality of a Language	11
1.3.14 Subformulas	11
1.3.15 Free and Bound Variables	12
1.3.16 Sentences	13
1.3.17 Notation Regarding Free Variables	13
1.3.18 Expansion of a Language	13
1.4 Semantics	13
1.4.1 On Defining Truth in a Structure	14
1.4.2 Interpretation of a Term in a Structure	14
1.4.3 Satisfaction of a Formula in a Structure	15

1.4.4	Defined Logical Symbols	17
1.4.5	Definable Sets	17
1.4.6	Semantic Entailment	17
1.4.7	Examples for Validity	18
1.5	Related Reading	18
2	Hilbert's Deductive System	19
2.1	Axioms	19
2.1.1	Preliminaries	19
2.1.2	Propositional Axioms	20
2.1.3	Quantifier Axioms	20
2.1.4	Equality Axioms	21
2.1.5	Universal Validity	22
2.2	Rules of Inference	23
2.2.1	Modus Ponens	23
2.2.2	Generalisation	23
2.2.3	Preserving Validity	23
2.3	Syntactic Entailment	24
2.3.1	Derivations	24
2.3.2	Soundness Theorem	25
2.4	Properties of Derivations	25
2.4.1	An Equivalence Relation on Constant Symbols	25
2.4.2	Existential Generalisation	26
2.4.3	Deduction Theorem	27
2.5	Material To Include	28
3	Gödel's Completeness Theorem	29
3.1	Overview	29
3.2	Consistency	29
3.3	Gödel's Completeness Theorem – Preliminaries	33
3.4	Gödel's Completeness Theorem – Proof	34
3.4.1	Step 1 Adding a Set $\mathcal{C}$ of New Constant Symbols to $\mathcal{L}$	34
3.4.2	Step 2 Extend Σ to Σ^* with $\mathcal{C}$ a Set of Henkin Witnesses	34
3.4.3	Step 3 Enlarge Σ^* to a Complete Theory	36
3.4.4	Step 4 An Equivalence Relation on $\mathcal{C}$	36
3.4.5	Step 5 Constructing the Universe $\mathcal{C}/\sim$ for a Structure $\mathfrak{A}^*$	37
3.4.6	Step 6 Proof that $\mathfrak{A}^* \models \Sigma^*$	39
3.4.7	Step 7 Definition of $\mathfrak{A}$ & Proof that $\mathfrak{A} \models \Sigma$	41
3.4.8	Step 8 Proof that $ \mathfrak{A} \leq \aleph_0$	42
3.5	Wrap-Up	42
4	Basics: Set Theory	44
4.1	Axiom of Choice	44
4.2	A Little on Ordinals & Well-Orderings	44
4.2.1	Introduction	45
4.2.2	Integers and Induction	45
4.2.3	Well-Orderings and Some Early Ordinals	45
4.2.4	Properties of Well-Orderings	46
4.2.5	Ordinals	48

4.2.6	Ordinals and Transfinite Induction	49
4.2.7	Well-Ordering Principle Equivalent to Axiom of Choice	49
4.3	A Little on Cardinals	50
4.3.1	Cardinality	50
4.3.2	Comparing Cardinals	51
4.3.3	Cardinals as Ordinals	53
5	Extended Completeness and Applications	54
5.1	Extended Completeness Theorem	54
5.1.1	Step 1 Adding a Set $\mathcal{C}$ of New Constant Symbols to $\mathcal{L}$	55
5.1.2	Step 2 Extend Σ to Σ^* with $\mathcal{C}$ a Set of Henkin Witnesses	55
5.1.3	Step 3 Enlarge Σ^* to a Complete Theory	55
5.1.4	Step 4 An Equivalence Relation on $\mathcal{C}$	55
5.1.5	Step 5 Constructing the Universe $\mathcal{C}/\sim$ for a Structure $\mathfrak{A}^*$	56
5.1.6	Step 6 Proof that $\mathfrak{A}^* \models \Sigma^*$	56
5.1.7	Step 7 Definition of $\mathfrak{A}$ & Proof that $\mathfrak{A} \models \Sigma$	56
5.1.8	Step 8 Proof that $ \mathfrak{A} \leq \kappa$	56
5.2	Compactness	56
5.2.1	Compactness Theorem	56
5.2.2	An Application of Compactness	56
5.3	Lowenheim–Skolem Theorems	57
6	Basics: Peano Axioms; Primitive Recursion; Representability	58
6.1	The Three Topics	58
6.2	Peano Axioms and their Models	58
6.2.1	The Peano Axioms [PA]	58
6.2.2	Variants and Expanded Language	59
6.2.3	How Strong are the Peano Axioms?	60
6.2.4	Nonstandard Models of Arithmetic	60
6.2.5	(Optional) Countable Dense Linear Orderings	62
6.3	Primitive Recursive — Computability	64
6.3.1	For and While Loops	64
6.3.2	Computability Classes	65
6.3.3	Function vs Algorithm	66
6.3.4	Primitive Recursive Functions and Relations – Kleene Approach	66
6.3.5	Equivalence of Two Definitions of Primitive Recursive	69
6.4	Arithmetic Definability and Representability	70
6.4.1	Overview	70
6.4.2	Definable/Expressible in $\mathfrak{N}$	70
6.4.3	Representable in PA	71
6.4.4	(Optional) Stronger Notions of Function Representability	72
6.4.5	$\Delta_0, \Sigma_1, \Pi_1$ Formulas	73
6.4.6	Semantic Properties of $\Delta_0, \Sigma_1, \Pi_1$ Formulas	74
6.4.7	(Optional) $\Sigma_n, \Pi_n, \Delta_n$: Formulas and Sets	75
6.4.8	Representability of P.R. Functions/Relations; β -Function	77
7	Gödel’s Incompleteness Theorem	81
7.1	Gödel Coding and Arithmetic Representation	81
7.1.1	Coding Formulas – The Idea	81

7.1.2	<i>Coding Formulas – Some Details</i>	81
7.1.3	<i>Primitive Recursive Syntactic Relations and Constructions</i>	82
7.1.4	<i>Coding Proofs</i>	83
7.1.5	<i>Arithmetic Representation</i>	83
7.1.6	<i>Provability</i>	84
7.2	Gödel's First Incompleteness Theorem	85
7.3	Summary	86
7.3.1	<i>Overview of Incompleteness Proof</i>	86
7.3.2	<i>Incompleteness for Extensions of PA</i>	86
7.3.3	<i>Non Self-Referential Examples</i>	86
7.3.4	<i>Syntactic Versions of the Incompleteness Theorem</i>	87
7.3.5	<i>Gödel's Second Incompleteness Theorem</i>	87
7.3.6	<i>For Contemplation</i>	88
7.3.7	<i>More Contemplation</i>	88
8	Computability	89
8.1	Summary of Computability Section 6.3	89
8.2	Computability Overview	89
8.3	Turing Machines	91
8.3.1	<i>Why Turing Machines?</i>	91
8.3.2	<i>Description</i>	91
8.3.3	<i>Partial Computable Functions and Turing Machines</i>	92
8.3.4	<i>Total Computable Functions and Turing Machines</i>	92
8.4	Computable Functions — Other Approaches	93
8.4.1	<i>While Loops</i>	93
8.4.2	<i>Kleene Approach</i>	93
8.5	Church–Turing Thesis	93
8.6	Halting Problem	94
8.7	Arithmetic Representation	94
8.8	Entscheidungsproblem	95
8.9	(Optional) More on Computably Enumerable Sets	96
8.10	(Optional) Ackermann Function	97
8.10.1	<i>History</i>	97
8.10.2	<i>Applications</i>	97
8.10.3	<i>The Sequence of Unary Ackermann Functions</i>	97
8.10.4	<i>Computing the Ackermann Sequence</i>	100
8.10.5	<i>Knuth Up-Arrow Notation</i>	100
8.10.6	<i>Binary Function Formulation</i>	101
8.10.7	<i>The Ackermann Function is Not Primitive Recursive</i>	101
9	Propositional Logic	102
9.1	The Basics of Propositional Logic	102
9.1.1	<i>Propositional Symbols and Formulas.</i>	102
9.1.2	<i>Complexity of a Propositional Formula.</i>	102
9.1.3	<i>Truth Values.</i>	103
9.1.4	<i>Defined Connectives.</i>	103
9.1.5	<i>Truth Tables</i>	103
9.1.6	<i>Tautologies</i>	104
9.1.7	<i>Disjunctive Normal Form</i>	104

9.2	Optional: A Deductive System	105
9.3	Optional: Deduction Theorem	106
9.4	Optional: Completeness Theorem	109
10	Peano Axioms and Arithmetic	114
11	Model Theory	115
12	Axiom of Choice	116
13	(Optional) Ordinals	117
13.1	Linear Orderings	117
13.2	Ordinals as Sophisticated Counting	119
13.3	Ordinals and Iterated Processes	120
13.4	Equivalent Definitions of Well-Orderings	122
13.5	Properties of Well-Orderings	124
13.6	Transfinite induction and recursion	125
14	Cardinals	126
14.1	Cardinal numbers	126
15	Zermelo–Frankel Set Theory	129
15.1	Metatheory	129
15.2	Structures (V, ϵ)	130
	15.2.1 <i>Notation for first-order language v. metalanguage</i>	131
15.3	ZFC Axioms	131
	15.3.1 <i>Extensionality Axiom</i>	131
	15.3.2 <i>Pair Axiom</i>	131
	15.3.3 <i>Separation Axiom-Schema</i>	132
	15.3.4 <i>Empty Set Axiom</i>	132
	15.3.5 <i>Union Axiom</i>	132
	15.3.6 <i>Power Set Axiom</i>	132
	15.3.7 <i>Axiom of Infinity</i>	133
	15.3.8 <i>Definition of ω</i>	133
	15.3.9 <i>Non-Standard Integers – Some Weird Stuff</i>	133
	15.3.10 <i>Order Type of Non-Standard Integers</i>	134
	15.3.11 <i>Replacement Axiom Schema</i>	135
	15.3.12 <i>Axiom of Foundation</i>	136
15.4	Classes	137
	15.4.1 <i>Sets, Classes and Metasets</i>	137
16	Extras	139
	References	140

Introduction

In mathematics we typically prove results from a set of axioms, as for example in the study of groups, fields, rings, topology or the real number system. In mathematical logic we formalise the methods of proof, and provide axioms for the underlying logic and rules of inference

The first emphasis in these notes is on the Completeness and Incompleteness Theorems for first-order logic. This is covered in Chapters 1–4 and I have provided a direct route through to these results. This is conceptually the most challenging part of the course.

You will find the material from Chapters 5 onwards, at least to the extent it is treated here, to be more similar to the mathematics you have previously learnt.

The beginning and introductory material from the later chapters is used in Chapters 1–4. You may already know some of this material. But I always point to this material explicitly when we proceed through the first four chapters, and we will cover it in class as it is needed. I have left a more complete (and yet still very incomplete!) development of model theory, computability, the axiom of choice, ordinals, cardinals and the Zermelo–Fraenkel axioms of set theory until Chapters 5 and beyond. Most of these chapters can be expanded into a book in their own right!

There are many *Exercises*. They are intended primarily to help you understand the material, and to check that you do indeed understand. Please attempt them all, and write out your solutions as you move through the text.

Chapter 1

Structures and Languages

1.1 TRUTH AND PROOF

What is the relationship between proof and truth? Does every truth have a proof?

The following introductory comments use terminology¹ that has not yet been defined. Hopefully it will be reasonably clear from the context what is intended. Defining these concepts carefully will be a major undertaking.

1.1.1 TRUTH

Suppose for example that $\mathfrak{G}$ ² is a mathematical structure (such as a group) and $\varphi(x_1, \dots, x_n)$ is an assertion (i.e. formula, in standard logic terminology) in some appropriate language $\mathcal{L}$ (for groups in this case). Then $\varphi(x_1, \dots, x_n)$ will be true or false in $\mathfrak{G}$ once we assign an element a_i of $\mathfrak{G}$ to each x_i .

We will define carefully what it means for $\varphi(x_1, \dots, x_n)$ to be *true* in $\mathfrak{G}$ for such an assignment. We write this as

$$\mathfrak{G} \models \varphi[a_1, \dots, a_n] \quad \text{eq sat (1.1)}$$

and read it as “ $\varphi(x_1, \dots, x_n)$ is *true* (or *is satisfied*) in $\mathfrak{G}$ at $(a_1, \dots, a_n)$ ”.

Suppose now that φ is a sentence (a formula with no free variables)³, and Σ is a set of sentences. By

$$\Sigma \models \varphi, \quad \text{eq log imp (1.2)}$$

which we read as “ Σ semantically implies φ ”, it is meant that φ is true in every structure in which all sentences from Σ are true.

Note that (1.2) does not indicate how one might establish that $\Sigma \models \varphi$, just that it *is* the case for some reason or other.

1.1.2 FIRST-ORDER LOGIC

To define (1.1) and (1.2) carefully we first need to define the set of all possible formulas in the language $\mathcal{L}$. For us, this set of formulas in $\mathcal{L}$ is the set of formulas in the *first-order logic*⁴ for $\mathcal{L}$. We carry out that program in this Chapter.

¹In particular: Proof, truth, structure, language, formula, sentence.

² $\mathfrak{G}$ is the letter G in the Fraktur font. See Subsection 1.2.3

³Such as $\forall x \forall y (x \cdot y = y \cdot x)$. But not $x \cdot y = y \cdot x$, in which the variables x and y are free.

⁴“First-order” means that quantifiers $\forall$ and $\exists$ range over the elements of a structure, rather than over subsets of a structure.

First-order logic may seem to be too restrictive in the case of group theory since we cannot express the notion of an arbitrary subset or subgroup in first-order logic.

But all of mathematics can be developed in set theory, and set theory is most naturally developed in an appropriate first-order logic. *Much* more needs to be said about this matter, and this we will do as the course proceeds.

*The notion of truth is a semantic⁵ notion.
The notion of proof is a syntactic⁶ notion.*

In principle one can verify and analyze a proof as a purely syntactic object, without a concept of meaning and without interpreting the language in a structure.

1.1.3 PROOF

In comparison with (1.2), with Σ and φ as there, one defines

$$\Sigma \vdash \varphi \quad \text{eq: log ded (1.3)}$$

to mean there is a formal proof of φ from Σ , using some deductive calculus of logical rules and axioms. Equation (1.3) is read as “there is a (formal) proof from Σ to φ ”.

In Chapter 2 we develop the Hilbert proof system. We study formal proofs not with the intention of writing such proofs, but in order to understand the nature of proof and its capabilities and limits. This is currently significant in relation to proof checking and theorem proof assistant programs such as [Lean](#).

1.1.4 “TRUTH = PROOF”

Gödel’s Completeness Theorem in Chapter 3 is the amazing result:

$$\Sigma \models \varphi \iff \Sigma \vdash \varphi \quad \text{eq: completeness (1.4)}$$

In words: The sentence φ is true whenever all the sentences (think “axioms”) in Σ are true, if and only if, there is a formal proof of φ from Σ .

The direction “ $\Leftarrow$ ” is straightforward, it just says that the logical rules and axioms preserve truth. But the direction “ $\Rightarrow$ ” is impressive: if we can deduce φ from Σ for whatever reason then there is a formal logical proof in (for example) the Hilbert system.

I need to clarify the statement “Truth = Proof”. It is true in first-order logic. But all of mathematics can (essentially) be done in set theory, and set theory is done in first-order logic. I will elaborate as the course proceeds.

A Final Remark: We are interested in an analysis at the meta-logic level of formal proofs in first-order logic. Hilbert systems for first-order logic are particularly convenient since its formal proofs are linear sequences of strings of symbols, and are readily codeable arithmetically.

Gentzen [natural deduction](#) systems and Gentzen [sequent calculus](#) systems, both of which are tree-like, are more appropriate for proof construction and proof analysis.

⁵Semantic: adjective, relating to meaning in language or logic. Origin: mid 17th century, from French ‘sémantique’, from Greek *sēmantikos* ‘significant’.

⁶Syntactic: adjective, relating to the arrangement and rules for words and phrases to create well-formed sentences in a language. Origin: mid 16th century: via late Latin from Greek ‘suntaxis’, from *sun* ‘together’ + *tassein* ‘arrange’.

1.2 MATHEMATICAL STRUCTURES

sec s1

1.2.1 WHAT IS A STRUCTURE?

A (*mathematical*) *structure* $\mathfrak{A}$ (for example, a group, field, vector space, set of complex numbers, etc.) can be represented as follows:

$$\mathfrak{A} = \langle A, R_i, F_j, C_k \rangle_{i \in I, j \in J, k \in K} \quad (1.5) \quad \text{str}$$

where:

- (i) A is the universe under consideration (e.g., the set of group elements, the set of real numbers, etc.), and $A \neq \emptyset$ for technical reasons,
- (ii) each R_i is an n_i -ary relation on A for some natural number $n_i \geq 1$, i.e. $R_i \subseteq A^{n_i}$,
- (iii) each F_j is an n_j -ary function on A for some natural number $n_j \geq 1$, i.e. $F_j : A^{n_j} \rightarrow A$,
- (iv) each C_k is a member of A , i.e. $C_k \in A$.

We shall try to be consistent in our notation. Thus the universe of a structure $\mathfrak{A}$ is A , of $\mathfrak{B}$ is B , of $\mathfrak{C}$ is C , etc. (See the following NOTES regarding the Fraktur Font used here.)

1.2.2 EXAMPLES

1. The structure for a group with multiplication $\circ$ and identity element e might be written $\mathfrak{G} = \langle G, \circ, e \rangle$.

We could include a unary inverse operator denoted by the symbol ι or $^{-1}$, so the structure is then $\mathfrak{G} = \langle G, \circ, \iota, e \rangle$ or $\mathfrak{G} = \langle G, \circ, ^{-1}, e \rangle$. Alternatively we could treat the inverse as a defined operator via the axioms for group theory. There is considerable flexibility and we will not need to be concerned with this from a foundational perspective.

2. The structure for a linear ordering is of the form $\mathfrak{L} = \langle L, < \rangle$, where L is a set and $<$ is a linear ordering of L .

3. The structure for a simple undirected graph is $\mathfrak{G} = \langle V, E \rangle$, where V is the set of vertices and E is a binary relation interpreted by $E(v, w)$ iff there is an edge between v and w .

4. Zermelo–Frankel set theory is discussed in Chapter 15. The language is particularly simple as it has just a single binary relation symbol, interpreted as membership and usually written $\in$. If we want to distinguish between the symbol and its interpretation in a structure, we occasionally write E for the symbol.

What makes this impressive is that all of mathematics can be formulated in set theory. What makes this strange is that there are countable models of the axioms, by the Lowenheim–Skolem Theorem.

Exercise 1.1. If membership is just treated as a binary relation, what are some of the properties that it should have? $\square$

5. The (first-order) structure for the real number system is $\mathfrak{R} = \langle \mathbb{R}, +, \cdot, <, 0, 1, \rangle$ where $+$ and $\cdot$ are binary operators on $\mathbb{R}$, $<$ is a binary relation, 0 and 1 are constants, all with their standard interpretation.

However, the completeness axiom for the real number system refers to arbitrary subsets of $\mathbb{R}$ and so is not a first-order concept. On the other hand, we can imbed $\mathfrak{R}$ within set theory, which *is* first-order. Again, we discuss this later.

1.2.3 FONTS & ALPHABETS – AN ASIDE

1. *Fraktur Font*: For structures/models in mathematical logic and for Lie algebras, the **Fraktur Font** is standard usage. In LaTeX use `\mathfrak{ }{ }`. sec fraktur

Here it is:

$\mathfrak{A}, \mathfrak{B}, \mathfrak{C}, \mathfrak{D}, \mathfrak{E}, \mathfrak{F}, \mathfrak{G}, \mathfrak{H}, \mathfrak{I}, \mathfrak{J}, \mathfrak{K}, \mathfrak{L}, \mathfrak{M}, \mathfrak{N}, \mathfrak{O}, \mathfrak{P}, \mathfrak{Q}, \mathfrak{R}, \mathfrak{S}, \mathfrak{T}, \mathfrak{U}, \mathfrak{V}, \mathfrak{W}, \mathfrak{X}, \mathfrak{Y}, \mathfrak{Z}.$
 $\mathfrak{a}, \mathfrak{b}, \mathfrak{c}, \mathfrak{d}, \mathfrak{e}, \mathfrak{f}, \mathfrak{g}, \mathfrak{h}, \mathfrak{i}, \mathfrak{j}, \mathfrak{k}, \mathfrak{l}, \mathfrak{m}, \mathfrak{n}, \mathfrak{o}, \mathfrak{p}, \mathfrak{q}, \mathfrak{r}, \mathfrak{s}, \mathfrak{t}, \mathfrak{u}, \mathfrak{v}, \mathfrak{w}, \mathfrak{x}, \mathfrak{y}, \mathfrak{z}.$

The script version in Figure 1.1 is for handwritten notes, blackboard/whiteboard talk and discussion, or to impress your friends.

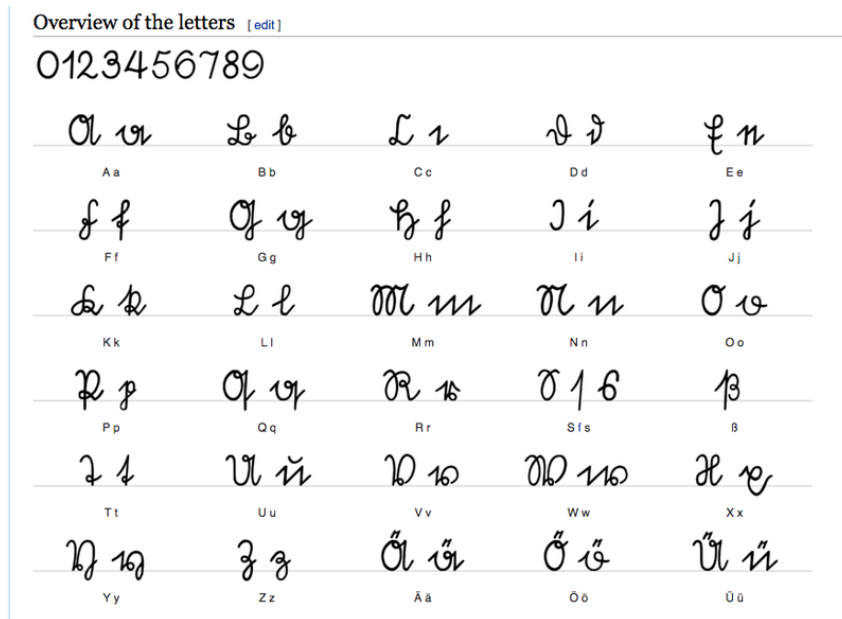


Figure 1.1: Handwritten version of the Fraktur font. fig fraktur_script

Only the capitals $\mathfrak{A}$, $\mathfrak{B}$ and $\mathfrak{C}$ are commonly used in mathematical logic, while $\mathfrak{G}$, $\mathfrak{M}$, $\mathfrak{N}$ and $\mathfrak{L}$ are occasionally used. This should not be a major stumbling block in your learning process.

2. *Greek Alphabet*: Used extensively.

α (alpha), β (beta), γ (gamma), δ (delta), ϵ (epsilon), ε (varepsilon), ζ (zeta),
 η (eta), θ (theta), ϑ (varthetaeta), ι (iota), κ (kappa), λ (lambda), μ (mu), ν (nu),
 ξ (xi), $\omicron$ (omicron), π (pi), ϖ (varpi), ρ (rho), ϱ (varrho), σ (sigma), ς (varsigma),
 τ (tau), υ (upsilon), φ (phi), φ (varphi), χ (chi), ψ (psi), ω (omega).

and

Γ (Gamma), Δ (Delta), Θ (Theta), Λ (Lambda), Ξ (Xi), Π (Pi), Σ (Sigma),
 Υ (Upsilon), Φ (Phi), Ψ (Psi), Ω (Omega).

Omicron is typeset the same as the roman letter o, and the missing capitals are typeset the same as their roman equivalents.

3. *Hebrew Alphabet*: Only need $\aleph$ (Aleph).

1.2.4 VARIANTS FOR STRUCTURES

1. *Nonempty Universe*: The restriction that $A \neq \emptyset$ is standard in mathematical logic, although not universal. See (Johnstone, 1987, page 24) for the price one must pay in terms of a modified version of modus ponens, if one allows an empty universe.

Exercise 1.2. How about $\forall x \varphi(x) \rightarrow \exists x \varphi(x)$? What if the universe is empty? $\square$

2. *Many-Sorted Structure*: In the case of a vector space, the universe could be the (disjoint) union of the set of scalars and the set of vectors. Unary relations S, V could be used to single out these two sets/sorts.

3. *Relational Structure*: This is a structure in which there are only constants and relations. An n -ary function can be identified with an $(n + 1)$ -ary relation in the natural way.

One could restrict considerations to relational structures, with essentially no loss of generality, as it does make some details easier. But it is less convenient when working with functions.

Constants are also particular (unary) relations, or 0-ary functions. But it is inconvenient to work without constants.

1.2.5 FOUNDATIONAL ISSUES

In matters of a constructive and foundational nature we restrict I, J, K to be finite. The languages, formulas, etc. can then be coded up algorithmically by integers. We say more about this as we proceed.

Another useful situation is to restrict I and J to be finite while allowing the index K for constant symbols to be infinite (for example, having one constant symbol for *every* element in some non-finite structure). See Chapter 3.

The generalisation of results to higher cardinalities usually involves no significant new ideas, other than using ordinals⁷ for enumeration/iteration purposes.

1.3 LANGUAGE AND SYNTAX

We will see in this Section how the symbols of a language $\mathcal{L}$ are used to construct terms, formulas, sentences, etc.

Syntax refers to the manner in which such meaningful expressions are built up in first-order logic, and to the notion and properties of formal proofs.

⁷See Chapter 13.

1.3.1 SYMBOLS FOR A LANGUAGE

A language $\mathcal{L}$ consists of a set of symbols. The language to describe the previous structure $\mathfrak{A}$ in (1.5) consists of:

- a set of relation symbols $\{r_i : i \in I\}$, each of some assigned arity,
- a set of function symbols $\{f_j : j \in J\}$, each of some assigned arity,
- a set of constant symbols $\{c_k : k \in K\}$.

In this case we write

$$\mathcal{L} = \{r_i (i \in I), f_j (j \in J), c_k (k \in K)\} \quad \text{or} \quad \mathcal{L} = \{r_i, f_j, c_k\} \quad (1.6)$$

or something similar, if the meaning is clear from context.

1.3.2 STRUCTURES FOR A LANGUAGE

The structure $\mathfrak{A} = \langle A, R_i, F_j, C_k \rangle_{i \in I, j \in J, k \in K}$ in (1.5) is an $\mathcal{L}$ -structure if there is a one-one correspondence $r_i \leftrightarrow R_i$, $f_j \leftrightarrow F_j$ and $c_k \leftrightarrow C_k$, such that the arities match up.

The interpretation in $\mathfrak{A}$ of a relation symbol $r \in \mathcal{L}$ is often written $r^{\mathfrak{A}}$. Similarly the interpretation of a function symbol f is often written $f^{\mathfrak{A}}$. And the interpretation of a constant symbol c is often written $c^{\mathfrak{A}}$.

So we will often write instead of (1.5),

$$\mathfrak{A} = \langle A, r_i^{\mathfrak{A}}, f_j^{\mathfrak{A}}, c_k^{\mathfrak{A}} \rangle_{i \in I, j \in J, k \in K} \quad \text{or just} \quad \mathfrak{A} = \langle A, r^{\mathfrak{A}}, f^{\mathfrak{A}}, c^{\mathfrak{A}} \rangle, \quad \text{str2} \quad (1.7)$$

if it is clear from context what is intended.

Remark 1.3.

1. *Important Distinction:* The relation/function/constant *symbols* in a formal language $\mathcal{L}$, and their *interpretation* as actual relations/functions/constants in a structure $\mathfrak{A}$, should not be confused with each other.

2. *Different Structures – Same Language:* Think of groups and the language of group theory.

Even the *Peano Axioms* for arithmetic have infinitely many non-isomorphic models apart from the standard model. Their study can often tell us something significant about the standard model. See Chapter 10.

3. *Uncluttering:* Eventually we may just rely on context to distinguish between symbols in a language and their interpretations in a structure, and use the same notation in both cases.

Some authors do not like this. But always making a distinction via notation between symbol and interpretation can lead to very cluttered writing. $\square$

1.3.3 LOGICAL SYMBOLS

Besides the relation, function and constant symbols specific to the language $\mathcal{L}$, we also have the following *logical symbols* common to all first-order languages, and which are used to build up the terms and formulas of $\mathcal{L}$:

$(,)$	parentheses,
$v_0, v_1, v_2, \dots$	variable symbols (usually just called variables),
$\neg$	not,
$\wedge$	and,
$\forall$	for all,
$=$	equals.

The intention, as we explain subsequently, is that variables will be interpreted as elements of the universe A of some structure $\mathfrak{A}$ under consideration.

Instead of taking $v_0, v_1, \dots, v_n, \dots$ as variables, we could take $v, v', v'', v''', \dots$ if we wish to make do with a finite number of symbols.

We normally use $x, y, u, v, \dots$ to denote variables.

def var

Definition 1.4. The set of variables for any language $\mathcal{L}$ is denoted by $\mathcal{V}$.

The symbols “ $\neg$ ” and “ $\wedge$ ” are *connectives*, “ $\forall$ ” is a *quantifier*, “ $=$ ” is a binary relation symbol (which will always be interpreted in a structure with the meaning “is the same as”, rather than as a more general equivalence relation).

We take these logical symbols as *primitive* and define other logic symbols from them, see [Subsection 1.3.5](#).

1.3.4 METALANGUAGE AND METALANGUAGE SYMBOLS

We have a mathematical/formal language $\mathcal{L}$ to describe our mathematical structures $\mathfrak{A}$. But we also have to discuss $\mathcal{L}$ itself and its relationship with structures $\mathfrak{A}$, and that we do in our *metalinguage*, which is English.

Unless we are doing concrete foundational matters the symbols of $\mathcal{L}$ are often treated as *set-theoretic objects*. This is particularly the case if we have an infinite number of symbols in $\mathcal{L}$ and we expand $\mathcal{L}$ to a larger language $\mathcal{L}^*$ — as is the case when we are considering infinite structures and we add a new constant symbol for each element of the structure. See the proof of Gödel’s Completeness Theorem [3.23](#).

Exercise 1.5. So we will regard symbols either as marks on a piece of papers (in Gödel’s Incompleteness Theorem), or as more abstract mathematical objects (in Gödel’s Completeness Theorem). Comment? $\square$

It will be convenient to use the following symbols in our metalanguage: $\implies$ or $\Rightarrow$ for “implies”, $\iff$ for “if and only if”, & for “and”.

Also, $A := B$ is an abbreviation for “ A by definition equals B ”, $A =: B$ is an abbreviation for “ B by definition equals A ”.

The symbol “ $=$ ” is used in a number of different ways. It is a logical symbol in the language $\mathcal{L}$.

But it is also used in a metalinguistic manner, when we might write for example that $\varphi = \varphi_1 \wedge \varphi_2$ or $\varphi = \varphi(x_1, \dots, x_n)$. This would indicate (perhaps depending on the context) that the formula φ is the conjunction of two formulas, or is a formula whose free variable

are among $x_1, \dots, x_n$, or just that we want to draw attention to the particular free variables $x_1, \dots, x_n$ for some reason or other, although there may be other free variables in φ .

Hopefully it will be clear from context and accompanying explanations what is intended.

1.3.5 DEFINED LOGICAL SYMBOLS

log sym

PROPOSITIONAL LOGIC

In the following I will assume a little knowledge on your part of propositional logic. If you know what is a truth table and what is a tautology, you should be fine. If not, please read and understand Section 9.1.

Definition 1.6. Taking $\neg, \wedge, \vee$ as primitive, the following definitions/abbreviations are justified in terms of truth tables as in Section 9.1 or just their standard meaning, and the standard meaning of $\exists$. par abbrev

- (*or*): $\varphi \vee \psi$ for $\neg(\neg\varphi \wedge \neg\psi)$
- (*implies*): $\varphi \rightarrow \psi$ for $\neg(\varphi \wedge \neg\psi)$
- (*if and only if*): $\varphi \leftrightarrow \psi$ for $(\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$
- (*there exists*): $\exists v \varphi$ for $\neg\forall v \neg\varphi$

Brackets are dropped where possible, and also sometimes added for clarity.

The convention is that $\neg$ is less binding than $\wedge$ and $\vee$, which are less binding than $\rightarrow$ and $\leftrightarrow$. For example,

$$\neg\varphi \vee \psi \rightarrow \theta \wedge \varphi \quad \text{means} \quad (\neg\varphi \vee \psi) \rightarrow (\theta \wedge \varphi) \quad \text{means} \quad ((\neg\varphi) \vee \psi) \rightarrow (\theta \wedge \varphi).$$

1.3.6 TERMS

A finite sequence (or string) of symbols is a *term* of $\mathcal{L}$ if and only if it is one of the following forms:

- (i) A variable v or constant symbol c ;
- (ii) $f(t_1, \dots, t_n)$, where f is an n -ary function symbol of $\mathcal{L}$, and each t_i is a term.

Alternatively, we can define terms set-theoretically:

The set of terms of $\mathcal{L}$ is the least set S of finite sequences such that:

- (i) each variable and constant symbol belongs to S ;
- (ii) if f is an n -ary function symbol of $\mathcal{L}$ and $t_1, \dots, t_n \in S$, then $f(t_1, \dots, t_n) \in S$.

Intuitively a *term* is a variable or a constant symbol, or is built from them by using function symbols. A term has a value in an $\mathcal{L}$ -structure once the variables in the term are assigned values in the structure. See Definition 1.14 for the precise definition.

1.3.7 ATOMIC FORMULAS

A finite sequence (or string) of symbols is an *atomic formula* of $\mathcal{L}$ iff it is one of the following forms:

- (i) $t_1 = t_2$, where t_1 and t_2 are terms;
- (ii) $r(t_1, \dots, t_n)$, where r is an n -ary relation symbol and the t_i are terms.

Intuitively an *atomic formula* is the simplest expression that will be either true or false in a $\mathcal{L}$ -structure once values are assigned to the variables in the formula.

1.3.8 FORMULAS

A finite sequence of symbols is a *formula* of $\mathcal{L}$ iff it is one of the following forms:

- (i) an atomic formula;
- (ii) $(\neg\varphi)$, where φ is a formula;
- (iii) $(\varphi_1 \wedge \varphi_2)$, $(\varphi_1 \vee \varphi_2)$, $(\varphi_1 \rightarrow \varphi_2)$, $(\varphi_1 \leftrightarrow \varphi_2)$, where φ_1 and φ_2 are formulas;
- (iv) $\forall v \varphi$, $\exists v \varphi$, where v is a variable and φ is a formula.

As for terms, we can define the sets of atomic formulas and of formulas set theoretically.

Intuitively a *formula* is an expression that is either true or false in a $\mathcal{L}$ -structure once values are assigned to the free⁸ variables in the formula. See Definition 1.18.

Keep the following distinction in mind. The difference between $\mathcal{L}$ -terms and $\mathcal{L}$ -formulas is the following: once values in an $\mathcal{L}$ -structure $\mathfrak{A}$ are assigned to (free) variables in a term or a formula, the term will represent an *element in* $\mathfrak{A}$ and the formula will be a *statement about* $\mathfrak{A}$ which will be either true or false.

See Subsections 1.4.2 and 1.4.3.

1.3.9 COMPLEXITY OF TERMS AND FORMULAS

It is convenient to use a small number of logical symbols when proving results about formulas.

For this reason we will normally just prove results for formulas built up using the logical symbols $\neg, \wedge, \vee, \forall$, and regard $(\varphi_1 \vee \varphi_2)$, $(\varphi_1 \rightarrow \varphi_2)$, $(\varphi_1 \leftrightarrow \varphi_2)$ and $\exists v \varphi$ as convenient abbreviations.

It is convenient to assign an integer $h(t)$ to terms t , and $h(\varphi)$ to formulas φ , where h corresponds to the maximum number of “steps” required in their construction. If you think of the construction as being given by a finite tree, then h is the height/complexity of this tree. See Figure 1.2.

For terms define the *term complexity* function h by:

- $h(v) = h(c) = 1$, where v is a variable symbol and c is a constant symbol.⁹

⁸See Subsection 1.3.15.

⁹Notice that “=” as used here is equality in the metalanguage. It is of course *not* the equality “=” for the formal language $\mathcal{L}$.

Some authors use a different symbol such as “ $\equiv$ ” within the formal language. I think ambiguity and relying on context leads to a cleaner and less-cluttered exposition, but be careful!

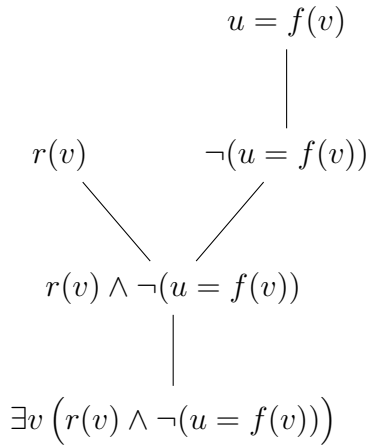
- $h(f(t_1, \dots, t_n)) = 1 + \max\{h(t_1), \dots, h(t_n)\}$, where $f(t_1, \dots, t_n)$ is a term.

Similarly for formulas φ , define a (different) *formula height/complexity* function h by:

- $h(\varphi) = 1$ if φ is atomic,
- $h(\neg\varphi) = 1 + h(\varphi)$,
- $h(\varphi \wedge \psi) = 1 + \max\{h(\varphi), h(\psi)\}$,
- $h(\forall v \varphi) = 1 + h(\varphi)$.

Here we have taken $\neg, \wedge, \forall$ as primitive symbols.

See Figure 1.2.



In the diagram the atomic formulas $r(v)$ and $u = f(v)$ both have (formula) complexity 1.

The formula $\neg(u = f(v))$ has complexity 2.

The formula $\exists v (r(v) \wedge \neg(u = f(v)))$ has complexity 4, corresponding to the 4 levels required for its construction.

The terms $v, u, f(v)$ have (term) complexity 1, 1, 2 respectively.

Figure 1.2: Decomposition of a first-order formula into subformulas. fig tree

Exercise 1.7. What is the complexity of $r(v) \wedge \neg(u = f(v))$?

1.3.10 PROVING RESULTS ABOUT TERMS AND FORMULAS

Structural properties of terms and formulas can be established by induction on their complexity. To show every formula has some property P , we must establish four things:

1. Every atomic formula has property P ;
2. If ψ is $\neg\varphi$ and φ has property P , then so does ψ ;
3. If ψ is $\varphi_1 \wedge \varphi_2$ and both φ_1 and φ_2 have property P , then so does ψ ;
4. If ψ is $\forall v \varphi$ and φ has property P , then so does ψ .

1.3.11 UNIQUENESS OF PARSING

If terms or formulas are regarded as *concatenated strings* of symbols, uniqueness of parsing becomes an issue. Could a string of symbols be represented, say, as both $(\varphi_1 \wedge \varphi_2)$ and $(\varphi'_1 \wedge \varphi'_2)$, etc.? The answer is no, by a suitably induction on their complexity.

We will often drop, or sometimes add, brackets for the sake of readability and according to standard mathematical practice.

If we are interested in formulas primarily as concatenated strings, it is preferable to write $\wedge\varphi_1\varphi_2$ instead of $(\varphi_1 \wedge \varphi_2)$, $\neg\varphi$ instead of $(\neg\varphi)$, $\forall v\varphi$ instead of $(\forall v)\varphi$. This is known as *Polish notation*. We won't bother about doing this.

1.3.12 OTHER LOGICS

The language $\mathcal{L}$, the notions of term and formula of $\mathcal{L}$, the interpretation of these terms and formulas in structures for $\mathcal{L}$ (Section 1.4), and provability in $\mathcal{L}$ (Chapter 2), together determine what we refer to as the *first-order logic* over $\mathcal{L}$. Terminology is not universally consistent — often “language” and “logic” are used interchangeably.

Various other logics are possible over $\mathcal{L}$. If we add a second class of variables, interpreted as ranging over *subsets* of the interpretation, we obtain *second-order logic*. Likewise, higher-order logics.

Alternatively, new logical symbols can be added, such as a quantifier Q where $Qv\dots$ is interpreted as meaning “there are uncountably many v such that $\dots$ ”, etc. Or, we may allow conjunctions and disjunctions of sets of formulas of certain infinite cardinalities. Also, various combinations of these ideas.

These and other logics have been studied extensively. Although they are not nearly as well-behaved as first-order logic, some of them do have nice properties.

From the perspective of studying the foundations of mathematics, first-order logic is the most important.

Moreover, all of mathematics can be done within set theory, and set theory has a first-order theory, as we discuss in Chapter 15.

1.3.13 CARDINALITY OF A LANGUAGE

The *cardinality* of a language $\mathcal{L}$ is

$$|\mathcal{L}| := \max\{\aleph_0, |I|, |J|, |K|\}, \quad \text{eq card } \mathcal{L} \quad (1.8)$$

where $|I|, |J|, |K|$ are the cardinalities of the sets I, J, K of relation, function and constant symbols respectively of $\mathcal{L}$. See (1.5), (1.7)

Exercise 1.8. $|\mathcal{L}|$ is the cardinality of the set of formulas of $\mathcal{L}$.

(If you have not at this stage studied any cardinality arithmetic, just treat the case where $|\mathcal{L}| = \aleph_0$.)

1.3.14 SUBFORMULAS

The set of *subformulas* of a given formula is defined inductively. In particular, the set of subformulas:

- of an atomic formula just contains the formula itself,
- of $\neg\varphi$ is $\{\neg\varphi\} \cup \{\text{subformulas of } \varphi\}$,
- of $\varphi \wedge \psi$ is $\{\varphi \wedge \psi\} \cup \{\text{subformulas of } \varphi\} \cup \{\text{subformulas of } \psi\}$,
- of $\forall v \varphi$ is $\{\forall v \varphi\} \cup \{\text{subformulas of } \varphi\}$.

And similarly for other logical symbols.

Exercise 1.9. What is the set of subformulas of $r(v) \wedge \neg(u = f(v))$? See Figure 1.2. $\square$

1.3.15 FREE AND BOUND VARIABLES

Intuitively, the *free* variables of a formula φ are those variables that occur in φ but are not within the scope of a quantifier. They are the variables on whose values, in a given structure, the truth or falsity of φ depends (see Section 1.4).

The *bound* variables are those that occur within the scope of a quantifier.

Definition 1.10. The set $FV(\varphi)$ of *free variables* of a formula φ is defined inductively:

- If φ is an atomic formula then $FV(\varphi)$ is the set of all variables that occur in φ .
- $FV(\neg\varphi) = FV(\varphi)$.
- $FV(\varphi \wedge \psi) = FV(\varphi) \cup FV(\psi)$.
- $FV(\forall v \varphi) = FV(\varphi) \setminus \{v\}$.

The symbol “ $\setminus$ ” denotes set-theoretic subtraction.

Regarding defined logical symbols, it follows that

- $FV(\varphi \vee \psi) = FV(\varphi \rightarrow \psi) = FV(\varphi \leftrightarrow \psi) = FV(\varphi) \cup FV(\psi)$.
- $FV(\exists v \varphi) = FV(\varphi) \setminus \{v\}$.

The set $BV(\varphi)$ of *bound variables* of φ is similarly defined by induction on complexity, and is the set of variables of φ occurring within the scope of a quantifier.

Exercise 1.11. Look at the following examples. Then write out the definition for $BV(\varphi)$. $\square$

Example 1.12 (Free and Bound variables).

$\varphi_1 := r(u, v) \rightarrow u = w$	$FV(\varphi_1) = \{u, v, w\}$	$BV(\varphi_1) = \emptyset$
$\varphi_2 := \exists v (r(u, v) \rightarrow u = w)$	$FV(\varphi_2) = \{u, w\}$	$BV(\varphi_2) = \{v\}$
$\varphi_3 := \forall u \exists v (r(u, v) \rightarrow u = w)$	$FV(\varphi_3) = \{w\}$	$BV(\varphi_3) = \{u, v\}$
$\varphi_4 := \forall u \exists v (r(u, v) \rightarrow u = w) \vee (u = u)$	$FV(\varphi_4) = \{u, w\}$	$BV(\varphi_4) = \{u, v\}$
$\varphi_5 := \forall x \exists v (r(x, v) \rightarrow x = w) \vee (u = u)$	$FV(\varphi_5) = \{u, w\}$	$BV(\varphi_5) = \{x, v\}$

It is often more convenient to speak of *free and bound occurrences* of a variable. In particular, for φ_4 as compared to φ_3 , there are 2 additional occurrences of the symbol “u”, both of which are free. The first 3 occurrences of u in both cases are bound. The last two occurrences of u in φ_4 are free occurrences.

It is also conventional to say in φ_2 that the 2 occurrences of the symbol v are bound, in φ_3 the 3 occurrences of u are bound, etc.

Finally, note that φ_4 and φ_5 both have the same intended meaning. In fact, it is always possible and often convenient, to change the bound variables without changing any intended meaning. In this way one can ensure that no variable symbol in a formula has both free and bound occurrences.

1.3.16 SENTENCES

A *sentence* is a formula with no free variables.

So a sentence is always true or false in a structure, and this is independent of any assignment of values to variables.

1.3.17 NOTATION REGARDING FREE VARIABLES

The expression $t(x_1, \dots, x_n)$ is often used to denote a term t whose variables are a subset of $\{x_1, \dots, x_n\}$.

But sometimes we will instead write, for example $t(v)$ or $t(u, v)$ etc., to denote a term t where u is *only one* of its free variables, or u and v are *only two* of its free variables, etc.

This will be convenient when we make substitutions. For example, if in the term $t(u)$ we replace all variables u by w , we might write $t(w)$ for the new term. It would be understood from context in this case that the free occurrences of u in t have been replaced throughout by w .

Similarly as for terms, the expression $\varphi(x_1, \dots, x_n)$ denotes a formula φ whose *free* variables are a subset of $\{x_1, \dots, x_n\}$.

Sometimes we write $\varphi(u)$ to draw attention to the fact that u is a free variable of φ , although there may also be other free variables in φ . This is again convenient if we wish to replace a free occurrence of u by another variable v or constant symbol c , in which case we write $\varphi(v)$ or $\varphi(c)$ respectively. If there is any danger of ambiguity I will be careful to explain what is intended.

1.3.18 EXPANSION OF A LANGUAGE

Sometimes we are interested in passing from a language $\mathcal{L}$ to a larger language $\mathcal{L}' \supseteq \mathcal{L}$. We say $\mathcal{L}'$ is an *expansion* of $\mathcal{L}$ and $\mathcal{L}$ is a *reduction* of $\mathcal{L}'$.

We will be particularly interested in the case where $\mathcal{L}' \setminus \mathcal{L}$ consists of new constant symbols, typically an infinite set of constant symbols, corresponding to a name for each element of some $\mathcal{L}$ -structure. See the proof of Theorem 3.23.

1.4 SEMANTICS

1.4.1 ON DEFINING TRUTH IN A STRUCTURE

The notion of truth of a sentence in a structure, and more generally of satisfaction of a formula for some assignment of values to its free variables, both of which are defined in this section, are *semantic* notions.

Unless mentioned otherwise, we consider a fixed language $\mathcal{L}$. All formulas, sentences, and structures are for $\mathcal{L}$.

Suppose φ is a *sentence*, and $\mathfrak{A}$ a structure for $\mathcal{L}$. In order to define

$$\mathfrak{A} \models \varphi, \quad (1.9)$$

which we read as “ φ is true in $\mathfrak{A}$ ” or “ $\mathfrak{A}$ satisfies φ ”, we need to more generally consider *formulas* with free variables.

The definition of $\mathfrak{A} \models \varphi$ proceeds by an induction on the complexity of φ . For example, to decide whether $\mathfrak{A} \models \forall v \varphi(v)$ we need to consider the truth value of $\varphi(v)$ in $\mathfrak{A}$ for *all* assignments of members of A (the universe of $\mathfrak{A}$) to the variable v .

The precise notion of *truth* as defined here is due to Tarski (Tarski, 1931; 1944). While it may appear relatively natural now, as Tarski remarks there are considerable difficulties in constructing *a materially adequate and formally correct definition of the term ‘true sentence’*.

1.4.2 INTERPRETATION OF A TERM IN A STRUCTURE

Definition 1.13 (Variable Assignment). ^{sub its} A *variable assignment* σ from the set of variables $\mathcal{V} = \{v_0, v_1, \dots, v_n, \dots\}$ into the structure $\mathfrak{A}$ is a function

$$\sigma : \mathcal{V} \rightarrow A, \quad (1.10)$$

mapping $\mathcal{V}$ into the universe A of $\mathfrak{A}$.

The value/interpretation of a term t under an assignment σ is defined in the natural way using the complexity of t .

Definition 1.14 (Interpretation of a Term). ^{def int term} Let t be a term and let σ be a variable assignment into $\mathfrak{A}$. Then the interpretation $t^{\mathfrak{A}}[\sigma]$ of t in $\mathfrak{A}$ under the assignment σ is defined by induction on the complexity of t as follows:

- (i) if t is a variable x , then $t^{\mathfrak{A}}[\sigma] = \sigma(x)$,
- (ii) if t is a constant symbol c , then $t^{\mathfrak{A}}[\sigma] = c^{\mathfrak{A}}$,
- (iii) if t is a term $f(t_1, \dots, t_m)$, then

$$t^{\mathfrak{A}}[\sigma] = f^{\mathfrak{A}}(t_1^{\mathfrak{A}}[\sigma], \dots, t_m^{\mathfrak{A}}[\sigma]).$$

Remark 1.15 ^{rem wavm} (*Which assignment values matter?*). If $t = t(x_1, \dots, x_n)^{10}$ is a term all of whose variables are in the set $\{x_1, \dots, x_n\}$, then $t^{\mathfrak{A}}[\sigma]$ depends *only* on the values $\sigma(x_1), \dots, \sigma(x_n)$ and *not* on $\sigma(u)$ for any other variable u .

¹⁰Once again, the “=” here is a symbol in the metalanguage, not in the formal language $\mathcal{L}$. The statement $t = t(x_1, \dots, x_n)$ is a shorthand way of saying that the variables occurring in t belong to the set $\{x_1, \dots, x_n\}$.

That is, if $\sigma, \sigma' : \mathcal{V} \rightarrow A$ and $\sigma(x_i) = \sigma'(x_i)$ for $i = 1, \dots, n$, then $t^{\mathfrak{A}}[\sigma] = t^{\mathfrak{A}}[\sigma']$.

If $\sigma(x_i) = a_i$ for $i = 1, \dots, n$, then one often writes

$$t^{\mathfrak{A}}[a_1, \dots, a_n] \quad \text{or} \quad t[a_1, \dots, a_n] \quad (1.11)$$

for $t^{\mathfrak{A}}[\sigma]$, where the fact $x_i \mapsto a_i$ is understood from context, and where $\mathfrak{A}$ is also understood from context in the second case. $\square$

Exercise 1.16. The proof that $t^{\mathfrak{A}}[\sigma] = t^{\mathfrak{A}}[\sigma']$ in the previous Remark is a simple induction on the complexity of t . Write out the proof in a succinct manner. $\square$

Exercise 1.17. Is the “=” in the statement $t^{\mathfrak{A}}[\sigma] = t^{\mathfrak{A}}[\sigma']$ a symbol in the language $\mathcal{L}$ or a symbol in the metalanguage? Explain. $\square$

1.4.3 SATISFACTION OF A FORMULA IN A STRUCTURE

Let $\varphi = \varphi(x_1, \dots, x_n)$ be a formula of $\mathcal{L}$, where the set $\{x_1, \dots, x_n\}$ contains all free variables from φ . sub sfs

Let $\sigma : \mathcal{V} \rightarrow A$ be a variable assignment into the universe A of the $\mathcal{L}$ -structure $\mathfrak{A}$, where $\sigma(x_i) = a_i$ for $i = 1, \dots, n$. In Definition 1.18 we will define

$$\mathfrak{A} \models \varphi[\sigma] \quad \text{or equivalently} \quad \mathfrak{A} \models \varphi[a_1, \dots, a_n], \quad (1.12) \quad \text{form sat}$$

so that it agrees with the informal idea that $\varphi(x_1, \dots, x_n)$ is true in $\mathfrak{A}$ if each variable $x_i \in \mathcal{V}$ is interpreted as $a_i \in A$.

We read (1.12) as

$\mathfrak{A}$ satisfies φ at (the assignment) σ , or
 $\mathfrak{A}$ satisfies φ at $(a_1, \dots, a_n)$, or
 something similar.

If two assignments $\sigma, \sigma' : \mathcal{V} \rightarrow A$ agree on all free variables in φ then it will follow from Definition 1.18 that

$$\mathfrak{A} \models \varphi[\sigma] \iff \mathfrak{A} \models \varphi[\sigma'] \quad (1.13) \quad \text{form sat equiv}$$

This justifies the use of the notation $\mathfrak{A} \models \varphi[a_1, \dots, a_n]$ in (1.12).

If φ is a sentence and so φ has no free variables, then from (1.13) the truth or falsity of $\mathfrak{A} \models \varphi[\sigma]$ is independent of the choice of assignment σ , and so we can simply write this as $\mathfrak{A} \models \varphi$.

We now give the formal definition of $\mathfrak{A} \models \varphi[\sigma]$ by induction on the complexity of φ . At each stage we will obtain either that $\mathfrak{A} \models \varphi[\sigma]$ is true or otherwise that $\mathfrak{A} \models \varphi[\sigma]$ is false. In the second case we write $\mathfrak{A} \not\models \varphi[\sigma]$ and read it as “ $\mathfrak{A}$ does not satisfy φ at (the assignment) σ ”.

In the following we are, as usual, taking $\neg, \wedge, \forall$ as the primitive logical symbols for the language $\mathcal{L}$.

Definition 1.18 (Satisfaction). Let $\mathfrak{A}$ be an $\mathcal{L}$ -structure. We define for every variable assignment σ , by induction on the complexity of a formula φ , the relation $\mathfrak{A} \models \varphi[\sigma]$. def sat

- (i) $\mathfrak{A} \models (t_1 = t_2)[\sigma]$ iff $t_1^{\mathfrak{A}}[\sigma] = t_2^{\mathfrak{A}}[\sigma]$,¹¹
- (ii) $\mathfrak{A} \models r(t_1, \dots, t_m)[\sigma]$ iff $(t_1^{\mathfrak{A}}[\sigma], \dots, t_m^{\mathfrak{A}}[\sigma]) \in r^{\mathfrak{A}}$,
- (iii) $\mathfrak{A} \models (\neg\varphi)[\sigma]$ iff $\mathfrak{A} \not\models \varphi[\sigma]$,
- (iv) $\mathfrak{A} \models (\varphi_1 \wedge \varphi_2)[\sigma]$ iff $\mathfrak{A} \models \varphi_1[\sigma]$ and $\mathfrak{A} \models \varphi_2[\sigma]$,
- (v) $\mathfrak{A} \models \forall x \varphi[\sigma]$ iff $\mathfrak{A} \models \varphi[\sigma']$ for all σ' such that $\sigma(u) = \sigma'(u)$ whenever $u \neq x$. □

Remark 1.19 (Universal Validity). rem univ valid

If $\varphi = \varphi(x_1, \dots, x_n)$ is a formula, then by $\mathfrak{A} \models \varphi$ we mean that $\mathfrak{A} \models \varphi[\sigma]$ for all assignments $\sigma : \mathcal{V} \rightarrow A$.

This is equivalent to $\mathfrak{A} \models \forall x_1 \dots \forall x_n \varphi$. We say φ is true/valid in $\mathfrak{A}$.

We call $\forall x_1 \dots \forall x_n \varphi$ the *universal closure* of φ . There is some ambiguity, depending on the order of the x_i , but it makes no difference to whether or not $\mathfrak{A} \models \forall x_1 \dots \forall x_n \varphi$.

If $\mathfrak{A} \models \varphi[\sigma]$ for all $\mathfrak{A}$ and all σ , we say the formula φ is *universally valid*. □

Exercise 1.20. Explain why “... the order of the x_i ... makes no difference to whether or not $\mathfrak{A} \models \forall x_1 \dots \forall x_n \varphi$ ”. □

Remark 1.21 (Which assignment values matter? Notational Simplification). rem wavm2 It is a straightforward induction, based on the complexity of φ , to show that the truth value of $\mathfrak{A} \models \varphi[\sigma]$ depends only on the assignments $\sigma(x_i)$ for the *free* variables $x_1, \dots, x_n$ in φ .

In this case, if $\sigma(x_i) = a_i$ for $i = 1, \dots, n$, then we often write

$$\mathfrak{A} \models \varphi(a_1, \dots, a_n) \quad \text{or} \quad \mathfrak{A} \models \varphi[a_1, \dots, a_n] \quad \text{for} \quad \mathfrak{A} \models \varphi[\sigma]. \quad (1.14)$$

One says “ $\varphi(a_1, \dots, a_n)$ ” is true in $\mathfrak{A}$, in accordance with standard mathematical practice.

Similarly, as in Remark 1.15, if the variables in the term t are a subset of $x_1, \dots, x_n$ and $\sigma(x_i) = a_i$, then one writes

$$t(a_1, \dots, a_n) \quad \text{or} \quad t[a_1, \dots, a_n] \quad \text{for} \quad t[\sigma]. \quad (1.15)$$

Exercise 1.22. Write down the induction step noted in Remark 1.21 for (iii) and (v). (Assume we already know the result for the relevant subformulas.) □

Remark 1.23 (Universal quantifiers). Implicitly or explicitly inserting universal quantifiers is common mathematical practice. For example, we might write

$$u = v \rightarrow v = u,$$

where it is clear from the context that this is intended to apply to all interpretations of u and v in the relevant structures. □

¹¹This is yet another example of using notation, here “=”, both in the formal language as in $t_1 = t_2$, and in the metalanguage as in $t_1^{\mathfrak{A}}[\sigma] = t_2^{\mathfrak{A}}[\sigma]$. The latter means $t_1^{\mathfrak{A}}[\sigma]$ and $t_2^{\mathfrak{A}}[\sigma]$ are the same element in the universe A of $\mathfrak{A}$.

1.4.4 DEFINED LOGICAL SYMBOLS

We took $\neg, \wedge, \forall$ as primitive and defined $\vee, \rightarrow, \leftrightarrow, \exists$ from them, see Section 1.3.5. It follows from Definition 1.6 that:

- (vi) $\mathfrak{A} \models (\varphi_1 \vee \varphi_2)[\sigma]$ iff $\mathfrak{A} \models \varphi_1[\sigma]$ or $\mathfrak{A} \models \varphi_2[\sigma]$,
- (vii) $\mathfrak{A} \models (\varphi_1 \rightarrow \varphi_2)[\sigma]$ iff $\mathfrak{A} \models \varphi_2[\sigma]$ whenever $\mathfrak{A} \models \varphi_1[\sigma]$,
- (viii) $\mathfrak{A} \models (\varphi_1 \leftrightarrow \varphi_2)[\sigma]$ iff $\mathfrak{A} \models \varphi_2[\sigma]$ whenever $\mathfrak{A} \models \varphi_1[\sigma]$, and conversely,
- (ix) $\mathfrak{A} \models \exists x \varphi[\sigma]$ iff $\mathfrak{A} \models \varphi[\sigma']$ for *some* σ' such that $\sigma(u) = \sigma'(u)$ whenever $u \neq x$.

Exercise 1.24. Prove (vii) and (ix) from Definition 1.6. ex vii

□

1.4.5 DEFINABLE SETS

Definition 1.25. If $\mathfrak{A} = \langle A, \dots \rangle$ is an $\mathcal{L}$ -structure and $X \subseteq A^n$ then X is *definable* in $\mathfrak{A}$ from the $\mathcal{L}$ -formula $\varphi(x_1, \dots, x_n, y_1, \dots, y_m)$, with parameters $b_1, \dots, b_m \in A$, if

$$(a_1, \dots, a_n) \in X \iff \mathfrak{A} \models \varphi[a_1, \dots, a_n, b_1, \dots, b_m]. \quad (1.16)$$

That is $\bar{a} \in X$ iff $\varphi(\bar{a}, \bar{b})$ is true in $\mathfrak{A}$, where $\bar{a} = (a_1, \dots, a_n)$ and $\bar{b} = (b_1, \dots, b_m)$.

1.4.6 SEMANTIC ENTAILMENT

In the following you might think of Σ as being the axioms for a group¹² or for a field.

Definition 1.26 (Models). If Σ is a (possibly infinite) set of formulas, by

$$\mathfrak{A} \models \Sigma$$

it is meant that $\mathfrak{A} \models \varphi$ for every $\varphi \in \Sigma$. One says that $\mathfrak{A}$ is a *model* for/of Σ . □

Remark 1.27 (Structures vs Models). One speaks of a *structure* for a language $\mathcal{L}$, and a *model* for a set of sentences Σ in the language $\mathcal{L}$. So every model for Σ is a structure for $\mathcal{L}$, but not conversely.

To add to any confusion, some authors use the words interchangeably. But it should be clear from the context what is happening. □

Definition 1.28 (Semantic Entailment). Suppose Σ is a set of formulas (possibly infinite) and φ is a formula from a language $\mathcal{L}$. Then $\Sigma \models \varphi$ if for every $\mathcal{L}$ -structure $\mathfrak{A}$, $\mathfrak{A} \models \Sigma \implies \mathfrak{A} \models \varphi$. def sem ent

We say Σ *semantically entails* or *logically implies* or *logically entails* φ .

If $\Sigma = \emptyset$ we write $\emptyset \models \varphi$ or $\models \varphi$, and say φ is *logically valid*. □

Equivalently, by Definition 1.18 and Remark 1.19, $\Sigma \models \varphi$ if for every $\mathcal{L}$ -structure $\mathfrak{A}$, $\mathfrak{A} \models \Sigma' \implies \mathfrak{A} \models \varphi'$, where Σ' and φ' are the universal closures of the formulas in Σ and of φ respectively.

¹²Almost all interesting results in group theory are not expressible in the first-order logic corresponding to the language of group theory. But they *are* expressible and provable in Zermelo Fraenkel set theory, which in turn *is* a first-order theory.

Remark 1.29 (Semantic Deduction “Theorem”). For a set of sentences Σ , and for sentences φ and ψ ,

$$\Sigma \models \varphi \quad \text{iff} \quad \Sigma \cup \{\neg\varphi\} \text{ has no models.} \quad \text{remsdt (1.17)} \quad \text{syn 1}$$

Moreover,

$$\Sigma, \psi \models \varphi \quad \text{iff} \quad \Sigma \models \psi \rightarrow \varphi. \quad \text{syn 2 (1.18)}$$

The analogous, deeper *Syntactic Deduction Theorem*, is Theorem 2.33.

Moreover, in analogy with the Syntactic Deduction Theorem, in (1.18) we can allow Σ and φ to be a set of formulas, and a formula, respectively. But ψ should be a sentence. $\square$

Exercise 1.30. Justify (1.17). $\square$

Exercise 1.31. Justify (1.18). $\square$

1.4.7 EXAMPLES FOR VALIDITY

Useful examples of logical validity and non-validity, are the following:

- (i) $\models \forall v(\varphi \rightarrow \psi) \leftrightarrow (\varphi \rightarrow \forall v \psi), \quad v \text{ not free in } \varphi,$
- (ii) $\models \exists v(\varphi \rightarrow \psi) \leftrightarrow (\varphi \rightarrow \exists v \psi), \quad v \text{ not free in } \varphi,$
- (iii) $\models \forall v(\varphi \rightarrow \psi) \leftrightarrow (\exists v \varphi \rightarrow \psi), \quad v \text{ not free in } \psi,$
- (iv) $\models \exists v(\varphi \rightarrow \psi) \leftrightarrow (\forall v \varphi \rightarrow \psi), \quad v \text{ not free in } \psi,$
- (v) $\models \exists u \forall v \varphi(u, v) \rightarrow \forall v \exists u \varphi(u, v),$
- (vi) $\not\models \forall v \exists u \varphi(u, v) \rightarrow \exists u \forall v \varphi(u, v).$ $\square$

Exercise 1.32. Give an argument to establish this in cases (iv), (v) and (vi). $\square$

1.5 RELATED READING

You may find the following sources helpful supplements for this Chapter.

In general they follow a Hilbert system similar to, but not the same as, that in these Notes.

1. (Hamkins, 2021, §§5.1–5.3) Discusses the material in this and the next chapter. Non-technical, with asides of a philosophical nature.
2. (Hils and Loeser, 2019, §§2.1–2.5) A slow introduction for beginners.
3. (Marker, 2015, pp 1-15) Well-written and succinct, many examples from mathematics.
4. (Leary and Kristiansen, 2015, Chapter 1) Notation is somewhat cumbersome. Mathematically sophisticated style.
5. (Enderton, 2001, Chapter 2 §§2.0–2.2) Takes a much more expansive and discursive approach. Chapter 1 is on propositional logic. You can probably just refer back to it as necessary when reading Chapter 2, or look at Section 9.1 in the current Notes.

Chapter 2

Hilbert's Deductive System

The notions of *axioms*, *rules of inference* and *formal proof* discussed in this chapter, are ch ds *syntactic*. The notions of satisfaction and truth in a structure are *semantic*.

The axioms, the rules and the definition of proof were designed in such a way that it was hoped, and was later shown to be true, that $\Sigma \models \varphi$ if and only if $\Sigma \vdash \varphi$. See Chapter 3.

The deductive system we use here is one of many *Hilbert-system* variants. It is mathematically simple in that proofs are linear (rather than tree-like) and so well adapted to coding as sequences of natural numbers. See Chapter 7.

But from the perspective of computer science and proof theoretic studies the approach (using tree structures) due to Gentzen and others, in particular *natural deduction* and the *sequent calculus*, are much more useful.

2.1 AXIOMS

As usual, unless stated otherwise we work in a fixed language $\mathcal{L}$.

2.1.1 PRELIMINARIES

Remark 2.1 (Propositional Logic). If you know a little about propositional logic, truth tables and tautologies, keep reading. If not, read and understand Subsections 9.1.1–9.1.6.

Remark 2.2 (Logical Axioms). The *logical axioms* for first-order logic are divided into three groups. The propositional axioms, the quantifier axioms and the equality axioms. interp $\square$

If an instance of an axiom has free variables then *the axiom should be true for all interpretations of the free variables in the structure*. Equivalently, it means the same as its universal generalisation obtained by universally quantifying over all its free variables.

For example, if we take $u = v \rightarrow v = u$ as an axiom then we intend this to mean $\forall u \forall v (u = v \rightarrow v = u)$.

The intention is that the axioms should be true in every $\mathcal{L}$ -structure for all assignments. Moreover, the rules of inference which follow the axioms should (and do) preserve truth. $\square$

Remark. At this point it would probably be helpful to look forward to Definitions 2.19 and 2.20, in order to better understand the purpose for the following Axioms and the Rules of Inference. $\square$

2.1.2 PROPOSITIONAL AXIOMS

prop ax

Definition 2.3 (Propositional Axioms).

Suppose φ is a formula of $\mathcal{L}$ obtained from a *propositional tautology* ψ (see Definition 9.6), by substituting simultaneously and uniformly, formulas of $\mathcal{L}$ for the propositional symbols in ψ .

Then φ is a *propositional axiom* for first-order logic.

Remark 2.4 (Examples). In particular, $\varphi(v) \vee \neg\varphi(v)$ is a propositional axiom.

But $\forall v (\varphi(v) \vee \neg\varphi(v))$ is not a propositional axiom. However, it is immediately deducible by Universal Generalisation, see (2.8).

(Remember that in first-order logic, if a deduction gives a formula with free variables then the variables can be interpreted as being any element in the relevant structures.)

Remark 2.5 (Possible Axiom Variations). We could just take as propositional axioms the substitution instances of the three axioms for propositional logic in Section 9.2, rather than substitution instances for *all* tautologies.

However, for our purposes there is no need to be restrictive in this manner. The significant point is that truth tables provide an algorithmic procedure for determining if a formula in propositional logic is indeed a tautology. This can then be used to decide if a formula of $\mathcal{L}$ is a propositional axiom. $\square$

2.1.3 QUANTIFIER AXIOMS

quant ax

Definition 2.6 (Quantifier Axioms).

1. Suppose φ and ψ are formulas of $\mathcal{L}$ and v is a variable not free in φ .

Then

$$\forall v(\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \forall v \psi) \quad \text{eq_ga1 (2.1)}$$

is a *quantifier axiom*, sometimes called *distribution of $\forall$ over implication* (under the “variable not free” side condition on φ).

2. Suppose φ (written $\varphi(v)$ here, but it may have other free variables) is a formula of $\mathcal{L}$. Suppose $\varphi(t)$ is obtained from φ by substituting the term t for each *free occurrence* of v in φ , and that *no variable of t occurs bound in φ at the places where it is substituted*.¹

Then

$$\forall v \varphi(v) \rightarrow \varphi(t) \quad \text{eq_ga2 (2.2)}$$

is the axiom of *universal instantiation* or of *specialisation*.

Remark 2.7 (Restrictions on Quantifier Axioms). These restrictions are necessary.

(a) For example, in (2.1) suppose φ and ψ are the same formula $s(v)$, where s is a unary relation symbol in the language $\mathcal{L}$.

Then

$$\forall v (s(v) \rightarrow s(v)) \rightarrow (s(v) \rightarrow \forall v s(v))$$

cannot be an axiom as it is not logically valid.

¹fn sub

¹This is often expressed as “ t is substitutable for v in φ ” or “ t is free for v in φ ”.

This is so since

$$\mathfrak{A} \models \forall v (s(v) \rightarrow s(v))$$

for every $\mathcal{L}$ -structure $\mathfrak{A}$. But it is *not* the case in general that

$$\mathfrak{A} \models s(v) \rightarrow \forall v s(v),$$

since the *free* occurrence of v could be assigned to some $a \in A$ such that $\mathfrak{A} \models s[a]$, while $\mathfrak{A} \not\models \forall v s(v)$.

(b) In (2.2) suppose φ (i.e. $\varphi(v)$) is the formula $\exists u r(u, v)$, where r is a binary relation symbol in the language $\mathcal{L}$.

Suppose the term t is the variable u . Then $\varphi(t)$ is $\exists u r(u, u)$ and (2.2) becomes

$$\forall v \exists u r(u, v) \rightarrow \exists u r(u, u),$$

which is *not* true in some $\mathcal{L}$ -structures.

The problem is that the term t contains (and indeed is) the variable u , which become bound when substituted for v in φ .

(c) Note that in (2.2) it is acceptable that the term t is identical to v . So in particular, $\forall v \varphi \rightarrow \varphi$ (i.e. $\forall v \varphi(v) \rightarrow \varphi(v)$) is an axiom.

In particular, taking the example from (b) where $\varphi(v)$ is the formula $\exists u r(u, v)$, but now taking t to be the variable v , we get the formula

$$\forall v \exists u r(u, v) \rightarrow \exists u r(u, v). \quad \text{tias (2.3)}$$

This *is* true in all $\mathcal{L}$ -structures, for all assignments of a value to the free occurrence of v . $\square$

Exercise 2.8. Why is (2.3) true in all $\mathcal{L}$ -structures, for all assignments of a value to the *free* occurrence of v ? $\square$

2.1.4 EQUALITY AXIOMS

Definition 2.9 (Equality Axioms).

id ax

1. Suppose u, v, w are variables. Then

$$\begin{aligned} u &= u, \\ u = v &\rightarrow v = u, \\ (u = v \wedge v = w) &\rightarrow u = w, \end{aligned} \quad \text{eq equality (2.4)}$$

are *equality axioms*.

2. Suppose $u_1, \dots, u_n$ and $v_1, \dots, v_n$ are variables and f is an n -ary function symbol. Then

$$(u_1 = v_1 \wedge \dots \wedge u_n = v_n) \rightarrow f(u_1, \dots, u_n) = f(v_1, \dots, v_n) \quad \text{IA4 (2.5)}$$

is an *equality axiom*.

3. Suppose $u_1, \dots, u_n$ and $v_1, \dots, v_n$ are variables and r is an n -ary relation symbol. Then

$$(u_1 = v_1 \wedge \dots \wedge u_n = v_n) \rightarrow (r(u_1, \dots, u_n) \rightarrow r(v_1, \dots, v_n)) \quad \text{IA5 (2.6)}$$

is an *equality axiom*.

Remark 2.10 (Equality Axioms and the Interpretation of “=”). Axioms (2.4) ensure that if “=” is interpreted in a structure $\mathfrak{A} = \langle A, \dots \rangle$ just as a binary relation “ $\sim$ ”, then “ $\sim$ ” is an equivalence relation.

Axioms (2.5) and (2.6) then ensure that the interpretations of the function and relation symbols in $\mathfrak{A}$ respect the equivalence classes corresponding to “ $\sim$ ”.

However, *we will require* that an $\mathcal{L}$ -structure $\mathfrak{A}$ interprets “=” to mean “is the same element as”. In other words, “=” is interpreted as equality in the standard sense, not just as an equivalence relation.

In particular, “=” is treated in a special manner, differently from any other binary relation. $\square$

2.1.5 UNIVERSAL VALIDITY

Recall that an $\mathcal{L}$ -formula φ is universally valid if $\mathfrak{A} \models \varphi[\sigma]$ for all $\mathcal{L}$ -structures $\mathfrak{A}$ and all assignments $\sigma : \mathcal{V} \rightarrow A$. See Remark 1.19.

Theorem 2.11 (Universal Validity of the Logical Axioms). *The logical axioms are universally valid.*

Proof. Let $\mathfrak{A}$ be an $\mathcal{L}$ -structure and $\sigma : \mathcal{V} \rightarrow A$ be an assignment of variables into the universe of $\mathfrak{A}$.

Propositional Axioms. Suppose ψ is a propositional axiom, say $\theta \rightarrow (\chi \rightarrow \theta)$. This corresponds to a tautology in Propositional Logic and so is true in $\mathfrak{A}$, regardless of σ and the truth or falsity of $\mathfrak{A} \models \theta[\sigma]$ and $\mathfrak{A} \models \chi[\sigma]$. Similarly for any propositional axiom.

Quantifier Axiom 1. Next suppose ψ is the quantifier axiom $\forall v(\theta \rightarrow \chi) \rightarrow (\theta \rightarrow \forall v \chi)$ where v is not free in θ .

We need to show² that $\mathfrak{A} \models (\forall v(\theta \rightarrow \chi) \rightarrow (\theta \rightarrow \forall v \chi))[\sigma]$. (*)

For this we can assume $\mathfrak{A} \models \forall v(\theta \rightarrow \chi)[\sigma]$.

Then $\mathfrak{A} \models (\theta \rightarrow \chi)[\sigma']$ for every σ' agreeing with σ except possibly at v . (**)

We now have to prove $\mathfrak{A} \models (\theta \rightarrow \forall v \chi)[\sigma]$. For this we can assume $\mathfrak{A} \models \theta[\sigma]$.

But since v is not free in θ , it follows $\mathfrak{A} \models \theta[\sigma']$ for every σ' agreeing with σ except possibly at v .

From (**) it now follows $\mathfrak{A} \models \chi[\sigma']$ for all such σ' and so $\mathfrak{A} \models \forall v \chi[\sigma]$.

So we have proved $\mathfrak{A} \models (\theta \rightarrow \forall v \chi)[\sigma]$. This was under the assumption that $\mathfrak{A} \models \forall v(\theta \rightarrow \chi)[\sigma]$. So we have proved (*).

Quantifier Axiom 2. We next need to show $\mathfrak{A} \models (\forall v \theta(v) \rightarrow \theta(t))[\sigma]$, where $\theta(t)$ is obtained from θ by substituting the term t for each free occurrence of v in θ , and no variable of t occurs bound in θ ³ at the places where it is substituted.

For this we can assume $\mathfrak{A} \models \forall v \theta(v) [\sigma]$, and so $\mathfrak{A} \models \theta(v) [\sigma']$ for every σ' agreeing with σ except possibly at v . (***)

We need to show that $\mathfrak{A} \models \theta(t)[\sigma]$.

²The argument here, and for the next quantifier axiom, is not much more than a small formalisation of what I hope is your intuitive understanding of why the axiom is universally valid. The formal argument tends to obscure this fact.

³But it can occur bound in $\forall v \theta(v)$, so for example t could be v itself, or $g(v)$ for some function symbol g .

But the conditions on t in $\theta(t)$ ensure that if $\sigma'(v) = t^{\mathfrak{A}}[\sigma]$, and otherwise σ' agrees with σ , then $\mathfrak{A} \models \theta(v)[\sigma']$ implies $\mathfrak{A} \models \theta(t)[\sigma]$.

From $(***)$ it follows that $\mathfrak{A} \models \theta(t)[\sigma]$. Hence $\mathfrak{A} \models \forall v \theta(v) \rightarrow \theta(t) [\sigma]$.

Equality Axioms. The equality axioms are similarly true in any $\mathfrak{A}$ for any assignment to the (free) variables. I leave this for you to think about in the next Exercise 2.12. $\square$

Exercise 2.12. Explain why the equality axioms are universally valid. $\square$

ex eq ax

2.2 RULES OF INFERENCE

There are two rules of inference in this version of a Hilbert system.

rule inf

2.2.1 MODUS PONENS

Definition 2.13 (Modus Ponens). Suppose φ and ψ are formulas of $\mathcal{L}$, then

from φ and $\varphi \rightarrow \psi$ infer ψ . inf mp (2.7)

Remark 2.14 (Etymology). From the Latin *modus ponendo ponens* – “method that by affirming affirms”. Goes back to the ancient Greeks, perhaps Theophrastus (371–287 BC). $\square$

rem usage

Remark 2.15 (Usage). For example, one often establishes φ and then immediately claims ψ on the basis $\varphi \rightarrow \psi$ is a propositional axiom. $\square$

2.2.2 GENERALISATION

Definition 2.16 (Generalisation). Suppose φ is a formula of $\mathcal{L}$, then

from φ infer $\forall v \varphi$. inf gen (2.8)

Remark 2.17 (Universal Quantifiers). A consequence of the Generalisation Rule is that we can put universal quantifiers in front of the propositional and identity axioms. rem univ quant

This accords with how we should think of free variables in the axioms — the axioms should hold in any structure independently of how the free variables are interpreted.

It also accords with standard mathematical practice: if we can prove a result $\varphi(v)$ about a variable v without any particular assumptions on v , we can deduce $\forall v \varphi(v)$. $\square$

2.2.3 PRESERVING VALIDITY

In the following, remember that $\mathfrak{A} \models \varphi$ means $\mathfrak{A} \models \varphi[\sigma]$ for every assignment $\sigma : \mathcal{V} \rightarrow A$, where A is the universe of $\mathfrak{A}$.

thm pv

Theorem 2.18 (Inference Preserves Validity). *The two rules of inference preserve validity in the following sense:*

- (i) if $\mathfrak{A} \models \varphi$ and $\mathfrak{A} \models \varphi \rightarrow \psi$ then $\mathfrak{A} \models \psi$,
- (ii) if $\mathfrak{A} \models \varphi$ then $\mathfrak{A} \models \forall v \varphi$.

Proof. Suppose $\mathfrak{A} \models \varphi$ and $\mathfrak{A} \models \varphi \rightarrow \psi$. Then for each assignment σ , $\mathfrak{A} \models \varphi[\sigma]$ and $\mathfrak{A} \models \varphi \rightarrow \psi[\sigma]$. From Section 1.4.4(vii) and Exercise 1.24, $\mathfrak{A} \models \psi[\sigma]$. Since σ was arbitrary, $\mathfrak{A} \models \psi$. This proves (i).

Suppose $\mathfrak{A} \models \varphi$. Then for every $\sigma : \mathcal{V} \rightarrow A$, $\mathfrak{A} \models \varphi[\sigma]$. Since $\mathfrak{A} \models \forall v \varphi[\sigma]$ iff $\mathfrak{A} \models \varphi[\sigma']$ for all those σ' which agree with σ except on the variable v , it follows that $\mathfrak{A} \models \forall v \varphi[\sigma]$. This proves (ii). $\square$

2.3 SYNTACTIC ENTAILMENT

sec syn ent

2.3.1 DERIVATIONS

def fp

Definition 2.19 (Derivation / Formal Proof). Suppose φ is a *formula* and Σ is a set of formulas in the language $\mathcal{L}$.

A *deduction/derivation/(formal) proof of φ from Σ* , or alternatively a Σ -*derivation of φ* , is a finite sequence $\varphi_1, \dots, \varphi_n$ of *formulas* such that $\varphi_n = \varphi$ and such that each φ_i is a logical axiom or a member of Σ , or deducible from earlier φ_j 's by means of one of the rules of inference. $\square$

def syn ent

Definition 2.20 (Syntactic Entailment). Suppose Σ is a set of formulas (possibly infinite) and φ is a formula from the language $\mathcal{L}$. Then

$$\Sigma \vdash \varphi \quad (2.9)$$

means there is a formal proof from Σ to φ . We say Σ *syntactically entails/implies* φ .

If Σ is the empty set we write $\emptyset \vdash \varphi$ or $\vdash \varphi$, and say φ is a *logical theorem*. $\square$

Exercise 2.21. Show $\Sigma \vdash \varphi \iff \Sigma \vdash \forall v \varphi$.

More generally, show that $\Sigma \vdash \varphi(v_1, \dots, v_n) \iff \Sigma \vdash \forall v_1, \dots, \forall v_n \varphi(v_1, \dots, v_n)$.

(One direction uses repeated applications of the Quantifier *Axiom* (2.2). The other uses repeated applications of the *Rule* for Generalisation (2.8).) $\square$

Remark 2.22 (Finite Nature of Proofs). Since derivations are finite, it is an important immediate consequence that $\Sigma \vdash \varphi$ iff $\Sigma_0 \vdash \varphi$ for some *finite* $\Sigma_0 \subseteq \Sigma$. $\square$

rem st

Remark 2.23 (Why these Axioms and Inference Rules?). As noted previously, we are working within a Hilbert-style system.

Of course, the axioms should be true in any $\mathcal{L}$ -structure under any assignment for the free variables, and the inference rules should preserve truth. This is so, and it is the main content of the *Soundness Theorem* 2.26. $\square$

ex st

Exercise 2.24. For each axiom try to convince yourself that it is indeed the case that the axiom is true in every $\mathcal{L}$ -structure under every assignment for the free variables, and the inference rules preserve this truth.

The proof of the Soundness Theorem 2.26 writes out the details. $\square$

Remark 2.25. Just about every text has its own variant of the axioms and rules. The main point is that the axioms and rules should be sufficient to establish all other “valid” logical formulas. This requires that they should be sufficient to prove Gödel’s *Completeness Theorem*. And this is what we will show in Section 3.4.

Showing that the Hilbert System is equivalent to any other standard system is not relatively difficult. In this way one has completeness for any reasonable first-order logic system. $\square$

2.3.2 SOUNDNESS THEOREM

Theorem 2.26 (Soundness Theorem). *If Σ is a set of formulas and φ is a formula (all in the same language $\mathcal{L}$), then $\Sigma \vdash \varphi$ implies $\Sigma \models \varphi$.* thm sound

Proof. Assume $\Sigma \vdash \varphi$ and $\varphi_1, \dots, \varphi_{n-1}, \varphi_n (= \varphi)$ is a corresponding derivation.

To prove that $\Sigma \models \varphi$ we need to show that if $\mathfrak{A} \models \Sigma$ then $\mathfrak{A} \models \varphi$.

So assume $\mathfrak{A} \models \Sigma$.

We will show by induction on i that $\mathfrak{A} \models \varphi_i$ for $i = 1, \dots, n$.

Since φ_1 is either a logical axiom or $\varphi_1 \in \Sigma$, it follows that $\mathfrak{A} \models \varphi_1$.

Suppose $i > 1$ and $\mathfrak{A} \models \varphi_j$ for all $j < i$.

If φ_i is a logical axiom or $\varphi_i \in \Sigma$, then as for φ_1 , $\mathfrak{A} \models \varphi_i$.

If φ_i is obtained from previous φ_j s by modus ponens or generalisation, then $\mathfrak{A} \models \varphi_i$ by Theorem 2.18.

It follows by induction that $\Sigma \models \varphi_n$, that is $\Sigma \models \varphi$. □

2.4 PROPERTIES OF DERIVATIONS

We discuss just a few techniques for constructing formal derivations/proofs from the Hilbert axioms, with an emphasis on what is needed for the proof of Gödel's Completeness Theorem 3.17.

Once we have the Completeness Theorem (and the Soundness Theorem), it is usually easier to check $\Sigma \models \varphi$ than $\Sigma \vdash \varphi$. But of course, this is not a constructive argument for showing $\Sigma \vdash \varphi$.

Formal derivations of Hilbert type are discussed in great detail in various logic texts. See (Enderton, 2001; Hils and Loeser, 2019; Johnstone, 1987; Mendelson, 2015)

Contemporary developments of proof theory concentrate on different axiomatic systems, natural deduction and sequent calculus in particular.

2.4.1 AN EQUIVALENCE RELATION ON CONSTANT SYMBOLS

With an eye to what will be needed in Step 4 of the proof of Gödel's Completeness Theorem 3.17, let $\mathcal{C}$ be the set of constant symbols (or some subset thereof) in the language $\mathcal{L}$. Let Σ be a set of sentences in $\mathcal{L}$. For $c_1, c_2 \in \mathcal{C}$ define sec equivrel C

$$c_1 \sim c_2 \iff \Sigma \vdash c_1 = c_2. \quad (2.10)$$

The formal deduction proving *reflexivity*, $c \sim c$, is

$u = u$	Equality axioms (2.4)
$\forall u (u = u)$	Generalisation (2.8)
$\forall u (u = u) \rightarrow c = c$	Quantifier axiom (2.2)
$c = c$	Modus Ponens (2.7)

Hence $\vdash c = c$, and in particular $\Sigma \vdash c = c$, and so $c \sim c$.

To show *symmetry*, $c \sim d \Rightarrow d \sim c$, suppose $c \sim d$. Then by definition $\Sigma \vdash c = d$.

Extend such a derivation to a derivation of $\Sigma \vdash d = c$ by *concatenating* at its end the following:

$$\begin{array}{ll}
 u = v \rightarrow v = u & \text{Equality axioms (2.4)} \\
 \forall u \forall v (u = v \rightarrow v = u) & \text{Generalisation (2.8), twice} \\
 (c = d \rightarrow d = c) & \text{Quantifier axiom (2.2), twice} \\
 d = c & \text{Modus Ponens (2.7) and using } \Sigma \vdash c = d
 \end{array}$$

Hence $\Sigma \vdash d = c$, and so $d \sim c$.

Exercise 2.27. So far we have seen that $\sim$ is reflexive and symmetric. The proof of *transitivity*, that $(c \sim d \ \& \ d \sim e) \implies c \sim e$, follows a similar pattern. Do it.

2.4.2 EXISTENTIAL GENERALISATION

The conditions on φ in the following are exactly as for universal instantiation, see (2.2).

Proposition 2.28. (*Existential Generalisation*) Suppose $\varphi(v)$ is a formula of $\mathcal{L}$, $\varphi(t)$ is obtained from φ by substituting t for each free occurrence of v in φ , and no variable of t occurs bound in φ at the places where it is substituted. Then prp EG

$$\vdash \varphi(t) \rightarrow \exists v \varphi(v). \quad (2.11)$$

Proof. From the definition of “ $\exists$ ” we need to establish

$$\vdash \varphi(t) \rightarrow \neg \forall v \neg \varphi(v).$$

As in Remark 2.15 it is sufficient⁴ to show

$$\vdash \forall v \neg \varphi(v) \rightarrow \neg \varphi(t).$$

But this is an instance of universal instantiation, quantifier axiom (2.2). □

Remark. The condition “no variable of t occurs bound in $\varphi(v)$ at the places where it is substituted” allows the good behaviour under substitution in the following example and prevents the BAD/unintended behaviour.

Let r be a binary relation symbol in the language $\mathcal{L}$.

Take $\varphi(v)$ to be the formula $\forall u r(u, v)$. In an $\mathcal{L}$ -structure it says that every element is “ r -related” to (the interpretation of/assignment for) v . (Good, ✓)

Then $\varphi(w)$ is the formula $\forall u r(u, w)$. In an $\mathcal{L}$ -structure it says that every element is “ r -related” to (the interpretation of/assignment for) w . (Good, ✓)

Also, $\varphi(x)$ is the formula $\forall u r(u, x)$. In an $\mathcal{L}$ -structure it says that every element is “ r -related” to (the interpretation of/assignment for) x . (Good, ✓)

But $\varphi(u)$ is the *sentence* $\forall u r(u, u)$, which says every element is related to itself. (BAD, unintended, ✗) □

⁴In propositional logic $\vdash (P \rightarrow \neg Q) \rightarrow (Q \rightarrow \neg P)$.

So in first-order logic $\vdash (\forall v \neg \varphi(v) \rightarrow \neg \varphi(t)) \rightarrow (\varphi(t) \rightarrow \neg \forall v \neg \varphi(v))$.

2.4.3 DEDUCTION THEOREM

sec dt

Remark 2.29 (Why σ is a Sentence in the Deduction Theorem 2.33).

Suppose $\mathcal{L}$ has a single unary relation symbol U and σ is the formula $U(x)$.

Then $U(x) \vdash \forall x U(x)$, since “ $U(x), \forall x U(x)$ ” is a deduction of $\forall x U(x)$ from $U(x)$ according to Definitions 2.19 and 2.20. But if the Deduction Theorem were true with σ replaced by $U(x)$, it would follow that $\vdash U(x) \rightarrow \forall x U(x)$, and so by Generalisation

$$\vdash \forall x (U(x) \rightarrow \forall x U(x)). \quad \text{risky ded (2.12)}$$

However this is not the case by the Soundness Theorem (see Remark 2.23 and Exercise 2.24), because $\not\models \forall x (U(x) \rightarrow \forall x U(x))$ and so $\not\models \forall x (U(x) \rightarrow \forall x U(x))$.

To see that $\not\models \forall x (U(x) \rightarrow \forall x U(x))$ consider an $\mathcal{L}$ -structure $\mathfrak{A}$ with universe A , for which $a, b \in A$, $a \in U^{\mathfrak{A}}$ and $b \notin U^{\mathfrak{A}}$. Then $\mathfrak{A} \not\models \forall x (U(x) \rightarrow \forall x U(x))$ by the following Exercise. $\square$

Exercise 2.30. Explain why $\mathfrak{A} \not\models \forall x (U(x) \rightarrow \forall x U(x))$. $\square$

Remark 2.31 (But what If σ is Not a Sentence?).

For simplicity suppose $\sigma \vdash \tau$, so Σ is $\emptyset$.

One can obtain a version of $\vdash \sigma \rightarrow \tau$ provided one restricts the class of allowable further deductions. In particular, generalisation has to be restricted to prevent (2.12). See (Mendelson, 2015, pp 70–72) for a careful analysis

While this might allow cleaner legitimate deductions in the Hilbert system, that is not our goal. And if formal deductions were the goal, then one should really be using sequent calculus or natural deduction. $\square$

Remark 2.32 (Connection between $\rightarrow$, $\vdash$ and $\models$).

We will show for sentences φ and ψ that

$$\vdash \varphi \rightarrow \psi \iff \varphi \vdash \psi \iff \varphi \models \psi \iff \models \varphi \rightarrow \psi. \quad (2.13)$$

The first $\iff$ is the Deduction Theorem 2.33, the third $\iff$ is Remark 1.29. The middle $\iff$ is the Gödel Completeness Theorem 3.17 in Chapter 3, and the middle $\implies$ is just the Soundness Theorem 2.26.

One can also add Σ to the left side of each $\vdash$ and $\models$, where Σ is a set of sentences. $\square$

thm dt

Theorem 2.33 (Deduction Theorem). *If Σ is a set of formulas, σ a sentence, and τ a formula, then*

$$\Sigma \cup \{\sigma\} \vdash \tau \iff \Sigma \vdash \sigma \rightarrow \tau.$$

Proof For the (easy) “ $\Leftarrow$ ” direction suppose $\Sigma \vdash \sigma \rightarrow \tau$ and let $\varphi_1, \dots, \varphi_n, \sigma \rightarrow \tau$ be a corresponding derivation.

Then, using modus ponens, $\varphi_1, \dots, \varphi_n, \sigma \rightarrow \tau, \sigma, \tau$ is a derivation for $\Sigma \cup \{\sigma\} \vdash \tau$.

For the “ $\Rightarrow$ ” direction suppose $\Sigma \cup \{\sigma\} \vdash \tau$ and let $\varphi_1, \dots, \varphi_n (= \tau)$ be a derivation. We show by induction on i that there is a derivation of $\Sigma \vdash \sigma \rightarrow \varphi_i$.

- (i) If φ_i is an *axiom* or $\varphi_i \in \Sigma$, then $\varphi_i, \varphi_i \rightarrow (\sigma \rightarrow \varphi_i), \sigma \rightarrow \varphi_i$ is such a derivation. (See Exercise 2.34)

- (ii) If $\varphi_i = \sigma$, then $\sigma \rightarrow \sigma$ is such a derivation ($\sigma \rightarrow \sigma$ is a tautology of $\mathcal{L}$).
- (iii) If φ_i is deduced by *modus ponens* from φ_j and from $\varphi_k = \varphi_j \rightarrow \varphi_i$, ($j, k < i$), let $\theta_1, \dots, \theta_n, \sigma \rightarrow \varphi_j$ be the derivation for $\Sigma \vdash \sigma \rightarrow \varphi_j$, and $\theta'_1, \dots, \theta'_m, \sigma \rightarrow (\varphi_j \rightarrow \varphi_i)$ be the derivation for $\Sigma \vdash \sigma \rightarrow (\varphi_j \rightarrow \varphi_i)$.

Then the following is a derivation for $\Sigma \vdash \sigma \rightarrow \varphi_i$.

$$\begin{array}{ll}
 \theta_1, \dots, \theta_n, \sigma \rightarrow \varphi_j, \theta'_1, \dots, \theta'_m, \sigma \rightarrow (\varphi_j \rightarrow \varphi_i), & \\
 \left(\sigma \rightarrow (\varphi_j \rightarrow \varphi_i) \right) \rightarrow \left((\sigma \rightarrow \varphi_j) \rightarrow (\sigma \rightarrow \varphi_i) \right), & \text{(prop. axiom, Exercise 2.35)} \\
 (\sigma \rightarrow \varphi_j) \rightarrow (\sigma \rightarrow \varphi_i), & \text{(modus ponens)} \\
 \sigma \rightarrow \varphi_i. & \text{(modus ponens)}
 \end{array}$$

- (iv) If φ_i is deduced from φ_j by *generalisation*, i.e. $\varphi_i = \forall v \varphi_j$, let $\theta_1, \dots, \theta_n, \sigma \rightarrow \varphi_j$ be a derivation for $\Sigma \vdash \sigma \rightarrow \varphi_j$.

Then the following is a derivation for $\Sigma \vdash \sigma \rightarrow \varphi_i$.

(Note that in the application of quantifier axiom 1, v is not free in σ since σ is a sentence.)

$$\begin{array}{ll}
 \theta_1, \dots, \theta_n, \sigma \rightarrow \varphi_j, & \\
 \forall v (\sigma \rightarrow \varphi_j), & \text{(generalisation)} \\
 \forall v (\sigma \rightarrow \varphi_j) \rightarrow (\sigma \rightarrow \forall v \varphi_j), & \text{(quantifier axiom 1)} \\
 \sigma \rightarrow \forall v \varphi_j. & \text{(modus ponens)}
 \end{array}$$

□

ex for (i)

Exercise 2.34. (The following is used in (i) in the previous proof.)

Show for any two formulas φ and ψ , that $\varphi \rightarrow (\psi \rightarrow \varphi)$ is a propositional axiom. □

ex for (iii)

Exercise 2.35. (The following is used in (iii) in the previous proof.)

Show for any three formulas φ, ψ, χ , that $(\varphi \rightarrow (\psi \rightarrow \chi)) \rightarrow ((\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \chi))$ is a propositional axiom. □

Remark 2.36 (Using the Axioms and Rules of Inference). Notice how certain propositional axioms, the first of the two quantifier axioms, and both rules of inference, were all used in the proof of the Deduction Theorem 2.33.

Regarding the propositional axioms, the first two of the three axiom schemas for Propositional Logic itself were used directly, see Section 9.2. The contrapositive schema (iii) was not used.

Proving the Deduction Theorem must surely have provided a strong incentive for the original axioms and rules of inference used in the Hilbert system. □

2.5 MATERIAL TO INCLUDE

old sec ds

Chapter 3

Gödel's Completeness Theorem

ch fol

3.1 OVERVIEW

The goal is to provide a system that is sound, verifiable and complete, such that

$$\Sigma \vdash \varphi \iff \Sigma \models \varphi$$

I find it incredible that we have formal proof systems for which semantic validity, a sweeping concept ranging across all possible semantic contexts, aligns exactly with provability in the system. Universal truth is thereby reduced to a finite reasoning process—every universal truth is true for a finite reason and this surely informs further philosophical questions and analysis.

David Hamkins ([Hamkins, 2021](#), Section 5.1)

3.2 CONSISTENCY

Remark 3.1 (Fixed Countable Language). In the following, unless otherwise clear from context, all formulas are in the same fixed language $\mathcal{L}$.

The cardinality of $\mathcal{L}$ is defined in (1.8), and prior to Section 5.1, is assumed countable. This implies that the set of formulas of $\mathcal{L}$ is also countable. However, the cardinality of the set of function and relation symbols, and often but not always of the set of constant symbols, will be assumed finite.

The significant exception to this is Section 5.1 and beyond. □

Remark 3.2 (Syntactic Notions). The notions of a *consistent* set of sentences (Definition 3.4) and of a *maximal consistent* set of sentences (Definition 3.10) are syntactic. That is, they involve formal derivations from axioms. □

Definition 3.3 (Absurdum). It is sometimes convenient to have a sentence, usually denoted by “ $\perp$ ” and called *falsum*, *absurdity* or *contradiction*, and which is false in all structures for the language $\mathcal{L}$.

We define $\perp$ by

$$\perp := \varphi \wedge \neg\varphi \tag{3.1} \text{def absurd}$$

for any sentence φ . For example, φ can be taken to be $\forall x (x = x)$. □

The essential point is that $\perp$ is false in every $\mathcal{L}$ -structure.

On the other hand $\neg\perp$ is true in every $\mathcal{L}$ -structure. It is sometimes denoted by $\top$.

Definition 3.4 (Consistency). A set of sentences Σ is *consistent* if a contradiction cannot be derived from Σ . That is, if $\Sigma \not\vdash \perp$. def consis

Σ is *inconsistent* if $\Sigma \vdash \perp$. □

Exercise 3.5. Prove the definition of consistency is independent of the choice of φ in (3.1)

Using this terminology we have the following version of the Soundness Theorem 2.26.

Corollary 3.6 (Soundness). Σ has a model $\implies \Sigma$ is consistent. cor sound

Proof. If Σ is inconsistent then $\Sigma \vdash \perp$. Hence $\Sigma \models \perp$ by the Soundness Theorem 2.26, and so Σ has no model. □

The following gives three equivalent criteria for consistency, and the corresponding three criteria for inconsistency.

Proposition 3.7 (Consistency, Equivalent Versions). Suppose Σ is a set of sentences in the language $\mathcal{L}$. def con

Then

$$\begin{aligned} \Sigma \text{ is consistent} &\iff \text{for every sentence } \varphi, \Sigma \not\vdash \varphi \text{ or } \Sigma \not\vdash \neg\varphi \\ &\iff \text{for some sentence } \psi, \Sigma \not\vdash \psi \\ &\iff \Sigma \not\vdash \perp. \end{aligned}$$

The set Σ is inconsistent if it is not consistent. That is,

$$\begin{aligned} \Sigma \text{ is inconsistent} &\iff \text{for some sentence } \varphi, \Sigma \vdash \varphi \text{ and } \Sigma \vdash \neg\varphi \\ &\iff \text{for every sentence } \psi, \Sigma \vdash \psi \\ &\iff \Sigma \vdash \perp. \end{aligned}$$

Proof.

To prove that the three definitions of “inconsistent” are equivalent note that for any two sentences φ and ψ , $\vdash \varphi \rightarrow (\neg\varphi \rightarrow \psi)$ since it is a propositional tautology.

(a) So if for some sentence φ , $\Sigma \vdash \varphi$ and $\Sigma \vdash \neg\varphi$, it follows by juxtaposing the two deductions and using modus ponens twice that, for every ψ , $\Sigma \vdash \psi$.

(b) If $\Sigma \vdash \psi$ for every ψ , then in particular $\Sigma \vdash \perp$.

(c) If $\Sigma \vdash \perp$, since $\perp \rightarrow \varphi$ and $\perp \rightarrow \neg\varphi$ as each is a propositional axiom, it follows that $\Sigma \vdash \varphi$ and $\Sigma \vdash \neg\varphi$ (for every and hence for some φ).

The three definitions of “consistent” are also equivalent since Σ is consistent iff it is not inconsistent. But the first/second/third definitions for consistency are the negation of the first/second/third definitions for inconsistency. □

The following gives connections between consistency and derivability.

Theorem 3.8 (Properties of Consistency). Suppose Σ is a set of sentences and σ is a sentence. thm conseq con

1. Σ is consistent iff every finite subset of Σ is consistent;
2. Suppose Σ is consistent. Then $\Sigma \vdash \sigma \implies \Sigma \cup \{\sigma\}$ is consistent, but not conversely.
3. $\Sigma \cup \{\sigma\}$ is inconsistent $\iff \Sigma \vdash \neg\sigma$, $\Sigma \cup \{\sigma\}$ is consistent $\iff \Sigma \not\vdash \neg\sigma$;

4. $\Sigma \cup \{\neg\sigma\}$ is inconsistent $\iff \Sigma \vdash \sigma$, $\Sigma \cup \{\neg\sigma\}$ is consistent $\iff \Sigma \not\vdash \sigma$.

Proof.

1. Immediate, since proofs have finite length.
2. Suppose Σ is consistent and $\Sigma \vdash \sigma$.
If $\Sigma \cup \{\sigma\}$ is inconsistent, that is $\Sigma \cup \{\sigma\} \vdash \perp$, then $\Sigma \vdash \sigma \rightarrow \perp$ by the Deduction Theorem 2.33. Hence $\Sigma \vdash \perp$ by modus ponens, which contradicts the consistency of Σ .
Hence $\Sigma \cup \{\sigma\}$ is consistent.
3. For the first statement:
(a) If $\Sigma \cup \{\sigma\}$ is inconsistent, then $\Sigma \cup \{\sigma\} \vdash \neg\sigma$ and so $\Sigma \vdash \sigma \rightarrow \neg\sigma$ by the Deduction Theorem. But $\vdash (\sigma \rightarrow \neg\sigma) \rightarrow \neg\sigma$ as this is a propositional tautology.¹
Hence $\Sigma \vdash \neg\sigma$ by modus ponens.
(b) If $\Sigma \vdash \neg\sigma$ then $\Sigma \cup \{\sigma\} \vdash \neg\sigma$, but $\Sigma \cup \{\sigma\} \vdash \sigma$, and so $\Sigma \cup \{\sigma\}$ is inconsistent.
The second statement in 3 follows immediately.
4. This follows from 3, using $\Sigma \vdash \sigma$ iff $\Sigma \vdash \neg\neg\sigma$. □

Definition 3.9 (Theory of a Structure). The *theory* of an $\mathcal{L}$ -structure $\mathfrak{A}$ is the set of all sentences true in the structure. That is,

$$\text{Th } \mathfrak{A} := \{\varphi \mid \mathfrak{A} \models \varphi\}, \quad (3.2)$$

where the φ are $\mathcal{L}$ -sentences. (For every φ , either $\varphi \in \text{Th } \mathfrak{A}$ or $\neg\varphi \in \text{Th } \mathfrak{A}$, and *not* both.) □

def maxcon

Definition 3.10 (Maximal Consistency). A set of sentences Σ is *maximal consistent* in the language $\mathcal{L}$ iff Σ is consistent and has no proper consistent extension in $\mathcal{L}$.

That is, Σ is consistent and $\Sigma \cup \{\sigma\}$ is consistent iff $\sigma \in \Sigma$.

Remark 3.11 (Every Example of a Maximal Consistent Set of Sentences). It follows from Theorem 3.12 that $\text{Th } \mathfrak{A}$ is a maximal consistent set of sentences for every $\mathcal{L}$ -structure $\mathfrak{A}$.

Conversely, it will follow from Gödel's Completeness Theorem 3.20, that *every* maximal consistent set of $\mathcal{L}$ -sentences is $\text{Th } \mathfrak{A}$ for some $\mathcal{L}$ -structure $\mathfrak{A}$, namely the model of $\mathfrak{A}$ constructed in the proof of Gödel's Completeness Theorem! □

thm maxcon

Theorem 3.12 (Maximal Consistency). A set of sentences Σ is maximal consistent iff Σ is consistent and for each sentence σ either $\sigma \in \Sigma$ or $\neg\sigma \in \Sigma$.

Proof. First suppose Σ is maximal consistent and so in particular is consistent.

For a sentence σ , if $\sigma \in \Sigma$ we are done.

If $\sigma \notin \Sigma$ then $\Sigma \cup \{\sigma\}$ is inconsistent by maximality. So $\Sigma \vdash \neg\sigma$ by Theorem 3.8.3. Hence $\Sigma \cup \{\neg\sigma\}$ is consistent and so $\neg\sigma \in \Sigma$ by maximality.

Next suppose Σ is consistent and for each sentence σ either $\sigma \in \Sigma$ or $\neg\sigma \in \Sigma$.

Consider an arbitrary sentence σ .

If $\Sigma \cup \{\sigma\}$ is consistent then $\Sigma \not\vdash \neg\sigma$ by Theorem 3.8.3. Assume $\sigma \notin \Sigma$ then $\neg\sigma \in \Sigma$ by assumption, so $\Sigma \vdash \neg\sigma$. This contradicts $\Sigma \not\vdash \neg\sigma$ and so $\sigma \in \Sigma$.

¹This follows easily by truth tables. To motivate it, note that $\sigma \rightarrow \neg\sigma$, $\neg\sigma \vee \neg\sigma$ and $\neg\sigma$ are tautologically equivalent

If $\Sigma \cup \{\sigma\}$ is inconsistent then $\Sigma \vdash \neg\sigma$ by Theorem 3.8.3. Assume $\sigma \in \Sigma$, then $\Sigma \vdash \sigma$. But this contradicts $\Sigma \vdash \neg\sigma$ and Σ is consistent, and so $\sigma \notin \Sigma$.

From Definition 3.10 it follows that Σ is maximal consistent. $\square$

cor max con

Corollary 3.13. *If Σ is maximal consistent then for any sentences σ, τ :*

$$\begin{aligned} \sigma \in \Sigma &\iff \Sigma \vdash \sigma && \text{max con 1 (3.3)} \\ \sigma \notin \Sigma &\iff \neg\sigma \in \Sigma \iff \Sigma \vdash \neg\sigma && \text{max con 2 (3.4)} \\ \sigma \wedge \tau \in \Sigma &\iff \sigma \in \Sigma \text{ and } \tau \in \Sigma. && \text{max con 3 (3.5)} \end{aligned}$$

Exercise 3.14. Prove the Corollary. Keep it as brief as you can — just a few lines. $\square$

The following result due to Lindenbaum will be applied in the proof of Gödel's Completeness Theorem in Section 3.4.

Lindenbaum's Theorem is easy to prove with the current set-up, it is essentially the same as for propositional logic and is just Theorem 9.18. But since that is in an optional section, let's prove it here.

thm Lind

Theorem 3.15 (Lindenbaum's Theorem). *Suppose Σ is a consistent set of sentences in the language $\mathcal{L}$. Then Σ can be extended to a maximal consistent set Σ^* in the same language $\mathcal{L}$.*

Proof. Let $\varphi_1, \varphi_2, \varphi_3, \dots$ be an enumeration of all sentences from $\mathcal{L}$. Define

$$\begin{aligned} \Sigma_1 &= \Sigma, \\ \Sigma_{n+1} &= \begin{cases} \Sigma_n \cup \{\varphi_n\} & \text{if } \Sigma_n \cup \{\varphi_n\} \text{ is consistent,} \\ \Sigma_n & \text{otherwise,} \end{cases} && \text{def sign2 (3.6)} \\ \Sigma^* &= \bigcup_{n \geq 1} \Sigma_n. \end{aligned}$$

Each Σ_n is consistent by construction. Hence Σ^* is consistent since derivations are finite in length.

To see that Σ^* is *maximal* consistent, suppose $\Sigma^* \cup \{\varphi\}$ is consistent. We want to show $\varphi \in \Sigma^*$.

But $\varphi = \varphi_n$ for some n , and because $\Sigma_n \cup \{\varphi_n\}$ must also be consistent since $\Sigma_n \subset \Sigma^*$, it follows by construction that $\varphi_n \in \Sigma_{n+1}$. Hence $\varphi_n \in \Sigma^*$ and so Σ^* is maximal consistent. $\square$

Remark 3.16 (“Construct” vs. “Constructivism”). The argument used in the proof of Lindenbaum's theorem is called a *constructive* process in the following sense. At each stage in the construction, one of two (mutually exclusive) alternatives are available which determine whether or not to add φ_n to Σ_n . However, there is not normally an algorithm to decide which alternative is the case.²

While this usage of the word “construction” as here is standard mathematical practice, the proof of Lindenbaum's Theorem is not an acceptable process within the *Constructivism* view of mathematics. Similarly, the proof of Gödel's Theorem in Section 3.3 is *not* acceptable within the Constructivist program. $\square$

²See Chapter 5.3.

3.3 GÖDEL'S COMPLETENESS THEOREM – PRELIMINARIES

In Section 3.4 we prove the following major result due to Gödel. sec gct

Theorem 3.17 (Completeness Theorem, Version 1). *Suppose Σ is a set of sentences in the language $\mathcal{L}$, φ is a sentence in $\mathcal{L}$, and $\Sigma \models \varphi$. Then $\Sigma \vdash \varphi$.* thm gct

Remark 3.18 (Significance of Completeness). Suppose we know (by any means whatsoever) that φ is true in a structure whenever all sentences in Σ are true in the structure.

That is, we know φ is true in all models of Σ .

Then by the Completeness Theorem there is a formal first-order logic proof (in the sense previously discussed) of φ from Σ .

Although the theorem refers to first-order logic, its significance is enhanced by the fact that all of mathematics can be treated within (*first-order*) Zermelo Fraenkel set theory, perhaps with additional axioms such as the axiom of choice, the generalised continuum hypothesis, etc. □

Remark 3.19 (An Important Case). Suppose Σ has *no* models, i.e. Σ is not satisfiable. rem imp case

This is the case iff $\Sigma \models \perp$. *Why?*

Then by Theorem 3.17, $\Sigma \vdash \perp$. That is, Σ is inconsistent. □

An equivalent, and equally important version of the Completeness Theorem, is the following.

Theorem 3.20 (Completeness Theorem, Version 2). *Suppose Σ is a consistent set of sentences in the language $\mathcal{L}$. Then Σ is satisfiable (i.e. has a model).* thm gct2

Proposition 3.21 (Versions 1 & 2 are Equivalent).

Proof. Version 1 implies Version 2 by Remark 3.19.

Next assume Version 2 (Theorem 3.20).

To prove Version 1 (Theorem 3.17), suppose Σ is a set of sentences, φ is a sentence, and $\Sigma \models \varphi$.

$\therefore \Sigma \cup \{\neg\varphi\}$ has no model.

$\therefore \Sigma \cup \{\neg\varphi\}$ is not consistent by Theorem 3.20.

$\therefore \Sigma \vdash \varphi$ by Theorem 3.8.

So Version 2 implies Version 1. □

Remark 3.22 (Summary). From Definition 3.7 and the Definition of satisfiability,

$$\Sigma \text{ is consistent} \iff \Sigma \not\models \perp, \quad \Sigma \text{ is satisfiable} \iff \Sigma \not\models \perp. \quad \text{con sat (3.7)}$$

Theorems 3.17 and 3.20 respectively, together with the Soundness Theorem, can now be summarised as follows (using the contrapositive for Theorem 3.20):

$$\text{for all } \Sigma \text{ and } \varphi, \quad \Sigma \models \varphi \iff \Sigma \vdash \varphi; \quad (3.8)$$

$$\text{for all } \Sigma, \quad \Sigma \models \perp \iff \Sigma \vdash \perp. \quad (3.9)$$

□

3.4 GÖDEL'S COMPLETENESS THEOREM – PROOF

sec prf gct

In this Section, all languages are finite or countably infinite.

The ideas extend to uncountable languages, and this is important for applications to algebra. See Section 5.1.

We now prove the version of Gödel's Completeness Theorem given by Theorem 3.20. Namely:

thm GCT

Theorem 3.23 (Gödel's Completeness Theorem). *Suppose Σ is a consistent set of sentences in the language $\mathcal{L}$. Then Σ has a model $\mathfrak{A}$.*

Assuming the cardinality³ $|\mathcal{L}| = \aleph_0$ (as we do at this stage), then $|\mathfrak{A}| \leq \aleph_0$ where $|\mathfrak{A}|$ is the cardinality of the universe of $\mathfrak{A}$.

Since all we have is the set Σ of consistent sentences, how do we go about building/constructing such a model $\mathfrak{A}$? In particular, what should we take for the universe A of $\mathfrak{A}$. The method here is due to Henkin and is somewhat different from that of Gödel, and is of broader applicability.

The proof is contained in the next 8 Steps.

Proof

3.4.1 STEP 1 ADDING A SET $\mathcal{C}$ OF NEW CONSTANT SYMBOLS TO $\mathcal{L}$

Introduce a language $\mathcal{L}^* = \mathcal{L} \cup C$ which extends $\mathcal{L}$, by adding a countably infinite set $C = \{c_1, c_2, \dots\}$ of new constant symbols not in $\mathcal{L}$.

3.4.2 STEP 2 EXTEND Σ TO Σ^* WITH $\mathcal{C}$ A SET OF HENKIN WITNESSES

add Henk wit

This is the key result, due to Henkin. In the following Lemma, the constant c is called a *Henkin witness* for the formula $\exists v \varphi(v)$.

lem HW

Lemma 3.24 (Henkin Witnesses). *There is a consistent extension Σ^* of Σ in the extended language $\mathcal{L}^*$, with the following property:*

For every formula $\varphi = \varphi(v)$ of $\mathcal{L}^$ (not just of $\mathcal{L}$) with at most one free variable depending on φ (here it is v), there is a constant $c \in \mathcal{C}$ (c depending on φ) such that*

$$\exists v \varphi(v) \rightarrow \varphi(c) \in \Sigma^*. \quad \text{eg hw (3.10)}$$

Proof. List all formulas of $\mathcal{L}^*$ (not just formulas of $\mathcal{L}$) with one free variable:

$$\varphi_1, \varphi_2, \dots, \varphi_n, \dots \quad \text{def hw2 (3.11)}$$

Define a sequence of consistent extensions of Σ , in the language $\mathcal{L}^*$:

$$\Sigma =: \Sigma_0^* \subset \Sigma_1^* \subset \Sigma_2^* \subset \dots \subset \Sigma_n^* \subset \dots,$$

by

$$\Sigma_n^* = \Sigma_{n-1}^* \cup \{\exists v \varphi_n(v) \rightarrow \varphi_n(c)\} \quad \text{for } n \geq 1, \quad \text{def signp (3.12)}$$

³Recall the definition of the cardinality $|\mathcal{L}|$ of $\mathcal{L}$ in (1.8).

where (depending on n), v is the free variable appearing in φ_n and c is the first constant symbol in C not already appearing in Σ_k^* for $k < n$. This is possible since only one new sentence is added at each stage, and so only a finite number of constant symbols from C have already been added at any (finite) stage in the process.

We claim that Σ_n^* is consistent for all n .⁴

This is true for $n = 0$ since $\Sigma_0^* = \Sigma$ and Σ is consistent.

Next assume Σ_{n-1}^* is consistent but Σ_n^* is not. Then:

$$\begin{aligned}
& \Sigma_{n-1}^* \vdash \neg(\exists v \varphi_n(v) \rightarrow \varphi_n(c)) && \text{(Theorem 3.8.2)} \\
\therefore & \Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(c) && \text{(propositional logic and modus ponens)} \\
\therefore & \Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(w) && \text{(in the previous derivation, replace } c \text{ everywhere by} \\
& && \text{a variable } w \text{ not previously used, free or bound)}^5 \\
\therefore & \Sigma_{n-1}^* \vdash \exists v \varphi_n(v), \quad \Sigma_{n-1}^* \vdash \neg \varphi_n(w) && \text{(propositional logic and modus ponens)} \\
\therefore & \Sigma_{n-1}^* \vdash \exists v \varphi_n(v), \quad \Sigma_{n-1}^* \vdash \forall w \neg \varphi_n(w) && \text{(generalisation)} \\
\therefore & \Sigma_{n-1}^* \vdash \exists v \varphi_n(v), \quad \Sigma_{n-1}^* \vdash \neg \exists w \varphi_n(w) && \text{(definition of } \forall \text{ from } \exists)
\end{aligned}$$

But $\vdash \exists v \varphi_n(v) \rightarrow \exists w \varphi_n(w)$ ⁶ and so $\Sigma_{n-1}^* \vdash \exists w \varphi_n(w)$ by modus ponens. This is a

⁴This consistency is not surprising. See “Plausibility Argument” in the following boxed discussion.

⁵To see more carefully that this changes the derivation for $\Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(c)$ into a (valid) derivation for $\Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(w)$, argue as follows:

Consider each formula ψ in the derivation $\Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(c)$. If ψ contains one or more occurrences of c , denote ψ by $\psi(c)$ and denote the changed formula in the proposed derivation for $\Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(w)$ by $\psi(w)$.

Now consider all the ways in which the formula ψ (or $\psi(c)$) can occur (as described in Definition 2.19) in the derivation $\Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(c)$, and can then change after replacing c by w in the proposed derivation $\Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(w)$.

1. If $\psi(c)$ is a propositional axiom (Definition 2.3) then so is $\psi(w)$, since it is similarly constructed from the same propositional tautology.
2. If $\psi(c)$ is a logical axiom of type (2.1) then so is $\psi(w)$, since it also is constructed as in (2.1).
3. If $\psi(c)$ is a logical axiom of type (2.2) then again so is $\psi(w)$, as it is also constructed as in (2.2). If the term t contains c then it will change to contain w , but the new t will still be substitutable for v as in (2.2) and footnote 1, since w does not occur in $\psi(c)$ by choice of w .
4. If ψ is an equality axiom then it does not contain any constant symbols, including c , and so ψ is unchanged in the proposed derivation $\Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(w)$.
5. If $\psi \in \Sigma_{n-1}^*$ then c does not occur in ψ by choice of c in (3.14), and so again ψ is unchanged.
6. If $\psi(c)$ is obtained by modus ponens or generalisation from previous formulas in the derivation of $\Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(c)$, then $\psi(w)$ is similarly obtained by modus ponens or generalisation from previous formulas in the derivation of $\Sigma_{n-1}^* \vdash \exists v \varphi_n(v) \wedge \neg \varphi_n(w)$.

⁶We need to be a little careful here. We passed from $\varphi_n(v)$ to $\varphi_n(c)$ to $\varphi_n(w)$. Moreover, c and v (as a free variable) do not occur in $\varphi_n(w)$. It follows that v is free for w in the sense of the universal instantiation axiom (2.2) and so

$\vdash \forall w \neg \varphi_n(w) \rightarrow \neg \varphi_n(v)$	by (2.1) as v not free in $\varphi_n(w)$
$\vdash \forall v (\forall w \neg \varphi_n(w) \rightarrow \neg \varphi_n(v))$	generalisation (2.8)
$\vdash \forall w \neg \varphi_n(w) \rightarrow \forall v \neg \varphi_n(v)$	axiom (2.2)
$\vdash \neg \forall v \neg \varphi_n(v) \rightarrow \neg \forall w \neg \varphi_n(w)$	propositional axioms
$\vdash \exists v \varphi_n(v) \rightarrow \exists w \varphi_n(w)$	definition of $\exists$

contradiction.

Hence Σ_n^* is consistent for all n and so

$$\Sigma^* := \bigcup_{n \geq 0} \Sigma_n^* \quad \text{def sign (3.13)}$$

is consistent.

Finally, C is a set of witnesses for Σ^* , since any formula φ from $\mathcal{L}^*$ with one free variable is φ_n for some n , and so a witnessing constant c for φ from C is introduced at the n -th stage in the previous procedure.

That is

$$\Sigma^* \vdash \exists v \varphi(v) \rightarrow \varphi(c), \quad \text{eg HC (3.14)}$$

where for each $\varphi \equiv \varphi(v)$, a formula with one free variable, the constant symbol c depends on the previous construction process. $\square$

PLAUSIBILITY ARGUMENT

It is natural to expect that we can consistently add $\exists v \varphi_n(v) \rightarrow \varphi_n(c)$ in (3.12) at the n th stage in the construction process. Here is a semantic argument.

(a) If we have a model for Σ_{n-1}^* in which $\exists v \varphi_n(v)$ is *true*, then interpreting the “new” symbol c as some such v , will make $\varphi_n(c)$ true and hence make the implication $\exists v \varphi_n(v) \rightarrow \varphi_n(c)$ true.

(b) If we have a model for Σ_{n-1}^* in which $\exists v \varphi_n(v)$ is *false*, then interpreting the “new” symbol c as any element in the model will make $\varphi_n(c)$ false, but the sentence $\exists v \varphi_n(v) \rightarrow \varphi_n(c)$ is again true. (*Why?*)

But of course we cannot argue semantically in this manner.

The correct argument is achieved by ensuring that there are enough axioms from first-order logic to justify the formal syntactic argument.

3.4.3 STEP 3 ENLARGE Σ^* TO A COMPLETE THEORY

By Lindenbaum’s Theorem 3.15, the theory Σ^* in the language $\mathcal{L}^* = \mathcal{L} \cup \mathcal{C}$ has a *complete* consistent extension in the same language $\mathcal{L}^*$.

Henceforth, *we will use the same notation Σ^* to denote this complete extension.*

As in Lindenbaum’s Theorem in general, *the extension is neither unique nor canonical.*

3.4.4 STEP 4 AN EQUIVALENCE RELATION ON $\mathcal{C}$

Define a relation “ $\sim$ ” on $\mathcal{C}$ by the first “ $\iff$ ” in the following,

$$c \sim c' \iff (c = c') \in \Sigma^* \iff \Sigma^* \vdash c = c' \iff \Sigma^* \not\vdash c \neq c', \quad \text{sim props (3.15)}$$

where the second “ $\iff$ ” is by the maximality of Σ^* , and the last “ $\iff$ ” is from Theorem 3.12.

Taking the negation of these statements,

$$c \not\sim c' \iff (c = c') \notin \Sigma^* \iff \Sigma^* \not\vdash c = c' \iff \Sigma^* \vdash c \neq c'. \quad \text{nsim props (3.16)}$$

We will show “ $\sim$ ” is an equivalence relation⁷. The essential point in the argument is to use the equality axioms in (2.4). Details are in Section 2.4.1.

Definition 3.25. The equivalence class corresponding to “ $\sim$ ” and containing c is written $[c]$. def eq rel

Exercise 3.26. It follows from the construction of Σ^* that each equivalence class contains a countably infinite set of constant symbols from $\mathcal{C}$. Why? □

3.4.5 STEP 5 CONSTRUCTING THE UNIVERSE $\mathcal{C}/\sim$ FOR A STRUCTURE $\mathfrak{A}^*$

The extension Σ^* of Σ is not canonical or uniquely determined. But once we have some such extension Σ^* , a structure $\mathfrak{A}^*$ for which $\mathfrak{A}^* \models \Sigma^*$ is uniquely determined from Σ^* , as we will see in this and the following Step following this one.

In this Step we build the $\mathcal{L}^*$ -structure $\mathfrak{A}^*$. In the next step we show $\mathfrak{A}^* \models \Sigma^*$.

The problem to be addressed in this Step is that the axioms for equality only ensure that the equality relation symbol “ $=$ ” in an $\mathcal{L}^*$ -structure will be interpreted as an equivalence relation, and not as the identity relation. So it is perhaps not so surprising that we will build the desired $\mathcal{L}^*$ -structure by taking equivalence classes. But what will be critical is the use of the Henkin axioms from (3.10).

The universe A of $\mathfrak{A}^*$ is defined by

$$A := \mathcal{C}/\sim := \{[c] : c \in \mathcal{C}\}, \quad (3.17) \quad \text{de}$$

where the relevant equivalence relation and the corresponding equivalence classes are from (3.15) and Definition 3.25.

The *interpretation* in $\mathfrak{A}^*$ of each constant symbol $c \in \mathcal{C}$ is, unsurprisingly, $[c]$.

$$c^{\mathfrak{A}^*} = [c]. \quad (3.18) \quad \text{def c interp}$$

The *interpretation* of the remaining constant, function and relation symbols from $\mathcal{L}^* \setminus \mathcal{C} = \mathcal{L}$, are determined as follows:

1. *Constant Symbols in $\mathcal{L}$:* Let $d \in \mathcal{L}$ be a constant symbol.

We need to show there is an equivalence class $[c]$ as in (3.17) which, in a natural way, will be the interpretation of d .

For this first note that $\vdash d = d$ by the same argument as in the “Details” box showing $\vdash c = c$.

But $\vdash d = d \rightarrow \exists v (v = d)$ from Proposition 2.28, existential generalisation.

Also $\Sigma^* \vdash \exists v (v = d) \rightarrow c = d$ for some (Henkin witness) $c \in \mathcal{C}$, by Lemma 3.24.

From two applications of modus ponens, $\Sigma^* \vdash c = d$ for some $c \in \mathcal{C}$.

It follows as in Section 2.4.1 that, if $c' \in \mathcal{C}$, then

$$\Sigma^* \vdash c' = d \iff \Sigma^* \vdash c' = c \iff [c] = [c'].$$

⁷That is, $c \sim c$, $c \sim d \implies d \sim c$, and $(c \sim d \ \& \ d \sim e) \implies c \sim e$.

So we define

$$d^{\mathfrak{A}^*} = [c] \iff \Sigma^* \vdash c = d, \quad \text{def d interp} \quad (3.19)$$

and we have shown that the definition is independent of the equivalence class representative of $[c]$.

2. *Function Symbols in $\mathcal{L}$* : Let $f \in \mathcal{L}$ be an n -ary function symbol. Define the interpretation $f^{\mathfrak{A}^*} : A^n \rightarrow A$ of f in $\mathfrak{A}$ by

$$f^{\mathfrak{A}^*}([c_1], \dots, [c_n]) = [c] \iff \Sigma^* \vdash f(c_1, \dots, c_n) = c, \quad \text{def f interp} \quad (3.20)$$

where $c_1, \dots, c_n, c \in \mathcal{C}$.

To justify this we need to show:

- (a) if $c_1, \dots, c_n \in \mathcal{C}$ then there exists $c \in \mathcal{C}$ such that $\Sigma^* \vdash f(c_1, \dots, c_n) = c$,
- (b) if also $c'_1, \dots, c'_n \in \mathcal{C}$, $c_1 \sim c'_1, \dots, c_n \sim c'_n$ then $f(c_1, \dots, c_n) \sim f(c'_1, \dots, c'_n)$.

This then implies $f^{\mathfrak{A}^*} : A^n \rightarrow A$ is a total (i.e. everywhere defined) function, and that the definition of $f^{\mathfrak{A}^*}$ is independent of choice of equivalence class representatives.

The argument, similar to that for constant symbols, is as follows:

For (a) first note $\vdash f(c_1, \dots, c_n) = f(c_1, \dots, c_n)$ by a similar argument to that in Section 2.4.1 showing $\vdash c = c$.

But $\vdash f(c_1, \dots, c_n) = f(c_1, \dots, c_n) \rightarrow \exists v (v = f(c_1, \dots, c_n))$ from Proposition 2.28, existential generalisation.

Also $\Sigma^* \vdash \exists v (v = f(c_1, \dots, c_n)) \rightarrow c = f(c_1, \dots, c_n)$ for some (Henkin witness) $c \in \mathcal{C}$, by Lemma 3.24.

From two applications of modus ponens, $\Sigma^* \vdash c = f(c_1, \dots, c_n)$ for some $c \in \mathcal{C}$.

This establishes (a).

For (b) we have

$$\begin{aligned} c_1 \sim c'_1, \dots, c_n \sim c'_n & \\ \implies \Sigma^* \vdash c_1 = c'_1, \dots, \Sigma^* \vdash c_n = c'_n & \quad \text{using (3.15)} \\ \implies \Sigma^* \vdash c_1 = c'_1 \wedge \dots \wedge c_n = c'_n & \quad \text{basic properties of “} \vdash \text{”} \\ \implies \Sigma^* \vdash f(c_1, \dots, c_n) = f(c'_1, \dots, c'_n) & \quad \text{using equality axiom (2.5)} \\ \implies f(c_1, \dots, c_n) \sim f(c'_1, \dots, c'_n) & \quad \text{using (3.15)} \end{aligned}$$

3. *Relation Symbols in $\mathcal{L}$* : Let $r \in \mathcal{L}$ be an n -ary relation symbol. Define the interpretation $r^{\mathfrak{A}^*} \subset A^n$ of r in $\mathfrak{A}^*$ by

$$([c_1], \dots, [c_n]) \in r^{\mathfrak{A}^*} \iff \Sigma^* \vdash r(c_1, \dots, c_n), \quad \text{def r} \quad (3.21)$$

where $c_1, \dots, c_n \in \mathcal{C}$.

To show this is well defined we need to show the definition of $r^{\mathfrak{A}^*}$ is independent of choice of equivalence class representatives. But this is by essentially the same argument as for a function symbol.

$$\begin{aligned}
c_1 \sim c'_1, \dots, c_n \sim c'_n & \\
\implies \Sigma^* \vdash c_1 = c'_1 \wedge \dots \wedge c_n = c'_n & \quad (3.15), \text{ properties of “} \vdash \text{”} \\
\implies \Sigma^* \vdash r(c_1, \dots, c_n) \leftrightarrow r(c'_1, \dots, c'_n) & \quad \text{using equality axiom (2.6)} \\
\implies \Sigma^* \vdash r(c_1, \dots, c_n) \text{ iff } \Sigma^* \vdash r(c'_1, \dots, c'_n) & \quad \text{properties of “} \vdash \text{”} \\
\implies ([c_1], \dots, [c_n]) \in r^{\mathfrak{A}^*} \text{ iff } ([c'_1], \dots, [c'_n]) \in r^{\mathfrak{A}^*} & \quad \text{by (3.21)}
\end{aligned}$$

3.4.6 STEP 6 PROOF THAT $\mathfrak{A}^* \models \Sigma^*$

Because *every* element in the universe A of $\mathfrak{A}^*$ is the interpretation of a constant symbol $c \in \mathcal{C}$ (in fact many such constant symbols, but all of which are equivalent via “ $\sim$ ”), we will *not* need to consider free variables when interpreting terms in $\mathfrak{A}^*$, *nor* to consider formulas with free variables when defining the truth values of sentences in $\mathfrak{A}^*$.

Definition 3.27. Suppose t is a term in the language $\mathcal{L}^*$ (with no free variables). Then the interpretation $t^{\mathfrak{A}^*} (\in A)$ of t is defined by induction on the complexity of t as follows:

- If t is a constant symbol $c \in \mathcal{C}$ or $d \in \mathcal{L}$, then $c^{\mathfrak{A}^*}$ and $d^{\mathfrak{A}^*}$ have already been defined in (3.18) and (3.19). In particular

$$t^{\mathfrak{A}^*} = [c] \iff \Sigma^* \vdash t = c.$$

- If t is $f(t_1, \dots, t_n)$ then

$$t^{\mathfrak{A}^*} := f^{\mathfrak{A}^*}(t_1^{\mathfrak{A}^*}, \dots, t_n^{\mathfrak{A}^*}).$$

Lemma 3.28. If t is a term in $\mathcal{L}^*$ and $c \in \mathcal{C}$, then

$$t^{\mathfrak{A}^*} = [c] \iff \Sigma^* \vdash t = c. \quad \text{lem t prop (3.22)}$$

Proof. We have already seen this for t a constant symbol.

Let t be $f(t_1, \dots, t_n)$. Suppose $t^{\mathfrak{A}^*} = [c]$.

Assume (the inductive hypothesis) that the result corresponding to (3.22) holds with t replaced by $t_1, \dots, t_n$.

Let $t_i^{\mathfrak{A}^*} = [c_i]$ and so $\Sigma^* \vdash t_i = c_i$, for $1 \leq i \leq n$.

Then

$$\begin{aligned}
t^{\mathfrak{A}^*} = [c] & \iff f^{\mathfrak{A}^*}(t_1^{\mathfrak{A}^*}, \dots, t_n^{\mathfrak{A}^*}) = [c] && \text{Definition 3.27} \\
& \iff f^{\mathfrak{A}^*}([c_1], \dots, [c_n]) = [c] && \text{by assumption for each } t_i^{\mathfrak{A}^*} \\
& \iff \Sigma^* \vdash f(c_1, \dots, c_n) = c && \text{by (3.20)} \\
& \iff \Sigma^* \vdash f(t_1, \dots, t_n) = c && \text{by } \Sigma^* \vdash t_i = c_i \text{ and equality axiom (2.5)} \\
& \iff \Sigma^* \vdash t = c && \text{definition of } t
\end{aligned}$$

This establishes the result. □

Theorem 3.29. For every sentence φ in the language $\mathcal{L}^*$,

$$\mathfrak{A}^* \models \varphi \iff \Sigma^* \vdash \varphi \iff \varphi \in \Sigma^*. \quad \text{thm imp (3.23)}$$

That is,

$$\mathfrak{A}^* \models \Sigma^*. \quad (3.24)$$

Proof. The second $\iff$ in (3.23) is by (3.3) in Corollary 3.13, since Σ^* is maximal consistent.

For the first $\iff$ it will be more convenient to work with sentences built using $\exists$ rather than $\forall$ as a primitive logical symbol. So we consider $\forall$ as a defined symbol via $\forall := \neg\exists\neg$.

The proof of the first $\iff$ will be by induction on the complexity of φ .

Moreover, unlike the situation with the definition and properties of “ $\models$ ” for a general structure, since every element in A corresponds to a constant (in fact many) constant symbols in $\mathcal{C}$, we will be able to work just with sentences.

Atomic Sentences:

(a) $r(t_1, \dots, t_n)$: We claim

$$\mathfrak{A}^* \models r(t_1, \dots, t_n) \iff \Sigma^* \vdash r(t_1, \dots, t_n). \quad (3.25)$$

Since $t_i^{\mathfrak{A}^*} \in A$ for $1 \leq i \leq n$, let $t_i^{\mathfrak{A}^*} = [c_i]$.

Then

$$\begin{aligned} \mathfrak{A}^* \models r(t_1, \dots, t_n) &\iff (t_1^{\mathfrak{A}^*}, \dots, t_n^{\mathfrak{A}^*}) \in r^{\mathfrak{A}^*} && \text{Definition 1.18 (ii) for “}\models\text{”} \\ &\iff ([c_1], \dots, [c_n]) \in r^{\mathfrak{A}^*} \\ &\iff \Sigma^* \vdash r(c_1, \dots, c_n) && \text{definition in (3.21) of } r^{\mathfrak{A}^*} \end{aligned}$$

(b) $t_1 = t_2$: We claim

$$\mathfrak{A}^* \models t_1 = t_2 \iff \Sigma^* \vdash t_1 = t_2 \quad (3.26)$$

Let $t_1^{\mathfrak{A}^*} = [c_1]$, $t_2^{\mathfrak{A}^*} = [c_2]$.

Then $\Sigma^* \vdash t_1 = c_1$ and $\Sigma^* \vdash t_2 = c_2$ by Lemma 3.22.

Hence⁸

$$\begin{aligned} \mathfrak{A}^* \models t_1 = t_2 &\iff t_1^{\mathfrak{A}^*} = t_2^{\mathfrak{A}^*} && \text{Definition 1.18 (i) for } \models \\ &\iff [c_1] = [c_2] \\ &\iff c_1 \sim c_2 && \text{Definition 3.25 of } [\cdot] \text{ from } \sim \\ &\iff \Sigma^* \vdash c_1 = c_2 && \text{definition of } \sim \text{ in (3.15)} \\ &\iff \Sigma^* \vdash t_1 = t_2 && \text{see below} \end{aligned}$$

The last equivalence uses

$$\begin{aligned} &\vdash (c_1 = c_2 \wedge t_1 = c_1 \wedge t_2 = c_2) \rightarrow t_1 = t_2 \\ &\vdash (t_1 = t_2 \wedge t_1 = c_1 \wedge t_2 = c_2) \rightarrow c_1 = c_2 \end{aligned}$$

This, in turn, uses the logical axioms for equality.

⁸Note that the second “=” in the first line of the following display, and the “=” in the second line, are *not* equality symbols in the formal language $\mathcal{L}^*$. They are abbreviations in the metalanguage used to discuss the structure $\mathfrak{A}^*$. They are saying that $t_1^{\mathfrak{A}^*}$ and $t_2^{\mathfrak{A}^*}$ denote the same element in the universe A , and similarly that $c_1^{\mathfrak{A}^*}$ and $c_2^{\mathfrak{A}^*}$ denote the same element in A .

Exercise 3.30. Explain. □

Negation $\neg\varphi$: Assume (3.23) is true for φ . It follows that the analogue is true for $\neg\varphi$:

$$\begin{aligned} \mathfrak{A}^* \models \neg\varphi &\iff \mathfrak{A}^* \not\models \varphi \\ &\iff \Sigma^* \not\vdash \varphi && \text{induction assumption} \\ &\iff \Sigma^* \vdash \neg\varphi && \text{Corollary 3.13, since } \Sigma^* \text{ is maximal consistent} \end{aligned}$$

Conjunction $\varphi \wedge \psi$: Assume (3.23) is true for φ and ψ . It follows that the analogue is true for $\varphi \wedge \psi$:

$$\begin{aligned} \mathfrak{A}^* \models \varphi \wedge \psi &\iff \mathfrak{A}^* \models \varphi \ \& \ \mathfrak{A}^* \models \psi && \text{Definition 1.18 (iv) for } \models \\ &\iff \Sigma^* \vdash \varphi \ \& \ \Sigma^* \vdash \psi && \text{induction assumption} \\ &\iff \Sigma^* \vdash \varphi \wedge \psi && \text{see below} \end{aligned}$$

The last equivalence uses

$$\vdash \varphi \wedge \psi \iff \vdash \varphi \ \& \ \vdash \psi$$

For $\Rightarrow$ use the propositional axioms $\varphi \wedge \psi \rightarrow \varphi$ and $\varphi \wedge \psi \rightarrow \psi$ and modus ponens.

For $\Leftarrow$ concatenate deductions for $\vdash \varphi$ and $\vdash \psi$, follow this by the propositional axiom $\varphi \rightarrow (\psi \rightarrow \varphi \wedge \psi)$ and then two applications of modus ponens.

Existential Quantifier $\exists v \varphi$: Since $\exists v \varphi$ is a *sentence* in the language $\mathcal{L}^*$, it follows that φ has at most v as a free variable. For this reason we here write $\varphi(v)$ for φ , and by $\varphi(c)$ mean the sentence obtained by replacing all free occurrences of v in $\varphi(v)$ by c .

Then since *every* element in the universe of $\mathfrak{A}$ is $[c]$ for some $c \in \mathcal{C}$,

$$\begin{aligned} \mathfrak{A}^* \models \exists v \varphi(v) &\iff \mathfrak{A}^* \models \varphi(c) && \text{for some } c \in \mathcal{C}, \\ &\iff \Sigma^* \vdash \varphi(c) && \text{by induction hypothesis, some } c \in \mathcal{C}, \\ &\iff \Sigma^* \vdash \exists v \varphi(v), \end{aligned}$$

where for “ $\Rightarrow$ ” of the last $\iff$, use the first-order logic result $\vdash \varphi(c) \rightarrow \exists v \varphi(v)$, together with modus ponens; and for “ $\Leftarrow$ ” use the Henkin Witness property (3.10) of $\mathcal{L}^*$, namely $\exists v \varphi(v) \rightarrow \varphi(c) \in \Sigma^*$, together with modus ponens.

This completes the proof of Theorem 3.23 and hence of Step 6. □

3.4.7 STEP 7 DEFINITION OF $\mathfrak{A}$ & PROOF THAT $\mathfrak{A} \models \Sigma$

This is now easy.

We have seen that

$$\mathfrak{A}^* \models \Sigma^*$$

Since $\Sigma \subset \Sigma^*$, it follows that

$$\mathfrak{A}^* \models \Sigma.$$

Since Σ is a set of sentences in the original language $\mathcal{L}$, Σ does not refer to the new constants $c \in \mathcal{C}$ which occur in the extended language $\mathcal{L}^* = \mathcal{L} \cup \mathcal{C}$.

We now *define* the structure $\mathfrak{A}$ to be the same as $\mathfrak{A}^*$, *except* that it is a structure just for the symbols in $\mathcal{L}$ and not for the new symbols in $\mathcal{C}$.

Since nothing else is changed, it is the case that

$$\mathfrak{A} \models \Sigma.$$

More precisely, we could argue by induction on the complexity of formulas φ that $\mathfrak{A} \models \varphi[\sigma]$ iff $\mathfrak{A}^* \models \varphi[\sigma]$ for arbitrary assignments σ .

3.4.8 STEP 8 PROOF THAT $|\mathfrak{A}| \leq \aleph_0$

We assumed that $|\mathcal{L}|$ is countable.

In particular the set of formulas with one free variable is countable, and hence so is the set of new constant symbols $\mathcal{C}$. But the universe of $\mathfrak{A}$ is given by equivalence classes from $\mathcal{C}$ and so is also countable (and possibly finite).

This completes the proof of Gödel's Completeness Theorem 3.17. ■

3.5 WRAP-UP

AN INTERESTING JOURNEY

We began with a consistent set of sentences Σ in the language $\mathcal{L}$.

We then extended $\mathcal{L}$ to a language $\mathcal{L}^* = \mathcal{L} \cup \mathcal{C}$, by adding a countably infinite set $\mathcal{C}$ of new constant symbols. *Step 1*

We next extended Σ to Σ^* , a *consistent* set of sentences in $\mathcal{L}^*$, by consistently adding for *every* sentence in $\mathcal{L}^*$ of the form $\exists v \varphi(v)$, a Henkin sentence $\exists v \varphi(v) \rightarrow \varphi(c)$ for some $c \in \mathcal{C}$ depending on φ . *Step 2*

We then extended Σ^* to a *maximal* consistent set of sentences, which we also denoted by Σ^* . *Step 3*

We then defined an equivalence relation on $\mathcal{C}$ by $c \sim c'$ iff $(c = c') \in \Sigma^*$, or equivalently by maximality, $\Sigma^* \vdash c = c'$. *Step 4*

We next used the equality axioms and the Henkin sentences to show that $\mathcal{C}/\sim$ was closed under the interpretation of all constant and function symbols in Σ^* , and that these and the relation symbols were well-defined on $\mathcal{C}/\sim$. This provided a natural structure $\mathfrak{A}^*$ for $\mathcal{L}^*$ in which “=” was interpreted by “the same as” rather than as just an equivalence relation. *Step 5*

We next showed that $\mathfrak{A}^* \models \Sigma^*$ by again using the Henkin sentences in Σ^* and the maximal consistency of Σ^* . *Step 6*

By *ignoring* the interpretation of the symbols in $\mathcal{C}$ we obtained the desired structure $\mathfrak{A}$ such that $\mathfrak{A} \models \Sigma$. *Step 7*

Since the cardinality of $\mathcal{C}$ is $\aleph_0$, the cardinality of $\mathcal{C}/\sim$ is $\leq \aleph_0$, but $\mathcal{C}/\sim$ is just the universe of $\mathfrak{A}$. *Step 8*

ALTERNATIVE APPROACHES

The approach in most texts is to iterate the production of Henkin Constants a countable number of times, take unions of the extended languages and extended sets of sentences thus obtained, and finally to iterate this iteration process itself a countable number of times. See (Johnstone, 1987; Leader, 2012; Zsák, 2025), and to a lesser extent

(Enderton, 2001; Hils and Loeser, 2019; Leary and Kristiansen, 2015).

For the approach here see (Chang and Keisler, 2012) and comments in (Henkin, 1996, Observation C pp 156–157).^a

^a(Henkin, 1996) “The Discovery of My Completeness Proof” has an interesting historical discussion, comments on the nature of mathematical discovery, and on Henkin’s own indirect route to his proof of the Completeness Theorem. Observation C on pages 156,157 discusses Henkin’s improvements in his proof as a result of his teaching the material. This is the essentially the proof used here in the Notes.

Chapter 4

Basics: Set Theory

In this chapter we do just enough on the Axiom of Choice, Ordinals and Cardinals to extend the Completeness Theorem to languages which may be uncountable. ch bst

The chapter is also an introduction to the later chapters on the Axiom of Choice 12, Ordinals 13, and Cardinals 14.

4.1 AXIOM OF CHOICE

The Axiom of Choice appears to be quite innocuous. But as we see in Theorem 4.29 and again in Chapter 12 it has some rather surprising consequences. sec acc

Nonetheless it is assumed in most of mathematics, although attention may be called to the fact when it is being used in a proof.

See also the quotes at the beginning of Chapter 12.

Definition 4.1 (Axiom of Choice). Suppose the sets A_i are all nonempty for $i \in I$. Then there exists a function f such that

$$\text{for all } i \in I, \quad f(i) \in A_i. \quad (4.1)$$

Equivalently, the product set

$$\prod_{i \in I} A_i := \left\{ f \mid f : I \rightarrow \bigcup_{i \in I} A_i, f(i) \in A_i \text{ if } i \in I \right\} \quad (4.2)$$

is nonempty.

If I is finite, say $I = \{1, \dots, n\}$, then up to notational changes this is just stating

$$A_1 \times \dots \times A_n \neq \emptyset. \quad (4.3)$$

This does *not* require the Axiom of Choice. It essentially follows from the much more basic and concrete Pairing Axiom applied the appropriate number of times, see Chapter 15.

4.2 A LITTLE ON ORDINALS & WELL-ORDERINGS

sec low

4.2.1 INTRODUCTION

Here I discuss the ideas behind ordinals in an informal manner. That is about all you will need at this stage.

Ordinals were introduced by Cantor in 1883 in his work on trigonometric series. See also [Aside/Cantor 13.3](#).

4.2.2 INTEGERS AND INDUCTION

Denote by ω the sequence of integers $\{0, 1, 2, \dots\}$ together with the ordering $0 < 1 < 2 < 3 < \dots$.

When we prove by induction that a property P holds for all integers, or define a function f over the integers by induction, the essential fact is that:

Every nonempty subset of ω contains a least element.

For example, suppose $P(n)$ states that n has property P . If we know $P(0)$ is true and that $P(n)$ is true whenever $P(n-1)$ is true, then we know by induction that $P(n)$ is true for all n . *For if not*, consider the least n such that $P(n)$ is false and thereby obtain a contradiction.

4.2.3 WELL-ORDERINGS AND SOME EARLY ORDINALS

We can use the following definition to extend the ordering on ω in a very convenient manner.

Definition 4.2. A (nonempty) linear ordering $\langle L, < \rangle$ is a *well-ordering* if every nonempty subset of L contains a least element. $\square$

By an informal argument (or formally by induction, see [Theorem 4.20](#) or [Theorem 13.17](#)) any *infinite* well-order has an isomorphic copy of ω as an initial segment.

Exercise 4.3. Give an informal argument. $\square$

The next smallest well-ordering after ω is isomorphic to the well-order on $\{0, 1, 2, \dots, \omega\}$ with ordering $0 < 1 < 2 < \dots < \omega$. We write this well-ordering as $\omega + 1$.

Exercise 4.4. Why is there a “next smallest”? $\square$

Remark 4.5 (Deliberate Notational Ambiguity). We denoted the set of natural numbers together with its natural ordering by ω , and also used ω to denote the next element in the “transfinite” sequence $0, 1, 2, 3, \dots, \omega$.

We similarly wrote $\omega + 1$ for both the set $0, 1, 2, 3, \dots, \omega$ with its ordering, and for the next element in the transfinite sequence after this set. $\square$

Remark 4.6 (The Transfinite Sequence of Ordinals). After $\omega + 1$, $\omega + 2$ is next, $\omega + 3$ is next, $\dots$. And after all these there is again a unique next, $\omega + \omega$ (written $\omega \cdot 2$, which we read as 2 copies of ω). Then $\omega \cdot 2 + 1$, $\omega \cdot 2 + 2$, $\dots$, and after all these $\omega \cdot 2 + \omega$ (written $\omega \cdot 3$).

Corollary 4.13. *If $\langle L, < \rangle$ is a well-ordering and $f : L \rightarrow L$ is an automorphism, then f is the identity function.* cor wo auto

Proof. Since both f and f^{-1} are order-preserving, from Lemma 4.11, $x \leq f(x)$ and $x \leq f^{-1}(x)$. From the second inequality and since f is order-preserving, $f(x) \leq x$. $\square$

Exercise 4.14. Give counter examples if $\langle L, < \rangle = \langle \mathbb{Z}, < \rangle$ and if $\langle L, < \rangle = \langle \mathbb{Q}^+, < \rangle$. $\square$

Corollary 4.15. *If $\langle L, < \rangle$ and $\langle L', <' \rangle$ are isomorphic well-orderings then the isomorphism is unique.*

Proof. Let $f, \tilde{f} : L \rightarrow L'$ be two such isomorphisms. Then $\tilde{f}^{-1} \circ f : L \rightarrow L$ is an automorphism. Hence $\tilde{f}^{-1} \circ f = \text{id}_L : L \rightarrow L$ by Corollary 4.13, where id_L is the identity automorphism. Applying $\tilde{f}$ to both sides of the equality gives $f = \tilde{f}$. $\square$

Exercise 4.16. Give a counter example if both $\langle L, < \rangle$ and $\langle L', <' \rangle$ are $\langle \mathbb{Z}, < \rangle$, and if both are $\langle \mathbb{Q}^+, < \rangle$. $\square$

Definition 4.17. A (proper) *initial segment* of a well-ordering $\langle L, < \rangle$ is a subset of L of the form $L_v = \{u : u < v\}$ for some $v \in L$.

Exercise 4.18. Show that $I \subseteq L$ is an initial segment iff $I \neq L$ and

$$\forall x \forall y ((x < y \ \& \ y \in I) \rightarrow x \in I).$$

Show that the two possible definitions can disagree if $\langle L, < \rangle$ is a linear ordering but not a well-ordering.

Lemma 4.19. *No well-ordering is isomorphic to an initial segment of itself.* lem not isom

Proof. Suppose the well-ordering $\langle L, < \rangle$ is isomorphic to the initial segment $\{u : u < v\}$ for some $v \in L$ via the isomorphism $f : L \rightarrow \{u : u < v\}$.

Then $f(v) < v$, contradicting Lemma 4.11. $\square$

Theorem 4.20. *If $\langle L, < \rangle$ and $\langle L', <' \rangle$ are two well-orderings, then exactly one of the following three cases holds* thm comp wo

1. $\langle L, < \rangle$ is isomorphic to $\langle L', <' \rangle$,
2. $\langle L, < \rangle$ is isomorphic to an initial segment of $\langle L', <' \rangle$,
3. $\langle L', <' \rangle$ is isomorphic to an initial segment of $\langle L, < \rangle$.

Proof. Let

$$f = \{(u, v) : u \in L, v \in L', L_u \text{ is isomorphic to } L'_v\}.$$

If L_{u_1} and L_{u_2} are both isomorphic to L'_{v_1} then they are isomorphic to each other, and so $u_1 = u_2$ from Lemma 4.19. Similarly if L'_{v_1} and L'_{v_2} are both isomorphic to L_{u_1} . So f is a one-one order-preserving function, and similarly so is f^{-1} .

If $\text{dom } f = L$ and $\text{ran } f = L'$ then the first case holds.

Suppose the first case does not hold. If both $\text{dom } f = L_u \subsetneq L$ and $\text{ran } f = L'_v \subsetneq L'$ then we could extend f by adding the pair (u, v) to f .

So either $\text{dom } f = L$ and the second case holds, or $\text{ran } f' = L'$ and the third case holds. $\square$

4.2.5 ORDINALS

One can identify each ordinal with the well-ordering on its set of predecessors.

This leads to the following definition of an ordinal, due to von Neumann. A nice treatment is in (Jech, 2003, Chapter 2). We consider ordinals in more detail in Chapter 13.

However, we do not need, in this chapter, to work closely with the following formal definition.

Definition 4.21 (Ordinals). The set α is an *ordinal* if α is a transitive set³ and $\in$ is a well-ordering of α .

So for ordinals, the well-ordering $<$ is just the membership relation $\in$, and every ordinal is the set of previous ordinals. For any two ordinals α and β ,

$$\alpha < \beta \iff \alpha \in \beta \iff \alpha \subsetneq \beta. \quad (4.5)$$

Exercise 4.22. Explain what this means for $\alpha, \beta \in \{1, 2, 3, \omega, \omega + 1, \omega + 3\}$. $\square$

Definition 4.23 (Types of Ordinals). Every ordinal α is one of the three following types:

- the *initial ordinal*, $\alpha = 0$,
- a *successor ordinal*, $\alpha = \beta + 1$ for some ordinal β ,
- a *limit ordinal* if α is neither of the above. $\square$

In (4.4) the explicitly displayed *limit* ordinals are

$$\omega, \omega \cdot 2, \omega \cdot 3, \omega \cdot 4, \omega^2, \omega^2 + \omega, \omega^2 + \omega \cdot 2, \omega^3, \omega^n, \omega^\omega, \omega^{\omega^\omega}, \omega^{\omega^{\omega^\omega}}, \omega^{\omega^{\omega^{\omega^\omega}}}, \epsilon_0.$$

The *class of all ordinals*, usually denoted On , is ordered as in its construction, that is

$$0 < 1 < 2 < 3 < \dots < \omega < \omega + 1 < \omega + 2 < \dots < \omega \cdot 2 < \dots < \omega^2 < \dots,$$

etc.

But this transfinite list does not even “scratch the surface” of the class On of *all* ordinals.

rem class ord

Remark 4.24 (Class of all Ordinals). This collection/class On of all ordinals is too large to be a set.

Informally, if there were a *set* of all ordinals then that set would itself be an ordinal, and we could get an even larger ordinal by taking its successor. But that gives a contradiction, since an ordinal cannot be isomorphic to a proper initial segment of itself.

This is the *Burali-Forti Paradox*. Informally, like the Russell paradox, it shows that a collection described by a property may lead to a collection too large to be a set. $\square$

³A set x is *transitive* if $\forall z \forall y (z \in y \in x \implies z \in x)$.

4.2.6 ORDINALS AND TRANSFINITE INDUCTION

Just as we have induction over the set ω of integers (with its standard ordering), more generally we have transfinite induction over any ordinal α , or even over the class On of all ordinals (with the ordering previously discussed).

Theorem 4.25 (Transfinite Induction). *Let $P(\beta)$ be a statement about ordinals β and let α be a fixed ordinal.*

Suppose for all $\beta < \alpha$ (for all $\beta \in On$),

$$((\forall \gamma < \beta) P(\gamma)) \implies P(\beta). \quad \text{hyp tfi (4.6)}$$

Then $P(\beta)$ for all $\beta < \alpha$ (for all $\beta \in On$).

Proof. If not, consider the least β such that $P(\beta)$ is false, which contradicts (4.6). $\square$

Remark 4.26. Note that (4.6) in case $\beta = 0$ is just the assertion $P(0)$. $\square$

Remark 4.27 (Applications). In applications it is common to split the proof of the hypothesis (4.6) into three cases:

1. Prove $P(0)$.
2. Prove that if $\beta = \gamma + 1$ and $P(\gamma)$ is true, then $P(\beta)$ is true.
3. Prove that if β is a limit ordinal and $P(\gamma)$ is true for every $\gamma < \beta$, then $P(\beta)$ is true. $\square$

There is a natural extension of the notion of a sequence, a function whose domain is ω , to a transfinite sequence, a function whose domain is an ordinal.

Definition 4.28 (Transfinite Sequence). A *transfinite sequence* is a function f whose domain is either an ordinal α or the class On of all ordinals.

A transfinite sequence f is usually denoted by $\langle a_\xi : \xi < \alpha \rangle$ or by $\langle a_\xi : \xi \in On \rangle$, where $f(\xi) = a_\xi$.

4.2.7 WELL-ORDERING PRINCIPLE EQUIVALENT TO AXIOM OF CHOICE

We first assume the Axiom of Choice and prove the Well-Ordering Principle.

Theorem 4.29 (Well-Ordering Principle). *Every set X can be well-ordered.* thm wop

Proof. Let f be a choice function for the nonempty subsets of X . That is, for every nonempty set $A \subseteq X$, $f(A) \in A$.

Define a transfinite sequence by

$$a_0 = f(X), \quad a_\beta = f(X \setminus \{a_\xi \mid \xi < \beta\}),$$

if $X \setminus \{a_\xi \mid \xi < \beta\}$ is nonempty.

Let θ be the first (and only by the construction) ordinal such that $\{a_\xi : \xi < \theta\} = X$.

There is such a θ , since otherwise there would be a one-one mapping from the class On of all ordinals into the set X . But this would imply On is a set by the standard axioms of set theory.⁴ And we know this is not the case.

The required well-ordering is given by

$$a_\xi < a_\beta \iff \xi < \beta,$$

where the ordering on the right side is the ordering for ordinals, with both $\xi, \beta < \theta$. $\square$

Remark 4.30. This is rather strange. What does a well-ordering of $\mathbb{R}$ look like?

The Axiom of Choice looks innocuous, but the Well-Ordering Principle does not.

In fact the two are equivalent, as the following theorem shows. $\square$

We next assume the Well-Ordering Principle and prove the Axiom of Choice.

Theorem 4.31. *Suppose the sets A_i are all nonempty for $i \in I$. Then there exists a function f such that*

$$\text{for all } i \in I, \quad f(i) \in A_i. \quad (4.7)$$

Proof. Let $A = \bigcup_{i \in I} A_i$. By the well-ordering principle there is a transfinite sequence $\langle a_\beta : \beta < \theta \rangle$ enumerating A .

For $i \in I$ define

$$f(i) = a_\beta, \quad \beta \text{ is the least ordinal such that } a_\beta \in A_i.$$

This is a suitable choice function. $\square$

4.3 A LITTLE ON CARDINALS

In this section we again assume the Axiom of Choice, essentially because of Theorem 4.29.

4.3.1 CARDINALITY

Definition 4.32 (Cardinality). Two sets A and B have the same *cardinality* or *size*, and one writes

$$|A| = |B|, \quad (4.8)$$

if there is a bijection between A and B .

This defines an equivalence relation on sets.

Exercise 4.33. Prove this is an equivalence relation. $\square$

The next Theorem shows we can get larger and larger cardinalities, in the natural ordering given by Definition 4.35.

⁴We have not yet discussed the axioms of set theory. But if you do look at the axioms, here is the argument.

If there is a one-one mapping $\xi \mapsto a_\xi$ from On onto $\tilde{X} \subseteq X$, then the inverse mapping $a_\xi \mapsto \xi$ from $\tilde{X}$ (which is a set by the Separation Axiom) implies On is a set (by the Replacement Axiom). But On is not a set by the Burali-Forti “paradox”.

Theorem 4.34 (Cantor). *For every set X there is no surjective map from X to $\mathcal{P}(X)$. In particular, there is no bijection from X to $\mathcal{P}(X)$, and so $|X| \neq |\mathcal{P}(X)|$. But there is clearly an injection, namely $x \in X \mapsto \{x\} \in \mathcal{P}(X)$.* thm Cant

Proof. Assume (in order to obtain a contradiction) that $f : X \rightarrow \mathcal{P}(X)$ is surjective.

Let $A = \{x \in X : x \notin f(x)\} (\subseteq X)$. Since f is surjective, $A = f(a)$ for some $a \in X$.

So by the definition of A ,

$$\begin{aligned} a \in A &\implies a \notin f(a) \implies a \notin A, \\ a \notin A &\implies a \in f(a) \implies a \in A, \end{aligned}$$

giving a contradiction either way. □

4.3.2 COMPARING CARDINALS

There is a natural ordering on cardinalities, defined as follows.

Definition 4.35 (Cardinality Ordering). The *ordering* on cardinalities is defined by def card ord

$$|A| \leq |B| \tag{4.9}^t$$

if there is a one-to-one map from A into B . The *strict ordering* is defined by

$$|A| < |B| \tag{4.10}$$

if $|A| \leq |B|$ and $|A| \neq |B|$.

It is straightforward to check that $\leq$ is *transitive*. That is

$$|A| \leq |B| \ \& \ |B| \leq |C| \implies |A| \leq |C|. \tag{4.11}^{\text{card trans}}$$

Exercise 4.36. Prove (4.11). □

Remark 4.37 (Are sets always comparable in size?). At this stage we do not know if, for arbitrary sets A and B , at least one of the following is true:

$$|A| \leq |B| \text{ or } |B| \leq |A|. \tag{4.12}$$

This is in fact the case, but only if we assume the Axiom of Choice!

In fact, as we will see and assuming the Axiom of Choice, for any two sets A and B , exactly one of the following is true:

$$|A| < |B| \text{ or } |B| < |A| \text{ or } |A| = |B|. \tag{4.13}$$

Remark 4.38. From Theorem 4.34, $|A| < |\mathcal{P}(A)|$ for every set A .

By successively taking the power set operation we can then obtain larger and larger cardinalities. □

The following Cantor–Bernstein Theorem 4.40 is elementary but not obvious. It shows the ordering on cardinalities is a partial ordering. In fact it is a linear ordering, and even more it is a well-ordering. See Remark 4.45.

Remark 4.39 (Motivation). In the following proof, starting from (4.14), you might find it helpful to think of Z^0 as the set of generation zero elements, and Z^n as the set of generation n elements, beginning from Z^0 and moving forward by iterating the function g .

The “picture” is that h acts on $C = \bigcup_{n \geq 0} Z^n$ by pushing Z^n forward bijectively by one generation to Z^{n+1} for each $n \geq 0$, so $h : C \rightarrow C \setminus Z^0$ is a bijection.

Moreover $h : B \setminus C \rightarrow B \setminus C$ is the identity bijection.

So $h : B \rightarrow B \setminus Z^0$, that is $h : B \rightarrow A$, is a bijection. $\square$

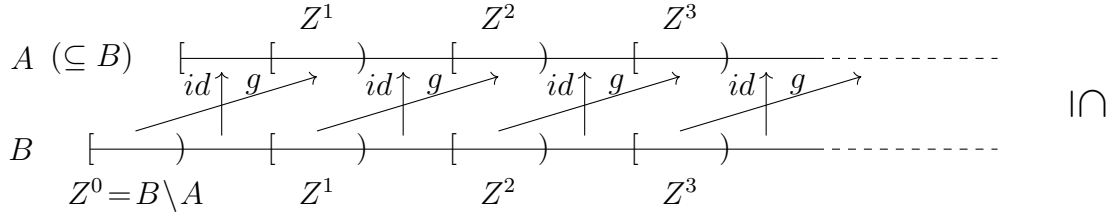


Figure 4.1: The disjoint sets Z^n in the proof of the Cantor–Bernstein Theorem, g mapping each generation to the next. $C := \bigcup_{n \geq 0} Z^n \subseteq B$. Define $h(x) = g(x)$ if $x \in C$, $h(x) = x$ if $x \in B \setminus C$.

Theorem 4.40 (Cantor–Bernstein Theorem). If $|A| \leq |B|$ and $|B| \leq |A|$ then $|A| = |B|$. thm cb

Proof. We use $\hookrightarrow$ for injective maps.

Suppose $f : A \hookrightarrow B$ and $g : B \hookrightarrow A$. We want to show $|A| = |B|$.

It follows from the data that $f(A) \subseteq B$ and that $f \circ g : B \hookrightarrow f(A)$ is an injective map. So it is sufficient and equivalent to show $|f(A)| = |B|$.

Changing notation, it is sufficient (and equivalent) to prove that

$$A \subseteq B \quad \& \quad g : B \hookrightarrow A \quad \implies \quad |A| = |B|. \quad \text{diff cb (4.14)}$$

Assume that $A \subseteq B$ and $g : B \hookrightarrow A$. We want a bijective map $h : B \rightarrow A$.

For this purpose (see Figure 4.1) define

$$Z^0 := B \setminus A, \quad Z^n := \{g^n(x) : x \in Z^0\} \text{ if } n \geq 1, \quad C := \bigcup_{n \geq 0} Z^n \subset B. \quad (4.15)$$

Then the sets Z^n are mutually disjoint and form a partition of C .

To see this first observe that Z^0 is disjoint from Z^n for $n \geq 1$ since the former is disjoint from $\text{ran}(g)$ and the latter is a subset of $\text{ran}(g)$.

If $g^n(x) = g^{n+k}(y)$ for $n \geq 1$, $k \geq 0$ and $x, y \in Z_0$, then $x = g^k(y)$ because g is injective implies g^n is injective. If $k \geq 1$ this contradicts the fact Z^0 is disjoint from Z^k . Hence $k = 0$ and so $x = y$.

Hence the Z^n are mutually disjoint for $n \geq 0$ and form a partition of C .

It follows that $g : Z^n \hookrightarrow Z^{n+1}$ is bijective for $n \geq 0$.

Hence $g : C \hookrightarrow \bigcup_{n \geq 1} Z^n = C \setminus Z^0$ is also bijective.

Define $h : B \rightarrow A$ by

$$h(x) = \begin{cases} g(x) & x \in C \\ id(x) (=x) & x \in B \setminus C \end{cases} \quad (4.16)$$

Then h is a bijection. This establishes (4.14) and hence the theorem. $\square$

4.3.3 CARDINALS AS ORDINALS

So far we have not actually defined what is meant by a cardinal number, but only just what is meant by the associated equivalence relation.

It is very convenient to have a canonical representative for each cardinality equivalence class. And this is how we will define cardinal numbers.

Using our results on ordinals we can do this.

Definition 4.41 (Cardinal Number). The *cardinality* $|A|$ of a set A is the least ordinal for which there is a bijection between $|A|$ and A . $|A|$ is also called the *cardinal number* for A . $\square$

Example 4.42. For any countably infinite set A , $|A| = \omega$. However, it is standard to denote this cardinal by $\aleph_0$. $\square$

Exercise 4.43. Show that every ordinal α in (4.4) is countable, and so $|\alpha| = \aleph_0$. ex many count $\square$

Remark 4.44 (Sets have many well-orderings). Since every set A can be well-ordered there is always an ordinal which is in one-one correspondence with A . In fact there are many such ordinals if A is non-finite. See Example 4.43. $\square$

Remark 4.45 (Cardinals are well-ordered). Since the cardinals are a subclass of the ordinals card wo they are *well-ordered*, and in particular the cardinals are *linearly ordered*. $\square$

Remark 4.46 (Indexing cardinals by ordinals). There are cardinals larger than any given ordinal (just take the cardinality of the power set of the ordinal).

So the cardinals are a subclass of the ordinals, too many to be a set. They are themselves indexed by the ordinals, and so form a transfinite sequence indexed by the ordinals and usually written

$$\aleph_0, \aleph_1, \aleph_2, \dots, \aleph_\omega, \aleph_{\omega+1}, \dots, \aleph_\alpha, \dots \quad (\alpha \in \text{On}). \quad (4.17)$$

$\square$

Chapter 5

Extended Completeness and Applications

ch conseq comp

5.1 EXTENDED COMPLETENESS THEOREM

In the proof of Gödel's Completeness Theorem 3.17 for a countable language $\mathcal{L}$ and a countable set $\mathcal{C}$ of new constant symbols, we listed in a sequence all formulas of $\mathcal{L}^* = \mathcal{L} \cup \mathcal{C}$ with one free variable:

$$\varphi_1, \varphi_2, \dots, \varphi_n, \dots \quad \text{def hw3 (5.1)}$$

Beginning with a consistent set of sentences Σ we sequentially and consistently added sentences $\exists v \varphi_n(v) \rightarrow \varphi_n(c)$, where v was the free variable in φ_n ($\varphi_n(v) := \varphi_n$) and $c \in \mathcal{C}$ did not appear so far in the sequence or in φ_n .

If the language $\mathcal{L}$ has infinite cardinality κ then the new set of constant symbols $\mathcal{C}$ will be chosen to also have cardinality κ . The sequence (5.1) is then replaced by a *transfinite* sequence:

$$\varphi_1, \varphi_2, \dots, \varphi_n, \dots, \varphi_\omega, \dots, \varphi_\alpha \dots \quad (\alpha < \kappa). \quad \text{def hw4 (5.2)}$$

We can still consistently add sentences $\exists v \varphi_\alpha(v) \rightarrow \varphi_\alpha(c)$, but this time it is because less than κ constant symbols have been added at each stage. And this time we need to use transfinite induction to define the members of the transfinite sequence (5.2).

The general result is the following.

Theorem 5.1 (Completeness Theorem, General Version). *Suppose Σ is a consistent set of sentences in the language $\mathcal{L}$ with cardinality $|\mathcal{L}| = \kappa$.¹*

Then Σ has a model $\mathfrak{A}$ with cardinality $|\mathfrak{A}| \leq \kappa$, where $|\mathfrak{A}|$ is the cardinality of the universe of $\mathfrak{A}$.

The proof of the Completeness Theorem in this more general setting is analogous to the proof in the countable case.

The difference is that instead of a sequence of new constants $\mathcal{C} = \{c_n : n < \omega\}$, one takes a transfinite sequence of new constants $\mathcal{C} = \{c_\alpha : \alpha < \kappa\}$. Instead of each proper initial segment of $\{n : n < \omega\}$ having (finite) cardinality $< \aleph_0$, each proper initial segment of $\{\alpha : \alpha < \kappa\}$ will have cardinality $< \kappa$, by a standard property of ordinals and cardinals.

The treatment in (Chang and Keisler, 2012, pp 61–66) is essentially the same as here.

¹Recall the definition of the cardinality $|\mathcal{L}|$ of $\mathcal{L}$ in (1.8).

5.1.1 STEP 1 ADDING A SET $\mathcal{C}$ OF NEW CONSTANT SYMBOLS TO $\mathcal{L}$

Introduce a language $\mathcal{L}^* = \mathcal{L} \cup C$ which extends $\mathcal{L}$, by adding a *cardinality* κ set $C = \{c_\alpha : \alpha < \kappa\}$ of new constant symbols not in $\mathcal{L}$.

5.1.2 STEP 2 EXTEND Σ TO Σ^* WITH $\mathcal{C}$ A SET OF HENKIN WITNESSES

add Henk wit2

This key result is as before, but using transfinite sequences rather than countably infinite sequences, in its proof.

lem HW2

Lemma 5.2 (Henkin Witnesses). *There is a consistent extension Σ^* of Σ in the extended language $\mathcal{L}^*$, with the following property:*

For every formula $\varphi = \varphi(v)$ of $\mathcal{L}^$ (not just of $\mathcal{L}$) with at most one free variable depending on φ (for example v), there is a constant $c \in C$ (c depending on φ) such that*

$$\exists v \varphi(v) \rightarrow \varphi(c) \in \Sigma^*. \quad \text{eq hw2 (5.3)}$$

Proof. List all formulas of $\mathcal{L}^*$ (not just formulas of $\mathcal{L}$) with one free variable, as a transfinite sequence indexed by the ordinals $\alpha < \kappa$:

$$\langle \varphi_\alpha : \alpha < \kappa \rangle. \quad \text{def hw5 (5.4)}$$

Define a transfinite sequence of consistent extensions of Σ , in the language $\mathcal{L}^*$:

$$\Sigma_\alpha^* = \begin{cases} \Sigma & \alpha = 0 \\ \Sigma_{\alpha-1}^* \cup \{\exists v \varphi_\alpha(v) \rightarrow \varphi_\alpha(c)\} & \alpha \text{ a successor ordinal} \\ \bigcup_{\xi < \alpha} \Sigma_\xi^* & \alpha \text{ a limit ordinal,} \end{cases} \quad (5.5)$$

where (depending on α), v is the free variable appearing in φ_α and c is the first constant symbol in C not already appearing in any Σ_η^* for $\eta < \alpha$. This is possible since less than κ symbols from C have been added prior to stage α in the process.

We claim that Σ_α^* is consistent for all α . The argument is as before for $\alpha = 0$ or α a successor ordinal (this was the tricky case).

For α a limit ordinal the result is immediate, since any finite subset of Σ_α^* was constructed at a previous stage, and so is consistent. $\square$

5.1.3 STEP 3 ENLARGE Σ^* TO A COMPLETE THEORY

Previously we used Lindenbaum's Theorem 3.15 to obtain a complete consistent extension in the case of a countable language.

The argument is essentially the same here except that instead of the countable sequence of extensions in (3.6) we have a transfinite sequence of length κ .

Exercise 5.3. State and prove Lindenbaum's theorem for a language $\mathcal{L}$ of cardinality κ . $\square$

We will use the same notation Σ^ to denote this complete extension.*

5.1.4 STEP 4 AN EQUIVALENCE RELATION ON $\mathcal{C}$

Define

$$c \sim c' \iff \Sigma^* \vdash c = c' \quad \text{sim props2 (5.6)}$$

as before. Similarly, “ $\sim$ ” is an equivalence relation.

5.1.5 STEP 5 CONSTRUCTING THE UNIVERSE $\mathcal{C}/\sim$ FOR A STRUCTURE $\mathfrak{A}^*$

The arguments are as before.

Every constant symbol in the original language $\mathcal{L}$, as well as each new symbol in $\mathcal{C}$, is interpreted as an appropriate equivalence class. Moreover, the function and constant symbols in the original language $\mathcal{L}$ can also be interpreted in a natural manner.

5.1.6 STEP 6 PROOF THAT $\mathfrak{A}^* \models \Sigma^*$

This is unchanged

5.1.7 STEP 7 DEFINITION OF $\mathfrak{A}$ & PROOF THAT $\mathfrak{A} \models \Sigma$

As before.

5.1.8 STEP 8 PROOF THAT $|\mathfrak{A}| \leq \kappa$

Essentially as before.

5.2 COMPACTNESS

5.2.1 COMPACTNESS THEOREM

Theorem 5.4 (Compactness). *A set of sentences Σ has a model if and only if every finite subset has a model.*

Proof. If Σ has a model, then this model is also a model of every finite subset of Σ .

Conversely, if every finite subset of Σ has a model, then by the Soundness Theorem (version in Corollary 3.6) every finite subset of Σ is consistent. Hence Σ is consistent, and so Σ has a model by the Completeness Theorem. $\square$

Exercise 5.5. Why “Hence Σ is consistent”? $\square$

Remark 5.6 (Semantic Proof of Compactness Theorem). It is perhaps surprising that this theorem, which says nothing about derivations (syntax) and is only concerned the models (semantics), uses the Completeness Theorem in its proof.

There is, however, a proof which combines the (usually different) models of each finite subset of Σ to obtain a model of Σ . This is called the *ultraproduct construction*, and it uses the notion of an *ultrafilter*.

See (Marker, 2015, pp 36–41) or (Chang and Keisler, 2012, Chapter 4), for example. $\square$

5.2.2 AN APPLICATION OF COMPACTNESS

Theorem 5.7. *If a set of sentences Σ has finite models of arbitrarily large finite cardinality, then Σ has an infinite model.*

Proof. Let

$$\Sigma^* = \Sigma \cup \left\{ \exists x_1 \dots \exists x_n \bigwedge_{1 \leq i < j \leq n} x_i \neq x_j \mid n \geq 2 \right\}. \quad (5.7)$$

Every finite subset of Σ^* has a model and so Σ^* has a model by compactness.

But this model is infinite and is a model of Σ . $\square$

Exercise 5.8. Why does this prove the theorem?

5.3 LOWENHEIM–SKOLEM THEOREMS

Theorem 5.9 (Downward Lowenheim–Skolem–Tarski Theorem). *If a set of sentences Σ in the language $\mathcal{L}$ has a model then it has a model of cardinality at most $|\mathcal{L}|$.*

Proof. Since Σ has a model it is consistent. The Completeness Theorem 5.1 then gives a model of cardinality at most $|\mathcal{L}|$. $\square$

Theorem 5.10 (Upward Lowenheim–Skolem–Tarski Theorem). *If a set of sentences Σ in the language $\mathcal{L}$ has an infinite model then it has models of every cardinality $\kappa \geq |\mathcal{L}|$.*

Proof. Let $\mathcal{L}^*$ be the expanded language

$$\mathcal{L}^* = \mathcal{L} \cup \{c_j : j \in K\}, \quad (5.8)$$

where the c_j are new and distinct constant symbols, and the index set K has cardinality κ . Note that $|\mathcal{L}^*| = \kappa$.

Define

$$\Sigma^* = \Sigma \cup \{c_j \neq c_k : j \neq k\}. \quad \text{mod card-1} \quad (5.9)$$

If F is any finite subset of $\{c_j \neq c_k : j \neq k\}$ then $\Sigma \cup F$ has a model and so is consistent. It follows that Σ^* is consistent.

By the Completeness Theorem, Σ^* has a model $\mathfrak{A}$ of cardinality at most κ , and by (5.9) the cardinality is at least κ .

Hence the cardinality of $\mathfrak{A}$ is precisely κ . By ignoring the interpretations of the new symbols c_k we obtain a model $\mathfrak{A}$ of Σ in the language $\mathcal{L}$ and which has cardinality κ . $\square$

Exercise 5.11. Justify: “then $\Sigma \cup F$ has a model and so is consistent. It follows that Σ^* is consistent.”

[This chapter to be completed]

ch rbcctmp

Chapter 6

Basics: Peano Axioms; Primitive Recursion; Representability

6.1 THE THREE TOPICS

In this chapter we look at the basics for three topics to the extent needed for the proof of Gödel's Incompleteness Theorem in Chapter 7.

The first topic concerns the Peano Axioms and their nonstandard models. This is very interesting in its own right, see (Kaye, 1991). For us it provides a framework to understand the nonstandard models of PA and the nonstandard models of all true sentences in $\mathfrak{N}$ other than $\mathfrak{N}$ itself!

Nonstandard models also help us understand, and motivate, the proof and constructions used in the Incompleteness Theorem

The second topic provides background information about computability for the proof of Gödel's Incompleteness Theorems. It is also background material for the Computability Chapter 8.

The third topic will enable us to code up the notions of proof and provability as a result of representing them by formulas in the language of arithmetic.

6.2 PEANO AXIOMS AND THEIR MODELS

6.2.1 THE PEANO AXIOMS [PA]

We take as the language of arithmetic, $\mathcal{L} = \{S, +, \cdot, 0\}$.

The intended interpretation is that S is the successor function, $+$ is addition, $\cdot$ is multiplication, and 0 is the number zero.

We denote by $\mathfrak{N} = \langle \mathbb{N}, S, +, \cdot, 0 \rangle$ the standard model of arithmetic. So we are abusing notation by using $S, +, \cdot, 0$ as symbols in the language $\mathcal{L}$, and also as functions and as a specific element in $\mathfrak{N}$.

Think of $\mathfrak{N}$ in the same informal manner as you may think of a group or a field or the real number system, or any other mathematical structure. More formally, you might think of $\mathfrak{N}$ as a set defined within ZFC set theory.

In the following, $\bar{y}$ is an abbreviation for $y_1, \dots, y_n$, and so $\varphi(x, \bar{y})$ is a formula with free variables in the set $\{x, y_1, \dots, y_n\}$.

As usual, axioms are to be interpreted as abbreviations for the sentences obtained by taking their universal closure. See Remark 2.17.

def PA

Definition 6.1 (Peano Axioms). The axioms for arithmetic are

1. $S(x) \neq 0$.
2. $x \neq 0 \rightarrow \exists y (x = S(y))$.
3. $x + 0 = x$.
4. $x + S(y) = S(x + y)$.
5. $x \cdot 0 = 0$.
6. $x \cdot S(y) = x \cdot y + x$.

Plus the axiom schema for induction:

$$\left[\varphi(0, \bar{y}) \wedge \forall x (\varphi(x, \bar{y}) \rightarrow \varphi(S(x), \bar{y})) \right] \rightarrow \forall x \varphi(x, \bar{y}). \quad (6.1)$$

The axiom schema is for all first-order definable subsets of the natural numbers. It is a countably infinite set of axioms.

6.2.2 VARIANTS AND EXPANDED LANGUAGE

sec var

We can define the binary relation “ $<$ ” by

$$x < y \quad \text{iff} \quad \exists z (z \neq 0 \wedge x + z = y). \quad \text{df 1o} \quad (6.2)$$

The standard properties for “ $<$ ” and its interaction with addition and multiplication can be checked:

1. $<$ is a linear ordering,
2. $0 < x \wedge y < z \rightarrow x \cdot y < x \cdot z$,
3. $0 < x \wedge y < z \rightarrow x + y < x + z$,
4. etc. .

This is all standard, and we won’t be doing it. But see the next Section 6.2.3.

It is often convenient to add $<$ as a new “primitive” symbol to the language $\mathcal{L}$, rather than as a definition as in (6.2), in which case we have an expanded language

$$\mathcal{L} = \{S, +, \cdot, <, 0\}. \quad (6.3)$$

In this case we add (6.2) as an axiom, after replacing “iff” by “ $\leftrightarrow$ ”.

Or alternatively take a new language

$$\mathcal{L} = \{S, +, \cdot, \leq, 0\}. \quad (6.4)$$

and add the axiom

$$x \leq y \iff \exists z (x + z = y). \quad \text{df 1o2} \quad (6.5)$$

It does not make any essential difference, but it is helpful in Section 6.4.5 when we define the notion of a formula with a particular structure, such as a Δ_0 -formula or a Σ_1 -formula.

Exercise 6.2. Explain how some of the above properties of “ $<$ ” and “ $\leq$ ” follow from the relevant definitions and axioms.

6.2.3 HOW STRONG ARE THE PEANO AXIOMS?

Extremely! All the basic first-order properties of addition, multiplication, exponentiation, etc. can be expressed and proved in PA. Arguably, every result proved in either classical number theory or combinatorics about the natural numbers can be formalized in PA. (Marker, 2024, Chapter 13)

Versions of the Prime Number Theorem are expressible and provable in PA.

The Riemann Hypothesis has PA equivalent versions, and if true it “probably” also has a PA proof.

Fermat’s last Theorem is readily expressed in $\mathcal{L}$. Although its proof uses methods far from PA, it seems probable that its proof can, in principle, be done in PA.

In fact, there is a major current project within the Lean community to unfold a proof of Fermat’s Last Theorem down to basic Lean assumptions. This would bring one much closer to knowing if there is, in principle, a proof of that theorem from the Peano Axioms. The consensus seems to be that there is indeed such a proof.

It is quite difficult to obtain a first-order sentence which is true in $\mathfrak{N}$ but not provable in PA. We will do this in the proof of Gödel’s Incompleteness Theorem in Chapter 7.

There are, however, more recent examples of “natural” first-order mathematical statements, concerning the Hydra Problem and the behaviour of the Goodstein functions, which are true but unprovable from the Peano axioms, and which we will later discuss.

See also [Goodstein’s Theorem](#). For interesting related material see [Representation of \$\epsilon_0\$ by rooted trees](#).

6.2.4 NONSTANDARD MODELS OF ARITHMETIC

In the following, “ $<$ ” is listed as one of the symbols of the language $\mathcal{L}$. Instead, we could regard it as a defined relation, as in Section 6.2.2. Either way it will be interpreted as a linear ordering (which is a property expressible in first-order logic) in any model of PA.

Theorem 6.3. *Let T be the Peano Axioms PA, or any extension of PA including $Th(\mathfrak{N})$, the set of all sentences true in the standard model of arithmetic*

$$\mathfrak{N} = \langle \mathbb{N}, +, \cdot, <, 0, 1 \rangle. \quad (6.6)$$

Then there are models $\mathfrak{A} \models T$ containing an element a such that

$$1 < a, 1 + 1 < a, 1 + 1 + 1 < a, \dots \quad \text{eq. ncc (6.7)}$$

are all true in $\mathfrak{A}$.

Proof. Extend the language $\mathcal{L}$ to $\mathcal{L}^* = \mathcal{L} \cup \{c\}$, where c is a new constant symbol.

Let T^* be the set of sentences

$$T^* := T \cup \{c > 1, c > 1 + 1, c > 1 + 1 + 1, \dots\}$$

Every finite subset of T^* has a model (why?) and so T^* has a model by the compactness theorem.

If a is the interpretation of the symbol c in any such model, then (6.7) will be satisfied in the model. $\square$

Remark 6.4. The models in the previous theorem are called *nonstandard models of arithmetic*. $\square$

Remark 6.5. In the proof, $\mathfrak{A}$ is a structure for the expanded language $\mathcal{L}^*$ as well as being a structure for the language $\mathcal{L}$.

Note that $\mathcal{L}^*$ is *NOT* the language $\mathcal{L}$, it is an *expansion* of $\mathcal{L}$. $\square$

Remark 6.6 (Structure Results for Nonstandard Models). Suppose

$$\mathfrak{A} = \langle A, S^{\mathfrak{A}}, +^{\mathfrak{A}}, \cdot^{\mathfrak{A}}, 0^{\mathfrak{A}} \rangle = \langle A, S, +, \cdot, 0 \rangle \models T. \quad (6.8) \text{ nsm}$$

The superscripts $*^{\mathfrak{A}}$ are dropped when it is clear that we are referring to the model $\mathfrak{A}$ and not to $\mathfrak{N}$ or to some other model of T .

Equation (6.8) means that $\mathfrak{A}$ satisfies exactly the same sentences from the first-order language $\mathcal{L}$ of arithmetic as does the standard model $\mathfrak{N}$.

In particular, it is convenient to identify the interpretations in $\mathfrak{N}$ and $\mathfrak{A}$ of the terms

$$0, 1, 2, 3, \dots \quad (6.9)$$

Exercise 6.7 (End Extension). Explain why the definition of $<$ in (6.2) leads to a linear ordering in $\mathfrak{A}$. $\square$ ex EE

Show that if a is any nonstandard element in a nonstandard model $\mathfrak{A}$ of T , where T is as in equation (6.8), then $n < a$ for any standard integer n .

For this reason we say that $\mathfrak{A}$ is an *end extension* of $\mathfrak{N}$. $\square$

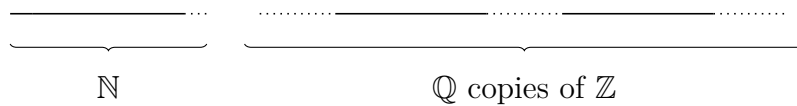


Figure 6.1: The order type $\mathbb{N} + \mathbb{Z} \times \mathbb{Q}$.

fig end ext

Remark 6.8 (Order Type of Countable End Extensions). It is not too difficult to **prove** that the order type of *countable* end extensions, see Figure 6.1, is rem EE

$$\mathbb{N} + \mathbb{Z} \times \mathbb{Q}. \quad (6.10) \text{ eq EE}$$

By this we mean the linear ordering is isomorphic to $\mathbb{N}$ followed by $\mathbb{Q}$ copies of $\mathbb{Z}$.

$\mathbb{Q}$ is the countable dense linear ordering of the rationals (in Section 6.2.5 we show all countable dense linear orderings are isomorphic) and $\mathbb{Z}$ is the discrete linear ordering of the integers (positive, zero and negative). $\square$

Remark 6.9 (Sketch of Proof). Suppose $\mathfrak{A}$ is an end extension of $\mathfrak{N}$ (not necessarily countable at this stage).

Suppose $a \in A$ is nonstandard. Then a has a successor, and an immediate predecessor, etc.:

$$\dots < a - 2 < a - 1 < a < a + 1 < a + 2 \dots$$

These elements are called the *orbit* of a .

Between any two orbits, of a and b say, there is a third orbit determined by $c = (a+b)/2$ or $c = (a+b+1)/2$ (depending on which of $a+b$ or $a+b+1$ is even, a multiple of another element by 2).

After any orbit determined by a there is another orbit determined by $2 \cdot a$, and similarly there is a previous orbit determined by $a/2$ or by $(a+1)/2$.

This is enough to establish (6.10) for *some* dense linear ordering $\mathbb{Q}$ without endpoints.

If $\mathfrak{A}$ is countable then so is the ordering labelled $\mathbb{Q}$. From Theorem 6.15, $\mathbb{Q}$ is the *unique countable* dense linear ordering without endpoints. $\square$

6.2.5 (OPTIONAL) COUNTABLE DENSE LINEAR ORDERINGS

sec dlos

Here are the natural first-order definitions.

df dlo

Definition 6.10 (Dense Linear Orderings). A linear ordering $\mathfrak{L} = \langle L, < \rangle$ is *dense* if

$$\forall x \forall y ((x < y) \rightarrow \exists z (x < z < y)). \quad (6.11)$$

$\mathfrak{L}$ has *no endpoints* if

$$\forall x (\exists y (y < x) \wedge \exists z (x < z)). \quad (6.12)$$

Definition 6.11 (Isomorphic Orderings). Two (linear) orderings $\mathfrak{A} = \langle A, <_A \rangle$ and $\mathfrak{B} = \langle B, <_B \rangle$ are isomorphic, written $\mathfrak{A} \cong \mathfrak{B}$, if there exists a bijection $f : A \rightarrow B$ such that $a <_A a'$ iff $f(a) <_B f(a')$.

Example 6.12 (Some Dense Linear Orderings). Both $\langle \mathbb{R}, < \rangle$ and $\langle \mathbb{Q}, < \rangle$ are dense linear orderings without endpoints. But they are certainly not isomorphic as their cardinalities are not the same.

A more surprising example is $\langle \mathbb{Q} \setminus \{0\}, < \rangle$. It *does* satisfy the conditions in Definition 6.10.

We will show the surprising result that it is in fact isomorphic to $\langle \mathbb{Q}, < \rangle$. This follows from the next theorem. But I also give an explicit isomorphism in the next example. $\square$

Exercise 6.13. Why is $\langle \mathbb{Q}, < \rangle$ a dense linear ordering? $\square$

Example 6.14 (An isomorphism $g : \mathbb{Q} \cong \mathbb{Q} \setminus \{0\}$). By “ $\cong$ ” we mean an isomorphism, which in this case is an order preserving bijection.

We begin with a monotone increasing function $f : \mathbb{R} \rightarrow \mathbb{R}$ defined as follows.

Consider two double sequences of *rational* numbers

$$\begin{array}{lll} r_0 < r_1 < r_2 < \dots < \sqrt{2} < \dots < s_2 < s_1 < s_0 & r_n \uparrow \sqrt{2} & s_n \downarrow \sqrt{2}, \\ r'_0 < r'_1 < r'_2 < \dots < 0 < \dots < s'_2 < s'_1 < s'_0 & r'_n \uparrow 0 & s'_n \downarrow 0, \end{array}$$

each monotone and converging to either $\sqrt{2}$ or 0 respectively.

Define $f : \mathbb{R} \rightarrow \mathbb{R}$ by

$$f(x) = \begin{cases} x - r_0 + r'_0 & x \leq r_0 \\ r'_n + \frac{x - r_n}{r_{n+1} - r_n} (r'_{n+1} - r'_n) & r_n \leq x \leq r_{n+1}, \quad n \geq 0 \\ f(\sqrt{2}) = 0 & \\ s'_{n+1} + \frac{x - s_{n+1}}{s_n - s_{n+1}} (s'_n - s'_{n+1}) & s_{n+1} \leq x \leq s_n, \quad n \geq 0 \\ x - s_0 + s'_0 & s_0 \leq x. \end{cases}$$

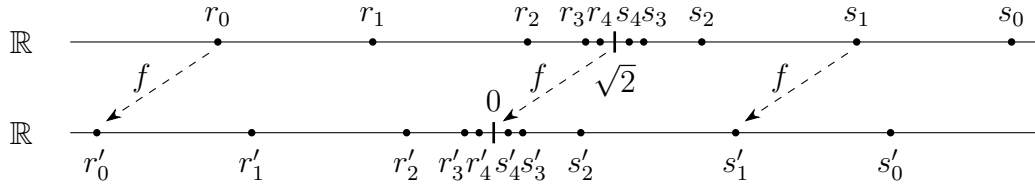


Figure 6.2: The order-isomorphism $f : \mathbb{R} \rightarrow \mathbb{R}$.

fig cge

Note that $f : [r_n, r_{n+1}] \rightarrow [r'_n, r'_{n+1}]$ is a monotone increasing linear bijection for each n , similarly for $f : [s_{n+1}, s_n] \rightarrow [s'_{n+1}, s'_n]$, similarly for $f : (-\infty, r_0] \rightarrow (-\infty, r'_0]$ and similarly for $f : (-\infty, s_0] \rightarrow (-\infty, s'_0]$.

So $f(x)$ is monotone increasing and continuous on all of $\mathbb{R}$, and piecewise linear on $\mathbb{R} \setminus \{\sqrt{2}\}$.

Since the interval endpoints are all rational, it follows that if x is rational then $f(x)$ is rational. Conversely, if $f(x)$ is rational then x is rational with the **single exception** that $f(\sqrt{2}) = 0$.

So, denoting the restriction of f to $\mathbb{Q}$ by g , it follows $g : \mathbb{Q} \rightarrow \mathbb{Q} \setminus \{0\}$ is a monotone increasing bijection, and so is the required isomorphism. $\square$

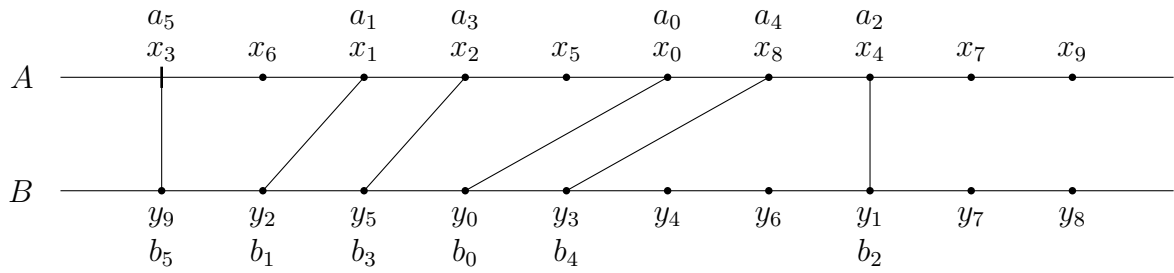


Figure 6.3: The order-isomorphism between $\mathfrak{A}$ and $\mathfrak{B}$ constructed in Theorem 6.15. fig q-minuszero thm ctbl dlo

Theorem 6.15. Suppose $\mathfrak{A} = \langle A, <_A \rangle$ and $\mathfrak{B} = \langle B, <_B \rangle$ are countable dense linear orders without end points. Then they are isomorphic.

Proof. See Figure 6.3

Since A and B are countably infinite there exist enumerations $A = (x_n \mid n \geq 0)$ and $B = (y_n \mid n \geq 0)$. (These enumerations bear no relation to the orderings $<_A$ and $<_B$.)

We will define increasing sequences of finite subsets

$$\{a_0, a_1, a_2, \dots, a_n\} \quad \text{and} \quad \{b_0, b_1, b_2, \dots, b_n\} \quad \text{eq fin approx (6.13)}$$

which preserve the order structure in that

$$a_i <_A a_j \iff b_i <_B b_j. \quad \text{eq preserve order (6.14)}$$

Moreover,

$$A = \{a_0, a_1, a_2, \dots, a_n, \dots\}, \quad B = \{b_0, b_1, b_2, \dots, b_n, \dots\}.$$

Step 0.

Define $a_0 = x_0$ and $b_0 = y_0$.

Step 1.

Define $a_1 = x_1$.

Define $b_1 = y_j$ where j is the least index such that for $\{a_0, a_1\}$ and $\{b_0, b_1\}$ the *order structure is preserved* as in (6.14)

Step 2.

Define $b_2 = y_j$ where j is the least index such that y_j is *not yet used*.

Define $a_2 = x_i$ where i is the least index such that for $\{a_0, a_1, a_2\}$ and $\{b_0, b_1, b_2\}$ the *order structure is preserved* as in (6.14).

Step 3.

Define $a_3 = x_i$ where i is the least index such that x_i is *not yet used*.

Define $b_3 = y_j$ where j is the least index such that the *order structure is preserved*.

Step 4

Define $b_4 = y_j$ where j is the least index such that y_j is *not yet used*.

Define $a_4 = x_i$ where i is the least index such that the *order structure is preserved*.

etc.

By step $2n$ the initial $x_0, \dots, x_{n-1}$ and the initial $y_0, \dots, y_{n-1}$ have been used.

And after each step the two finite structures are order isomorphic.

It follows that $a_i \leftrightarrow b_i$ is a bijection and is an order-isomorphism between $\mathfrak{A}$ and $\mathfrak{B}$. $\square$

Remark 6.16 (Back-and-Forth). The technique used here is called the back-and-forth method and is frequently used in model theory. $\square$

Remark 6.17 (Countability is Essential). The proof used the countability of the orderings. The result is not true for uncountable dense linear orderings without endpoints. $\square$

Exercise 6.18. Can you construct counterexamples? $\square$

6.3 PRIMITIVE RECURSIVE — COMPUTABILITY

sec comp

6.3.1 FOR AND WHILE LOOPS

We will be concerned with the computability properties of various functions $f : \mathbb{N}^k \rightarrow \mathbb{N}$. For now you should be thinking about those functions generated by means of some software program.

As well as the “for” and “while” loops discussed below, the programs may call on certain constants and elementary functions. In principle, the constant 0, the successor function S , and the arithmetic operations $+$ and $\times$ are sufficient.

See Remark 6.32 for references.

Definition 6.19 (Loop Types). In programming terminology a *for loop* requires that the number of steps in the loop is bounded, prior to entering the loop, in terms of functions and parameters previously computed.

An *until/while* loop has no such prior bound and is allowed to continue looping until some condition is met, and so it may require an unbounded search and also may never halt. $\square$

6.3.2 COMPUTABILITY CLASSES

Definition 6.20 (Primitive Recursive). A function $f : \mathbb{N}^k \rightarrow \mathbb{N}$ is *primitive recursive* if it can be computed using only “for” loops. The number of loops can depend on the function. def PR

A relation/set $R \subseteq \mathbb{N}^k$ is *primitive recursive* if its characteristic function¹ is primitive recursive.

See Section 6.3.4 for an alternative definition.

Definition 6.21. Functions which are computable but whose computation may require the use of “until” loops (as well as “for” loops) are *partial computable*. They are “partial” in the sense that their domain is the subset of $\mathbb{N}^k$ on which the program eventually halts. def comp

If a partial computable function is defined for all inputs then it is said to be *total computable*.

A relation $R \subseteq \mathbb{N}^k$ is *decidable/computable* if its characteristic function is computable. (Note that a characteristic function is everywhere defined.)

For alternative definitions see Chapter 8.

Remark 6.22. In this chapter and Chapter 7 we only need primitive recursive functions and relations, except for occasional side-comments and remarks. $\square$

Remark 6.23 (Three Families of Computable Functions). From the definitions:

$$\text{partial computable} \supset (\text{total}) \text{ computable} \supset \text{primitive recursive.} \quad \text{term_comp (6.15)}$$

The inclusions are strict:

One can obtain total computable, but not primitive recursive, functions by “diagonalising out” of the primitive recursive class, see Remark 6.27. Also, the Ackermann function as discussed in Section 8.10.7, is total computable but not primitive recursive.

The existence of a partial computable but not computable function is a consequence of the results on Turing machines in Section ??.

Sometimes we may refer to computable functions when we mean partial computable functions in general, rather than meaning just total computable functions. But hopefully this will be clear from context. $\square$

¹The characteristic function χ_R is define by

$$\chi_R(a_1, \dots, a_k) = \begin{cases} 1 & (a_1, \dots, a_k) \in R, \\ 0 & (a_1, \dots, a_k) \notin R. \end{cases}$$

We think of the statement $R(a_1, \dots, a_k) = 1$ as saying R is true of $(a_1, \dots, a_k)$.

Remark 6.24 (Older Terminology). Earlier terminology is

$$\text{partial recursive} \supset (\text{total}) \text{ recursive} \supset \text{primitive recursive.} \quad \text{term rec (6.16)}$$

The usage of “recursive” has been currently discontinued in the first two cases in (6.16) as it clashes with uses of the word in other contexts, such as a recursive definition or as in a procedure which calls on itself.

But the terminology “primitive recursive” is firmly entrenched in the literature! $\square$

6.3.3 FUNCTION VS ALGORITHM

We are primarily concerned with properties of functions (in particular the existence of certain types of algorithms necessary for their computation), rather than with the specific algorithms needed to compute them (other than their type). We may not even know the function, apart from certain properties it may have.

For example, suppose²

$$f(n) = \begin{cases} 0 & \text{if the Goldbach conjecture is true} \\ 1 & \text{if the Goldbach conjecture is false} \end{cases}$$

The function f is either the constant function f_0 which is everywhere 0 or is the constant function f_1 which is everywhere 1. So f is primitive recursive, even though we do not know if it is f_1 or f_2 .

Similar comments apply in the cases of a total computable function and a partial computable function.

6.3.4 PRIMITIVE RECURSIVE FUNCTIONS AND RELATIONS – KLEENE APPROACH

Here we give the standard definition of a primitive recursive functions as developed by Kleene. sec prfr

Definition 6.25 (Initial Functions, Composition, Primitive Recursion). The following functions $f : \mathbb{N}^n \rightarrow \mathbb{N}$ ($n \geq 1$) are *initial functions*: def ifcpr

- the *zero function*, $Z(x) = 0$,
- the *successor function* $S(x) = x + 1$,
- the *projection functions* $p_i^n(x_1, \dots, x_n) = x_i$ for $1 \leq i \leq n$.

The function $f : \mathbb{N}^n \rightarrow \mathbb{N}$ ($n \geq 1$) is *obtained by composition* from $h, g_1, \dots, g_p$, if

$$f(x_1, \dots, x_n) = h(g_1(x_1, \dots, x_n), \dots, g_p(x_1, \dots, x_n)).$$

The function $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ ($n \geq 0$) is *obtained by primitive recursion* from g and h if

$$\begin{aligned} f(x_1, \dots, x_n, 0) &= g(x_1, \dots, x_n), \\ f(x_1, \dots, x_n, y) &= h(x_1, \dots, x_n, y, f(x_1, \dots, x_n, y-1)) \quad (\text{for all } y \geq 1). \end{aligned} \quad \text{def pr (6.17)}$$

That is,

$$f(\bar{x}, 0) = g(\bar{x}), \quad f(\bar{x}, y) = h(\bar{x}, y, f(\bar{x}, y-1)) \text{ if } y \geq 1.$$

²The Goldbach conjecture asserts that every even natural number greater than 2 is the sum of two primes.

Definition 6.26 (Primitive Recursive Functions and Relations). def prfr A function $f : \mathbb{N}^n \rightarrow \mathbb{N}$ is *primitive recursive* if it is an initial function, or is obtained from primitive recursive functions by composition or by primitive recursion.

Equivalently, the function f is *primitive recursive* if there is a sequence of functions

$$f_1, f_2, \dots, f_k (= f) \quad \text{eq, prf} \quad (6.18)$$

such that f is f_k and such that each f_j is an initial function or is obtained from previous f_i by composition or primitive recursion.

A relation/set $R \subseteq \mathbb{N}^n$ is *primitive recursive* iff its characteristic function χ_R is primitive recursive.

rem pr comp
Remark 6.27 (Diagonalising out of Primitive Recursive Functions). Clearly a primitive recursive function is computable in any informal sense.

On the other hand, we can give an algorithm for listing all primitive recursive functions (e.g. first list all appropriate definitional sequences of length at most 100 in which all functions are at most 10-ary, then those of length 200 in which all functions are at most 20-ary, etc.). From this we can effectively list all unary primitive recursive functions.

Let $\varphi(n) = \varphi_n(n) + 1$ where φ_n is the n^{th} unary primitive recursive function. Clearly φ is computable but not primitive recursive.

Thus the primitive recursive functions are a proper subset of the set of all computable functions. (Although we have yet to give a precise definition of this latter set.) $\square$

Exercise 6.28. We have just seen that $n \mapsto \varphi(n)$ is not primitive recursive. But apart from this, explain why we expect that its definition cannot be recast into the form of a primitive recursive definition. $\square$

Definition 6.29. *Non-negative subtraction* is defined by

$$x \dot{-} y = \max\{0, x - y\}. \quad (6.19)$$

Theorem 6.30 (Primitive Recursive Examples). prop pr

(1) The functions $x + y$, $x \cdot y$, x^y , $x!$, $x \dot{-} y$, and the constant functions, are primitive recursive.

The relations $<$, $\leq$, $>$, $\geq$ and $=$ are primitive recursive.

(2) If R and S are primitive recursive subsets of $\mathbb{N}^k$, then so are $\mathbb{N}^k \setminus R$, $R \cap S$, $R \cup S$.

(3) (Bounded Quantification) If $R(x_1, \dots, x_n, y)$ is primitive recursive, then so are

$$S(x_1, \dots, x_n, x_{n+1}) := \forall y \leq x_{n+1} R(x_1, \dots, x_n, y),$$

$$T(x_1, \dots, x_n, x_{n+1}) := \exists y \leq x_{n+1} R(x_1, \dots, x_n, y),$$

and

$$W(x_1, \dots, x_n, x_{n+1}) := \mu y \leq x_{n+1} R(x_1, \dots, x_n, y),$$

where $\mu y \leq x_{n+1} R(x_1, \dots, x_n, y)$ is the least $y < x_{n+1}$ such that $R(x_1, \dots, x_n, y)$ if such a y exists, and is x_{n+1} otherwise.

(4) *(Definition by Cases)* Suppose $g_1, \dots, g_k : \mathbb{N}^n \rightarrow \mathbb{N}$ are primitive recursive functions, $Q_1, \dots, Q_k$ are primitive recursive subsets of $\mathbb{N}^n$, and $R_1, \dots, R_k$ are primitive recursive subsets forming a partition of $\mathbb{N}^n$.

Writing $\bar{x}$ for $x_1, \dots, x_n \in \mathbb{N}^n$, let $f : \mathbb{N}^n \rightarrow \mathbb{N}$ and $P \subseteq \mathbb{N}^n$ be defined by

$$f(\bar{x}) = \begin{cases} g_1(\bar{x}) & \text{if } \bar{x} \in R_1, \\ \vdots \\ g_k(\bar{x}) & \text{if } \bar{x} \in R_k, \end{cases}$$

and

$$\bar{x} \in P \iff \begin{cases} \bar{x} \in Q_1 & \text{if } \bar{x} \in R_1, \\ \vdots \\ \bar{x} \in Q_k & \text{if } \bar{x} \in R_k. \end{cases}$$

Then f and P are primitive recursive.

Proof.

(1) & (2) We do $x + y$ in detail. Informally we have

$$\begin{aligned} x + 0 &= x, \\ x + S(y) &= S(x + y), \end{aligned}$$

as a recursive definition of addition.

More formally, writing $f(x, y)$ for $x + y$, we have

$$\begin{aligned} f(x, 0) &= p_1^2(x), \\ f(x, S(y)) &= S(f(x, y)). \end{aligned}$$

But $S(f(x, y))$ can be written as $(S \circ p_3^3)(x, y, f(x, y))$. Thus we see addition is obtained in a primitive recursive way from the sequence f_1, f_2, f_3, f_4, f_5 where $f_1 = p_1^2$, $f_2 = p_3^3$, $f_3 = S$, $f_4 = f_3 \circ f_2$, and f_5 is obtained by primitive recursion from f_1 and f_4 .

But we would not normally be so formal.

Likewise for $x \cdot y$, x^y , $x!$, and the constant functions.

$x \dot{-} 1$ is a primitive recursive function of x since $0 \dot{-} 1 = 0$ and $(x + 1) \dot{-} 1 = x$. Then $x \dot{-} y$ is primitive recursive since $x \dot{-} 0 = x$, $x \dot{-} (y + 1) = (x \dot{-} y) \dot{-} 1$.

By composition, it follows that if the characteristic function χ_R of R is primitive recursive, then so is $1 \dot{-} \chi_R$, the characteristic function of $\mathbb{N}^n \setminus R$.

The characteristic function of $R \cap S$ is $\chi_R \cdot \chi_S$, hence if R and S are primitive recursive then so is $R \cap S$.

Similarly for $R \cup S = \mathbb{N}^n \setminus (\mathbb{N}^n \setminus R) \cap (\mathbb{N}^n \setminus S)$.

The characteristic function of $x = y$ is

$$1 \dot{-} ((x \dot{-} y) + (y \dot{-} x)).$$

The characteristic function of $x \leq y$ is $1 \dot{-} (x \dot{-} y)$.

Thus $\leq$ and $=$ are primitive recursive relations, and hence so are $<$, $\geq$, $>$, $\neq$.

(3) If

$$S(x_1, \dots, x_n, x_{n+1}) := \forall y \leq x_{n+1} R(x_1, \dots, x_n, y),$$

then

$$\chi_S(x_1, \dots, x_n, x_{n+1}) = \prod_{y=1}^{x_{n+1}} \chi_R(x_1, \dots, x_n, y).$$

A recursive definition of χ_S gives the result that if R is primitive recursive then S is primitive recursive.

We have also

$$\exists y \leq x_{n+1} R(x_1, \dots, x_n, y) \iff \neg \forall y \leq x_{n+1} \neg R(x_1, \dots, x_n, y),$$

so

$$(x_1, \dots, x_n, x_{n+1}) \mapsto \exists y \leq x_{n+1} R(x_1, \dots, x_n, y)$$

is primitive recursive.

(4) For the definition by cases part of the theorem we have

$$f(x) = g_1(\bar{x}) \cdot \chi_{R_1}(\bar{x}) + \dots + g_k(\bar{x}) \cdot \chi_{R_k}(\bar{x}).$$

Hence f is primitive recursive.

Since χ_P may be defined by

$$\chi_P(\bar{x}) = \begin{cases} \chi_{Q_1}(\bar{x}) & \text{if } \bar{x} \in R_1, \\ \vdots \\ \chi_{Q_k}(\bar{x}) & \text{if } \bar{x} \in R_k, \end{cases}$$

it follows P is primitive recursive.

Finally, W is primitive recursive since it is the following definition by cases:

$$\begin{aligned} \mu y \leq 0 \ R(x_1, \dots, x_n, y) &= 0, \\ \mu y \leq x_{n+1} \ R(x_1, \dots, x_n, y) &= \begin{cases} \mu y \leq x_n \ R(x_1, \dots, x_n, y) & \text{if } \exists y \leq x_{n+1} R(x_1, \dots, x_n, y), \\ x_{n+1} & \text{if } \neg \exists y \leq x_{n+1} R(x_1, \dots, x_n, y). \end{cases} \end{aligned}$$

□

Exercise 6.31. What are the recursive definitions for $(x, y) \mapsto x^y$ and for $x \mapsto x!$, thereby showing these functions are primitive recursive. □

6.3.5 EQUIVALENCE OF TWO DEFINITIONS OF PRIMITIVE RECURSIVE

Remark 6.32 (History). rem hist The fact that primitive recursive functions are those computed by programs using “for loops” but not “while loops” was first shown in (Meyer and Ritchie, 1967).

See (Brock and Ritchie, 2020) for an interesting history article concerning Ritchie and his “Lost Dissertation”.

For an informal discussion of primitive recursive functions corresponding to programs using “for loops” but not “while loops” see (Smith, 2020, Theorems 27, 28 p. 65). There is a briefer discussion in (Smith, 2007, Theorem 4.1, p.104) and the following paragraph.

To formulate and prove the following result in detail, see (Kozen, 1997, pp 269–273). □

Theorem 6.33. *The two definitions of a primitive recursive function, Definition 6.20 (no unbounded search) and Definitions 6.25 + 6.26 (standard formal definition), are equivalent.*

Proof.

Definitions 6.25 + 6.26 $\implies$ Definition 6.20:

First note that the zero, successor and projection functions are built-in, or are computable with at most for loops from other built-in functions.

Composition of functions is obtained by passing the output of one computation to the input of another. Recursion uses a prior fixed number of iterative calls to a loop construction – think of computing $n!$, which requires n iterations of multiplication.

Definition 6.20 $\implies$ Definitions 6.25 + 6.26:

Suppose a program sets a value g for $f(0)$ and then goes into a for loop which on the n -th loop calls on a previously computed function h to compute $f(Sn)$ in terms of n and $f(n)$. This corresponds to a definition of f from g and h by primitive recursion, probably also involving composition. $\square$

The utility of this result is that we can usually readily convince ourselves that various functions and relations are primitive recursive by imagining how to write a suitable computer program

6.4 ARITHMETIC DEFINABILITY AND REPRESENTABILITY

sec ADR

6.4.1 OVERVIEW

The goal of this Section is to prove that primitive recursive functions and relations can be defined in $\mathfrak{N}$ (a semantic notion) and represented in Peano Arithmetic PA (a syntactic notion) by a Σ_1 -formula. See Theorem 6.63 and Corollary 6.68, in Section 6.4.8.

Also, see Section 6.4.2 for “defined” and Section 6.4.3 for “represented” in the current context of Arithmetic. See Section 6.4.5 for “ Σ_1 -formula”.

Remark 6.34 (Convention). Here, and elsewhere, we may refer to $R \subseteq \mathbb{N}^n$ as either a set or a relation. In the latter case we may equivalently write

$$\begin{aligned} (a_1, \dots, a_n) \in R & \quad \text{or} \quad R(a_1, \dots, a_n), \\ (a_1, \dots, a_n) \notin R & \quad \text{or} \quad \neg R(a_1, \dots, a_n). \end{aligned} \quad \square$$

6.4.2 DEFINABLE/EXPRESSIBLE IN $\mathfrak{N}$

sec den

We have already defined the general notion of a set $R \subseteq A^n$, where A is the universe of an $\mathcal{L}$ -structure $\mathfrak{A}$, being defined via an $\mathcal{L}$ -formula $\varphi(x_1, \dots, x_n)$. See Definition 1.4.5.

Here we are concerned with the standard model $\mathfrak{N} = \langle \mathbb{N}, \dots \rangle$ for the language $\mathcal{L}$ of arithmetic. We do not need parameters $b_1, \dots, b_m$ as in Definition 1.4.5 regarding φ , since all elements of $\mathbb{N}$ already correspond to *variable-free* terms.

For completeness, here is the relevant definition in the current situation.

Remark 6.35 (Abuse of Notation). In the following we are again abusing notation.

For example in (6.23) and (6.24), within $R(a_1, \dots, a_n)$ the a_i refer to natural numbers, that is to elements of $\mathbb{N}$. In the expression $PA \vdash \varphi(a_1, \dots, a_n)$ the a_i refer to terms in the language $\mathcal{L}$ of arithmetic, where the term a_i is $S \dots S(0)$ built from the symbol 0 and the function symbol S written (the number) a_i times. $\square$

Definition 6.36 (Definable Relations and Functions).

Suppose $\mathfrak{N} = \langle \mathbb{N}, S, +, \cdot, 0 \rangle$, $R \subseteq \mathbb{N}^n$ and $f : \mathbb{N}^n \rightarrow \mathbb{N}$.

Then R and f are defined (or expressed) in $\mathfrak{N}$ by the formulas $\varphi(x_1, \dots, x_n)$ and $\psi(x_1, \dots, x_n, y)$ respectively, if

$$\begin{aligned} R(a_1, \dots, a_n) &\iff \mathfrak{N} \models \varphi(a_1, \dots, a_n), & \text{eq satn 1 (6.20)} \\ f(a_1, \dots, a_n) = b &\iff \mathfrak{N} \models \psi(a_1, \dots, a_n, b). & \text{eq satn 2 (6.21)} \end{aligned}$$

Remark 6.37. It follows from (6.20) and (6.21) respectively, that

$$\begin{aligned} \neg R(a_1, \dots, a_n) &\iff \mathfrak{N} \models \neg \varphi(a_1, \dots, a_n), & \text{eq satn 3 (6.22)} \\ f(a_1, \dots, a_n) \neq b &\iff \mathfrak{N} \models \neg \psi(a_1, \dots, a_n, b). \end{aligned}$$

6.4.3 REPRESENTABLE IN PA

Definition 6.38 (Representable Relation). Suppose $R \subseteq \mathbb{N}^n$ and PA are the Peano Axioms.

Then the relation $R \subseteq \mathbb{N}^n$ is represented in PA , or equivalently PA represents R , by the $\mathcal{L}$ -formula $\varphi(v_1, \dots, v_n)$ if for all $a_1, \dots, a_n \in \mathbb{N}$,

$$\begin{aligned} R(a_1, \dots, a_n) &\implies PA \vdash \varphi(a_1, \dots, a_n), & \text{eq RR1 (6.23)} \\ \neg R(a_1, \dots, a_n) &\implies PA \vdash \neg \varphi(a_1, \dots, a_n). & \text{RR2 (6.24)} \end{aligned}$$

Remark 6.39. Note the structural difference between definition (6.20) on one hand, and definition (6.23) plus (6.24) on the other hand.

The reason is that for every $\varphi(a_1, \dots, a_n)$ exactly one of $\mathfrak{N} \models \varphi(a_1, \dots, a_n)$ and $\mathfrak{N} \models \neg \varphi(a_1, \dots, a_n)$ will be the case.

On the other hand, it will often be the case that neither $PA \vdash \varphi(a_1, \dots, a_n)$ nor $PA \vdash \neg \varphi(a_1, \dots, a_n)$ will be the case.

In order to capture the notion of membership by PA , Definition 6.38 is the natural definition. $\square$

Exercise 6.40. Assuming PA is consistent, and both (6.23) and (6.24) are true, show that the $\implies$ in (6.23) and (6.24) can be replaced by $\iff$.

Solution. Assume PA is consistent, and both (6.23) and (6.24) are true.

Suppose $PA \vdash \varphi(a_1, \dots, a_n)$. If $\neg R(a_1, \dots, a_n)$ then by (6.24) it follows that $PA \vdash \neg \varphi(a_1, \dots, a_n)$. This contradicts the consistency of PA . Hence

$$PA \vdash \varphi(a_1, \dots, a_n) \implies R(a_1, \dots, a_n).$$

Similarly

$$PA \vdash \neg \varphi(a_1, \dots, a_n) \implies \neg R(a_1, \dots, a_n).$$

Exercise 6.41. Assuming PA is sound and R is represented in PA by φ , then R is defined/expressed in $\mathfrak{N}$ by φ . $\square$

Solution. By the assumptions that PA represents R and that PA is sound, respectively,

$$R(a_1, \dots, a_n) \implies PA \vdash \varphi(a_1, \dots, a_n) \implies \mathfrak{N} \models \varphi(a_1, \dots, a_n).$$

Similarly,

$$\neg R(a_1, \dots, a_n) \implies PA \vdash \neg \varphi(a_1, \dots, a_n) \implies \mathfrak{N} \models \neg \varphi(a_1, \dots, a_n).$$

Suppose $\mathfrak{N} \models \varphi(a_1, \dots, a_n)$. If $\neg R(a_1, \dots, a_n)$ then by the preceding $\mathfrak{N} \models \neg \varphi(a_1, \dots, a_n)$, contradiction. Hence

$$\mathfrak{N} \models \varphi(a_1, \dots, a_n) \implies R(a_1, \dots, a_n).$$

So R is defined by φ , see Definition 6.36.

Definition 6.42 (Representable Function). A function $f : \mathbb{N}^n \rightarrow \mathbb{N}$ is *represented* in PA , or equivalently PA represents f , by the $\mathcal{L}$ -formula $\varphi(v_1, \dots, v_n, w)$ if for all $a_1, \dots, a_n, b \in \mathbb{N}$:

$$\begin{aligned} f(a_1, \dots, a_n) = b &\implies PA \vdash \varphi(a_1, \dots, a_n, b), & \text{eq. RF1 (6.25)} \\ f(a_1, \dots, a_n) \neq b &\implies PA \vdash \neg \varphi(a_1, \dots, a_n, b). & \text{eq. RF2 (6.26)} \end{aligned}$$

Exercise 6.43. Assuming PA is consistent, and both (6.25) and (6.26) are true, show that the $\implies$ in (6.25) and (6.26) can be replaced by $\iff$. (Essentially the same argument as for Exercise 6.40.) $\square$

6.4.4 (OPTIONAL) STRONGER NOTIONS OF FUNCTION REPRESENTABILITY

Remark 6.44 (Two Variants of “Representable Function”). Notice that (6.25) and (6.26) only refer to standard integers. rem 2var

A stronger condition is to keep (6.25) and replace (6.26) by

$$PA \vdash \forall v_1 \dots \forall v_n \exists! w \varphi(v_1, \dots, v_n, w), \quad \text{eq. RF2a (6.27)}$$

which states that φ encodes a function in every model of PA , not necessarily standard.

Weaker than this but still stronger than assuming (6.26) is to replace (6.26) by, for all $a_1, \dots, a_n \in \mathbb{N}$,³

$$PA \vdash \exists! w \varphi(a_1, \dots, a_n, w). \quad \text{eq. RF2b (6.28)}$$

This ensures φ encodes a function when restricted to standard integers in its domain. $\square$

Exercise 6.45. Think about what might happen if (6.26) is true in a model of PA , but its stronger version (6.27) is false. $\square$

³ “ $\exists!$ ” is the uniqueness operator. $\exists! w \varphi(v_1, \dots, v_n, w)$ is an abbreviation for the sentence $\exists w [\varphi(v_1, \dots, v_n, w) \wedge \forall z (\varphi(v_1, \dots, v_n, z) \rightarrow (z = w))]$.

Remark 6.46.

$$(6.25) + (6.28) \implies (6.25) + (6.26)$$

To see this, first simplify notation as follows.

In (6.28) suppress $a_1, \dots, a_n$ and write $\exists!w \varphi(w)$ for $\exists!w \varphi(a_1, \dots, a_n, w)$. (Note that $a_1, \dots, a_n$ in the formula $\varphi(a_1, \dots, a_n, v_{n+1})$ are just constant terms).

Now assume (6.25) + (6.28) and assume moreover that $f(a_1, \dots, a_n) \neq b$. We want to show that $PA \vdash \neg\varphi(a_1, \dots, a_n, b)$.

Since $f(a_1, \dots, a_n) \neq b$, $f(a_1, \dots, a_n) = k$ for some $k \neq b$ and so from (6.25), $PA \vdash \varphi(k)$.

Moreover, $PA \vdash k \neq b$ (if two integers are different, this is certainly provable in PA).

From (6.28), $PA \vdash \exists!w \varphi(w)$.

Summarising,

$$PA \vdash \varphi(k), \quad PA \vdash k \neq b, \quad PA \vdash \exists!w \varphi(w). \quad \text{eq PA prop (6.29)}$$

We claim that

$$\vdash [\varphi(k) \wedge (k \neq b) \wedge \exists!w \varphi(w)] \rightarrow \neg\varphi(b). \quad \text{bad der (6.30)}$$

Since $\neg\varphi(b)$ is $\neg\varphi(a_1, \dots, a_n, b)$, this will establish (6.26).

But (6.30) would be a lengthy derivation from the logical axioms, and we are not doing that in this course. Instead we argue semantically and use the Completeness Theorem.

From (6.29) for every $\mathcal{L}$ -structure $\mathfrak{A}$, if $\mathfrak{A} \models PA$ then

$$\mathfrak{A} \models \varphi(k), \quad \mathfrak{A} \models \exists!w \varphi(w) \quad \mathfrak{A} \models k \neq b, .$$

From this, by just reasoning about the model $\mathfrak{A}$, it follows $\mathfrak{A} \models \neg\varphi(b)$.

Since $\mathfrak{A}$ was an arbitrary model of PA , $PA \models \neg\varphi(b)$.

It follows from the Completeness Theorem 3.17 that $PA \vdash \neg\varphi(b)$. That is $PA \vdash \neg\varphi(a_1, \dots, a_n, b)$, which is as required. $\square$

6.4.5 $\Delta_0, \Sigma_1, \Pi_1$ FORMULAS

In the following we take “ $\leq$ ” as a basic relation symbol. So as the language of arithmetic we take sec DSF

$$\mathcal{L} = \{S, +, \cdot, \leq, 0\}.$$

See Section 6.2.2.

This makes no essential difference to the expressive power of the language $\mathcal{L}$ of arithmetic since we could define

$$x \leq y \quad := \quad \exists z (x + z = y),$$

and then prove the standard linear ordering properties for “ $\leq$ ”.

However, it is convenient to do this in order to make the following definitions.

Definition 6.47 (Δ_0 -Formula). A formula φ is a Δ_0 -formula if

1. φ is atomic, or
2. φ is $\neg\psi$ where ψ is Δ_0 , or
3. φ is $\psi \wedge \chi$ where ψ and χ are Δ_0 , or

4. φ is $\exists x (x \leq t \wedge \psi)$ where t is a term not containing x and ψ is Δ_0 , or
5. φ is $\forall x (x \leq t \rightarrow \psi)$ where t is a term not containing x and ψ is Δ_0 .

It follows that $\psi \vee \chi$ and $\psi \rightarrow \chi$ are Δ_0 if both ψ and χ are Δ_0 . $\square$

So Δ_0 -formulas are those formulas built up using connectives and bounded quantifiers.

We use the standard abbreviations $\exists x \leq t \psi$ and $\forall x \leq t \psi$ for $\exists x (x \leq t \wedge \psi)$ and $\forall x (x \leq t \rightarrow \psi)$ respectively.

Definition 6.48 (Σ_1, Π_1 Formulas). A formula φ is a Σ_1 -formula or a Π_1 -formula if it is of the form **df SP1**

$$\exists x_1 \exists x_2 \dots \exists x_n \psi \quad \text{or} \quad \forall x_1 \forall x_2 \dots \forall x_n \psi$$

respectively, where ψ is Δ_0 .

Remark 6.49 ($\Delta_0 \subset \Sigma_1, \Delta_0 \subset \Pi_1$). We include the case $n = 0$ in Definition 6.48, by which we mean there are no quantifiers. So if a formula is Δ_0 then it is also Σ_1 and Π_1 . $\square$

6.4.6 SEMANTIC PROPERTIES OF $\Delta_0, \Sigma_1, \Pi_1$ FORMULAS

For a more detailed discussion of the following see (Kaye, 1991, Chapter 2).

Theorem 6.50 (Δ_0 -Formulas: Validity and End Extensions). **thm vald0** Suppose $\mathfrak{N} = \langle \mathbb{N}; S, +, \cdot, \leq, 0 \rangle$ is the standard model for arithmetic, $\varphi(x_1, \dots, x_n)$ is a Δ_0 -formula with free variables included in $x_1, \dots, x_n$, and $a_1, \dots, a_n \in \mathbb{N}$.

Then there is an explicit finite (“mechanical”) procedure, based on the syntactic form of φ , to check if $\mathfrak{N} \models \varphi(a_1, \dots, a_n)$ or $\mathfrak{N} \models \neg \varphi(a_1, \dots, a_n)$.

If $\mathfrak{A} = \langle A; S, +, \cdot, \leq, 0 \rangle$ is an end extension of $\mathfrak{N}$ with $\mathbb{N} \subset A$ (Exercise 6.7 and Remark 6.8), then

$$\mathfrak{N} \models \varphi(a_1, \dots, a_n) \iff \mathfrak{A} \models \varphi(a_1, \dots, a_n). \quad (6.31)$$

Proof. The proof of the first claim is by induction on the way φ is built up, that is by induction on the complexity of φ .

For example:

- Suppose $\varphi(x_1, x_2, x_3)$ is the *atomic formula* $(x_1 + 3) \cdot x_2 = x_3$, so $\varphi(a_1, a_2, a_3)$ is the assertion $(a_1 + 3) \cdot a_2 = a_3$. Given values for a_1, a_2, a_3 it is a mechanical procedure, following the way φ is built up, to check the validity of $\varphi(a_1, a_2, a_3)$ in $\mathfrak{N}$.
- Suppose $\varphi(x_1, x_2)$ is the *bounded existential formula* $\exists x_3 \leq t \psi(x_1, x_2, x_3)$, where $t(x_1, x_2)$ may depend on x_1, x_2 but not x_3 . Then $\varphi(a_1, a_2)$ is the assertion $\exists a_3 \leq t(a_1, a_2) \psi(a_1, a_2, a_3)$.

Given values for a_1, a_2 it is a mechanical procedure to compute $t =: t(a_1, a_2)$ as in the previous example, and then for each $a_3 \leq t(a_1, a_2)$, by the inductive hypothesis, to manually check the validity of $\psi(a_1, a_2, a_3)$.

This gives a manual check of the validity of $(x_1 + 3) \cdot x_2 = x_3$.

- The same argument, with $\exists$ replace by $\forall$, works for the *bounded ge* formula $\forall x_3 \leq t \psi(x_1, x_2, x_3)$.

For the second claim the essential point is that the validity of $\varphi(a_1, \dots, a_n)$ in both $\mathfrak{N}$ and in $\mathfrak{A}$ at some $(a_1, \dots, a_n) \in \mathbb{N}^n$ (remember that $\mathbb{N} \subset A$), depends only on the same finite initial segment of $\mathbb{N}$ in each case. The integer determining this segment is obtained from a composition of the terms in the bounded quantifiers of φ , applied to $a_1, \dots, a_n$. The arithmetic operations $+$ and $\cdot$ and the relation $\leq$ are obtained similarly from S and 0 in each case, and so are isomorphic on this initial segment. $\square$

Remark 6.51 (Δ_0 implies Primitive Recursive). The *mechanical* procedure discussed in the previous theorem is primitive recursive, as it can be carried out by a software program using no unbounded search.

Alternatively, one could do it directly from Definitions 6.25 and 6.26.

In particular, it follows that the set

$$\{(a_1, \dots, a_n) \in \mathbb{N}^n \mid \mathfrak{N} \models \varphi(a_1, \dots, a_n)\}$$

is primitive recursive. But it is not the case that every primitive recursive relation is Δ_0 , as we will see later. $\square$

Theorem 6.52 (Validity for Σ_1/Π_1 formulas). *Suppose $\mathfrak{N}$ is the standard model for arithmetic, $\varphi(x_1, \dots, x_n)$ is a formula with free variables $x_1, \dots, x_n$, and $(a_1, \dots, a_n) \in \mathbb{N}^n$.*

If $\mathfrak{A}$ is an end extension of $\mathfrak{N}$, then

$$\text{if } \varphi \text{ is } \Sigma_1, \quad \mathfrak{N} \models \varphi(a_1, \dots, a_n) \implies \mathfrak{A} \models \varphi(a_1, \dots, a_n), \quad (6.32)$$

$$\text{if } \varphi \text{ is } \Pi_1, \quad \mathfrak{A} \models \varphi(a_1, \dots, a_n) \implies \mathfrak{N} \models \varphi(a_1, \dots, a_n). \quad (6.33)$$

Exercise 6.53. Explain why this is true by using Theorem 6.50. $\square$

Corollary 6.54. *If $\varphi(x_1, \dots, x_n)$ is Σ_1 and $\mathfrak{N} \models \varphi(a_1, \dots, a_n)$ then $PA \vdash \varphi(a_1, \dots, a_n)$.*

Proof. If $\mathfrak{N} \models \varphi(a_1, \dots, a_n)$ then $\mathfrak{A} \models \varphi(a_1, \dots, a_n)$ for every end extension $\mathfrak{A}$. But every model of PA is an end extension. $\square$

Definition 6.55 ($\Delta_0, \Sigma_1, \Pi_1, \Delta_1$ Sets). A set $A \subseteq \mathbb{N}^k$ is Δ_0 if it is defined by a Δ_0 -formula, Σ_1 if defined by a Σ_1 -formula, Π_1 if defined by a Π_1 -formula, and Δ_1 if defined by both a Σ_1 -formula and by a Π_1 -formula.

Exercise 6.56. Why is the set A a Π_1 -set iff its complement $\mathbb{N}^k \setminus A$ is Σ_1 ?

Why is A a Δ_1 -set iff both A and A^c are Σ_1 sets? $\square$

6.4.7 (OPTIONAL) $\Sigma_n, \Pi_n, \Delta_n$: FORMULAS AND SETS

The following extends Definition 6.48 to arbitrary $n \geq 1$.

Definition 6.57 (Σ_n, Π_n Formulas). A formula φ is Σ_n or Π_n if it is in the form

$$\begin{aligned} &\exists x_1 \dots \exists x_n \forall y_1 \dots \forall y_m \exists z_1 \dots \exists z_k \dots \psi \quad \text{or} \\ &\forall x_1 \dots \forall x_n \exists y_1 \dots \exists y_m \forall z_1 \dots \forall z_k \dots \psi, \end{aligned}$$

respectively, where $n \geq 1$ is the number of alternating blocks of quantifiers and ψ is Δ_0 .

Remark 6.58 (Hierarchy of Formulas). We allow empty blocks of quantifiers, so that $\Sigma_n \subset \Sigma_{n+1}$ and $\Sigma_n \subset \Pi_{n+1}$. Similarly, $\Pi_n \subset \Sigma_{n+1}$ and $\Pi_n \subset \Pi_{n+1}$.

Other than the case for Δ_0 -formulas, one does not usually define Δ_1 -formulas (at least in a structural manner) for $n \geq 1$.

However, when one considers *sets* defined in $\mathfrak{N}$, or represented in PA , by $\mathcal{L}$ -formulas, there *is* a natural notion of a Δ_n -set. $\square$

Definition 6.59 (Σ_n , Π_n , Δ_n Sets). A *set/relation* $A \subseteq \mathbb{N}^k$ is Δ_0 if it is defined by a Δ_0 -formula, Σ_n if defined by a Σ_n -formula, Π_n if defined by a Π_n -formula, and Δ_n ($n \geq 1$) if defined by both a Σ_n -formula and a Π_n -formula. def Dsets

See Figure 6.4 for the set inclusions. All inclusions are strict as we discuss later.

Every definable set does

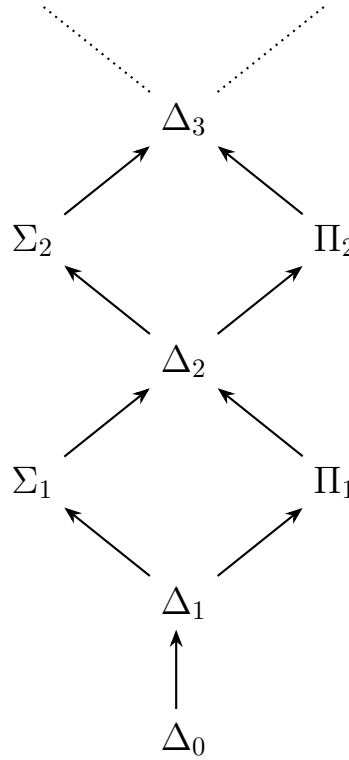


Figure 6.4: Arithmetic Hierarchy of Sets.

fig arith hier

Remark 6.60 (Hierarchies of Formulas and of Sets). A formula is in *prenex normal form* if it is of the form

$$\begin{aligned} & \exists x_1 \dots \exists x_n \forall y_1 \dots \forall y_m \exists z_1 \dots \exists z_k \dots \varphi \quad \text{or} \\ & \forall x_1 \dots \forall x_n \exists y_1 \dots \exists y_m \forall z_1 \dots \forall z_k \dots \varphi. \end{aligned}$$

The formula is Σ_j in the first case and Π_j in the second case, if j is the number of alternating blocks.

Every formula or arithmetic (using bound quantifiers) is logically equivalent to a formula in prenex normal form. One shows this by progressively moving quantifiers to the

front of the formula as follows:

$$\begin{aligned} (\exists x \varphi) \wedge \psi &\mapsto \exists y (\varphi \wedge \psi) \quad (\text{changing } x \text{ to } y \text{ in } \varphi \text{ if necessary, where } y \text{ not free in } \psi) \\ (\forall x \varphi) \wedge \psi &\mapsto \forall y (\varphi \wedge \psi) \quad (\text{changing } x \text{ to } y \text{ in } \varphi \text{ if necessary, where } y \text{ not free in } \psi) \\ \neg \exists x \varphi &\mapsto \forall x \neg \varphi, \quad \neg \forall x \varphi \mapsto \exists x \neg \varphi. \end{aligned}$$

A Σ_n formula is logically equivalent to a Σ_k formula and a Π_k formula for every $k \geq n$ by adding “junk” quantifiers that essentially say nothing.

A set $A \subseteq \mathbb{N}^k$ is Δ_0 if it is defined by a Δ_0 formula, Σ_n if defined by a Σ_n formula, Π_n if defined by a Π_n formula, and Δ_n ($n \geq 1$) if defined by both a Σ_n formula and a Π_n formula. See Figure 6.4 for the inclusions given by the arrows.

All inclusions are strict.

It does not make sense to refer to *formulas* in prenex normal form as being Δ_n for $n \geq 1$, since a formula cannot be more than one of Δ_0 or Σ_n or Π_n (for $n \geq 1$), although it certainly will be logically equivalent to any higher. $\square$

6.4.8 REPRESENTABILITY OF P.R. FUNCTIONS/RELATIONS; β -FUNCTION

Remark 6.61 (Overview). There are many details I am skipping over, but the main ideas are here. sec recprfr

The first goal is to show that primitive recursive functions and relations are *definable* in the standard model $\mathfrak{N} = \langle \mathbb{N}, \dots \rangle$ using the language $\mathcal{L}$ of arithmetic. (See Definition 6.36.)

The second is to show that primitive recursive functions and relations are *representable* in Peano Arithmetic. (See Definitions 6.38 and 6.42.) $\square$

Remark 6.62 (Additional Resources). The terminology “defines/definable” and “represents/representable” is standard. However, (Smith, 2007; 2020) use “expresses” and “captures” respectively with the same meanings.

For more details and discussion regarding this section on representability see (Smith, 2007, Chapter 10) and then (Smith, 2020, Chapters 15–17). They are quite readable, and should be looked at in that order.

(Leary and Kristiansen, 2015, Chapter 5) is the most elementary treatment, but achieves its simplicity by including exponentiation E and axioms for E in the language $\mathcal{L}$ and the corresponding extension of PA . This avoids using Gödel’s β -function, but does not show representability for the *standard* PA axioms.

Other useful references are (Hils and Loeser, 2019, §§4.7, 5.6), (Johnstone, 1987, pp. 49–51), (Marker, 2015, pp 75–81) and (Marker, 2024, Chapter 13, pp. 207–220). $\square$

Theorem 6.63 (Defining in $\mathfrak{N}$ and Representing in P.A. a P.R. function). thm prrep *If f is primitive recursive, then f is both definable (in $\mathfrak{N}$ in the language $\mathcal{L}$) and representable in PA , by a Σ_1 -formula.*

Proof. We use Definition 6.25 of primitive recursive function and Definition 6.42 of representable function.

The idea of the proof is to build up a formula (which represents the function f) by “mirroring” the sequential construction of f in Definition 6.26.

BASIC FUNCTIONS Following the notation in Definition 6.42, the zero, successor and projection functions, respectively $x \mapsto S(x)$, $x \mapsto 0$ and $(x_1, \dots, x_n) \mapsto x_m$, are represented by the $\mathcal{L}$ -formulas $y = S(x)$, $y = 0$ and $y = x_m$ respectively.⁴

(See also the discussion in (Smith, 2020, p76 E1 and p73 C1).)

COMPOSITION If $f(x_1, \dots, x_n) = g(h_1(x_1, \dots, x_n), \dots, h_m(x_1, \dots, x_n))$, g is defined and represented by $\varphi(v_1, \dots, v_m, w)$, and h_i is defined and represented by $\psi_i(v_1, \dots, v_n, w)$, then f is defined and represented by the following formula $\chi(v_1, \dots, v_n, w)$:

$$\exists w_1 \dots \exists w_m (\psi_1(v_1, \dots, v_n, w_1) \wedge \dots \wedge \psi_m(v_1, \dots, v_n, w_m) \wedge \varphi(w_1, \dots, w_m, w)).$$

It is straightforward to check the required facts. That is, if the two implications in Definition 6.42 are true for φ and the ψ then they are true for χ . This is clear if we replace “ $PA \vdash$ ” by “ $\mathfrak{N} \models$ ”, which establishes that definability is preserved under composition.

To establish that provability from PA is preserved, it is just a matter of checking the relevant substitution operations are provable from the logical axioms. See (Smith, 2020, p76 E2 and p73 C2).

PRIMITIVE RECURSION This defines the function $f : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$ ($n \geq 0$) from g and h by

$$\begin{aligned} f(x_1, \dots, x_n, 0) &= g(x_1, \dots, x_n), \\ f(x_1, \dots, x_n, y) &= h(x_1, \dots, x_n, y, f(x_1, \dots, x_n, y-1)) \quad (\text{for all } y \geq 1). \end{aligned} \quad \text{eq def pr (6.34)}$$

For notational simplicity we often write $\bar{x}$ for the sequence $x_1, \dots, x_n$.

It follows from (6.34) that for arbitrary y ,

$$f(x_1, \dots, x_n, y) = z$$

iff

$$\begin{aligned} \exists y \exists (\text{a sequence } z_0, z_1, \dots, z_y \text{ of length } y) \text{ such that} \\ z_0 = g(\bar{x}), \quad 1 \leq i \leq y \implies z_i = h(\bar{x}, i, z_{i-1}). \end{aligned} \quad \text{eq:sequence-def (6.35)}$$

At this stage we need to show PA is sufficiently strong to state and prove results about finite sequences of *arbitrary* length.⁵

We continue the proof following the next two theorems. □

⁴ Once again we are abusing notation. In “ $x \mapsto S(x)$, $x \mapsto 0$ and $(x_1, \dots, x_n) \mapsto x_m$ ” we are treating $x, x_1, \dots, x_n, x_m$ as names for arbitrary numbers.

In “ $y = S(x)$, $y = 0$ and $y = x_m$ ” we are treating “ y, x, x_m ” as names of variables (think of them just as variables) in the language $\mathcal{L}$.

⁵ In exp We can readily prove results from PA about addition $m + n$ and multiplication $m \cdot n$, but not yet about exponentiation, that is arbitrary powers m^n . If we did have exponentiation we could use the standard prime decomposition to uniquely represent an arbitrary sequence $z_0, z_1, \dots, z_q$ by the numeral

$$2^{z_0+1} \cdot 3^{z_1+1} \cdot \dots \cdot \pi_n^{z_n+1} \cdot \dots \cdot \pi_q^{z_q+1},$$

where π_j is the j -th prime, with 2 being prime number 0.

Even without exponentiation and its properties, we will see that we can represent arbitrary finite sequences of numerals by means of Gödel’s β -function, which relies on a version of the Chinese Remainder Theorem.

Remark 6.64 (Notation). Write

$$\text{Rem}(a, b) \text{ or } a \bmod b \quad (\in [0, b)) \quad (6.36)$$

for the remainder after dividing a by b , or equivalently for a modulo b . $\square$

Exercise 6.65. Show, and note, that $z = \text{Rem}(x, y)$ is definable in $\mathfrak{N}$ by a Δ_0 formula, that is by an $\mathcal{L}$ -formula using only *bounded* quantifiers.⁶ $\square$

Theorem 6.66 (Chinese Remainder Theorem). *Suppose $m_0, \dots, m_n$ are relatively prime. Then for any sequence of natural numbers $k_0, \dots, k_n$ with $k_i < m_i$, there is a c such that $k_i = \text{Rem}(c, m_i)$.*

Proof. Let

$$M_i = \prod_{j \neq i} m_j.$$

Then M_i and m_i are relatively prime, so there exist b_i such that

$$b_i M_i = 1 \bmod m_i.$$

Let

$$c = \sum_i k_i b_i M_i$$

Then

$$c = k_i b_i M_i \bmod m_i,$$

since $m_i \mid M_j$ if $j \neq i$.

Hence

$$c = k_i \bmod m_i.$$

Since $k_i < m_i$ this means $k_i = \text{Rem}(c, m_i)$. $\square$

Theorem 6.67 (Gödel's β -function). *Let*

$$\beta(u, v, i) = \text{Rem}(u, (1+i)v + 1) = u \bmod ((1+i)v + 1).$$

Then for any n and any sequence $k_0, \dots, k_n$ of natural numbers, there exist c and d (which depend on n and $k_0, \dots, k_n$ and can effectively be found) such that $\beta(c, d, i) = k_i$ for $0 \leq i \leq n$.

The function β is representable in PA. (It is Δ_0 .)

Proof. Consider any sequence $k_0, \dots, k_n$.

Let

$$K = \max\{n+1, k_0, \dots, k_n\}, \quad d = K!$$

Let

$$m_i = d(i+1) + 1.$$

Then the m_i are relatively prime.

For if not assume p is a prime, $p \mid m_i$, $p \mid m_j$ and $i < j$.

Then $p \mid (m_j - m_i)$ and so $p \mid (j-i)d$. But $j-i$ is a factor of d and so $p \mid d$.

⁶See Section 6.4.5 for a precise definition of Δ_0 formulas.

But then p leaves a remainder of 1 when dividing m_i and m_j , which contradicts our assumption.

So the m_i are relatively prime, and certainly $k_i < m_i$.

Now we can apply the Chinese Remainder Theorem to obtain c (explicitly). $\square$

Continuation of Proof (of Theorem 6.63).

Recall that (6.35) was

$$\exists y \exists (\text{a sequence } z_0, z_1, \dots, z_y \text{ of length } y) \text{ such that} \\ z_0 = g(\bar{x}), \quad 1 \leq i \leq y \implies z_i = h(\bar{x}, i, z_{i-1}).$$

We needed to show that this can be represented by a Σ_1 formula, given that

$$z_0 = g(\bar{x}), \quad 1 \leq i \leq y \implies z_i = h(\bar{x}, i, z_{i-1}).$$

can be represented in this manner (essentially the induction hypothesis).

But now we have Gödel's β -function, which is Δ_0 . It follows that (6.35) can be represented by

$$\exists y \exists u \exists v \left[\beta(u, v, 0) = g(\bar{x}) \wedge 1 \leq i \leq y \implies \beta(u, v, i) = h(\bar{x}, i, \beta(u, v, i-1)) \right].$$

This is the required Σ_1 -formula. $\square$

Corollary 6.68 (Representing a P.R. relation by a Σ_1 -Formula). ^{cor prrep} *If $A \subseteq \mathbb{N}^k$ is primitive recursive, then A is both definable (in $\mathfrak{N}$ in the language $\mathcal{L}$) and representable in PA , by a Σ_1 -formula.*

Proof. If A is p.r. then so is its characteristic function χ_A . If χ_A is represented by the Σ_1 -formula $\varphi(x_1, \dots, x_n, y)$ then A is represented by $\varphi(x_1, \dots, x_n, 1)$. $\square$

[This chapter to be completed]

Chapter 7

Gödel's Incompleteness Theorem

I realized, of course, that Gödel's work is of fundamental importance, but I was puzzled by it. It made me glad that I was no longer working at mathematical logic.¹

Bertrand Russell

For a summary of the important and surprising consequences of this theorem see Section 7.3.

7.1 GÖDEL CODING AND ARITHMETIC REPRESENTATION

7.1.1 CODING FORMULAS – THE IDEA

If θ is a formula of $\mathcal{L}$, the language of arithmetic, its *Gödel code* will be a number $\ulcorner \theta \urcorner \in \mathbb{N}$. The *code map* $\theta \mapsto \ulcorner \theta \urcorner$ is injective.

The code map is *effective/constructive/algorithmic/mechanical*. Given an $\mathcal{L}$ -formula θ (as a string of symbols) one can effectively find the corresponding natural number $\ulcorner \theta \urcorner \in \mathbb{N}$.

Conversely, given $n \in \mathbb{N}$ one can effectively decide if $n = \ulcorner \theta \urcorner$ for some formula θ , and in this case effectively find θ .

Appropriate software programs go in either direction, and while lengthy and tedious, could reasonably be written down (and probably have been).

In a similar manner we can effectively list all formulas with one free variable, v say:

$$\varphi_0(v), \varphi_1(v), \dots, \varphi_n(v), \dots \quad \text{eq list (7.1)}$$

We use this in the proof of Gödel's Incompleteness Theorem 7.2.

The point to all this is that we can code up and represent in PA the notions of proof and provability. This will lead to the proof of Gödel's Incompleteness Theorem 7.2.

7.1.2 CODING FORMULAS – SOME DETAILS

To each term or formula in the language $\mathcal{L}$ of number theory assign a unique integer in an effective way as follows.

¹Russell's letter to L. Henkin of April 1 1963, cited in Francisco Rodríguez-Consuegra, *Russell, Gödel, Tarski: Selected Papers on the Philosophy of Logic and Mathematics*, Studies in Logic, p. 102.

Coding Symbols: Assign to the variable v_i the integer $2i + 1$. Thus

$$v_0 \mapsto 1, v_1 \mapsto 3, v_2 \mapsto 5, \dots$$

For the other symbols of $\mathcal{L}$,

$$\neg \mapsto 2, \wedge \mapsto 4, \vee \mapsto 6, \rightarrow \mapsto 8, (\mapsto 10,) \mapsto 12, \\ \forall \mapsto 14, \exists \mapsto 16, = \mapsto 18, S \mapsto 20, + \mapsto 22, \cdot \mapsto 24, 0 \mapsto 26.$$

Let $a^\#$ be the integer assigned to the symbol a .

Coding Formulas: To the string of symbols $a_1 \dots a_n$ assign the *gödel code/number*

$$\ulcorner a_1 \dots a_n \urcorner := p_1^{a_1^\#} \cdot \dots \cdot p_n^{a_n^\#}, \quad \text{eq gcf (7.2)}$$

where p_i is the i^{th} prime.

For example, to the formula $\exists v_0 (v_0 = 2)$ which is an abbreviation of $\exists v_0 (v_0 = SS0)$, assign

$$\ulcorner \exists v_0 (v_0 = 2) \urcorner := 2^{16} \cdot 3^1 \cdot 5^{10} \cdot 7^1 \cdot 11^{18} \cdot 13^{20} \cdot 17^{20} \cdot 19^{26} \cdot 23^{12}.$$

Given a string of symbols, in particular a term or a formula, we can effectively find the associated gödel code. Conversely, given an integer, we can effectively find whether or not it is the gödel code of a string of symbols, and if so we can effectively write out this string.²

7.1.3 PRIMITIVE RECURSIVE SYNTACTIC RELATIONS AND CONSTRUCTIONS

The usual sets of formulas (atomic formulas, sentences, etc.) and operations on formulas (forming the conjunction of two formulas etc.), after coding, are clearly computable.

Moreover, they are *primitive recursive as their validity does not require unbounded search*.

For example, p is a prime iff $\neg \exists x < p (x \neq 1 \wedge x \mid p)$, and $x \mid p$ iff $\exists u \leq p (u \cdot x = p)$. So only bounded search is required and so the set of primes is primitive recursive.

There are also *primitive recursive functions* $\ell(x)$, $k(i, x)$, which assign to any code

$$x = \ulcorner a_1 \dots a_n \urcorner = p_1^{a_1} \cdot \dots \cdot p_n^{a_n}$$

the integer n (“length of x ”), and to (i, x) the integer a_i (“ i^{th} member of x ”) — essentially because these functions can be defined using only bounded search.

One can then check that the set of gödel codes of terms, of atomic formulas, of all formulas, of formulas with at most v free, of sentences, of PA axioms etc., are *primitive recursive*.

Likewise the relations C, N, E_n , such that

$$\begin{aligned} C(a, b, c) \text{ iff } a = \ulcorner \varphi \urcorner, b = \ulcorner \psi \urcorner, c = \ulcorner \varphi \wedge \psi \urcorner \text{ for some formulas } \varphi, \psi, \\ N(a, b) \text{ iff } a = \ulcorner \varphi \urcorner, b = \ulcorner \neg \varphi \urcorner \text{ for some formula } \varphi, \\ E_n(a, b) \text{ iff } a = \ulcorner \varphi \urcorner, b = \ulcorner \exists v_n \varphi \urcorner \text{ for some formula } \varphi, \end{aligned}$$

²We are working here in the metatheory, not in the formal language of arithmetic. We are not concerned here with what can be formalised *within* PA , as we were previously in Section 6.4.8 footnote 5, for example. In particular, we can here use elementary properties of primes and exponentiation.

are all *primitive recursive*.

Also, the relation Sub defined by

$$\text{Sub}(x, k, y) \text{ iff } x = \ulcorner \varphi \urcorner, y = \ulcorner \psi \urcorner, \quad \text{eq_sub (7.3)}$$

and ψ is obtained from φ by substituting k for each free occurrence of v in φ , is *primitive recursive*.

There is similarly a *primitive recursive* function f whose range is the set of gödel codes of formulas with at most v free. See (7.1)

That is,

$$f(n) = \ulcorner \varphi_n(v) \urcorner. \quad \text{eq_lst (7.4)}$$

Composing the primitive recursive functions in (7.3) and (7.4), the binary function $f(n, m)$ defined by

$$f(n, m) := \ulcorner \varphi_n(m) \urcorner, \quad (7.5)$$

is also *primitive recursive*.

7.1.4 CODING PROOFS

Just as we coded formulas as integers, so we can code finite sequences of formulas, and in particular formal PA proofs, as integers. sec cp

If p is a proof from PA , then p is a finite sequence of formulas $\varphi_1, \dots, \varphi_k$. Analogously to (7.2), we code p as the integer $[p]$, where

$$[p] := p_1^{\ulcorner \varphi_1 \urcorner} \cdot \dots \cdot p_k^{\ulcorner \varphi_k \urcorner}. \quad (7.6)$$

Conversely, given a natural number y one can (in principle) effectively obtain the corresponding proof, denoted by $\lfloor y \rfloor$.

Formula coding uses “ $\ulcorner \dots \urcorner$ ” and “ $\lfloor \dots \rfloor$ ” while proof coding uses “ $[\dots]$ ” and “ $\lfloor \dots \rfloor$ ”. This is no big deal, we could also use even integers and odd integers for formulas and proofs respectively, or just rely on context!

A particularly significant for us is the relation $\text{Prf}(x, y)$ defined by

$$\text{Prf}(x, y) \text{ iff } x = \ulcorner \varphi \urcorner, y = [p], p \text{ is a } PA \text{ proof of } \varphi. \quad (7.7)$$

It is *primitive recursive* by similar arguments to before.

7.1.5 ARITHMETIC REPRESENTATION

The relations discussed in Section 7.1.3 and the relation Prf in Section 7.1.4 are all primitive recursive.

Hence by Theorem 6.63 and Definition 6.38 they can be represented in PA by Σ_1 -formulas.

In particular, there is a Σ_1 -formula $\text{Prf}(x, y)$ (which represents Prf) such that for $a, b \in \mathbb{N}$,

$$\begin{aligned} \text{Prf}(a, b) &\implies PA \vdash \text{Prf}(a, b), \\ \neg \text{Prf}(a, b) &\implies PA \vdash \neg \text{Prf}(a, b). \end{aligned} \quad \text{eq_Prf (7.8)}$$

Similarly, the ternary proof relation $\text{Prf}(u, v, y)$, defined by

$$\text{Prf}(u, v, y) \quad \text{iff} \quad y = \lceil p \rceil, \text{ } p \text{ is a } PA \text{ proof of } \varphi_u(v), \quad (7.9)$$

is primitive recursive. It follows that we can effectively exhibit a Σ_1 -formula $\text{Prf}(u, v, w)$ in the language $\mathcal{L}$ of number theory, such that

$$\begin{aligned} \text{Prf}(m, n, y) &\implies PA \vdash \text{Prf}(m, n, y), \\ \neg \text{Prf}(m, n, y) &\implies PA \vdash \neg \text{Prf}(m, n, y). \end{aligned} \quad \begin{array}{l} \text{eq_PA1} \\ (7.10) \end{array}$$

(As usual, the formal PA proofs can be written out by mimicking the calculation of the primitive recursive characteristic function of Prf , which comes in turn from its given primitive recursive definition.)

7.1.6 PROVABILITY

Consider the set $\text{Prov} \subseteq \mathbb{N}$ of those $a \in \mathbb{N}$ for which there is a proof of the sentence corresponding to a . That is, sec prov

$$a \in \text{Prov} \text{ (that is, } \text{Prov}(a)) \iff \exists b \text{ such that } \text{Prf}(a, b). \quad \begin{array}{l} \text{eq_prov} \\ (7.11) \end{array}$$

(Note that this is a statement in the metalanguage.) “Prov” meaning “provable”.

Because of the existential quantifier “ $\exists$ ”, the set Prov is *not* primitive recursive, nor is it computable in the broader sense of Definition 6.21.

However, Prov is *computably enumerable* in the following sense: we can write a program which will enumerate all possible proofs, and hence enumerate all sentences which are provable

But one cannot know of an arbitrary sentence θ , how long it will take to be listed if it is indeed provable. As we will see later, for some θ it will be the case that both θ and its negation $\neg\theta$ will not be provable from PA . There is then no algorithmic procedure which will eventually give a “provability” answer for every input $\lceil \theta \rceil$.

Following the definition of Prov in (7.11) and introduction of the $\mathcal{L}$ -formula $\text{Prf}(x, y)$ in (7.8), we *define the $\mathcal{L}$ -formula $\text{Prov}(x)$* by

$$\text{Prov}(x) := \exists y \text{Prf}(x, y) \quad \begin{array}{l} \text{eq_prov2} \\ (7.12) \end{array}$$

The formula $\text{Prov}(x)$ is Σ_1 , although it does not correspond to a primitive computable (or even a total computable) function. It asserts there is a PA proof of the sentence with Gödel number x .

Similarly and importantly, analogously to (7.10) and (7.11), define for $m, n \in \mathbb{N}$

$$\text{Prov}(m, n) \iff \exists b \text{ such that } \text{Prf}(m, n, b), \quad (7.13)$$

which is the provability relation meaning, in this case, that there is a formal proof from PA of $\varphi_m(n)$.

Then there is a Σ_1 -formula $\text{Prov}(u, v)$ defined by

$$\text{Prov}(u, v) := \exists y \text{Prf}(u, v, y). \quad (7.14)$$

This is the formula in the language of arithmetic corresponding to “there is a PA proof of $\varphi_u(v)$ ”. More explicitly it corresponds to “there is a PA proof coded up by y of $\varphi_u(v)$ ”.

Exercise 7.1. Discuss and explain the paragraph beginning with “However, one cannot know of an arbitrary sentence θ , ...” □

7.2 GÖDEL'S FIRST INCOMPLETENESS THEOREM

We will now, finally, explicitly construct a sentence G which is true in the standard model $\mathfrak{N}$ (i.e. $\mathfrak{N} \models G$), but $PA \not\vdash G$ and $PA \not\vdash \neg G$. sec gfit

See Table 7.1 to follow the diagonal argument.

$\varphi_0(v)$	$\varphi_0(0)$	$\varphi_0(1)$	$\varphi_0(2)$	$\cdots$	$\varphi_0(k)$	$\cdots$
$\varphi_1(v)$	$\varphi_1(0)$	$\varphi_1(1)$	$\varphi_1(2)$	$\cdots$	$\varphi_1(k)$	$\cdots$
$\varphi_2(v)$	$\varphi_2(0)$	$\varphi_2(1)$	$\varphi_2(2)$	$\cdots$	$\varphi_2(k)$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\vdots$
$\Phi(v) \equiv \varphi_k(v)$	$\varphi_k(0)$	$\varphi_k(1)$	$\varphi_k(2)$	$\cdots$	$\varphi_k(k)$	$\cdots$
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\ddots$

Table 7.1: The diagonal argument for Theorem 7.2.

tab form coding
thm GIT

Theorem 7.2 (Gödel's Incompleteness Theorem). *There is a Π_1 -sentence G , which we can explicitly exhibit, such that $\mathfrak{N} \models G$, $PA \not\vdash G$ and $PA \not\vdash \neg G$.*

Proof. Let

$$\Phi(v) := \forall y \neg \text{Prf}(v, v, y),$$

which is logically equivalent to $\neg \exists y \text{Prf}(v, v, y)$, that is to $\neg \text{Prov}(v, v)$.

(So $\Phi(v)$ asserts “there is no formal PA proof of $\varphi_v(v)$ ”.)

We can (in principle) explicitly write down the formula $\Phi(v)$ and explicitly find $k \in \mathbb{N}$ such that

$$\varphi_k(v) \equiv \Phi(v). \quad (7.15)$$

The Gödel sentence is

$$G := \varphi_k(k) \equiv \forall y \neg \text{Prf}(k, k, y) \equiv \neg \exists y \text{Prf}(k, k, y) \equiv \neg \text{Prov}(k, k), \quad (7.16) \quad \text{GS state}$$

where “ $\equiv$ ” means is logically equivalent, and in particular is provable from PA .

Motivation (Going across the yellow row) for each n , $\varphi_k(n)$ says:

(going down the blue diagonal) $\varphi_n(n)$ is not provable.

So (at the green intersection point) $\varphi_k(k)$ says $\varphi_k(k)$ is not provable.

We now claim it follows, assuming PA is sound,

$$\mathfrak{N} \models G, \quad PA \not\vdash G, \quad PA \not\vdash \neg G. \quad (7.17)$$

For if $PA \vdash G$ then $PA \vdash \neg \exists y \text{Prf}(k, k, y)$.

But also if $PA \vdash G$ then there is a formal proof of G with Gödel code k say, and so $PA \vdash \text{Prf}(n, n, k)$.

Contradiction.

So $PA \not\vdash G$.

But $PA \not\vdash G$ means there is no formal proof of G , and so by (7.16) G is indeed true in the standard model $\mathfrak{N}$, that is $\mathfrak{N} \models G$

Finally, if $PA \vdash \neg G$ then $\mathfrak{N} \models \neg G$, which is a contradiction. □

7.3 SUMMARY

sec es

7.3.1 OVERVIEW OF INCOMPLETENESS PROOF

The proof of the Incompleteness Theorem involved the following major steps

1. [Section 6.3] Developing basic properties of the primitive recursive [p.r.] sets and functions. This used
 - (i) the “bounded search” characterisation (for showing various examples are p.r.),
 - (ii) the definition/characterisation from initial functions, composition and recursion (for later showing the definability and representability in arithmetic),
 and established the equivalence of the two.
2. [Section 6.4] Proving that p.r. relations and functions can be defined in $\mathfrak{N}$ and represented in PA by Σ_1 -formulas. This used the second definition of p.r. and required using Gödel’s β -function.
3. [Section 7.1] Coding up the notions of formula, proof and provability as p.r. sets of integers. Defining and representing these with Σ_1 -formulas.
4. [Section 7.2] Proof of Gödel’s First Incompleteness Theorem.

7.3.2 INCOMPLETENESS FOR EXTENSIONS OF PA

The Incompleteness Theorem applies not only to the Peano Axioms PA but to any extension of PA , provided the extension is a primitive recursive set of axioms after coding the axioms as integers.

Primitive Recursiveness is not particularly restrictive, because if we could not decide by means of a bounded search whether or not a sentence is an axiom, then the set of axioms would be of little practical significance. All axiomatic systems in mathematics satisfy this criteria.³

So essentially the same proof gives:

thm ext GCT

Theorem 7.3. *If T is a sound primitive recursive axiomatized theory containing PA , then there will be a true Π_1 -sentence G_T such that $T \not\vdash G_T$ and $T \not\vdash \neg G_T$.*

While the sentence G is complicated, in the arithmetic hierarchy it is in fact quite simple, just Π_1 !

7.3.3 NON SELF-REFERENTIAL EXAMPLES

Set Theory Theorem 7.3 applies to the ZF axioms in set theory.

But the most interesting examples of sentences φ in set theory such that neither φ nor $\neg\varphi$ follow from the ZF axioms, are the axiom of choice and the continuum hypothesis.

These results were established by Gödel in 1939 and Paul Cohen in 1962. The latter involved the completely new idea of “forcing” and has been extraordinarily influential since then. See (Jech, 2003, Chapters 13, 14).

³In fact, the results do still apply for computable sets of axioms in the sense of “total computable” in Definition 6.21, which allows unbounded search.

New Examples in PA There are now also interesting independence results in arithmetic which are independent of PA and are not self-referential: the *Paris–Harrington theorem* on a version of Ramsey’s theorem in combinatorics, *Goodstein’s theorem* and the related *Kirby–Paris Hydra Game*. See also *epsilon number* for the ϵ_0 -connection to finite trees and Gentzen’s consistency result.

See (Marker, 2024, Chapters 15, 16).

7.3.4 SYNTACTIC VERSIONS OF THE INCOMPLETENESS THEOREM

There is a version of the Incompleteness Theorem which does not involve truth in $\mathfrak{N}$ but requires that T be ω -consistent, a stronger requirement than consistency but very natural.

Specifically, T is ω -consistent if

$$T \vdash \psi(m) \text{ for all } m \in \mathbb{N} \implies T \not\vdash \neg \forall m \psi(m).$$

The proof is only a small variation of the previous proof.

There is a version of the sentence G due to Rosser that does not require ω -consistency to establish its independence from PA .

See (Leary and Kristiansen, 2015; Hils and Loeser, 2019; Smith, 2007; 2020), for example.

7.3.5 GÖDEL’S SECOND INCOMPLETENESS THEOREM

It is possible to formalise the proof of the first half of the Incompleteness Theorem as a proof in PA to obtain

$$PA \vdash \neg Prov(\perp) \rightarrow G, \text{ written } PA \vdash Con(PA) \rightarrow G \quad \text{eq goe 2 (7.18)}$$

Here $\perp$ is an “absurdity” such as $0 = 1$ or $\varphi \wedge \neg\varphi$ for some sentence φ . $Prov$ is the provability formula (7.11). G is the Gödel sentence. $Con(PA) := \neg Prov(\perp)$ is a sentence interpreted as consistency of PA .

This is far from straightforward, see (Smith, 2020, Chapters 20,21).

From (7.18), it follows that

$$PA \not\vdash Con(PA)$$

since if $PA \vdash Con(PA)$ then $PA \vdash G$, contradicting $PA \not\vdash G$ by Gödel’s first theorem.

Restating this, for an arbitrary theory T , by essentially the same argument:

Theorem 7.4 (Second Incompleteness Theorem). *If T is a consistent primitive recursive axiomatized theory containing PA , then*

$$T \not\vdash Con(T). \quad (7.19)$$

That is, T is not strong enough to prove its own consistency! Note this also applies to any axioms for set theory!

7.3.6 FOR CONTEMPLATION

The following is essentially verbatim from (Marker, 2015, p71).

The following three problems can be considered the basic problems in the foundations of mathematics.

1) **True imply Provable?** Does every mathematical truth about the natural numbers have a meaningful finitistic proof? While it is arguable what a “finitistic proof” is, one precise way of saying this is that there is a natural set of axioms from which we can derive all truths about the natural numbers. For example Peano Arithmetic (PA) would be a good candidate. Is every sentence that is true in $\mathfrak{N}$ provable in PA ?

2) **Hilbert’s Conservation Program:** If a mathematical truth can be proved by strong methods (say using set theoretic methods), then it can be proved by finitistic methods.

3) **Hilbert’s Consistency Program:** We should be able to give a finitistic proof of the consistency of our methods.

Gödel showed that all of this is impossible in the following.

Theorem 7.5 (First Incompleteness Theorem). *There is a sentence φ such that $\mathfrak{N} \models \varphi$ and $PA \not\models \varphi$. Indeed, if T is a recursive theory such that $T \supseteq PA$ and $\mathfrak{N} \models T$ then there is a sentence φ such that $T \not\models \varphi$ and $T \not\models \neg\varphi$.*

Theorem 7.6 (Second Incompleteness Theorem). *Let T be as above. Then T does not prove the consistency of T .*

The First Incompleteness theorem shows that (1) will fail.

The Second Incompleteness theorem shows that (3) fails.

Since using set theory we can show that Peano Arithmetic is consistent, (2) shows that the conservation program fails as well.

7.3.7 MORE CONTEMPLATION

(Ebbinghaus et al., 2013, Chapter VII, Exercise 4.4)

A reader who has been confused by the discussion of this chapter says, “Now I’m completely mixed up. How can ZFC be used as a basis for first-order logic, while first-order logic was actually needed in order to build up ZFC?” Help such a reader out of his dilemma.

Hint: Again, be careful in distinguishing between the object and the background level.

[This chapter to be completed]

Chapter 8

Computability

ch computability

8.1 SUMMARY OF COMPUTABILITY SECTION 6.3

In Section 6.3 we developed the theory of primitive recursive [p.r.] functions $f : \mathbb{N}^k \rightarrow \mathbb{N}$ and of primitive recursive relations $R \subseteq \mathbb{N}^k$, to an extent sufficient to apply to the proof of Gödel's Incompleteness Theorem.

We gave two approaches and established their equivalence.

Definition 6.20: The first approach identified p.r. functions and relations as those which can be obtained from a software program using only bounded search. The bound should be known before entering the loop, and not computed within the loop.

Definition 6.25 and 6.26: The second more precise approach defined p.r. functions as those functions built up from some very simple initial functions by means of composition and definition by recursion.

A relation is defined to be p.r. if its characteristic function is p.r.

Theorem 6.30 used the second definition to give standard and general examples: p.r. functions $x + y$, $x \cdot y$, x^y , $x!$, $x \dot{-} y$, constants; p.r. relations $<$, $\leq$, $>$, $\geq$, $=$; closure property under complement, intersection, union, bounded quantification, definition by cases.

We also noted briefly that there are larger classes of computable functions and relations:

primitive recursive $\subset$ (total) computable $\subset$ partial computable. term comp2 (8.1)

8.2 COMPUTABILITY OVERVIEW

For a more detailed overview see the first few chapters of (Soare, 2016) and sec comp over (Rogers, 1987).

Terminology See Table 8.1

We also use “relation” for subsets of $\mathbb{R}^k$ and use “algorithmic” for a computing procedure.

Various Equivalent Approaches There have been many attempts to delineate precisely the notion of a partial computable function. All these attempts, diverse as they may seem at first, lead to the same class of functions, the partial computable functions.

Current terminology	Old terminology
partial computable function	partial recursive function
computable function	recursive function
decidable set	recursive set
computably enumerable set	recursively enumerable set

Table 8.1: Terminology for computability notions.

tab comp term

In particular there is the approach via Turing machines, the approach due to Kleene and others as in Sections 6.3.4 and 8.4.2, the λ -calculus due to Church, and the class of functions that can be computed by any current programming language!

Partial Computable Functions These are those functions that can be computed by a software program that allows unbounded search. In this case such a program may not terminate for some inputs, and in this case the output is considered to be undefined.

We will later give examples of partial computable functions f for which there is *no* program that can say of each input $a_1, \dots, a_k$ what is the output $f(a_1, \dots, a_k)$ if it exists, and gives an output such as “will not converge” otherwise.

Remark 8.1 (Why “partial”?). We use the word “partial” to indicate that the algorithm for the function f may not terminate for all inputs, or in other words $f(a_1, \dots, a_k)$ is only defined for *some* inputs $(a_1, \dots, a_k)$. $\square$

Remark 8.2 ($\mathbb{N}^m$ -valued functions). We do not usually consider computability issues for functions $f : \mathbb{N}^k \rightarrow \mathbb{N}^m$ with $m > 1$, since this reduces to computability questions for the individual component functions. $\square$

Decidable and Computably Enumerable Sets We next define the notions of a *decidable* (sometimes called *computable*) subset of $\mathbb{N}^k$ and of a *computably enumerable* [c.e.] subset of $\mathbb{N}^k$.

A set $A \subseteq \mathbb{N}^k$ is *decidable* if its characteristic function $\chi_A : \mathbb{N}^k \rightarrow \mathbb{N}$ is computable. The terminology “decidable” corresponds to the fact that there is a decision procedure for deciding of each $(a_1, \dots, a_k)$ if $(a_1, \dots, a_k) \in A$ or $(a_1, \dots, a_k) \notin A$.

A set $A \subseteq \mathbb{N}^k$ is *computably enumerable* if $A = \emptyset$ or A is the range of a computable function $f : \mathbb{N} \rightarrow \mathbb{N}^k$.

Theorem 8.3. *If $A \subseteq \mathbb{N}^k$ is decidable then A is c.e.*

Proof. Enumerate all k -tuples. For each k -tuple $(a_1, \dots, a_k)$, it is computable whether or not $(a_1, \dots, a_k) \in A$. If $(a_1, \dots, a_k) \in A$ add it to the list. In either case proceed to the next k -tuple, etc.

This gives a c.e. listing of A . $\square$

Remark 8.4 (C.E. non-Decidable Sets). There are c.e. sets that are not decidable, see Theorem 8.12. An example is the set of consequences of the *PA* axioms!

But we do have the following tight relationship between the two concepts. $\square$

Theorem 8.5. $A \subseteq \mathbb{N}^k$ is decidable iff A and $\mathbb{N}^k \setminus A$ are both c.e.

Proof. If A is decidable then so is A^c (Why?). So by the previous theorem, A and A^c are both c.e.

Conversely, if A and A^c are both c.e. then we have a computable “zig-zag” procedure to decide if $(a_1, \dots, a_k) \in A$:

Run the first program for enumerating A for 10 steps, then the second program for enumerating A^c for 10 steps. Then back to the first, then the second, etc. etc.

Eventually one of the two programs will output $(a_1, \dots, a_k)$, thereby allowing us to conclude $(a_1, \dots, a_k) \in A$ or $(a_1, \dots, a_k) \notin A$. $\square$

Remark 8.6. Yes, this is a ridiculously inefficient procedure. But we cannot do better in general. $\square$

8.3 TURING MACHINES

A Working Turing machine

8.3.1 WHY TURING MACHINES?

We use Turing machines to investigate the nature of computability. No one would normally calculate with them.

But for fun, see the previous link.

All contemporary computing devices are equivalent to a Turing machine, provided it has a sufficiently long tape (that is, memory). We are not going to verify that all the usual arithmetical operations, addition, exponentiation etc. can be performed on some appropriate Turing machine. That would be excessively tedious.

Turing gives a very good explanation in his paper (Turing, 1936) as to why any algorithmic procedure has an equivalent Turing machine procedure.

8.3.2 DESCRIPTION

Definition 8.7 (Turing Machine). A Turing Machine consists of a linear tape, potentially infinite in both directions, divided into cells, with each cell containing a 1 or blank $\emptyset$, and at most a finite number of cells not blank.

There is a reading head which at time t ($t = 1, 2, 3, \dots$) is in one of a finite set $Q = \{q_0, \dots, q_n\}$ of possible states and is reading a particular cell.

Depending on this state q_i , and the cell content m_j ($= 1$ or $\emptyset$), and the fixed set of instructions initially supplied with the machine, the machine either (i) stops or (ii) changes to state q_k and moves the whole tape one cell to the left or one cell to the right or puts 1 into the cell it is reading or puts $\emptyset$ into the cell it is reading.

Turing Machine Instructions are given by a finite set of quadruples $q_i m_j O q_k$ where $q_i, q_k \in Q = \{q_0, \dots, q_n\}$, $m_j \in \{1, \emptyset\}$, $O \in \{1, \emptyset, L, R\}$.

Here Q is the set of possible states, which may vary from machine to machine.

A consistency condition is imposed; no two distinct quadruples can have the same first two entries.

Turing Machine in Operation The machine starts in state q_0 , the tape initially has a finite string of consecutive 1's, and the machine is reading the leftmost 1.

At time t suppose the machine is in state q_i reading a cell containing m_j ($= 1$ or $\emptyset$). If no quadruple $q_i m_j **$ is among the instructions the machine halts. If $q_i m_j o q_k$ is among the instructions, then the machine moves into state q_k and according as $O = L, R, 1$, or $\emptyset$ respectively, the tape is moved one unit left, or one unit right, or a 1 is placed in the cell being read, or the cell being read is made blank.

Thus at time $t + 1$ the machine is now in state q_k and the machine is reading the cell one unit to the right of previously, or one unit to the left, or the same cell now with a 1 in it, or the same cell now blank.

Turing Machine Output The machine proceeds in this way, and for each input either eventually stops or proceeds indefinitely.

If it stops, we take the output to be the leftmost block of 1's (this is somewhat arbitrary, it is convenient if our inputs and outputs are finite strings of consecutive 1's).

If it never stops, then, well, it never stops.

8.3.3 PARTIAL COMPUTABLE FUNCTIONS AND TURING MACHINES

We associate to the Turing machine, in the obvious way, a *partial* function $f : \mathbb{N} \rightarrow \mathbb{N}$. Such partial functions are called partial computable functions (a better terminology might be computable partial function!).

We can Turing compute the partial computable functions from $\mathbb{N}^m \rightarrow \mathbb{N}^n$ by taking inputs as m -strings of 1's each separated by a $\emptyset$, and output as the first such n strings of 1's.

Alternatively, m -tuples and n -tuples can themselves be coded as integers in some algorithmic way, and we thus identify partial functions from $\mathbb{N}^m \rightarrow \mathbb{N}^n$ with partial functions from $\mathbb{N} \rightarrow \mathbb{N}$. Either way gives the same set of partial computable (and hence computable) functions from $\mathbb{N}^m \rightarrow \mathbb{N}^n$.

8.3.4 TOTAL COMPUTABLE FUNCTIONS AND TURING MACHINES

The set of such *total* functions (that is, from machines which eventually stop on every input) is the set of (total) computable functions.

Proposition 8.8. *There is no algorithm to computably enumerate those Turing Machines which give the total computable functions.*

Proof. For suppose we had a c.e. list of precisely those instruction sets which gave an output for every input.

Let f_n be the function corresponding to the n th instructions set and define $g(n) = f_n(n) + 1$.

This gives an algorithmic function not in our list, and one for which we could write down, in principle, a set of instructions.

Contradiction. □

This is yet another example of a “diagonal” argument.

8.4 COMPUTABLE FUNCTIONS — OTHER APPROACHES

8.4.1 WHILE LOOPS

In Definition 6.21 we discussed generating primitive recursive functions using bounded search.

Partial computable functions in general corresponding to allowing unbounded search or “while loops”. This also corresponds to Turing machines, as they may not terminate on some inputs.

8.4.2 KLEENE APPROACH

In Section 6.3.4 we defined the p.r. functions as those obtained from the initial functions sec KAp by using composition and primitive recursion.

The partial computable functions are obtained by adding the following μ -operator, the *least number operator*.

Definition 8.9 (μ -Operator). Let $g : \mathbb{N}^{n+1} \rightarrow \mathbb{N}$. Then $f : \mathbb{N}^n \rightarrow \mathbb{N}$,

$$f(x_1, \dots, x_n) := \mu y (g(x_1, \dots, x_n, y) = 0),$$

is defined by

$$f(x_1, \dots, x_n) = \begin{cases} y & g(x_1, \dots, x_n, u) \text{ defined and } \neq 0 \text{ if } u < y, \\ & g(x_1, \dots, x_n, y) = 0, \\ \text{undefined} & \text{otherwise} \end{cases} \quad (8.2)$$

We say f is obtained from g by the μ -operator.

Remark 8.10. It is not particularly surprising that unbounded search (allowing while loops), the Kleene approach and the Turing machine approach all give the same class of functions, the partial computable functions. In particular, the μ -operator corresponds to unbounded search and to a Turing Machine not terminating on some inputs. $\square$

8.5 CHURCH–TURING THESIS

Definition 8.11 (Church-Turing Thesis). A partial numerical function can be effectively computed by some algorithmic procedure if and only if it is partial computable.

A total numerical function can be effectively computed by some algorithmic procedure if and only if it is total computable.

The Church-Turing Thesis asserts that our various notions of “computable function” completely capture our intuitive notion of an algorithmic procedure.

This is a statement of philosophy or physics, regarding what is possible, rather than a mathematical theorem or conjecture.

We have already noted that Turing gave a careful justification in his original paper.

The fact that the many very different definitions, by Turing, Church, Kleene and Post are equivalent is very strong evidence. No computer ever built has extended the boundary of what is possible, although there may be large improvements in efficiency and speed. This includes machines involving random algorithms and quantum computers.

8.6 HALTING PROBLEM

Each Turing machine is described by a finite list of instructions, so we can list all Turing machines and write down an algorithm which gives us the n^{th} machine. sec TM HP

The next result, known as the *Halting Problem*, explicitly constructs a c.e. but non-computable $S \subseteq \mathbb{N}$.

Theorem 8.12 (A c.e. but non-decidable set). thm ndec Suppose $n \in S$ iff the n^{th} Turing machine with input n eventually halts. Then S is c.e. but not decidable.

Proof. For each n let φ_n be the partial computable function corresponding to the n^{th} Turing machine. Then

$$S = \{n : \varphi_n(n) \text{ is defined}\} = \{n : n^{\text{th}} \text{ Turing machine with input } n \text{ eventually halts}\}.$$

Showing that S is c.e. is easy. By a standard looping operation:

Compute $\varphi_1(1)$ for 10 steps;

then each of $\varphi_1(1), \varphi_2(2)$ for 10 more steps;

then each of $\varphi_1(1), \varphi_2(2), \varphi_3(3)$, for 10 more steps;

etc.

In this way we can computably enumerate S .

Assume, in order to obtain a contradiction, that S is decidable.

Define $f : \mathbb{N} \rightarrow \mathbb{N}$ by

$$f(n) = \begin{cases} 0 & n \notin S, \\ \varphi_n(n) + 1 & n \in S. \end{cases}$$

Since S is decidable and $\varphi_n(n)$ is defined for every $n \in S$, it follows $f(n)$ is defined for every $n \in \mathbb{N}$, and so f is *total* computable.

It follows that f is *computed* by some Turing machine — suppose it is the k^{th} machine. Since f is total it follows that the k^{th} machine halts on every input. In particular, $\varphi_k(k)$ is defined and so $k \in S$.

But

$$f(k) = \varphi_k(k) + 1 \neq \varphi_k(k).$$

So f is *not computed* by the k^{th} Turing machine.

Contradiction

This means the original *assumption* is false and so S is *not decidable*. □

8.7 ARITHMETIC REPRESENTATION

The result in Corollary 6.68 for p.r. relations extends to computable relations.

Theorem 8.13 (Representing a decidable relation by a Σ_1 -Formula). thm rep dec If $A \subseteq \mathbb{N}^k$ is decidable, then A is both definable (in $\mathfrak{N}$ in the language $\mathcal{L}$) and representable in PA , by a Σ_1 -formula.

Proof. (See (Hils and Loeser, 2019, p128 ff) or (Smith, 2007, §39) for details)

The proof mirrors that for p.r. relations by first considering p.r. functions. See Corollary 6.68 and Theorem 6.63.

There are two significant differences:

- In the proof of Theorem 6.63 we were concerned with representing p.r. functions. With minor variations the proof actually gave the weaker of the stronger versions of representability as noted in Remark 6.44. That is needed here.
- The new additional case concerns the μ -operator. But this is, unsurprisingly, handled in a relatively straightforward way by the $\exists$ operator. $\square$

8.8 ENTSCHEIDUNGSPROBLEM

Entscheidungsproblem (German): *Decision Problem*, or more literally “resulting in separating (cases) problem”.

Following Gödel’s Incompleteness Theorem in 1931, and the general acceptance of the claim that Turing Machines and the other approaches to computability, all capture the same informal notion of an algorithmic procedure, the following question arises:

Given a collection Σ of (p.r. say) axioms in some language $\mathcal{L}$, is there an algorithm to decide if an arbitrary sentence φ in the language $\mathcal{L}$ is a consequence?

We know that the set of formally derivable consequences is the same as the set of logical consequences (that is, true in all models of Σ), by the Completeness Theorem.

We know that the set of consequences is c.e., since we can (in principle) generate all consequences of Σ by generating all proofs of size 10, then of size 10^{10} , then $10^{10^{10}}$, etc.

So the question is whether the set of consequences is decidable, as defined and discussed in Section 8.2.

Definition 8.14. If Σ is a set of sentences in a language $\mathcal{L}$, then $\text{Cn}(\Sigma)$ is the set of *consequences* of Σ .

Theorem 8.15 (Church’s Theorem). *If $PA \subseteq \Sigma$ and Σ is a consistent set of sentences, then $\text{Cn}(\Sigma)$ is not decidable.* thm chth

Proof. (Sketch)

There is a p.r. listing such that $\varphi_n(x)$ is the n^{th} formula in the language of number theory having at most x free.

Let

$$E_n = \{m : \Sigma \vdash \varphi_n(m)\} \quad \text{for each } n \in \mathbb{N}.$$

Let

$$E = \{m : \Sigma \not\vdash \varphi_m(m)\}.$$

Note that for each n , $E \neq E_n$ since

$$n \in E \iff \Sigma \not\vdash \varphi_n(n) \iff n \notin E_n.$$

Now suppose, in order to obtain a contradiction, that $\text{Cn}(\Sigma)$ is decidable.

This implies that all of the sets E_n , and also E , are decidable — not just c.e.

From Theorem 8.13 it follows E is represented in PA by some formula $\varphi_k(x)$,¹ that is $E = E_k$ for some k .

Contradiction. Hence $\text{Cn}(\Sigma)$ is *not* decidable. $\square$

Remark 8.16 (Extending the result). The representability results do not require the full PA axioms. It is sufficient to use Robinson's Axioms, which are Axioms 1–6 in Definition 6.1 plus

$$S(x) = S(y) \rightarrow x = y.$$

It follows that $\text{Cn}(R)$ is not decidable by essentially the same argument as in Theorem 8.15. Hence that there is no decision procedure for deciding of any sentence φ in the language $L = \{S, +, \cdot, 0\}$ of arithmetic if $R \vdash \varphi$.

But this means there is no decision procedure for deciding of an arbitrary φ if $\vdash R \rightarrow \varphi$, where R is now the conjunction of the 7 Robinson axioms (after universally generalising).

And this means there is no decision procedure for deciding of an arbitrary language with one unary function symbol and two binary function symbols and one constant symbol, if its set of purely logical consequences are decidable! $\square$

Remark 8.17. (Hils and Loeser, 2019, Exercise 5.8.3 p141) outlines a proof of why the logical consequences for a language $\mathcal{L} = \{U, c\}$ with a single unary relation symbol and a single constant symbol *is* decidable.

On the other hand, for a language with a single binary symbol, the set of logical consequences is not decidable.

Reference? $\square$

8.9 (OPTIONAL) MORE ON COMPUTABLY ENUMERABLE SETS

The following two Theorems can be proved properly, but we give informal proofs. sec ccmp enum

Theorem 1. The following are equivalent for $S \subseteq \mathbb{N}^k$.

- (i) S is c.e.
- (ii) $S = \emptyset$ or S is the range of a total computable function $f : \mathbb{N} \rightarrow \mathbb{N}^k$.
- (iii) $S = \emptyset$ is the range of a partial computable function $f : \mathbb{N} \rightarrow \mathbb{N}^k$.

Proof: The informal version of (ii) is essentially our original definition of effectively enumerable. The equivalence with the informal version of (i) was established in paragraph 2. (ii) $\Rightarrow$ (iii) is trivial. For (iii) $\Rightarrow$ (i) we will be content with showing that the range of a partial computable function $f : \mathbb{N} \rightarrow \mathbb{N}^k$ is the domain of a partial computable function $g : \mathbb{N}^k \rightarrow \mathbb{N}$. We calculate $g(n_1, \dots, n_k)$ as follows. First calculate $f(1)$ for 10 steps, then $f(2)$ for 20 steps, then $f(1)$ for 20 steps, then $f(3)$ for 30 steps, then $f(2)$ for 30 steps, then $f(1)$ for 30 steps, etc. If $(n_1, \dots, n_k)$ is ever an output, let $g(n_1, \dots, n_k) = 0$. Otherwise $g(n_1, \dots, n_k)$ is undefined.

¹In fact by a Σ_1 -formula, but that is not needed here. Note also that we are not claiming this corresponds to a listing of all Turing machines.

Theorem 2. $S \subseteq \mathbb{N}^k$ is recursive iff S and $\mathbb{N}^k - S$ are r.e.

Proof: By the proposition of p. 14, Chapter IA.

[This section to be completed]

8.10 (OPTIONAL) ACKERMANN FUNCTION

8.10.1 HISTORY

The *Ackermann function* $A : \mathbb{N} \times \mathbb{N} \rightarrow \mathbb{N}$ is computable and is defined for *all* $m, n \in \mathbb{N} \times \mathbb{N}$, but it is *not* primitive recursive. It was introduced in 1928 by Ackermann, working on Hilbert’s program, in order to show the existence of a “computable” function which was not captured by the concept of primitive recursive. At that time the precise notion of a computable function was unknown.

The version of the Ackermann function which is essentially the one used here, was introduced independently by Rózsa Péter and Raphael Robinson in the early 1930s.

8.10.2 APPLICATIONS

The Ackermann function $A(m, n)$ is the beginning of the theory of rapidly growing functions.

It occurs in term rewriting systems and termination proofs within the computer science study of algorithms, and arises in the ordinal analysis of hierarchies of consistency proofs for Peano Arithmetic and set theory.

The associated unary (diagonal) function is $A(n) := A(n, n)$. As we will see it is monotonically increasing and grows faster than any primitive recursive function.

Its (functional) inverse² is an extremely slowly growing function, essentially constant in any system of units, and is used in the analysis of [disjoint-set data structures](#).

For a “light” discussion of computing $A(m, n)$ see [“The Most Difficult Program to Compute?”](#). For the program itself see [Ackermann Function](#).

8.10.3 THE SEQUENCE OF UNARY ACKERMANN FUNCTIONS

As discussed in the next two subsections, the two-variable Ackermann function $A(m, n)$ can be seen as a sequence of one-variable functions $A_m(n) := A(m, n)$ for $m, n \in \mathbb{N}$.

In Definition 8.23 we define the functions A_m independently of A , using the standard iterated function notation

$$f^{(n)}(x) := \overbrace{f \circ f \circ \cdots \circ f}^{n \text{ times apply } f}(x). \quad \text{def iter (8.3)}$$

The following definition is motivated by the Remark which follows it. See also Figure 8.1.

²The functional inverse A^{-1} is defined by

$$A^{-1}(y) = \max\{x : A(x) \leq y\}, \quad \text{that is} \quad x \leq A^{-1}(y) \iff A(x) \leq y.$$

If one computation were done every nanosecond, and since it can be shown that the number of computations needed for $A(x)$ is comparable to $A(x)$, the function A^{-1} would equal 5 from just after the big bang to far beyond the big crunch. See Figure 8.1.

Definition 8.18 (Ackermann Sequence). The sequence of functions $A_m : \mathbb{N} \rightarrow \mathbb{N}$ is given by

$$\begin{aligned}
 A_1(0) &= 2, \quad A_2(0) = 0, \quad A_m(0) = 1 \text{ if } m \geq 3 & \text{def ack11 (8.4)} \\
 A_0(n) &= n + 1 & n \geq 0 \quad \text{def ack12 (8.5)} \\
 A_m(n) &= A_{m-1}^{(n)}(A_m(0)) & m \geq 1, n \geq 1 \quad \text{def ack13 (8.6)}
 \end{aligned}$$

Remark 8.19 (Notes on the Definition).

- A_0 is the successor function.
- The conditions in (8.4) and (8.5) can be regarded as “boundary” conditions, see Figure 8.1.
- The values in (8.4) are chosen here so that A_1 , A_2 and A_3 agree with addition, multiplication and exponentiation from the base 2 — see the following comments for these three functions.³
- For $m \geq 1$ and $n \geq 1$, $A_m(n)$ is defined by iterating n times the function A_{m-1} applied to $A_m(0)$.
- In particular, for $n \geq 1$, $A_1(n)$ is obtained by starting with $A_1(0) = 2$ and *iterating* n times the *successor* function A_0 . So A_1 is the *add 2* function $n \mapsto n + 2$.
- For $n \geq 1$, $A_2(n)$ is obtained by starting with $A_2(0) = 0$ and *iterating* n times the *add 2* function A_1 . So A_2 is the *multiply by 2* function $n \mapsto n \cdot 2$.
- For $n \geq 1$, $A_3(n)$ is obtained by starting with $A_3(0) = 1$ and *iterating* n times the *multiply by 2* function A_2 . So A_3 is the *exponentiation from 2* function $n \mapsto 2^n$.
- For $n \geq 1$, $A_4(n)$ is obtained by starting with $A_4(0) = 1$ and *iterating* n times the *exponentiation from 2* function A_3 . So A_4 is the *tetration from 2* function defined by $A_4(0) = 1$, and for $n \geq 1$, $A_4(n) := 2^{2^{\cdot^{\cdot^2}}}$ (tower of 2’s of height n).
- For $n \geq 1$, $A_5(n)$ is obtained by starting with $A_5(0) = 1$ and *iterating* n times the *tetration from 2* function A_4 .

To gain some idea of this, note that $A_4 \circ A_4(k)$ is the value of a tower of 2’s whose height itself is given by the value of a further tower of 2’s of height k . You might call this tower of towers a *power tower*.

- etc. (I think it impossible to wrap one’s head around the size of these numbers – and this is barely the start of what is possible, that is, is “impossible”!) $\square$

Remark 8.20 (Explosive Blow-Up). These functions grow faster and faster, with explosive blow-up quite soon. See Figure 8.1.

The growth behaviour of the A_m will become clearer after we discuss the Knuth Up-Arrow 8.10.5 notation .

³You will find other conventions, but this does not any significant difference. In some cases it is just a matter of subtracting the number 3 from $A_m(n)$, which in the circumstances is about as trivial a change as possible!

8.10.4 COMPUTING THE ACKERMANN SEQUENCE

Each A_m is primitive recursive. Here is the high level program to compute A_5 .

Note that the program consists of *nested primitive recursions*, three levels deep in this case, and $m - 2$ levels deep for A_m , if we take exponentiation as a built-in function.

Program	Explanation
Outer loop: calculate $A_5(n)$	
Input n	Defines a sequence $(x_i)_{0 \leq i \leq n}$ by
$x := 1$	
For $i = 1, \dots, n$	$x_0 = 1; \quad x_i = A_4^{(i)}(1) = A_5(i), \quad 1 \leq i \leq n.$
$x := A_4(x)$ (<i>call subroutine</i>)	
$\Rightarrow$ Output $x := A_5(n)$	
Middle loop: calculate $A_4(x)$	
Input x	Defines a sequence $(y_j)_{0 \leq j \leq x}$ by
$y := 1$	
For $j = 1, \dots, x$	$y_0 = 1; \quad y_j = A_3^{(j)}(1) = A_4(j), \quad 1 \leq j \leq x.$
$y := A_3(y)$ (<i>call subroutine</i>)	
$\Rightarrow$ Output $y := A_4(x)$	$A_4(x)$ is a power tower of 2's of height x . $A_4(x) = 2^{2^{\cdots^2}}$ } (tower of x 2's) =: $2 \uparrow\uparrow x$
Innermost loop: calculate $A_3(y)$	
Input y	Defines a sequence $(z_k)_{0 \leq k \leq y}$
$z := 1$	
For $k = 1, \dots, y$	$z_0 = 1; \quad z_k = 2^k, \quad 1 \leq k \leq y.$
$z := 2 \cdot z$	
$\Rightarrow$ Output $z := A_3(y)$	$A_3(y) = 2^y =: 2 \uparrow y$. (which is normally a built in function)
Output	
return x	The output is $x_n = A_5(n) = 2 \uparrow\uparrow\uparrow n$.

8.10.5 KNUTH UP-ARROW NOTATION

KUN

The up-arrow notation due to Knuth is very useful. In particular

$$\begin{aligned}
 2 \uparrow n &:= 2^n = A_3(n) \\
 2 \uparrow\uparrow n &:= A_4(n) \\
 2 \uparrow\uparrow\uparrow n &:= A_5(n) \\
 &\text{etc.}
 \end{aligned}$$

Note that

$$\begin{aligned}
 2 \uparrow\uparrow n &= \overbrace{2 \uparrow (2 \uparrow \dots (2 \uparrow 2) \dots)}^{n \text{ copies of } 2} \\
 2 \uparrow\uparrow\uparrow n &= \overbrace{2 \uparrow\uparrow (2 \uparrow\uparrow \dots (2 \uparrow\uparrow 2) \dots)}^{n \text{ copies of } 2} \\
 2 \uparrow\uparrow\uparrow\uparrow n &= \overbrace{2 \uparrow\uparrow\uparrow (2 \uparrow\uparrow\uparrow \dots (2 \uparrow\uparrow\uparrow 2) \dots)}^{n \text{ copies of } 2} \\
 &\text{and so on.}
 \end{aligned}$$

The role of 2 is not critical. Any integer $k \geq 2$ could be used, giving an Ackermann function with the additional parameter k . The growth rates are comparable.

See the Wiki article [Knuth's Up-Arrow Notation](#).

8.10.6 BINARY FUNCTION FORMULATION

The Ackermann Function is usually defined as a function $A(m, n)$ of two variables, (or with an additional parameter) even as a function of three variables, rather than as a sequence of unary functions A_m .

Whereas the individual A_m are primitive recursive from Proposition 8.22, the function $A(m, n)$ is computable but is *not* primitive recursive 8.10.7.

Definition 8.23 (Ackermann Function). For $m, n \in \mathbb{N}$, $A(m, n)$ is defined by def ack

$$A(0, n) = n + 1 \quad \text{def ack1} \quad (8.7)$$

$$A(m, 0) = \begin{cases} 1 & m = 0 \\ 2 & m = 1 \\ 0 & m = 2 \\ 1 & m \geq 3 \end{cases} \quad \text{def ack2} \quad (8.8)$$

$$A(m, n) = A(m - 1, A(m, n - 1)) \quad m, n \geq 1. \quad \text{def ack3} \quad (8.9)$$

$A(m, n)$ is thus defined by double/simultaneous recursion on m and n , with the ordering

$$(m_1, n_1) \prec (m_2, n_2) \iff m_1 < m_2 \text{ or } (m_1 = m_2 \text{ and } n_1 < n_2). \quad \text{ack on} \quad (8.10)$$

This is the lexicographic well-ordering on $\omega \times \omega$, which in Figure 8.1 can be seen as first proceeding up the n -axis from $(0, 0)$, then up the parallel vertical line from $(1, 0)$, then up from $(2, 0)$, etc.

It should now be clear from Definition 8.7 that $A(m, n)$ is well-defined for all $m, n \in \mathbb{N}$.

Exercise 8.24. Why is $A(m, n) = A_m(n)$, using Definitions 8.7 and 8.23 respectively ?

Exercise 8.25. Why is “ $\prec$ ” called “lexicographic” ?

Exercise 8.26. Explain why “ $\prec$ ” is a well-ordering in terms of the “no infinite descending sequence” property.

Exercise 8.27. Fill in the missing values for A_6, A_7, A_8 in Figure 8.1.

8.10.7 THE ACKERMANN FUNCTION IS NOT PRIMITIVE RECURSIVE

[This chapter to be completed]

afnpr

Chapter 9

Propositional Logic

9.1 THE BASICS OF PROPOSITIONAL LOGIC

The methods and results in this section, up to but not including Subsection 9.1.7^{sec pf}, are a precursor to the material in Subsection 1.3.5 and subsequently for first-order logic.

9.1.1 PROPOSITIONAL SYMBOLS AND FORMULAS.

We work with a *propositional language*

sub psf

$$\mathcal{P} = \{p_1, p_2, p_3, \dots\},$$

which is just a countable collection of *propositional symbols* p_i .

These symbols are combined by means of “ $\neg$ ” (not), “ $\wedge$ ” (and) and parentheses to form *propositional formulas* (usually denoted $\varphi, \psi, \theta, \dots$) and defined inductively to be of the following form:

- a propositional symbol p_i , or
- $(\neg\varphi)$, where φ is a propositional formula, or
- $(\varphi \wedge \psi)$, where φ and ψ are propositional formulas.

Notice there are no quantifiers.

Parentheses are usually dropped where no ambiguity arises and for ease of readability.

Remark 9.1 (Why $\{\neg, \wedge\}$?). We could use many other combinations, for example $\{\neg, \rightarrow\}$ or $\{\neg, \vee\}$. What is chosen depends on what is convenient in the particular situation.

See also subsection 9.1.7

The particular utility of $\{\neg, \rightarrow\}$ is that $\rightarrow$ corresponds more directly to $\vdash$ (*syntactic implication*) and $\models$ (*semantic implication*). $\square$

9.1.2 COMPLEXITY OF A PROPOSITIONAL FORMULA.

In order to prove results depending on the “complexity” of a formula it is often convenient to assign an integer $c(\varphi)$ to formulas as follows:

$$\begin{aligned} c(p_i) &= 1 \quad i = 1, 2, \dots \\ c(\neg\varphi) &= 1 + c(\varphi) \\ c(\varphi \wedge \psi) &= 1 + \max\{c(\varphi), c(\psi)\}. \end{aligned}$$

9.1.3 TRUTH VALUES.

subsec tv

The following table gives the truth values associated with the standard connectives.

φ	ψ	$\neg\varphi$	$\varphi \rightarrow \psi$	$\varphi \vee \psi$	$\varphi \wedge \psi$	$\varphi \leftrightarrow \psi$	$\varphi \uparrow \psi$
T	T	F	T	T	T	T	F
T	F	F	F	T	F	F	T
F	T	T	T	T	F	F	T
F	F	T	T	F	F	T	T

9.1.4 DEFINED CONNECTIVES.

subsec dc

Columns 5, 6, 7 justify the following *definitions* of $\vee$, $\wedge$, $\leftrightarrow$ in terms of $\neg$ and $\rightarrow$:

- (i) $\varphi \vee \psi := \neg(\neg\varphi \wedge \neg\psi)$,
- (ii) $\varphi \rightarrow \psi := \neg(\varphi \wedge \neg\psi)$,
- (iii) $\varphi \leftrightarrow \psi := (\varphi \rightarrow \psi) \wedge (\psi \rightarrow \varphi)$.

Exercise 9.2. Verify this via truth tables applied to each side of the “:=” symbols. □

The *Sheffer Stroke* symbol $\uparrow$, can be read as “not and” or “joint denial” or “neither nor”, and corresponds to the logical/electronic gate NAND. It is defined by

$$\varphi \uparrow \psi := \neg(\varphi \wedge \psi) = \varphi \rightarrow \neg\psi.$$

Moreover, $\neg$ and $\wedge$, and hence all connectives, can similarly be defined in terms of $\uparrow$:

$$\begin{aligned}\neg\varphi &:= \varphi \uparrow \varphi, \\ \varphi \wedge \psi &:= \neg(\varphi \uparrow \psi) := (\varphi \uparrow \psi) \uparrow (\varphi \uparrow \psi), \\ \varphi \rightarrow \psi &:= \varphi \uparrow (\psi \uparrow \psi).¹\end{aligned}$$

Exercise 9.3. Verify these via truth tables. □

9.1.5 TRUTH TABLES

The truth values of a propositional formula can be computed in a systematic manner using the table in subsection 9.1.3. For example, see Table 9.1 for $\varphi \rightarrow ((\psi \vee \theta) \rightarrow (\theta \rightarrow \neg\varphi))$.

Definition 9.4 (Truth Valuation). A (*truth*) *valuation/assignment* v is a function

$$v : \mathcal{P} = \{p_1, p_2, p_3, \dots\} \rightarrow \{T, F\}.$$

In a similar manner to that in the previous Truth Table, v *extends to all propositional formulas* φ . The value of $v(\varphi)$ depends only on $v(p_i)$ for those p_i occurring in φ , as follows by a simple induction on the complexity of formulas.

Definition 9.5 (Models).

¹I suppose somebody somewhere found this useful at sometime.

φ	ψ	θ	$\psi \vee \theta$	$\neg\varphi$	$\theta \rightarrow \neg\varphi$	$(\psi \vee \theta) \rightarrow (\theta \rightarrow \neg\varphi)$	$\varphi \rightarrow ((\psi \vee \theta) \rightarrow (\theta \rightarrow \neg\varphi))$
T	T	T	T	F	F	F	F
T	T	F	T	F	T	T	T
T	F	T	T	F	F	F	F
T	F	F	F	F	T	T	T
F	T	T	T	T	T	T	T
F	T	F	T	T	T	T	T
F	F	T	T	T	T	T	T
F	F	F	F	T	T	T	T

Table 9.1: Truth table for $\varphi \rightarrow ((\psi \vee \theta) \rightarrow (\theta \rightarrow \neg\varphi))$

table truth

- $v \models \varphi$ means φ is true under the valuation v .
We say v is a model for φ .
- If Σ is a set of propositional formulas and $v \models \varphi$ for all $\sigma \in \Sigma$,
we say v is a model for Σ .
- $\Sigma \models \varphi$ means every model of Σ is a model of φ .
That is, if all formulas in Σ are true under a valuation v , then φ is also true under the valuation v .
- $\models \varphi$ means φ is true in all models, (i.e. is a *tautology*).

The above dot points are analogous in first-order logic to respectively: $\mathfrak{A} \models \varphi$, $\mathfrak{A} \models \sigma$ for every $\sigma \in \Sigma$, $\Sigma \models \varphi$ and $\models \varphi$ as in Section 1.4.

Here $\mathfrak{A}$ is a structure for a first-order language $\mathcal{L}$, φ is an $\mathcal{L}$ -sentence and Σ is a set of $\mathcal{L}$ -sentences.

9.1.6 TAUTOLOGIES

Definition 9.6. A *propositional tautology* is a propositional formula φ which takes the value T for every valuation v . def prop taut

If the propositional formula A contains n distinct propositional symbols, there are 2^n possible assignments to consider, or 2^n rows as in the previous table with $n = 3$.

The truth table technique gives an algorithm for deciding of any formula whether or not it is a tautology.

9.1.7 DISJUNCTIVE NORMAL FORM

The following theorem shows that every truth table is truth-functionally equivalent to a formula built from $\wedge, \vee, \neg$ in a particularly nice manner. subsec dnf

The only “exception” is that if the truth table has output F for all inputs then one has the “empty disjunction”, which is always false anyway (*Why?*). We could also use $\varphi \wedge \neg\varphi$ as the relevant formula.

Theorem 9.7 (Disjunctive Normal Form). *Corresponding to every truth table whose output is not identically false, and hence corresponding to every propositional formula*

which is not a contradiction, there is a truth-functionally equivalent formula which is a disjunction of conjunctions of atomic formulas or their negations.

Proof. For example, consider Table 9.1.

Corresponding to the six rows 2 and 4–8, for which the (φ, ψ, θ) input gives output T , consider the following propositional formula:

$$(\varphi \wedge \psi \wedge \neg\theta) \vee (\varphi \wedge \neg\psi \wedge \neg\theta) \vee (\neg\varphi \wedge \psi \wedge \theta) \vee (\neg\varphi \wedge \psi \wedge \neg\theta) \vee (\neg\varphi \wedge \neg\psi \wedge \theta) \vee (\neg\varphi \wedge \neg\psi \wedge \neg\theta). \quad \text{thm dnf (9.1)}$$

The first disjunct is true for the input (T, T, F) and *only* this input, the second similarly for the input (T, F, F) , the third for (F, T, T) , the fourth for (F, T, F) , the fifth for (F, F, T) and the sixth for (F, F, F) .

Thus the entire disjunction is true for each input in the Truth Table which gives output T , and is false for each input which gives output F .

This method for constructing a propositional formula corresponding to a given truth table, clearly works in general. $\square$

Remark 9.8 (Truth-Functional Completeness). We saw in subsection 9.1.4 that the standard connectives, including $\vee$, are definable in terms of $\neg$ and $\wedge$. So it follows from the Theorem that the pair $\{\wedge, \neg\}$ alone is truth-functionally complete. But $\neg$ will no longer be applied only to atomic formulas. $\square$

Exercise 9.9. Show that any formula φ can be put in “Conjunctive Normal Form”.

1. Do this by first applying the previous theorem to $\neg\varphi$ and then apply $\neg$ to the result.
2. Explain what is meant by “Conjunctive Normal Form”. $\square$

9.2 OPTIONAL: A DEDUCTIVE SYSTEM

Axioms. Here is a set of *Axiom Schemas for Propositional Logic*:

sec ds2

- (i) $\varphi \rightarrow (\psi \rightarrow \varphi)$,
- (ii) $(\varphi \rightarrow (\psi \rightarrow \theta)) \rightarrow ((\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \theta))$,
- (iii) $(\neg\varphi \rightarrow \neg\psi) \rightarrow (\psi \rightarrow \varphi)$,

where φ, ψ, θ are arbitrary propositional formulas.

Exercise: Check the axioms are indeed tautologies.

Rule of Inference. The *sole* rule of inference here is *modus ponens*.

- From φ and $\varphi \rightarrow \psi$, infer ψ .

Exercise: Check that modus ponens preserves tautologies.

Formal Proofs. Suppose φ is a *propositional formula* and Σ is a set of *propositional formulas*.

A *proof* of φ from Σ is a finite sequence $\varphi_1, \dots, \varphi_n$ of *formulas* such that $\varphi_n = \varphi$ and such that each φ_i is a propositional axiom, or a member of Σ , or deducible from earlier φ_j 's by means of modus ponens.

We write $\Sigma \vdash \varphi$ in such a case, and say that φ is a *theorem* of Σ , or φ is *deducible* from Σ .

If Σ is the empty set we write $\vdash \varphi$, and say φ is a *propositional theorem*.

It is trivially true that if $\Sigma \vdash \varphi$ where Σ is infinite then $\Sigma_0 \vdash \varphi$ for some *finite* $\Sigma_0 \subset \Sigma$.

Soundness Theorem. It follows from the previous two *Exercises* regarding the axioms and the rule of inference for propositional logic, that all propositional theorems are tautologies.

More generally, if $\Sigma \vdash \varphi$ and all sentences in Σ have truth value T for some set of truth values assigned to the propositional symbols occurring in Σ , then φ also has truth value T for the same assignment.

This is called the *Soundness Theorem for Propositional Logic*. It is a trivial result – we would not choose axioms which were not tautologies, nor rules of inference that did not preserve tautologies!

Completeness Theorem. We show in Section 9.4 that the converse is also true: all tautologies are theorems, i.e. are logical consequences of Axioms (i), (ii) and (iii). This is called the *Completeness Theorem for Propositional Logic*. It is *not* trivial.

However...

All Tautologies as Axioms. *We will in future allow all tautologies as axioms for propositional logic.*

This is quite reasonable as it is a mechanical procedure to decide if a propositional formula is a tautology.

Moreover, there is little point in pursuing an axiomatic approach for Hilbert propositional logic, as indicated at the beginning of Section 2

* **“Simplifying” Axiom (iii).** In some older texts Axiom (iii) is replaced by $\neg\neg\varphi \rightarrow \varphi$. See (Johnstone, 1987, Pages 7,8,11,12), (Church, 1956, Chapter 1, §10).

But this only makes sense if $\neg\varphi$ is already defined in terms of other primitives, typically $\neg\varphi := \varphi \rightarrow \perp$, where $\perp$ is a propositional constant which is interpreted as “False”.

Otherwise $\neg\neg\varphi$ only gives a property of the operator $\neg$, and indeed $\neg\varphi$ could then consistently be interpreted as φ and the axiom would still be satisfied, which state of affairs is certainly not intended.

9.3 OPTIONAL: DEDUCTION THEOREM

As remarked previously, starred sections are not part of the main pathway through to the proofs of Gödel’s Completeness and Incompleteness Theorems. sec dtpl

In this and the following section, despite the fact that we will take all tautologies as axioms, I do go through the details of deriving the tautologies from the axioms in Section 9.2. Because:

- It is (sort of) fun.
- It should be part of your training.
- Many texts do it.
- The ideas are useful in the proof of the Completeness Theorem for first-order logic.

Deductions from the Axioms. Here are some examples of proofs from the axioms of propositional logic.

Example 1. $\vdash \varphi \rightarrow \varphi$

- | | | |
|----|---|----------------------|
| 1. | $(\varphi \rightarrow ((\varphi \rightarrow \varphi) \rightarrow \varphi)) \rightarrow ((\varphi \rightarrow (\varphi \rightarrow \varphi)) \rightarrow (\varphi \rightarrow \varphi))$ | axiom (ii) |
| 2. | $\varphi \rightarrow ((\varphi \rightarrow \varphi) \rightarrow \varphi)$ | axiom (i) |
| 3. | $(\varphi \rightarrow (\varphi \rightarrow \varphi)) \rightarrow (\varphi \rightarrow \varphi)$ | modus ponens on 1, 2 |
| 4. | $\varphi \rightarrow (\varphi \rightarrow \varphi)$ | axiom (i) |
| 5. | $\varphi \rightarrow \varphi$ | modus ponens on 3, 4 |

Example 2. $\varphi \rightarrow \psi, \psi \rightarrow \theta \vdash \varphi \rightarrow \theta$

- | | | |
|----|---|----------------------|
| 1. | $(\varphi \rightarrow (\psi \rightarrow \theta)) \rightarrow ((\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \theta))$ | axiom (ii) |
| 2. | $\psi \rightarrow \theta$ | hypothesis |
| 3. | $(\psi \rightarrow \theta) \rightarrow (\varphi \rightarrow (\psi \rightarrow \theta))$ | axiom (i) |
| 4. | $\varphi \rightarrow (\psi \rightarrow \theta)$ | modus ponens on 2, 3 |
| 5. | $(\varphi \rightarrow \psi) \rightarrow (\varphi \rightarrow \theta)$ | modus ponens on 1, 4 |
| 6. | $\varphi \rightarrow \psi$ | hypothesis |
| 7. | $\varphi \rightarrow \theta$ | modus ponens on 5, 6 |

Remarks re the Deduction Theorem.

1. The Deduction Theorem (to follow) shows that syntactic provability ($\vdash$) “acts like” logical implication ($\rightarrow$).
2. The Deduction Theorem simplifies *formal* proofs from the axioms in propositional logic. Consider the previous two examples.

Example 1. To show that $\vdash \varphi \rightarrow \varphi$, by the Deduction Theorem it is sufficient to show $\varphi \vdash \varphi$, which is immediate.

Example 2. To show

$$\varphi \rightarrow \psi, \psi \rightarrow \theta \vdash \varphi \rightarrow \theta,$$

using the Deduction Theorem it is sufficient to show that

$$\varphi \rightarrow \psi, \psi \rightarrow \theta, \varphi \vdash \theta.$$

But this is immediate by two applications of modus ponens.

3. There is nothing very profound about the following proof of the Deduction Theorem. We simply made sure there were enough axioms for the proof, and for the proof of the Completeness Theorem which uses the Deduction Theorem, to go through.
4. Notice that the proof of the Deduction Theorem only uses Axioms (i) and (ii), and not Axiom (iii).

Deduction Theorem. If Σ is a set of propositional formulas, and φ and ψ are propositional formulas, then

$$\Sigma \cup \{\varphi\} \vdash \psi \quad \text{iff} \quad \Sigma \vdash \varphi \rightarrow \psi.$$

Proof

- (a) Suppose $\Sigma \vdash \varphi \rightarrow \psi$, with proof

$$\theta_1, \dots, \theta_n (= \varphi \rightarrow \psi).$$

Then, using modus ponens,

$$\theta_1, \dots, \theta_{n-1}, (\theta_n =) \varphi \rightarrow \psi, \varphi, \psi,$$

is a proof for $\Sigma \cup \{\varphi\} \vdash \psi$.

- (b) Suppose $\Sigma \cup \{\varphi\} \vdash \psi$, with proof

$$\psi_1, \dots, \psi_n (= \psi).$$

We show by induction that there is a proof of $\Sigma \vdash \varphi \rightarrow \psi_i$ for each i .

- (i) If ψ_i is an axiom or $\psi_i \in \Sigma$, then by Axiom (i) and modus ponens,

$$\psi_i, \psi_i \rightarrow (\varphi \rightarrow \psi_i), \varphi \rightarrow \psi_i$$

is a proof for $\Sigma \vdash \varphi \rightarrow \psi_i$.

- (ii) If $\psi_i = \varphi$, then $\Sigma \vdash \varphi \rightarrow \psi_i$ since we saw a five line proof of $\vdash \varphi \rightarrow \varphi$ in the previous Example 1.

- (iii) If ψ_i is deduced by modus ponens from ψ_j and from $\psi_k = \psi_j \rightarrow \psi_i$ ($j, k < i$), let

$$S_1, S_2, \dots, S_n, \varphi \rightarrow \psi_j$$

be the proof for $\Sigma \vdash \varphi \rightarrow \psi_j$, and let

$$S'_1, S'_2, \dots, S'_m, \varphi \rightarrow (\psi_j \rightarrow \psi_i)$$

be the proof for $\Sigma \vdash \varphi \rightarrow (\psi_j \rightarrow \psi_i)$. Then the following is a proof for $\Sigma \vdash \varphi \rightarrow \psi_i$:

$$\begin{array}{ll} S_1, \dots, S_n, \varphi \rightarrow \psi_j, & \\ S'_1, \dots, S'_m, \varphi \rightarrow (\psi_j \rightarrow \psi_i), & \\ (\varphi \rightarrow (\psi_j \rightarrow \psi_i)) \rightarrow ((\varphi \rightarrow \psi_j) \rightarrow (\varphi \rightarrow \psi_i)), & \text{(logical axiom (ii))} \\ (\varphi \rightarrow \psi_j) \rightarrow (\varphi \rightarrow \psi_i), & \text{(modus ponens)} \\ \varphi \rightarrow \psi_i. & \text{(modus ponens)} \end{array}$$

More Deductions via the Deduction Theorem.

Example 3. $\vdash \neg\varphi \rightarrow (\varphi \rightarrow \psi)$, (equivalently by the Deduction Theorem) $\neg\varphi \vdash \varphi \rightarrow \psi$, and (again equivalently) $\{\varphi, \neg\varphi\} \vdash \psi$.

Take the hypothesis $\neg\varphi$.

1.	$\neg\varphi$	hypothesis
2.	$\neg\varphi \rightarrow (\neg\psi \rightarrow \neg\varphi)$	axiom (i)
3.	$\neg\psi \rightarrow \neg\varphi$	modus ponens on 1, 2
4.	$(\neg\psi \rightarrow \neg\varphi) \rightarrow (\varphi \rightarrow \psi)$	axiom (iii)
5.	$\varphi \rightarrow \psi$	modus ponens on 3, 4

Hence $\neg\varphi \vdash \varphi \rightarrow \psi$ and so $\vdash \neg\varphi \rightarrow (\varphi \rightarrow \psi)$.

Example 4. $\vdash \neg\neg\varphi \rightarrow \varphi$, or equivalently by the Deduction Theorem, $\neg\neg\varphi \vdash \varphi$.

Take the hypothesis $\neg\neg\varphi$.

1.	$\vdash \neg\varphi \rightarrow \neg\neg\varphi$	previous example with $\varphi \mapsto \neg\varphi$, $\psi \mapsto \neg\neg\varphi$
2.	$(\neg\varphi \rightarrow \neg\neg\varphi) \rightarrow (\neg\neg\varphi \rightarrow \varphi)$	axiom (iii)
3.	$\neg\neg\varphi \rightarrow \varphi$	modus ponens on 1, 2
4.	$\neg\neg\varphi$	hypothesis
5.	φ	modus ponens on 3, 4

Hence $\neg\neg\varphi \vdash \varphi$ and so $\vdash \neg\neg\varphi \rightarrow \varphi$.

Example 5. $\vdash \varphi \rightarrow \neg\neg\varphi$, or equivalently by the Deduction Theorem, $\varphi \vdash \neg\neg\varphi$.

1.	$\vdash \neg\neg\varphi \rightarrow \neg\varphi$	previous example with $\varphi \mapsto \neg\varphi$
2.	$(\neg\neg\varphi \rightarrow \neg\varphi) \rightarrow (\varphi \rightarrow \neg\neg\varphi)$	axiom (iii)
3.	$\varphi \rightarrow \neg\neg\varphi$	modus ponens on 1, 2

Hence $\vdash \varphi \rightarrow \neg\neg\varphi$ and so $\varphi \vdash \neg\neg\varphi$.

9.4 OPTIONAL: COMPLETENESS THEOREM

Overview Our goal in this section is to prove that syntactic implication is equivalent to semantic implication sec ctpl

That is,

$$\Sigma \vdash \varphi \iff \Sigma \models \varphi.$$

“A propositional formula φ is deducible from a set Σ of propositional formulas, using just the logical axioms (i), (ii) and (iii) and modus ponens, iff φ is true in all models of Σ .”

The direction $\implies$ says that the logical axioms and modus ponens are not absurd. It is called the Soundness Theorem and was discussed in Section 9.2.

The direction $\impliedby$ is much deeper and says that the logical axioms and modus ponens are sufficiently strong to prove from Σ *every* semantic consequence of Σ .

Falsum In the following we could take $\perp$ (“falsum” or “absurdity” or “contradiction”) to be the negation of any of the axioms in Section 9.2, or one of their consequences. The truth table for any such formula has output F (false) for all truth value inputs.

For simplicity we define

$$\perp := \neg(p_1 \rightarrow p_1),$$

where p_1 was the first propositional symbol.

In the following $\varphi, \psi, \theta, \dots$ are propositional formulas and Σ is a set of propositional formulas.

Definition 9.10. Σ is *consistent* if $\Sigma \not\vdash \perp$. Σ is *inconsistent* if $\Sigma \vdash \perp$.

th triv

Theorem 9.11. Σ is consistent iff every finite subset of Σ is consistent.

Proof. Trivial, since proofs are finite. □

th incon

Theorem 9.12. *The following are equivalent:*

1. Σ is inconsistent, i.e. $\Sigma \vdash \perp$,
2. There is some φ such that $\Sigma \vdash \varphi$ and $\Sigma \vdash \neg\varphi$,
3. For every φ , $\Sigma \vdash \varphi$.

Proof. (1 $\implies$ 2): Take $\varphi = p_1 \rightarrow p_1$. Then $\Sigma \vdash \varphi$ since $\vdash p_1 \rightarrow p_1$ from Example 1 in Section 9.3, and $\Sigma \vdash \neg\varphi$ by the definition of inconsistent.

(2 $\implies$ 3): Assume 2, and note from Example 3 in Section 9.3 that $\{\varphi, \neg\varphi\} \vdash \psi$ (for any ψ). Concatenating the proofs of $\Sigma \vdash \varphi$, $\Sigma \vdash \neg\varphi$ and $\{\varphi, \neg\varphi\} \vdash \psi$, gives a proof of $\Sigma \vdash \psi$.

(3 $\implies$ 1): Let $\varphi = p_1 \rightarrow p_1$. □

Corollary 9.13. *The following are equivalent:*

1. Σ is consistent, i.e. $\Sigma \not\vdash \perp$,
2. There is no φ such that $\Sigma \vdash \varphi$ and $\Sigma \vdash \neg\varphi$,
3. There is at least one φ such that $\Sigma \not\vdash \varphi$.

Proof. Immediate by the previous theorem. □

The following connects consistency and provability.

th cp

Theorem 9.14.

$$\begin{aligned} \Sigma \cup \{\neg\varphi\} \text{ is inconsistent} &\iff \Sigma \vdash \varphi, \\ \Sigma \cup \{\varphi\} \text{ is inconsistent} &\iff \Sigma \vdash \neg\varphi. \end{aligned}$$

Proof.

$$\begin{aligned} \Sigma \cup \{\neg\varphi\} \text{ inconsistent} &\implies \Sigma \cup \{\neg\varphi\} \vdash \neg(p_1 \rightarrow p_1) && \text{using } \perp := \neg(p_1 \rightarrow p_1) \\ &\implies \Sigma \vdash \neg\varphi \rightarrow \neg(p_1 \rightarrow p_1) && \text{deduction theorem} \\ &\implies \Sigma \vdash (p_1 \rightarrow p_1) \rightarrow \varphi && \text{axiom (iii)} \\ &\implies \Sigma \vdash \varphi && \text{example 1, modus ponens} \end{aligned}$$

Conversely, if $\Sigma \vdash \varphi$ then both $\Sigma \cup \{\neg\varphi\} \vdash \varphi$ and $\Sigma \cup \{\neg\varphi\} \vdash \neg\varphi$. By the previous Theorem this implies $\Sigma \cup \{\neg\varphi\}$ is inconsistent.

Replacing φ by $\neg\varphi$, $\Sigma \cup \{\neg\neg\varphi\}$ is inconsistent iff $\Sigma \vdash \neg\varphi$. But $\varphi \vdash \neg\neg\varphi$ and $\neg\neg\varphi \vdash \varphi$ by Examples 4 & 5 in Section 9.3, so $\Sigma \cup \{\neg\neg\varphi\}$ is inconsistent iff $\Sigma \cup \{\varphi\}$ is inconsistent. This completes the proof. $\square$

cor cp

Corollary 9.15.

$$\begin{aligned}\Sigma \cup \{\neg\varphi\} \text{ is consistent} &\iff \Sigma \not\vdash \varphi, \\ \Sigma \cup \{\varphi\} \text{ is consistent} &\iff \Sigma \not\vdash \neg\varphi.\end{aligned}$$

Theorem 9.16. Suppose Σ is a consistent set of propositional formulas and φ is any propositional formula. Then at least one of $\Sigma \cup \{\varphi\}$ and $\Sigma \cup \{\neg\varphi\}$ is consistent.

Proof. If both $\Sigma \cup \{\varphi\}$ and $\Sigma \cup \{\neg\varphi\}$ are inconsistent, then $\Sigma \vdash \neg\varphi$ and $\Sigma \vdash \varphi$ by Theorem 9.14. By Theorem 9.12, Σ is inconsistent. Contradiction. $\square$

The following is a key concept for proving the Completeness Theorem.

Definition 9.17. Σ is *maximal consistent* if

- Σ is consistent,
- $\Sigma \subseteq \tilde{\Sigma}$ and $\tilde{\Sigma}$ consistent $\implies \Sigma = \tilde{\Sigma}$.

thm prop Lind

Theorem 9.18. Suppose Σ is a consistent set of propositional formulas. Then Σ can be extended to a maximal consistent set $\tilde{\Sigma}$.

Proof. Let $\varphi_1, \varphi_2, \varphi_3, \dots$ be an enumeration of all propositional formulas. Define

$$\begin{aligned}\Sigma_1 &= \Sigma, \\ \Sigma_{n+1} &= \begin{cases} \Sigma_n \cup \{\varphi_n\} & \text{if } \Sigma_n \cup \{\varphi_n\} \text{ is consistent,} \\ \Sigma_n & \text{otherwise,} \end{cases} \\ \tilde{\Sigma} &= \bigcup_{n \geq 1} \Sigma_n.\end{aligned}$$

Each Σ_n is consistent by construction. Moreover $\tilde{\Sigma}$ is consistent, since if $\tilde{\Sigma} \vdash \perp$ then $\Sigma_n \vdash \perp$ for some n since derivations are finite in length. But this is a contradiction.

To see that $\tilde{\Sigma}$ is *maximal* consistent, suppose $\tilde{\Sigma} \cup \{\varphi\}$ is consistent. Then $\varphi = \varphi_n$ for some n , and because $\Sigma_n \cup \{\varphi_n\}$ must also be consistent it follows by construction that $\varphi_n \in \Sigma_{n+1}$. Hence $\varphi \in \tilde{\Sigma}$ and so $\tilde{\Sigma}$ is maximal consistent. $\square$

The next result shows that a maximal consistent set of sentences is closed under deductions.

th mcud

Theorem 9.19. *If Σ is maximal consistent, then $\Sigma \vdash \varphi$ implies $\varphi \in \Sigma$.*

Proof. If $\varphi \notin \Sigma$ then by definition of *maximal consistent* $\Sigma \cup \{\varphi\}$ is not consistent. Hence $\Sigma \vdash \neg\varphi$, and so Σ is inconsistent by Theorem 9.12. $\square$

th neg

Theorem 9.20. *If Σ is maximal consistent, then for every φ exactly one of the following holds: $\varphi \in \Sigma$ or $\neg\varphi \in \Sigma$.*

Proof. They cannot both hold since Σ is consistent.

If $\varphi \notin \Sigma$ then $\Sigma \cup \{\varphi\}$ is inconsistent and so $\Sigma \vdash \neg\varphi$ by Theorem 9.12. But then $\neg\varphi \in \Sigma$ by Theorem 9.19. $\square$

th imp

Theorem 9.21. *If Σ is maximal consistent then*

$$(\varphi \rightarrow \psi) \in \Sigma \iff (\varphi \in \Sigma \implies \psi \in \Sigma)$$

Proof. If $(\varphi \rightarrow \psi) \in \Sigma$ and $\varphi \in \Sigma$, then $\Sigma \vdash \varphi \rightarrow \psi$ and $\Sigma \vdash \varphi$. Hence $\Sigma \vdash \psi$ by modus ponens, and so $\psi \in \Sigma$ by Theorem 9.19.

Conversely, suppose $\varphi \in \Sigma \implies \psi \in \Sigma$. Then $\Sigma \vdash \varphi \implies \Sigma \vdash \psi$. Consider the two cases $\varphi \in \Sigma$ and $\varphi \notin \Sigma$

1. If $\varphi \in \Sigma$ then $\Sigma \vdash \varphi$, and so $\Sigma \vdash \psi$.

But $\vdash \psi \rightarrow (\varphi \rightarrow \psi)$ by axiom (i).

So $\Sigma \vdash \varphi \rightarrow \psi$ by modus ponens, and so $\varphi \rightarrow \psi \in \Sigma$ by Theorem 9.19

2. If $\varphi \notin \Sigma$ then $\neg\varphi \in \Sigma$ and so $\Sigma \vdash \neg\varphi$.

Hence $\Sigma \vdash \neg\psi \rightarrow \neg\varphi$, again by axiom (i) and modus ponens.

Hence $\Sigma \vdash \varphi \rightarrow \psi$ by axiom (iii) and modus ponens.

Hence $(\varphi \rightarrow \psi) \in \Sigma$ by Theorem 9.19. $\square$

Theorem 9.22. *Suppose Σ is maximal consistent. Let v be the valuation on propositional symbols p_n defined by*

$$v(p_n) = \begin{cases} T & \text{if } p_n \in \Sigma, \\ F & \text{if } p_n \notin \Sigma. \end{cases}$$

Using the primitive connectives “ $\neg$ ” and “ $\rightarrow$ ”, extend v to a valuation V on all propositional formulas via induction on formula complexity and using truth tables.

Then

$$V(\chi) = \begin{cases} T & \text{if } \chi \in \Sigma, \\ F & \text{if } \chi \notin \Sigma. \end{cases}$$

Proof. The result is true by definition if χ is p_n for some n .

By Theorem 9.20, $\neg\varphi \in \Sigma$ iff $\varphi \notin \Sigma$. It follows from the truth table for $\neg\varphi$ that the result is true for $\neg\varphi$ if it is true for φ .

Assuming the result true for φ and ψ , and using Theorem 9.21

$$\begin{array}{lll}
 V(\varphi \rightarrow \psi) = T & \text{iff} & V(\varphi) = T \implies V(\psi) = T & \text{truth tables} \\
 & \text{iff} & \varphi \in \Sigma \implies \psi \in \Sigma & \text{inductive hypothesis} \\
 & \text{iff} & (\varphi \rightarrow \psi) \in \Sigma & \text{Theorem 9.21}
 \end{array}$$

The result now follows by induction on formula complexity. □

Theorem 9.23 (Model Existence Theorem). Σ is consistent $\iff \Sigma$ has a model. th met

Proof. For $\Leftarrow$ suppose Σ has a model, but Σ is inconsistent and so $\Sigma \vdash \perp$. By the Soundness theorem $\Sigma \models \perp$. But $\neg \perp$ must be true in the model since it is an axiom. Contradiction.

For $\Rightarrow$ suppose Σ is consistent and extend Σ to a maximal consistent set and apply the previous theorem. □

Theorem 9.24 (Completeness Theorem). $\Sigma \models \varphi \iff \Sigma \vdash \varphi$.

Proof. The $\Leftarrow$ direction is just the Soundness Theorem.

For the opposite direction suppose $\Sigma \not\models \varphi$.

Then $\Sigma \cup \{\neg\varphi\}$ is consistent by Theorem 9.14.

Therefore $\Sigma \cup \{\neg\varphi\}$ has a model by Theorem 9.23.

Therefore $\Sigma \not\models \varphi$ by the very meaning/definition of $\Sigma \models \varphi$.

This completes the proof. □

Chapter 10

Peano Axioms and Arithmetic

[This chapter to be completed]

ch PAA

Chapter 11

Model Theory

[This chapter to be completed]

ch mod thy 2

Chapter 12

Axiom of Choice

See See 4.1.

ch ac

[This chapter to be completed]

Chapter 13

(Optional) Ordinals

See Section 4.2

ch ords

13.1 LINEAR ORDERINGS

Definition 13.1 (Linear ordering). A linear ordering $<$ on a set A is a subset of $A \times A$ such that def:linear-order

- (i) (Transitivity) for all $a, b, c \in A$, if $a < b$ and $b < c$ then $a < c$,
- (ii) (Trichotomy) for all $a, b \in A$, exactly one of $a = b$, $a < b$, $b < a$ holds.

We often write $\mathcal{A} = \langle A, < \rangle$ and call $\mathcal{A}$ a linear ordering.

If $S \subset A$ and $x \in A$ then $x < S$ means $x < y$ for all $y \in S$. Similarly for $S < x$.

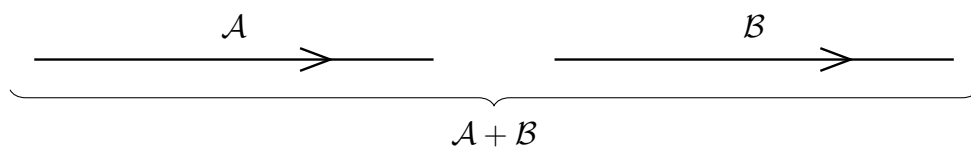
Exercise 13.2. Express the axioms for a linear ordering in an appropriate first-order language.

Definition 13.3 (Sum and product of linear orderings). If $\mathcal{A}$ and $\mathcal{B}$ are linear orderings, then: def:sum-prod

- (i) $\mathcal{A} + \mathcal{B}$ is the linear ordering obtained by following a copy of $\mathcal{A}$ with a copy of $\mathcal{B}$.

More precisely, let $\mathcal{A} = \langle A, <_1 \rangle$ and $\mathcal{B} = \langle B, <_2 \rangle$ where B is *disjoint*¹ from A . Then $\mathcal{A} + \mathcal{B} = \langle A \cup B, < \rangle$ where

$$u < v \quad \text{iff} \quad \begin{cases} u, v \in A \text{ and } u <_1 v, \\ \text{or } u \in A, v \in B, \\ \text{or } u, v \in B \text{ and } u <_2 v. \end{cases}$$



¹There is clearly no loss of generality in assuming this.

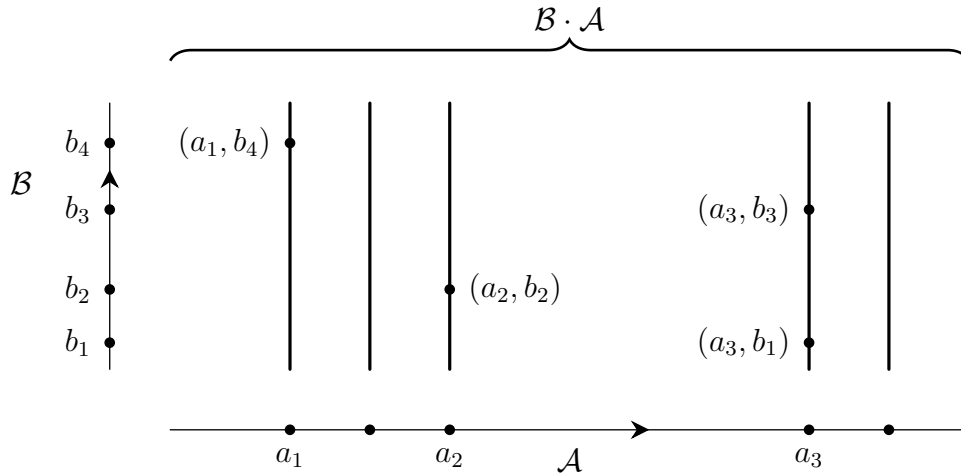
- (ii) $\mathcal{B} \cdot \mathcal{A}$ is the linear ordering obtained by placing a copy of $\mathcal{B}$ at each $a \in A$, and ordering between different copies of $\mathcal{B}$ according to their position on A .

We “read” $\mathcal{B} \cdot A$ from right to left according to the usual convention. Think of it as “ A copies of $\mathcal{B}$ ”.

More precisely, let $\mathcal{A} = \langle A, <_1 \rangle$ and $\mathcal{B} = \langle B, <_2 \rangle$.

Then $\mathcal{B} \cdot \mathcal{A} = \langle A \times B, < \rangle$ where

$$(a_1, b_1) < (a_2, b_2) \quad \text{iff} \quad \begin{cases} a_1 <_1 a_2, \\ \text{or } a_1 = a_2 \text{ and } b_1 <_2 b_2. \end{cases}$$



Here: $(a_1, b_4) < (a_2, b_2) < (a_3, b_1) < (a_3, b_3)$

Remark 13.4. The ordering $<$ in $\mathcal{B} \cdot \mathcal{A}$ is also called the *lexicographic ordering* or *dictionary ordering* on $A \times B$. And is called the *antilexicographic ordering* on $B \times A$. $\square$

Remark 13.5 (Algebraic Properties of Addition and Multiplication). It is straightforward to prove, that

$$\begin{aligned} \mathcal{A} + (\mathcal{B} + \mathcal{C}) &= (\mathcal{A} + \mathcal{B}) + \mathcal{C}, \\ \mathcal{A} \cdot (\mathcal{B} + \mathcal{C}) &= (\mathcal{A} \cdot \mathcal{B}) + (\mathcal{A} \cdot \mathcal{C}), \\ \mathcal{A} \cdot (\mathcal{B} \cdot \mathcal{C}) &= (\mathcal{A} \cdot \mathcal{B}) \cdot \mathcal{C}. \end{aligned}$$

But in general

$$(\mathcal{A} + \mathcal{B}) \cdot \mathcal{C} \neq \mathcal{A} \cdot \mathcal{C} + \mathcal{B} \cdot \mathcal{C}$$

For example, let $\mathcal{A} = \{0\}$, $\mathcal{B} = \{1\}$ and $\mathcal{C} = \omega$, with the trivial order on the two singleton sets and $\omega = \{0, 1, 2, \dots\}$ with the standard ordering $0 < 1 < 2 < \dots$.

Then $\mathcal{A} + \mathcal{B} = 2 := {}^2\{0, 1\}$ with the ordering $0 < 1$. So

$$\begin{aligned} (\mathcal{A} + \mathcal{B}) \cdot \mathcal{C} &= 2 \cdot \omega \\ &= (0, 0) < (0, 1) < (1, 0) < (1, 1) < (2, 0) < (2, 1) < (3, 0) < (3, 1) < \dots \\ &\cong \omega. \end{aligned}$$

² “:=” means “by definition equals”

where “ $\cong$ ” means isomorphic as linear orderings.

On the other hand,

$$\mathcal{A} \cdot \mathcal{C} + \mathcal{B} \cdot \mathcal{C} \cong \{0\} \cdot \omega + \{1\} \cdot \omega \cong \omega + \omega \not\cong \omega$$

The non-isomorphism is because every element in ω has an immediate predecessor, but this is not the case for $\omega + \omega$. *Why?*

Definition 13.6 (Initial segment). def:initial-segment I is an initial segment of the linear order $\mathcal{A} = (A, <)$ iff $I \subseteq A$ and

$$x \in I \wedge y < x \implies y \in I.$$

Definition 13.7 (Finite orders and ω). def:finite-omega

1. Besides denoting the respective integer, n also denotes the unique (up to isomorphism) linear ordering containing n elements. The domain of n is $\{0, 1, \dots, n-1\}$ with $0 < 1 < \dots < n-1$.
2. ω is the unique (up to isomorphism) linear ordering with infinite domain such that every proper initial segment is finite. The domain of ω is $\{0, 1, 2, \dots\}$ with the usual order $0 < 1 < 2 < \dots$.

13.2 ORDINALS AS SOPHISTICATED COUNTING

In this and the next section I discuss the ideas behind ordinals in an informal manner. In many cases where ordinals are being used, this is about all that you will need.

Ordinals were introduced by Cantor in 1883 in his work on trigonometric series. See [Aside/Cantor 13.3](#).

When we prove or define something by induction over ω (a “step by step” proof) we essentially know it works because ω has a first element, and because for every initial segment $I \subsetneq \omega$ there exists a first element $x > y$ for every $y \in I$.

Let’s temporarily call this the *segment property* of ω .

By an informal argument (or formally by induction, but do not bother — it is all in Theorem [13.17](#)) we see that any infinite linear order with the above *segment property* has an isomorphic copy of ω as a proper initial segment. Thus ω is the “smallest” linear ordering with the segment property, with domain and ordering as in Definition [13.7.2](#).

The next smallest such linear ordering has domain

$$\{0, 1, 2, \dots, \omega\}$$

with ordering

$$0 < 1 < 2 < \dots < \omega.$$

We write this as $\omega + 1$, consistent with Definitions [13.3](#) (and [13.7](#)).

Then $\omega + 2$ is the next, $\omega + 3$ the next, $\dots$. And after all these there is again a unique next, $\omega + \omega$ (i.e. $\omega \cdot 2$). Then $\omega \cdot 2 + 1$, $\omega \cdot 2 + 2$, $\dots$, and after all these $\omega \cdot 2 + \omega$ (i.e. $\omega \cdot 3$).

These order types are called *ordinals*. Writing out part of the list we have

$$\begin{aligned} &0, 1, 2, 3, \dots, \omega, \omega + 1, \omega + 2, \omega + 3, \dots, \omega + \omega = \omega \cdot 2, \\ &\omega \cdot 2 + 1, \omega \cdot 2 + 2, \omega \cdot 2 + 3, \dots, \omega \cdot 3, \omega \cdot 3 + 1, \dots, \omega \cdot 4, \dots, \\ &\omega \cdot \omega = \omega^2, \omega^2 + 1, \dots, \omega^2 + \omega, \omega^2 + \omega + 1, \dots, \omega^2 + \omega \cdot 2, \\ &\omega^2 + \omega \cdot 2 + 1, \dots, \omega^3, \dots, \omega^n, \dots, \omega^\omega, \dots, \omega^{\omega^\omega}, \dots, \omega^{\omega^{\omega^\omega}}, \dots, \epsilon_0, \dots \end{aligned}$$

Some ordinals α are *successor ordinals*, that is $\alpha = \beta + 1$ for some β . Otherwise α is a *limit ordinal*, except that 0 is the *initial ordinal*. In the above list the explicitly displayed limit ordinals are

$$\omega, \omega \cdot 2, \omega \cdot 3, \omega \cdot 4, \omega^2, \omega^2 + \omega, \omega^2 + \omega \cdot 2, \omega^3, \omega^n, \omega^\omega, \omega^{\omega^\omega}, \omega^{\omega^{\omega^\omega}}, \omega^{\omega^{\omega^{\omega^\omega}}}, \epsilon_0.$$

The class of ordinals is ordered as in its construction, that is

$$0 < 1 < 2 < 3 < \dots < \omega < \omega + 1 < \omega + 2 < \dots < \omega \cdot 2 < \dots < \omega^2 < \dots,$$

etc. Note that each ordinal has order type with domain its set of predecessors and ordering on this set as above.

The list of ordinals above does not even “scratch the surface” of the class of all ordinals.

In fact, the class of all ordinals is too big to even be a set, as we see later.*** ref***
Informally: if there were such a set it would itself be a set which is an ordinal, in which case we could get an even larger ordinal by taking its successor. But this gives a contradiction.

13.3 ORDINALS AND ITERATED PROCESSES

Ordinals arise naturally in the context of *transfinite iteration* of various operations.

For example, take some fixed set X (in particular, X may be $\mathbb{R}$). Let

$$V_0(X) = X, \quad V_{n+1}(X) = V_n(X) \cup \mathcal{P}(V_n(X)) \quad \text{for } n < \omega.$$

Then clearly

$$V_0(X) \subset V_1(X) \subset V_2(X) \subset \dots \subset V_n(x) \subset \dots$$

It is natural to extend this process up through the ordinals by defining

$$V_\omega(X) = \bigcup_{n < \omega} V_n(X), \quad V_{\omega+1}(X) = V_\omega(X) \cup \mathcal{P}(V_\omega(X)), \quad \dots$$

More generally, define

$$V_\alpha(X) = \begin{cases} X & \text{if } \alpha = 0, \\ V_\beta(X) \cup \mathcal{P}(V_\beta(X)) & \text{if } \alpha = \beta + 1, \\ \bigcup_{\beta < \alpha} V_\beta(X) & \text{if } \alpha \text{ is a limit ordinal.} \end{cases}$$

Then clearly

$$V_\beta(X) \subset V_\alpha(X) \quad \text{if } \beta < \alpha.$$

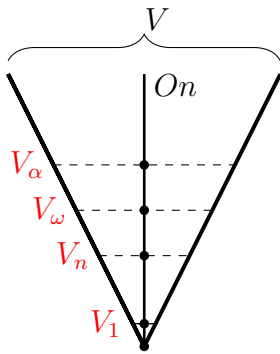
One surprising but useful observation is that one can, and it is standard to, take $X = \emptyset$ and build up all the sets used in set theory in this manner.

In this case

$$V_\alpha = \begin{cases} \emptyset & \text{if } \alpha = 0, \\ \mathcal{P}(V_\beta) & \text{if } \alpha = \beta + 1, \\ \bigcup_{\beta < \alpha} V_\beta & \text{if } \alpha \text{ is a limit ordinal.} \end{cases}$$

Exercise: I claim that $V_\alpha \subset V_{\alpha+1}$ for all α . Note that $V_0 \subset V_1$, and convince yourself by a sort of induction that it is always true. We will make the argument more rigorous later by “transfinite induction”, but it is good to think about it now with this straightforward example.

Use this result to show that $V_\alpha = V_\alpha(\emptyset)$, with $V_\alpha(\emptyset)$ as previously defined. $\square$



- **On**: collection of ordinals
- **Iterate power set** along On
- $V_0 = \emptyset$, $V_{\alpha+1} = \mathcal{P}(V_\alpha)$, $V_\lambda = \bigcup_{\alpha < \lambda} V_\alpha$ (limit λ)
- The sets in V_1 , V_n , V_ω , V_α , respectively are “points” below the corresponding dashed lines
- Black dots on the On line are ordinals $0, 1, n, \omega, \alpha$, respectively.
- $0 \in V_1$; $0, \dots, n-1 \in V_n$; $0, \dots, n, \dots \in V_\omega$;
 $\beta \in V_\alpha$ if $\beta < \alpha$

Remark 13.8 (Tetrational Growth). The size/cardinality of the V_n grows mind-bogglingly fast. Denoting by $|A|$ the number of elements in the set A , we have

- $|V_0| = 0$,
- $|V_1| = 1$,
- $|V_2| = 2$,
- $|V_3| = 2^2 = 4$,
- $|V_4| = 2^4 = 2^{2^2} =: 2 \uparrow\uparrow 3 = 16$,
- $|V_5| = 2^{16} = 2^{2^{2^2}} =: 2 \uparrow\uparrow 4 = 65,536$,
- $|V_6| = 2^{65,536} = 2^{2^{2^{2^2}}} =: 2 \uparrow\uparrow 5 > 10^{19728}$, ...

So $|V_6|$ is far, far greater than the $\approx 10^{17}$ seconds since the big bang, or the $\approx 10^{80}$ atoms in the observable universe. $\square$

CANTOR

Cantor introduced ordinals in the following similar setting for his analysis of trogonometric series. aside Cantor

If $A \subset \mathbb{R}$ then its derived set $A' \subseteq A$ is obtained by removing all isolated points and

keeping the limit points. Repeating this operation gives a decreasing ordinal sequence of sets:

$$\begin{aligned} A_0 &:= A \supset A_1 := A'_0 \supset A_2 := A'_1 \supset \dots \\ &\supset A_\omega := \bigcap_{n < \omega} A_n \supset A_{\omega+1} := A'_\omega \supset \dots \end{aligned}$$

The transfinite sequence is continued until it is no longer strictly decreasing.

See [Wikipedia: Trigonometric Series](#)

The set $\mathcal{B} \subset \mathcal{P}(\mathbb{R})$ of Borel subsets of the set $\mathbb{R}$ of real numbers, is defined to be the smallest collection of containing all open subsets, and which is closed under complements, countable intersections and countable unions.

The existence of $\mathcal{B}$ is obtained by taking the intersection of all such collections of subsets of $\mathbb{R}$.

A more constructive definition of $\mathcal{B}$ is obtained by defining, for any $\mathcal{C} \subseteq \mathcal{P}(\mathbb{R})$, $\mathcal{C}^*$ to consist of all sets in $\mathcal{C}$ or obtained by taking complements, countable intersections and countable unions.

Beginning with $\mathcal{B}_0$ to be the union of the set of all open sets and the set of all closed sets, define

$$\begin{aligned} \mathcal{B}_0 &\subset \mathcal{B}_1 := \mathcal{B}_0^* \subset \mathcal{B}_2 := \mathcal{B}_1^* \subset \dots \\ &\subset \mathcal{B}_\omega := \bigcup_{n < \omega} \mathcal{B}_n \subset \mathcal{B}_{\omega+1} := \mathcal{B}_\omega^* \subset \dots \end{aligned}$$

The transfinite sequence continues until it is no longer strictly increasing (which is up to the first uncountable ordinal as we will later see).

13.4 EQUIVALENT DEFINITIONS OF WELL-ORDERINGS

The next theorem gives three equivalent definitions for a well-ordered set, see Definition 13.10. The remark after Theorem 13.13 shows that well-ordered sets are precisely those linear orders over which one can do induction, and leads to yet a fourth definition.

Theorem 13.9. *If $\mathcal{A} = \langle A, < \rangle$ is a linearly ordered set, then the following are equivalent:* thm:wo-equiv

- (i) *A contains no infinite descending sequences $a_1 > a_2 > a_3 > \dots$.*
- (ii) *Every nonempty $S \subseteq A$ contains a least member.*
- (iii) *Every initial segment $I \neq A$ has a least next element $a \in A$, that is*

$$I < a \wedge (I < x \implies a \leq x)$$

Proof.

(i) $\implies$ (ii): Assume (i). If $\emptyset \neq S \subseteq A$ has no least member, construct a sequence

$$a_1 > a_2 > a_3 > \dots > a_n > \dots$$

of elements from S as follows:

choose $a_1 \in S$ (possible as $S \neq \emptyset$)

choose $a_2 \in S$ so that $a_2 < a_1$ (possible as otherwise a_1 is a least member of S)

choose $a_3 \in S$ so that $a_3 < a_2$ (possible as otherwise a_2 is a least member of S)

etc.

(ii) $\Rightarrow$ (i): Assume (i) is false. Then there exists an infinite descending sequences $a_1 > a_2 > a_3 > \dots$. Let $S = \{a_1, a_2, a_3, \dots\}$. Then S is nonempty and has no least member. So (ii) is false.

(ii) $\Rightarrow$ (iii): Assume (ii). If $I \neq A$ is an initial segment then the least element of $A \setminus I$ is the least next element for I .

(iii) $\Rightarrow$ (ii): Assume (iii). If $\emptyset \neq S \subseteq A$, let $I = \{x \in A : x < S\}$. Then $I \subsetneq A$ is an initial segment (perhaps $= \emptyset$). The least next element for I is the required least member of S . $\square$

def:wo

Definition 13.10 (Well-Ordered Set). A linearly ordered set $A = (A, <)$ is *well-ordered* iff it satisfies one (hence all) of the conditions of Theorem 13.9.

USE OF THE AXIOM OF CHOICE

The construction of an infinite sequence in the proof of (i) $\Rightarrow$ (ii) uses a form of the axiom of choice – we discuss this axiom in Chapter 12.

The use of the axiom of choice here is NOT in the individual choices made in the proof of (i) $\Rightarrow$ (ii). This is allowable by the axioms for set theory as we are choosing from a nonempty set in each individual case.

The problem arises in asserting that the countably infinite number of individual choices corresponds to a single (infinite) set. I will say more in Chapter 12.

Exercise 13.11. Every subset of a well-ordered set is well-ordered under the induced ordering.

Exercise 13.12. The sum and product of well-ordered sets are well-ordered.

thm:induction-wo

Theorem 13.13 (Transfinite induction on well-ordered sets). Suppose $\mathcal{A} = \langle A, < \rangle$ is a well-ordering, $S \subseteq A$, and

$$\forall x \in A \left(\forall u < x (u \in S) \implies x \in S \right).$$

Then $S = A$.

Proof. If $A \setminus S \neq \emptyset$, take its least member and derive an immediate contradiction from Theorem 13.9(ii). $\square$

rem:inductive

Remark 13.14. Call a linear order $A = (A, <)$ *inductive* if every $S \subseteq A$ satisfies the hypothesis of Theorem 13.13. Then A is well-ordered: for if $I \subsetneq A$ is an initial segment with no least next element, then for every $x \in A$ we have $(\forall u < x (u \in I)) \Rightarrow x \in I$ (consider $x \in I$ and $x \notin I$ separately), hence $I = A$, a contradiction. Thus well-orders are precisely those linear orders over which we have proof by induction.

13.5 PROPERTIES OF WELL-ORDERINGS

Remark 13.15. Equivalence classes of linear orders under order-isomorphism are called *ordinals* (ordinal numbers). Write $\alpha < \beta$ iff some representative of α is isomorphic to some proper initial segment of some representative of β . The next results show that $<$ is a well-order on ordinal numbers, and for every α the set of predecessors of α under $<$ is a representative of α . rem:ordinals-intro

Theorem 13.16 (No self-isomorphism to a proper initial segment). thm:iso-segment-not-self If $A = (A, <)$ and $B \subsetneq A$ is an initial segment of A , then $B = (B, <)$ is not isomorphic to A . In fact, the only order-isomorphism from B to a segment of A is the identity.

Proof. If $f : B \rightarrow B' \subseteq A$ is an order isomorphism and B' is a segment of A , then for the first element b of B (and hence of A) we must have $f(b) = b$. Then f fixes the second element, and so on. Formally, if $f(b) \neq b$ for some $b \in B$, consider the least $x \in B$ with $f(x) \neq x$; a contradiction arises immediately. $\square$

Theorem 13.17 (Comparability of well-orders). thm:wo-comparability If $A = (A, <)$ and $B = (B, <)$ are well-ordered sets, then one is isomorphic to an initial segment of the other.

Proof. *** See Theorem 12 proof, my old notes, p 115

Consider order-isomorphisms from segments of A onto segments of B . Any two agree on the smaller domain, so they paste to a largest one. If it is defined on all of A we are done. If not, its range must be all of B (else extend it), and we are still done. $\square$

Corollary 13.18. cor:wo-trichotomy Given any two well-orderings, exactly one holds: they are isomorphic, or the first is isomorphic to a unique proper initial segment of the second, or the second to a unique proper initial segment of the first.

It follows that $<$ is a linear ordering of ordinal numbers. More is true.

Theorem 13.19. thm:On-wo $<$ is a well-order of the class of ordinal numbers. Moreover, every ordinal has as a representative the set of ordinal numbers less than it together with the ordering induced from $<$.

Proof. For the second claim: if $(A, <)$ represents α , then by Corollary 13.18 the set $\{\beta : \beta < \alpha\}$ is order-isomorphic to $(A, <)$ via the map taking each β to the corresponding proper initial segment of A .

See my old notes botom p115, top p 116

For the first claim: if S is a nonempty set of ordinals, take $\alpha \in S$. If α is least we are done; otherwise $S \cap \{\beta : \beta < \alpha\}$ has a least element because it is a subset of the well-ordered set $\{\beta : \beta < \alpha\}$. $\square$

Definition 13.20. The well-ordered class of all ordinals is written On .

Definition 13.21 (Successor and limit). def:limit-successor An ordinal $\alpha > 0$ is a *limit ordinal* iff it has no immediate predecessor. Otherwise (if $\alpha > 0$) it is a *successor ordinal*.

If α is a successor ordinal, then $\alpha = \beta + 1$ where β is the immediate predecessor of α .

Using Theorem 13.9(ii), one can see that every ordinal can be written (uniquely) in the form $\Lambda + n$ where Λ is a limit ordinal and $n \in \mathbb{N}$ (perhaps 0). For if β is a limit ordinal we are done. If not, let $\beta_0 = \beta + 1$, $\beta_1 = \beta_0 + 1$, and so on. Since $\beta > \beta_0 > \beta_1 > \dots$ we must stop at some $\beta_m = \Lambda$, and $\beta = \Lambda + n$.

13.6 TRANSFINITE INDUCTION AND RECURSION

We give informal versions of these ideas, i.e. the versions used in everyday mathematics, as opposed to those formulated precisely in a specific set-theoretic language. sec:TI-TR

Definition 13.22 (Transfinite induction). Let S be an initial segment of the ordinals (the set of ordinals less than some ρ). Let $P(\alpha)$ be a property of ordinals. Suppose for every $\alpha \in S$, def:TI

$$(\forall \beta < \alpha P(\beta)) \Rightarrow P(\alpha)$$

(and in particular $P(0)$). Then $P(\alpha)$ holds for every $\alpha \in S$.

Idea. Consider the first $\alpha \in S$ (if any) where $P(\alpha)$ fails; this contradicts the hypothesis at α . □

Remark 13.23. In practice, verify separately:

- (i) $P(0)$;
- (ii) $P(\alpha) \Rightarrow P(\alpha + 1)$ for successors;
- (iii) if λ is a limit and $\forall \beta < \lambda P(\beta)$, then $P(\lambda)$.

Definition 13.24 (Transfinite recursion). Let S be an initial segment of the ordinals. Let G be a binary operation. There is a unique function F with domain S such that for every $\alpha \in S$, def:TR

$$F(\alpha) = G(\alpha, F \upharpoonright \alpha).$$

Idea. Let $\mathcal{F}$ be the family of functions f with (i) $\text{dom}(f)$ an initial segment of S , and (ii) $f(\alpha) = G(\alpha, f \upharpoonright \alpha)$ for all $\alpha \in \text{dom}(f)$. Any two are compatible (argue by the first place they differ). Their union F lies in $\mathcal{F}$ and must have domain S (else extend), giving existence; uniqueness is by the same minimum-counterexample argument. □

Remark 13.25 (Typical format). Often one specifies F by

$$F(0) = A, \quad F(\alpha + 1) = G(F(\alpha)), \quad F(\lambda) = H(\{F(\beta) : \beta < \lambda\})$$

for given A , unary G , and unary H , handling 0, successor, and limit stages separately.

[This chapter to be completed]

Chapter 14

Cardinals

ch card

14.1 CARDINAL NUMBERS

Definition 14.1. $A \sim B$ iff there is a bijection $f : A \rightarrow B$. This is an equivalence relation on sets. Each equivalence class is called a *cardinal number*. We write κ , or $|A|$, or $\text{card}(A)$ for the cardinal associated with A . Cardinal numbers are often written $\aleph_0, \aleph_1, \aleph_2, \dots$

Definition 14.2. Write $\kappa \leq \lambda$ iff there are sets A, B with $|A| = \kappa$, $|B| = \lambda$ and an injection $A \hookrightarrow B$. Write $\kappa < \lambda$ iff $\kappa \leq \lambda$ and $\kappa \neq \lambda$.

Remark 14.3. One shows easily: if $\kappa \leq \lambda$ and $\lambda \leq \mu$ then $\kappa \leq \mu$; and by Schröder–Bernstein, $\kappa \leq \lambda$ and $\lambda \leq \kappa$ imply $\kappa = \lambda$. Thus $\leq$ is a partial order on cardinals (no use of AC needed).

Remark 14.4. A cardinal number via equivalence classes is a proper class. Alternatively, identify cardinals with particular ordinals (initial ordinals); ordinals themselves can be identified with particular sets.

Remark 14.5 (Cardinals as initial ordinals). Identify each cardinal κ with the least ordinal α that well-orders some (hence every) set of size κ . By the Well-Ordering Principle this exists (here AC is used). Then the class Card of cardinals is a subclass of On and is well-ordered. It is not a set: for every ordinal α , $|\mathcal{P}(\alpha)|$ is a strictly larger cardinal (Theorem 14.14); also On is a proper class.

Remark 14.6 (Counting ordinals). It is conventional to enumerate the infinite cardinals as $\aleph_\alpha$ for ordinals α .

Remark 14.7. For each ordinal α , let $|\alpha|$ be the cardinality of the set of predecessors of α . Clearly $|\alpha| \leq \alpha$ and $|\alpha| = \alpha$ iff α is a cardinal. Hence if κ is a cardinal and $\alpha < \kappa$ then $|\alpha| < \kappa$.

In particular, every set has a well-order such that every initial segment has smaller cardinality than the whole set. If $|\alpha| = \aleph_0$ we say α is *countable*. All ordinals listed in §13.7–§13.17 are countable (or finite).

All ordinals listed in §13.7–13.17 are countable (or finite).

Definition 14.8 (Cardinal addition and multiplication). Let $|A| = \kappa$, $|B| = \lambda$ and $A \cap B = \emptyset$. Then

$$\kappa + \lambda = |A \cup B|, \quad \kappa \cdot \lambda = |A \times B|.$$

Remark 14.9. Cardinal addition and multiplication differ from ordinal addition and multiplication; the context should make it clear which is intended.

lem card mult

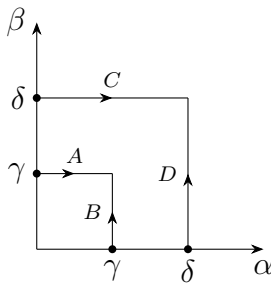
Lemma 14.10. *If κ is an infinite cardinal then $\kappa \cdot \kappa = \kappa$. (cardinal multiplication)*

Proof Idea. We will treat cardinals as particular ordinals.

We define a linear ordering on $On \times On$ as indicated in the diagram below and check that this is a well-ordering.

We then show by a cardinality argument that for any infinite cardinal κ , the restriction to $\kappa \times \kappa$ is isomorphic to the ordering on κ itself. Hence $\kappa \cdot \kappa = \kappa$.

The key idea is, assuming there is some cardinal which does not have this property, to consider the *first* cardinal which does not have this property, and obtain a contradiction.



Points in A, B, C, D are in the order shown. And $A \prec B \prec C \prec D$ in an obvious sense.

Points in $A \cup B$ are less than points in $C \cup D$ by condition (a) in the proof.

Points in A have their relative order determined by condition (b), as do points in C .

Points in B have their relative order determined by condition (c), as do points in D . $\square$

Proof. (See the previous diagram and explanation)

Define a linear ordering $\prec$ on $On \times On$ by $(\alpha_1, \beta_1) \prec (\alpha_2, \beta_2)$ iff

- (a) $\max\{\alpha_1, \beta_1\} < \max\{\alpha_2, \beta_2\}$ or
- (b) $\max\{\alpha_1, \beta_1\} = \max\{\alpha_2, \beta_2\}$ and $\alpha_1 < \alpha_2$ or
- (c) $\max\{\alpha_1, \beta_1\} = \max\{\alpha_2, \beta_2\}$ and $\alpha_1 = \alpha_2$ and $\beta_1 < \beta_2$.

It is immediate that $\prec$ is a linear ordering, and straightforward to show it is a well-ordering. *Exercise*

For any ordinal λ , the map $\gamma \mapsto (0, \gamma)$ for $0 \leq \gamma < \lambda$, is an injection from $\langle \lambda, < \rangle$ into $\langle \lambda \times \lambda, \prec \rangle$. It follows that the well-ordering $\langle \lambda, < \rangle$ is isomorphic to an initial segment of $\langle \lambda \times \lambda, \prec \rangle$.

However, it need *not* be the case that $\langle \lambda, < \rangle$ is isomorphic to a *proper* initial segment of $\langle \lambda \times \lambda, \prec \rangle$. For example, $\langle \omega, < \rangle$ is isomorphic to $\langle \omega \times \omega, \prec \rangle$, that is to $\langle \aleph_0 \times \aleph_0, \prec \rangle$.

Exercise

We claim that for *every* (infinite) cardinal κ the well-ordering $\langle \kappa, < \rangle$ is isomorphic to the well-ordering $\langle \kappa \times \kappa, \prec \rangle$, and so not to a *proper* initial segment.¹

If the claim is false, let κ be the least cardinal such that the well-ordering $\langle \kappa, < \rangle$ is isomorphic to a *proper* initial segment of $\langle \kappa \times \kappa, \prec \rangle$ (and so not to $\langle \kappa \times \kappa, \prec \rangle$ itself).

¹This is subtle. The well-orderings

$$0, 1, 2, \dots, n, \dots$$

$$(0, 0), (1, 1), (2, 2), \dots, (n, n)$$

n an integer

Let the proper initial segment be $\{(\alpha, \beta) : (\alpha, \beta) \prec (\gamma, \delta)\}$, where $\gamma, \delta < \kappa$. Let $\xi = \max\{\gamma, \delta\}$. Then

$$\langle \kappa, < \rangle \cong \{(\alpha, \beta) : (\alpha, \beta) \prec (\gamma, \delta)\} \subseteq \xi \times \xi,$$

where $\cong$ denotes isomorphism of order types.

Taking cardinalities of $\langle \aleph_\zeta, < \rangle$ and $\xi \times \xi$, $\aleph_\zeta \leq |\zeta|^2$ (where $|\zeta|$ is the cardinality of ζ). □

prop:RxR-wellorder

Proposition 14.11. *There is a well-order of $\text{On} \times \text{On}$ such that, for each fixed α , the set $\alpha \times \text{On}$ is an initial segment isomorphic to On . In particular, $|\alpha \times \alpha| = |\alpha|$ for every infinite ordinal α .*

Idea. Order pairs (α, β) by the maximum first, breaking ties by first then second coordinate. This is a well-order with the stated initial segment property. A minimal-counterexample argument yields a bijection $\alpha \times \alpha \cong \alpha$ for infinite α . □

thm:card-arith

Theorem 14.12 (Cardinal arithmetic for infinite cardinals). *If at least one of κ, λ is infinite, then*

$$\kappa \cdot \lambda = \kappa + \lambda = \max\{\kappa, \lambda\}.$$

Proof. Let $\mu = \max\{\kappa, \lambda\}$. Then $\mu \leq \kappa \cdot \lambda \leq \mu \cdot \mu = \mu$ by Proposition 14.11. For addition, $\mu \leq \mu + \mu \leq \mu \cdot 2 = \mu$; equivalently, partition μ into two disjoint subsets each equinumerous with μ . □

Definition 14.13. $2^\kappa := |\mathcal{P}(\kappa)|$.

thm:cantor

Theorem 14.14 (Cantor). $\kappa < 2^\kappa$ for every cardinal κ .

Proof. If $f : \kappa \rightarrow \mathcal{P}(\kappa)$ were surjective, let $A = \{\alpha \in \kappa : \alpha \notin f(\alpha)\}$. Then $A = f(\gamma)$ for some γ , giving $\gamma \in A \iff \gamma \notin A$, a contradiction. □

Remark 14.15. Thus $|\mathbb{R}| = 2^{\aleph_0}$. The Continuum Hypothesis asserts $2^{\aleph_0} = \aleph_1$; the Generalised Continuum Hypothesis asserts $2^\kappa = \kappa^+$ for all infinite κ .

prop:finite-seqs

Proposition 14.16. *If A is infinite with $|A| = \kappa$, then the set S of finite sequences of elements of A also has cardinality κ .*

Exercise. Show $\bigcup_{n < \omega} A^n$ has size κ when κ is infinite. □

Chapter 15

Zermelo–Frankel Set Theory

ch form st

Zermelo–Fraenkel set theory, usually ZF or ZFC, is a first-order theory with equality in the language $\mathcal{L}_\in$ having just the one non-logical binary relation $\in$, for membership. We discuss the axioms for set theory, and their models, in the following sections.

So how can there be a first-order theory for set theory, where we certainly *do have* quantifiers ranging over subsets? The idea is that in a model of set theory the sets will themselves be the individual members of the model of the axioms being considered.

The ZFC axioms for set theory are essentially sufficient to establish all of the properties of set theory needed in mathematics. And essentially all of mathematics *can* be described within set theory.

There is no “free lunch” however. The ZFC axioms for set theory do not fully mirror our informal intuition about what sets should be. There are infinitely many non-standard models of set theory also satisfying the ZFC axioms. Peano axioms for arithmetic and axioms for the real number system similarly have non-standard models. These non-standard models have some amazing and beautiful applications which are developed in the subject of non-standard analysis. For starters, you might look at (Keisler, 2000) and (Keisler, 2022)

GB SET THEORY

There *are* extensions of the ZF and ZFC axioms which include variables ranging over classes of sets. This is known as Bernays–Gödel set theory and denoted BG and BGC respectively. All sets are classes but not conversely, and classes are describable by first-order formulas in the original ZF language. There is even a class of all sets, but classes cannot be elements of other sets or of other classes. See (Jech, 2003, p 70). The BG and BGC extensions are convenient but they do not introduce any profound new ideas.

15.1 METATHEORY

DON'T

If you want to see detailed discussions of the metatheory, references are I STRONGLY recommend against wasting time by doing so at this stage. It would be like exploring underground caves without a flashlight.
We will discuss models of ZF (and so in particular of ZFC) by using the same simple

elementary set theory as we use to discuss models of group theory and models of linear orderings.

However, it may seem paradoxical to discuss set theory by using set theory, the latter being called the *metatheory* in this context. But the metatheory is usually simple and constructive and does not use sophisticated notions from ZFC.

In particular, the discussion of proofs in Chapter 3 is finitistic. Basically it is all about marks on a piece of paper and allowable ways to manipulate them. The proof of Gödel's completeness theorem concerning the construction of a model is not finitistic, but in the case of a countable language (as for ZFC, for groups and for linear orderings) it is still fairly constructive and non-controversial philosophically. Although one proceeds through a countably infinite number of steps, each step is of a similar type, and is not unlike the notion of constructing the positive integers by adding one at each step. The proofs in Chapter 9 of the Gödel incompleteness theorems are constructive and finitistic.

15.2 STRUCTURES (V, ϵ)

Just as there are many non-isomorphic models of the axioms for a linear ordering, and of the axioms for a group, there will be many different models for ZF and ZFC. But we are trying to capture in the ZFC axioms our intuitive idea of a universe of sets by means of a structure $\{V, \epsilon\}$ where V is a (meta)set in the meta-universe in which we discuss models of various first-order theories, and ϵ is a binary relation on V . We write $a \in b$ for $\epsilon(a, b)$.

So $\{V, \epsilon\}$ will be a model of (or occasionally just some of) the ZFC axioms. The axioms are discussed on pages 53-61 of (Johnstone, 1987) and will be discussed here.

As strange as it may seem, there will even be countable models $\{V, \epsilon\}$ of ZFC, although within ZFC we can prove the existence of uncountable sets. The point is that a one-one correspondence that exists in the metatheory between the natural numbers and the members of V will not be represented by any member of V .

Along with the axioms themselves, it is helpful to simultaneously think of their intended interpretation in a fixed structure $\{V, \epsilon\}$.

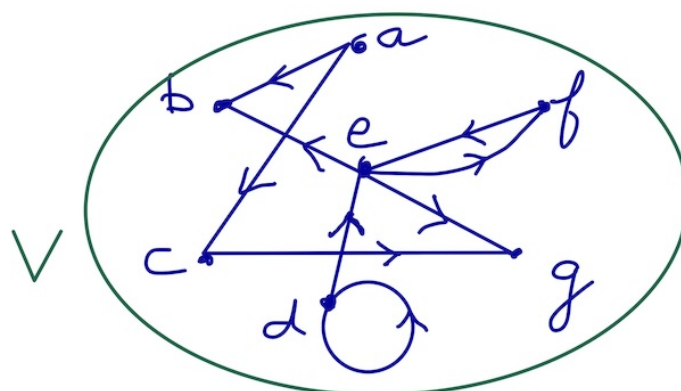


Figure 15.1: A very silly example of a $\{V, \epsilon\}$ structure. Read the arrow from a to b as “ b is a member of a ”. Probably no ZF axioms hold here.

fig:V_structure

Analogous to the situation for structures concerning the language of groups or the

language of linear orderings, V is a set in the metatheory and $\in$ is a binary relation on V called the *membership relation*.

It is sometimes helpful to have a diagram in mind. In Figure 15.1,

$$b \in a, \quad c \in a.$$

We could write $\in ba$, but since we are humans and not computers we do not do so. Noting the direction of the relevant arrows, we do sometimes write $a \ni b$ and $a \ni c$.

Similarly,

$$b \in a, \quad c \in a; \quad g \in c; \quad d \in d, \quad e \in d; \quad b \in e, \quad f \in e, \quad g \in e; \quad f \in f.$$

We can also write

$$a = \{b, c\}, \quad b = \varphi, \quad c = \{g\}, \quad d = \{d, e\}, \quad e = \{b, f, g\}, \quad f = \{f\}, \quad g = \varphi.$$

In particular, b and g have no members.

15.2.1 NOTATION FOR FIRST-ORDER LANGUAGE V. METALANGUAGE

In the previous example, following the notation conventions of (Johnstone, 1987, page 53), a, b, c, d, e, f, g are *elements* of V and so are called *sets*.

We read $b \in a$ as b *belongs* to a , b is a *member* of a , b is an *element* of a , or a *contains* b . More precisely, $b \in a$ is the assertion that the first-order formula $v_1 \in v_2$ interpreted in $\{V, \in\}$ with v_1 assigned the value b and v_2 assigned the value a , is true.

On the other hand, we say a, b, c, d, e, f, g are *in* (the meta-set) V . Note this is a statement in the *metalanguage*, since in particular V is not a set in V .

Often we will not be so careful in terms of our terminology. It should be clear from context what is intended.

15.3 ZFC AXIOMS

15.3.1 EXTENSIONALITY AXIOM

Two sets are equal if they have the same members.

$$\forall u (u \in x \leftrightarrow u \in y) \rightarrow x = y \quad (15.1)$$

This is not true in Figure 15.1, as b and g are different sets in V but have the same members, namely no members.

Conversely, if two sets are equal then they have the same members. But this is just because of the interpretation of the equality symbol “=” in first-order logic.

15.3.2 PAIR AXIOM

For any two sets x and y there is a set whose members are exactly x and y . This is the *Pair Axiom*:

$$\forall x, y \exists z \forall t (t \in z \leftrightarrow t = x \vee t = y) \quad (15.2)$$

We write $z = \{x, y\}$. If $x = y$ then we write $z = \{x\}$.

15.3.3 SEPARATION AXIOM-SCHEMA

sec:sep

In general there is never a set z such that

$$z = \{x \mid x \notin x\}. \quad \text{eq:Russell} \quad (15.3)$$

For if there were such a set z , then either $z \in z$ or $z \notin z$. In the first case it follows from the definition of z that $z \notin z$ and in the second it similarly follows that $z \in z$. Contradiction either way.

Separation Schema. ¹ if in (15.3) we restrict the x to some set y then there is no paradox. More generally, for any formula $\varphi(x, z_1, \dots, z_n)$,

$$\forall z_1, \dots, z_n \forall y \exists z \forall x (x \in z \leftrightarrow x \in y \wedge \varphi(x, z_1, \dots, z_n)) \quad (15.4)$$

We write

$$z = \{x \in y \mid \varphi(x, z_1, \dots, z_n)\}. \quad (15.5)$$

Free variables $z_1, \dots, z_n$ in φ will be interpreted as sets in V . In many cases there are no such free variables and $\varphi = \varphi(x)$ has just the one free variable x .

15.3.4 EMPTY SET AXIOM

So far we may not actually have any sets in our model. It is convenient to add the *Empty Set Axiom*

$$\exists x \forall y (y \notin x). \quad (15.6)$$

We write $x = \varnothing$. It is unique by extensionality.

15.3.5 UNION AXIOM

For every set x there is a set y whose members are the members of members of x . Equivalently, y is the union of all the members of x . This is the *Union Axiom*

$$\forall x \exists y \forall t (t \in y \leftrightarrow \exists z (t \in z \wedge z \in x)) \quad (15.7)$$

We write $y = \bigcup x$. If $x = \{z, w\}$ then we write $z \cup w$ for $\bigcup x$. See (Johnstone, 1987, page 56) for the intersection and set-theoretic difference of sets.

15.3.6 POWER SET AXIOM

Johnstone uses the notation ² $z \subseteq x$, which is read as “ z is a subset of x ”, as an abbreviation for

$$\forall t (t \in z \rightarrow t \in x).$$

The *Power Set Axiom* says that for every set x there is another set y whose members consist of all subsets of x .

$$\forall x \exists y \forall z (z \in y \leftrightarrow z \subseteq x). \quad (15.8)$$

We write $y = \mathcal{P}(x)$.

¹The term “Schema” refers to the fact that there is a separate axiom for each formula φ .

However, The word "separation" since the formula φ separates the set z into two parts.

²The more common notation now for “ z is a subset of x ” is $z \subset x$. If z is a *proper subset* of x , that is $z \subset x$ and $z \neq x$, then one writes $z \subsetneq x$.

15.3.7 AXIOM OF INFINITY

This axiom provides for the existence of an infinite set: ³

$$\exists x \left(\varphi \in x \wedge \forall y \in x (y \cup \{\varphi\} \in x) \right). \quad \text{ax:infnty (15.9)}$$

It follows from the axiom that such a set x has members

$$\varphi, \{\varphi\}, \{\varphi\} \cup \{\{\varphi\}\}, \dots$$

This is used to define the natural numbers: ⁴

$$\begin{aligned} 0 &:= \varphi \\ 1 &:= \{\varphi\} = \{0\} \\ 2 &:= \{0\} \cup \{\{0\}\} = \{0, 1\} \\ 3 &:= \{0, 1\} \cup \{\{0, 1\}\} = \{0, 1, 2\} \\ 4 &:= \{0, 1, 2\} \cup \{\{0, 1, 2\}\} = \{0, 1, 2, 3\} \\ &\vdots \end{aligned} \quad (15.10)$$

This turns out to be very useful, since both

$$0 \in 1 \in 2 \in 3 \in 4 \in 5 \in \dots \quad (15.11)$$

and

$$0 \subset 1 \subset 2 \subset 3 \subset 4 \subset 5 \subset \dots \quad (15.12)$$

give the standard linear ordering.

15.3.8 DEFINITION OF ω

There is not a unique such infinite set x in (15.9) in each model of ZF, but in a model we can take the *intersection* of all such infinite sets, to define another set *which we define to be* ω . This intersection can be justified in a straightforward manner because of the ZF axioms defined so far, see (Johnstone, 1987, page 58) following $(\text{Inf})_w$. Thus we *might* write

$$\omega = \{0, 1, 2, 3, 4, \dots\} \quad (15.13)$$

This can be misleading however, as ω may also contain other “non-standard integers” as we now discuss.

15.3.9 NON-STANDARD INTEGERS – SOME WEIRD STUFF

You can skip this subsection, but you really should not, even though the details might not be clear at this stage. It is motivation for the subject of non-standard analysis, which allows us to work rigorously with infinitesimals, positive “hyperreal” numbers less than any standard positive real number. See (Keisler, 2000; 2022) for a very

³Instead of $y \cup \{\varphi\} \in x$ one could just require $\{\varphi\} \in x$. But this is technically less useful. It does not immediately lead to a definition of the natural numbers such that each natural number n is the set containing all the previous natural numbers.

⁴The symbol $:=$ means “is defined to equal”.

approachable treatment.

This is what Johnstone is alluding to in (Johnstone, 1987, page 63, Example 5.2), first paragraph, beginning at “Since every subset of ω ...”

We cannot exclude the possibility, *by means of first-order formulas*, that in a model of ZF there will be members of ω not equal to any of the standard integers. We might want the model to satisfy

$$\forall x \in \omega (x = 0 \vee x = 1 \vee x = 2 \vee x = 3 \vee \dots) \quad (15.14)$$

But this is an infinitary formula, not a first-order formula.

In fact, for the ZF axioms or any consistent set of sentences Φ which extends ZF, consider the following infinite set of sentences with one new constant symbol c .

$$\Phi, c \in \omega, c \neq 0, c \neq 1, c \neq 2, c \neq 3, \dots \quad (15.15)$$

Every finite subset is consistent (why?) and so by the compactness theorem there is a model of all these sentences.⁵ In any such model the interpretation of the constant symbol c is then a non-standard integer, which is a set in ω distinct from all the standard integers.

15.3.10 ORDER TYPE OF NON-STANDARD INTEGERS

If the model $\{V, \in\}$ has non-standard sets/integers in ω , then the elements of ω will be *linearly ordered* under the membership relation $\in$ (and the set inclusion relation $\subsetneq$) which we now also write as $<$, since being linearly ordered is provable from the ZF axioms. Moreover, as well as $c \in \omega$ there will be other non-standard integers which we write as

$$\dots < c - 3 < c - 2 < c - 1 < c < c + 1 < c + 2 < c + 3 < \dots \quad \text{eq:} < \text{ in } \omega \quad (15.16)$$

and call the $\uparrow$ emphorbit of c . In fact it is not too difficult⁶ to show that the order type of all non-negative integers in any such model is

$$\mathbb{N} + \mathbb{Z} \times \mathbb{Q} \quad \text{order_nonstandard_integers} \quad (15.17)$$

See Figure 15.2. Here $\mathbb{N}$ has the order type of the natural numbers, $\mathbb{Q}$ has the order type of the rationals (the unique countable dense linear ordering) and $\mathbb{Z}$ has the order type of the standard integers, positive and negative. We read (15.17) as the linear order type of $\mathbb{N}$ followed by $\mathbb{Q}$ copies of $\mathbb{Z}$.

We will soon add an “Axiom of Foundation” to ZF, which implies that every set in $\{V, \in\}$ has an $\in$ -minimal element, see Section 15.3.12. This then implies that *the metaset* $\mathbb{N} = \{0, 1, 2, 3, \dots\}$ *is not normally a set/element in* $\{V, \in\}$,⁷ since if it were then the non-standard part $\omega \setminus \mathbb{N}$ of ω would also be a set in V . But $\omega \setminus \mathbb{N}$ has no $\in$ -minimal element because of the order type (15.17), contradicting the Axiom of Foundation.

⁵For the compactness theorem see (Johnstone, 1987, page 27, Corollary 3.8). The “magic” of first-order logic arises from the compactness theorem.

⁶The key points in establishing (15.17) are that for any non-standard integer c the integer $2c$ is greater than any integer in the orbit of c and so generates its own orbit. Moreover, between any two orbits generated by c and d there is another orbit generated by $(c + d)/2$ or $(c + d + 1)/2$, say.

⁷It follows easily from the previous axioms that if x and y are sets then so is $x \setminus y$. See (Johnstone, 1987), page 56, paragraph following $(\text{Un})_s$.

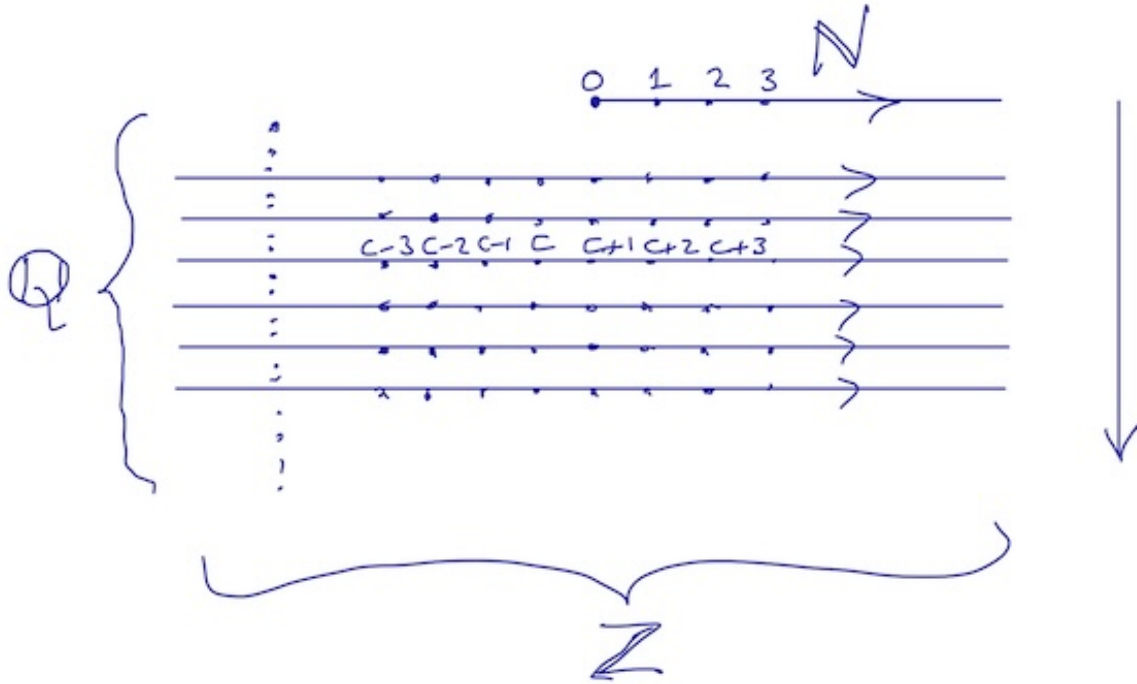


Figure 15.2: Order type of all positive integers in a model with non-standard integers. The ordering is to the right and then downwards.

fig:nonstandard_integers

A FINAL REMARK

Although models of ZF will normally contain non-standard integers, we can still speak of a *standard integer* model as being one in which the only members of ω are the standard integers

$$0, 1, 2, \dots, n \dots$$

We cannot specify this by means of first-order sentences, we need to do it in the metatheory.

15.3.11 REPLACEMENT AXIOM SCHEMA

For each formula $\varphi(x, y, v_1, \dots, v_k)$ we have the following axiom: ⁸

$$\begin{aligned} \forall x, y_1, y_2 \left(\left(\varphi(x, y_1, v_1, \dots, v_k) \wedge \varphi(x, y_2, v_1, \dots, v_k) \right) \rightarrow y_1 = y_2 \right) \\ \rightarrow \forall t \exists u \forall y \left(y \in u \leftrightarrow \exists x \in t \varphi(x, y, v_1, \dots, v_k) \right). \end{aligned} \quad \text{ax:repl (15.18)}$$

Moreover, as usual, it is understood that one universally quantifies over the free variables $v_1, \dots, v_k$.

In order to understand (15.18) first suppose there are no free variables $v_1, \dots, v_k$, and hence consider a formula $\varphi(x, y)$ and corresponding axiom:

$$\begin{aligned} \forall x, y_1, y_2 \left(\left(\varphi(x, y_1) \wedge \varphi(x, y_2) \right) \rightarrow y_1 = y_2 \right) \\ \rightarrow \forall t \exists u \forall y \left(y \in u \leftrightarrow \exists x \in t \varphi(x, y) \right). \end{aligned} \quad \text{ax:repl2 (15.19)}$$

⁸Here and elsewhere I have tried for comparison purposes to follow the choice of variable names at the corresponding point in (Johnstone, 1987)

Axiom (15.19) asserts the following in a model $\{V, \in\}$: Suppose that for every set x in the universe V there is *at most* one set y in V such that $\varphi(x, y)$ is true. (In this sense, $\varphi(x, y)$ defines a partial function⁹ $x \mapsto y$.) Then for every set t its image under the partial function φ is a set u in V .

Similarly Axiom (15.18) asserts the following in $\{V, \in\}$: Suppose that for some set of parameters $a_1, \dots, a_k$ in V , it is true for every set x in V that there is *at most* one set y such that $\varphi(x, y, a_1, \dots, a_k)$ is true. In this sense, $\varphi(x, y, a_1, \dots, a_k)$ defines a partial function $x \mapsto y$. Then for every set t its image under this partial function is a set u in V .

The *motivation* behind the replacement axioms is that the image of a set t under a well-defined (partial) map should also be a set, here called u . In particular, since t should not be larger in cardinality than u , u should also be a set – unlike the class V of all sets, which is too big to be a set.

15.3.12 AXIOM OF FOUNDATION

Definition 15.1. Suppose x is a set in V and $x \neq \varnothing$. Then y is $\in$ -minimal in/for x if $y \in x$ and there is no $z \in x$ such that $z \in y$. That is, $y \in x$ and $y \cap x = \varnothing$.

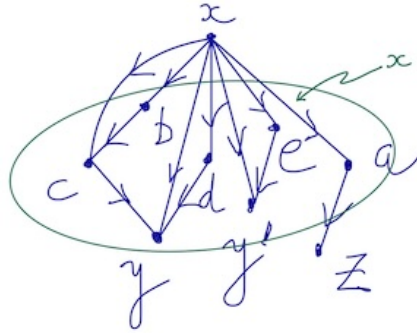


Figure 15.3: Another silly universe $\{V, \in\}$, with V containing the sets $x, b, c, d, e, y, y', a, z$. Here $x = \{b, c, d, e, y, y', a\}$, but $z \notin x$. The members of x are indicated in green. The sets y, y', a are all $\in$ -minimal in x . The sets y, y', z are $\in$ -minimal in V , but a is not $\in$ -minimal in V . See Definitions 15.1 and ??.

fig:minimal

The *Axiom of Foundation* is: Every non-empty set x contains a set y that is $\in$ -minimal for x .

That is

$$\forall x \neq \varnothing \exists y \in x \forall z \in x (z \notin y) \quad (15.20)$$

*Consequences*¹⁰

- No infinite descending chains. That is, no set $x = \{x_1, x_2, x_3, \dots\}$ such that $x_1 \ni x_2 \ni x_3 \ni \dots$.
- No cycles. That is, no set $x = \{x_1, x_2, x_3, \dots, x_n\}$ such that $x_1 \in x_2 \in x_3 \in \dots \in x_n \in x_1$.

⁹A partial function is one which is not necessarily everywhere defined on the universe V .

¹⁰In Section 15.3.10 we saw an example of a model $\{V, \in\}$ where the sets in ω are an infinite descending sequence, and similarly for the sets less than any non-standard integer. But these sequences do not correspond to a set in V , they are meta-sets in the meta-universe.

- No set x such that $x \in x$.

We will see later in the treatment of ordinals that the Foundation Axiom is critical in showing that the universe V is an increasing union $\bigcup_{\alpha \in On} V_\alpha$ of sets indexed by the ordinals.

15.4 CLASSES

Let $\{V, \in\}$ be a fixed model of the axioms of set theory.

Definition 15.2. If $\varphi(v, v_1, \dots, v_k)$ is a formula of $\mathcal{L}_\in$ with free variables $v, v_1, \dots, v_k$, and if $a_1, \dots, a_k$ are sets in V , then the *class* C defined by φ and $a_1, \dots, a_k$ is the meta(sub)set of V consisting of all sets x in V such that

$$\{V, \in\} \models \varphi[x, a_1, \dots, a_k]. \quad \text{def:class0 (15.21)}$$

We normally just write

$$C = \{x \mid \varphi(x, a_1, \dots, a_k)\}, \quad \text{def:class (15.22)}$$

which we read as “ C is the class of x such that $\varphi(x, a_1, \dots, a_k)$ ”. The structure $\{V, \in\}$ is understood from context.

We often write $x \in C$ instead of $\varphi(x, a_1, \dots, a_k)$.

Important Remark. When we discuss classes here, we are *not* working in (von Neumann)-Bernays-Gödel set theory. Statements such as $x \in C$ or $M \subseteq V$ or $x \in A \cup B$ are just very convenient abbreviations for first-order formulas.

15.4.1 SETS, CLASSES AND METASETS

If for some set a in V , $x \in a$ iff $\varphi(x, a_1, \dots, a_k)$, then we say C is (or corresponds to) the set a .

Every set is a class, but not conversely. A class which is not a set is called a proper class. ¹¹

If a is a set then $a = \{x \mid x \in \{a\}\}$ and so a is a class.

We saw in the discussion of (15.3) that the class $C = \{x \mid x \notin x\}$ is not a set, and so it is a proper class.

Note, however, that if a is a set in V and the x in (15.22) are restricted to a , then the corresponding class C is a set by the Separation Axiom. That is

$$\{x \in a \mid \varphi(x, a_1, \dots, a_k)\} \quad (15.23)$$

is a set in V .

Every class is a metaset, but not conversely.

A class is a meta-set, since a metasubset of V is *any* subset of V in the naive sense in the meta-universe.

¹¹The intuition is that a proper class is too large to be set. Because if a class is a subclass of some set then by the Axiom of Separation it is in fact a set.

A cardinality argument ¹² shows that if V is countable then the number of classes is countable. But the cardinality of the collection of all (meta)subsets of V , that is of all subsets in the naive sense from the countable metaset V , is uncountable. So there are certainly many metasets that are not classes

The definition of a class in the notes here is standard, see (Hils and Loeser, 2019, p147 §6.1) and (Jech, 2003, pp 5,6).

[This chapter to be completed]

¹²Countability of the collection of classes follows from the fact that if V is countable then there is only a countable number of parameters $a_1, \dots, a_k$ for each k , and only a countable number of formulas in the language $\mathcal{L}_\in$.

Chapter 16

Extras

[This chapter to be completed]

References

The books and notes closest to the material in this course are (Leary and Kristiansen, 2015; Hils and Loeser, 2019; Johnstone, 1987; Leader, 2012; Zsák, 2025; Smith, 2007; 2020).

Of the first two books, (Leary and Kristiansen, 2015) is slower and perhaps best for a first introduction. However, in the first two chapters the excessive amount of notation and detail leads to rather tedious reading. It takes 86 pages to build up the necessary framework and prove Gödel's Completeness Theorem.

The book (Hils and Loeser, 2019) is a clean but succinct treatment and contains a lot of material — good for the second time around. It takes a mere 25 (and smaller) pages for the corresponding proof of Gödel's Completeness Theorem.

The treatment in the current notes, together with class lectures, is somewhere between these two books. Perhaps you could consider this treatment as motivation for the material in Hils and Loeser's book, or as an overview of Leary and Kristiansen's book!

(Smith, 2007; 2020) are well-written with lots of helpful detail.

(Marker, 2015; 2024) are very good, at a higher level.

(Johnstone, 1987) is the text for a Cambridge course but is not so clearly written. Chapters have no further divisions and they read more like a detailed transcription of the lectures.

The notes (Leader, 2012) and (Zsák, 2025) by subsequent lecturers for the Cambridge course are in most part closely based on Johnstone's book and are easier to follow.

The above books/lecture notes are mostly freely (and legally) available through the author's websites or otherwise, with links below.

Batzoglou, Serafim. 2024. *Introduction to Incompleteness: From Gödel's Theorems to Forcing and the Continuum Hypothesis*, Birkhäuser.

Lots of material. Similar approach to Chang and Keisler for Gödel's Completeness Theorem.

Brock, David and Dennis Ritchie. 2020. *Discovering Dennis Ritchie's Lost Dissertation*.

The quaint story of Ri lost thesis. Ritchie was later heavily involved as a founder of Unix and C.

Chang, C. C. and H. Jerome Keisler. 2012. *Model Theory*, Dover. From 3rd ed Elsevier 1989

Well-written, the approach to Gödel's Completeness Theorem in these Notes is modelled on that in this book.

Church, Alonzo. 1956. *Introduction to Mathematical Logic*, Princeton University Press.

Primarily of historical interest.

Cunningham, Daniel W. 2023. *Mathematical Logic*, De Gruyter.

A more elementary approach with much detail.

Ebbinghaus, Heinz-Dieter, Jörg Flum, and Wolfgang Thomas. 2013. *Mathematical Logic*, 3rd ed., Springer.

Doxiadis, Apostolos and Christos H. Papadimitriou. 2009. *Logicomix: An Epic Search for Truth*, Bloomsbury.

A fun graphic novel in comic-book style about the life and work of Bertrand Russell and the foundations of logic.

Dudley, R. M. 2002. *Real analysis and probability*, Cambridge Studies in Advanced Mathematics, vol. 74, Cambridge University Press.

Revised reprint of the 1989 original.

Enderton, Herbert B. 2001. *A Mathematical Introduction to Logic*, 2nd ed., Academic Press.

A classic, somewhat dated.

Feferman, Anita Burdman. 1993. *From Trotsky to Gödel: The Life of Jean Van Heijenoort*, A.K. Peters.

With an Appendix by Solomon Feferman. The Fefermans knew Van Heijenoort professionally and socially.

Franzén, Torkel. 2005. *Gödel's Theorem: An Incomplete Guide to Its Use and Abuse*, A K Peters.

A well-written exposition at a mostly non-technical level.

———. 2006. *The Popular Impact of Gödel's Incompleteness Theorem*, Notices of the American Mathematical Society **53**, no. 4, 440–443.

Gaifman, Haim. 2006. *Naming and Diagonalization, from Cantor to Gödel to Kleene*.

User friendly survey.

Goldstern, Martin and Haim Judah. 1995. *The Incompleteness Phenomenon*, A K Peters. Corrected reprint, 2002.

Based on lectures given at Berkeley and Bar Ilan. Good informal exposition of various topics.

Hamkins, Joel David. 2021. *Lectures on the Philosophy of Mathematics*, MIT Press.

An informal overview of the mathematics. Well-written and informative.

Henkin, Leon. 1949. *The completeness of the first-order functional calculus*, J. Symbolic Logic **14**, no. 3, 159–166.

———. 1996. *The discovery of my completeness proofs*, Bulletin of Symbolic Logic **2**, no. 2, 127–158.

Interesting historical discussion, comments on the nature of mathematical discovery, Henkin's indirect route to his proof of the Completeness Theorem. Observation C (pp. 156–157) gives improvements in the proof as a result of Henkin's course teaching, and which is the method used in these Notes.

Hils, Martin and François Loeser. 2019. *A First Journey Through Logic*, Springer.

Similar Hilbert system and completeness proof as for these notes.

Hinman, Peter G. 2005. *Fundamentals of Mathematical Logic*, A K Peters.

Mostly well-written, introductory graduate level, enough material for 4 courses at that level.

Jech, Thomas. 2003. *Set Theory: The Third Millennium Edition, Revised and Expanded*, 3rd ed., Springer.

Well written extensive treatment of modern set theory.

Johnstone, Peter T. 1987. *Notes on Logic and Set Theory*, Cambridge University Press.

Sometimes difficult to follow. No section breaks within chapters. Reads like a transcription of Johnstone's lectures at Cambridge.

Kaye, Richard. 1991. *Models of Peano Arithmetic*, Oxford Logic Guides, vol. 15, Clarendon Press.

Keisler, H. Jerome. 2000. *Elementary Calculus: An Infinitesimal Approach*, Dover Publications. <https://people.math.wisc.edu/~hkeisler/calc.html>.

First year calculus based on non-standard techniques.

———. 2022. *Foundations of Infinitesimal Calculus*, Prindle, Weber & Schmidt. <https://people.math.wisc.edu/~hkeisler/foundations.pdf>.

Well-written and rigorous treatment of the material in (Keisler, 2000).

Knuth, Donald E. 1976. *Mathematics and Computer Science: Coping with Finiteness*, Science **194**, no. 4271, 1235–1242.

Introduces the up-arrow $\uparrow\uparrow$ notation. Well-written, entertaining, for the non-expert.

Kozen, Dexter C. 1997. *Automata and Computability*, Undergraduate Texts in Computer Science, Springer.

For a Cornell computer science course. Discusses much of the material in these Notes.

- Kunen, Kenneth. 2009. *The Foundations of Mathematics*, College Publications.
Well written, a lot of detail.
- Leader, Imre. 2012. *Logic and Set Theory*.
Based on Johnstone's book, easier to follow.
- Leary, Christopher C. and Lars Kristiansen. 2015. *A Friendly Introduction to Mathematical Logic*, 2nd ed., Milne Library.
- Marker, David. 2015. *Metamathematics*.
For a first year graduate level course, extracts from his book.
- . 2024. *An Invitation to Mathematical Logic*, Springer.
Well written graduate level text.
- Mendelson, Elliott. 2015. *Introduction to Mathematical Logic*, 6th ed., CRC Press.
A classic, but somewhat old-fashioned.
- Meyer, Albert R. and Dennis M. Ritchie. 1967. *The complexity of loop programs*, Proceedings of the ACM National Meeting, 465–469.
The first to explicitly develop the connection between primitive/general recursive functions and for/while loops in a computer program.
- Rathjen, Michael. *Proof Theory*, The Stanford Encyclopedia of Philosophy (Winter 2024 Edition).
See Section 2 for an overview discussion of Gentzen's Consistency Proof for Peano Arithmetic.
- Potter, Michael. 2004. *Set Theory and Its Philosophy: A Critical Introduction*, Oxford University Press.
The interplay between mathematical results and philosophical reflection.
- Rautenberg, Wolfgang. 2010. *A Concise Introduction to Mathematical Logic*, 3rd ed., Springer.
- Rogers, Hartley Jr. 1987. *Theory of Recursive Functions and Effective Computability*, MIT Press.
A classic. The first four Sections in particular, are a very readable introduction.
- Simpson, Stephen G. 2009. *Foundations of Mathematics*, Springer.
- Smith, Peter. 2007. *An Introduction to Gödel's Theorems*, Cambridge University Press.
Very detailed but user friendly.
- . 2020. *Gödel Without (Too Many) Tears*, Cambridge University Press.
A shorter version of his previous book *An Introduction to Gödel's Theorems*, and better for a first reading.
- Soare, Robert I. 2016. *Turing Computability: Theory and Applications*, Springer.
An excellent readable text, with extensive motivation and history.
- Srivastava, Shashi Mohan. 2013. *A Course on Mathematical Logic*, 2nd ed., Springer.
- Tanaka, Kazuyuki. 2024. *Logic and Computation 1*.
Detailed slides from a lecture course.
- Tarski, Alfred. 1931. *The concept of truth in formalized languages*, — Logic, Semantics, Metamathematics: Papers from 1923 to 1938 (edn 2, 1983), pp. 152–278.
A detailed analysis of truth from a philosophical and mathematical perspective.
- . 1944. *The Semantic Conception of Truth and the Foundations of Semantics*, Philosophy and Phenomenological Research 4, no. 3, 341–376.
A readable summary of Tarski's previous work (Tarski, 1931).
- Towsner, Henry. 2013. *Proof Theory Lecture Notes*.
Propositional and First-order Logic, Peano Arithmetic, Gentzen Sequent Calculus, Reverse Mathematics.
- . 2020a. *Goodstein's Theorem, Big Functions, and Unprovability*. Online Talks.
- . 2020b. *Goodstein's Theorem, ϵ_0 , and Unprovability*.
Notes related to the online talks.
- Turing, Alan. 1936. *On Computable Numbers, with an Application to the Entscheidungsproblem*.
Particularly see §9 for informal motivation and discussion.
- Uzquiano, Gabriel. 2022. *Quantifiers and Quantification*.

van Dalen, Dirk. 2013. *Logic and Structure*, 5th ed., Springer.

Wolf, Robert S. 2005. *A Tour Through Mathematical Logic*, Mathematical Association of America.
A user friendly survey.

Zach, Richard. 2016. *The Open Logic Text*, available at <https://openlogicproject.org/>.

Zsák, András. 2025. *Logic and Set Theory*.

Based on Johnstone's book, easier to follow.